

AURIX

Microcontroller Infineon Software Architecture

User's Manual

ADC Driver

Release V6.7

Feb 2017

Edition Feb 2017

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2017 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

CONFIDENTIAL

Date	Version	Change Description
2017-02-09	6.7	Updated section 4.5.14.5 for source information.
2016-08-08	6.6	Updated review comments #10612.
2016-08-08	6.5	Updated the point 4 statement in section 8.5 based on the JIRA 0000051124-322.
2016-05-31	6.4	Updated the below sections for review comments #9823 - In Section 6.2 API functions 'Origin' is added.
2016-04-01	6.3	Updated the below sections for Usermode support #0000051261-231 - Included below functions for newly added macros. 4.2.25 AdcRunningInUser0Mode 4.2.26 AdcUserModeInitApiEnable 4.2.27 AdcUserModeDeInitApiEnable 4.2.28 AdcUserModeRuntimeApiEnable - Updated the I/O mode section for all functions in 6.2 API Functions. - Updated figure 2 and figure 3 in section 2.2. - Updated figure 9 in section 3.
2015-11-30	6.2	Updated review comments #REV_009097 Removed the given comment for JIRA-0000051100-230 in below sections which are not required. - 4.5.8 AdcPostCal - 6.2.19 Adc_17_GetStartupCalStatus, - 6.2.20 Adc_17_SoftCalWrLongInit and - 6.2.21 Adc_17_SoftCalWrVagnd
2015-11-27	6.1	Updated ADC_TC.62 Errata related comments for JIRA - 0000051100-230 in below sections - Description section in 4.5.8 AdcPostCal and 4.5.16.2 AdcKernelChResolution - Implementation Comments section in 6.2.19 Adc_17_GetStartupCalStatus, 6.2.20 Adc_17_SoftCalWrLongInit and 6.2.21 Adc_17_SoftCalWrVagnd
2015-10-15	6.0	Updated review comment #8793. - Revision history and dates are updated
2015-10-13	5.9	Included the safety error for ADC interrupts based on JIRA 0000051100-167 - Adc_IsrSrn0AdcRS0 - Adc_IsrSrn1AdcRS1 - Adc_IsrSrn1CG0AdcRS3 - Adc_IsrSrn2AdcChEvt
2015-09-09	5.8	Updated review comments #REV_008476. Corrected the sentence in Adc_Init and Adc_DeInit functions.
2015-09-02	5.7	Updated the 6.2.1 Adc_Init and 6.2.3 Adc_DeInit, to include the safety error ADC_E_CLC_ERROR.
2015-05-28	5.6	Updated the description of parameter AdcResultAlignment for the handling result alignment Limitation of using continuous conversion is updated in description of parameter AdcGroupConversionMode and section 8.14, 8.16.2
2015-05-18	5.5	Updated the description of parameter AdcGtmTriggerRef with Gtm container name limitation
2015-04-29	5.4	Review comments updated REV_007564.

CONFIDENTIAL

Date	Version	Change Description
		Limitation section updated with the not supported apis and parameters in Non Autosar Result Polling mode.
2015-04-25	5.3	Added parameters AdcNonAutosarResultPolling and AdcResetSfrAtInit Added Apis Adc_17_GetGroupResult() and Adc_17_GetChannelStatus() Added Example usage of Non Autosar Result Polling Mode
2015-03-10	5.2	REV_007187 – corrected references.
2015-02-11	5.1	Table 86 AdcNotofication : Range is written for Selectable and loadable
2015-01-12	5.0	ADC1_RS0 interrupt node is used for Request Source 0 and Request Source 3 of Aurix EP devices for UTP AI00254394. Table 218 - SRN and Interrupt Source Mapping is updated
2014-12-03	4.9	I/O mode is added for all the APIs. UTP:AI00251090
2014-09-09	4.8	Table 73 - Analog Channel Mapping for Aurix EP updated for TC22x
2014-08-07	4.7	Chapter 9 added with the Assumption for TC27xDB
2014-08-05	4.6	Configuration parameter AdcCcu6Synchronization and AdcCcu6ModuleRef are added for the devices with metal fix for cross coupling effect
2014-07-14	4.5	Safety Information chapter removed Description of the parameter AdcAnalogClockDivider, AdcDigitalArbiterClockDivider and AdcArbitrationRoundLength are modified for cross coupling issue.
2014-05-30	4.4	Table 71 - Analog Channel Mapping for Aurix EP updated for TC237
2014-03-25	4.3	Updated the TC27xCA information.
2014-02-28	4.2	Updated Table 216 SRN and Interrupt Source Mapping in section 6.3 Interrupt Handling. Background Scan request source Interrupt handling SRN is modified for SAR ADC. Added configuration parameters AdcRefPrechargeControl, AdcLowPowerSupplyVoltage, AdcRequestSource3Prio, AdcRS3TriggerSeqCount, AdcRS3InternalTrigger Updated the configuration parameter AdcGroupRequestSource Updated for TC23x
2014-02-03	4.1	Added API Adc_17_SoftCalWrLongInit() and Adc_17_SoftCalWrVagnd(). Removed the API Adc_17_SoftCalibration()
2013-11-11	4.0	Review comments updated REV_004850
2013-10-11	3.5	Requirement traceability done for Condition of Use chapter.
2013-09-25	3.4	Added Adc_17_SoftCalibration API information in section 6.2.20.
2013-08-26	3.3	ISO26262 support and TC29x support
2013-07-11	3.2	Updated for TC26x
2013-05-30	3.1	Limitation of Adc_17_GetStartupCalStatus API is updated in section 6.2.18 and 10.2
2013-04-02	3.0	Review comments updated
2013-03-19	2.3	Updated for TC27xB-Step
2012-12-18	2.2	File structure updated
2012-11-19	2.1	AdcAnChannelNum description updated for UTP AI00099818
2012-08-09	2.0	Review comments updated File structure updated.
2012-06-27	1.3	Derived configuration parameter section updated
2012-06-06	1.2	Added example configuration in section 8 for Emux Scan Mode and ADC group trigger using Pwm signal

CONFIDENTIAL

Date	Version	Change Description
2012-05-16	1.1	Updated for TC27xA step
2011-11-09	1.0	IP name changed from "ADC" to "VADC" in Figure 1.
2011-10-21	0.2	Review comment updated. ADC_E_CHANNEL_SAMPLING_TIMEOUT Det information removed. Information about Suspend Resume functionality removed.
2011-10-18	0.1	Initial version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?

Your feedback will help us to continuously improve the quality of this document.

Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents	Page
1 Introduction	20
1.1 Scope	20
1.2 Abbreviations.....	20
1.3 References	21
2 ADC Driver Overview	22
2.1 Software Hardware Mapping	22
2.1.1 SYSTEM CLOCK	23
2.1.2 PORT	23
2.1.3 ERU	24
2.1.4 CCU6.....	24
2.1.5 GTM	24
2.1.6 DMA	24
2.1.6.1 EMUX SCAN mode DMA usage	24
2.1.6.2 NON AUTOSAR Result Handling mode DMA usage	24
2.1.7 VADC	24
2.2 Software Driver Description	25
2.2.1 ADC Driver Legend	29
2.2.2 Possible combinations of Group properties	31
2.2.3 Possible number of channels for Group request sources	31
2.2.4 EMUX SCAN	32
3 File Structure	32
4 Configuration Documentation	34
4.1 Configuration Concept	34
4.1.1 Configuration Class	34
4.1.2 Configuration Variant	35
4.1.3 How to read the Configuration Class field.....	35
4.2 Container: AdcGeneral.....	35
4.2.1 AdcDevErrorDetect	35
4.2.2 AdcDeinitApi.....	36
4.2.3 AdcEnableLimitCheck	36
4.2.4 AdcEnableQueuing	37
4.2.5 AdcEnableStartStopGroupApi.....	37
4.2.6 AdcGrpNotifCapability	38
4.2.7 AdcHwTriggerApi	38
4.2.8 AdcPriorityImplementation	39
4.2.9 AdcReadGroupApi	40
4.2.10 AdcResultAlignment.....	40
4.2.11 AdcVersionInfoApi.....	43
4.2.12 AdcUseEmux.....	44
4.2.13 AdcMaxChConvTimeCount.....	44
4.2.14 AdcNonAutosarDmaResultHandling	45
4.2.15 AdcResultAccumulation	46
4.2.16 AdcPBFixedAddress	47
4.2.17 AdcDebugSupport	47
4.2.18 AdcSameChannelQueuing.....	48
4.2.19 AdcMasterSlaveSync	48
4.2.20 AdcAlternateRefMode	49
4.2.21 AdcSampleAndHoldUnitAvailable	50
4.2.22 AdcCcu6Synchronization	50
4.2.23 AdcNonAutosarResultPolling	51
4.2.24 AdcResetSfrAtInit	52
4.2.25 AdcRunningInUser0Mode	52
4.2.26 AdcUserModeInitApiEnable	53
4.2.27 AdcUserModeDeInitApiEnable.....	53
4.2.28 AdcUserModeRuntimeApiEnable.....	54
4.3 Container: AdcSafety	54

Table of Contents	Page
4.3.1 AdcSafetyEnable	54
4.3.2 AdcInitCheckApi	55
4.4 Container: AdcConfigSet	55
4.4.1 AdcSleepMode	55
4.4.2 AdcSystemClock	56
4.4.3 AdcEmuxSelectGrp0	56
4.4.4 AdcEmuxSelectGrp1	57
4.4.5 AdcAnalogClockDivider	57
4.4.6 AdcDigitalArbiterClockDivider	58
4.4.7 AdcSHSClockDivider	58
4.4.8 AdcOperationMode	59
4.4.9 AdcCalibrationOrder	60
4.4.10 AdcCalMaxTime	60
4.4.11 AdcCluster0AltRefDef	60
4.4.12 AdcCluster1AltRefDef	61
4.4.13 AdcCluster2AltRefDef	62
4.4.14 AdcRefPrechargeControl	62
4.4.15 AdcLowPowerSupplyVoltage	63
4.4.16 AdcCcu6ModuleRef	63
4.4.17 Container: AdcGlobInputClass	63
4.4.17.1 AdcGlobChSampleTime	64
4.4.17.2 AdcGlobChResolution	64
4.4.17.3 AdcGlobEmuxChSampleTime	65
4.5 Container: AdcHWUnit	65
4.5.1 AdcClockSource	65
4.5.2 AdcHwUnitId	66
4.5.3 AdcPrescale	66
4.5.4 AdcArbitrationRoundLength	66
4.5.5 AdcRequestSource0Prio	67
4.5.6 AdcRequestSource1Prio	68
4.5.7 AdcRequestSource3Prio	68
4.5.8 AdcPostCal	69
4.5.9 AdcDmaChannelRef	70
4.5.10 AdcSyncConvMode	70
4.5.11 AdcAcceleratedTiming	70
4.5.12 AdcShortSampleTime	71
4.5.13 AdcSafetySignal	72
4.5.14 Container: AdcChannel	72
4.5.14.1 AdcChannelConvTime	72
4.5.14.2 AdcChannelHighLimit	73
4.5.14.3 AdcChannelId	73
4.5.14.4 AdcChannelLimitCheck	74
4.5.14.5 AdcChannelLowLimit	74
4.5.14.6 AdcChannelRangeSelect	75
4.5.14.7 AdcChannelRefVoltsrcHigh	76
4.5.14.8 AdcChannelRefVoltsrcLow	76
4.5.14.9 AdcAnChannelNum	77
4.5.14.10 AdcInputClassSelection	81
4.5.14.11 AdcChannelResolution	82
4.5.14.12 AdcChannelSampTime	82
4.5.14.13 AdcSyncChannel	82
4.5.15 Container: AdcGroup	83
4.5.15.1 AdcGroupAccessMode	83
4.5.15.2 AdcGroupConversionMode	84
4.5.15.3 AdcGroupId	85
4.5.15.4 AdcGroupPriority	85
4.5.15.5 AdcGroupReplacement	86
4.5.15.6 AdcGroupTriggSrc	87
4.5.15.7 AdcHwTrigSignal	87

Table of Contents	Page
4.5.15.8 AdcHwTrigTimer.....	88
4.5.15.9 AdcNotification	88
4.5.15.10 AdcStreamingBufferMode	89
4.5.15.11 AdcStreamingNumSamples	89
4.5.15.12 AdcGroupDefinition	90
4.5.15.13 AdcGroupRequestSource	91
4.5.15.14 AdcHwTrigType.....	91
4.5.15.15 AdcHwExtTrigSelect	92
4.5.15.16 AdcEruTriggRef.....	93
4.5.15.17 AdcEruGateRef.....	94
4.5.15.18 AdcHwGateSignal	94
4.5.15.19 AdcHwGatePin	95
4.5.15.20 AdcGtmTimerPeriod.....	98
4.5.15.21 AdcGtmTriggerRef	99
4.5.15.22 AdcEmuxScanEnable	100
4.5.15.23 AdcChannelEmuxSelect	101
4.5.15.24 AdcGroupBufferMarker	102
4.5.15.25 AdcRS3InternalTrigger.....	102
4.5.15.26 AdcRS3TriggerSeqCount.....	103
4.5.16 Container: AdcKernelInputClass	103
4.5.16.1 AdcKernelChSampleTime	103
4.5.16.2 AdcKernelChResolution	104
4.5.16.3 AdcKernelEmuxSampleTime	104
4.6 Container: AdcPublishedInformation	105
4.6.1 AdcChannelValueSigned	105
4.6.2 AdcGroupFirstChannelFixed	105
4.6.3 AdcMaxChannelResolution	106
4.7 Container: CommonPublishedInformation	106
4.7.1 ArMajorVersion.....	106
4.7.2 ArMinorVersion.....	106
4.7.3 ArPatchVersion	107
4.7.4 SwMajorVersion	107
4.7.5 SwMinorVersion	107
4.7.6 SwPatchVersion	108
4.7.7 ModuleId.....	108
4.7.8 VendorId.....	108
4.7.9 VendorApilInfix.....	108
4.7.10 Release	109
4.8 Derived Configuration Parameters.....	109
4.8.1 ADC_MAX_GROUPS	109
4.8.2 ADCx_QM_KERNEL_INDEX.....	110
4.8.3 ADCx_ASIL0_KERNEL_INDEX.....	110
4.8.4 ADCx_ASIL1_KERNEL_INDEX.....	111
4.8.5 ADC_VALUE_GROUP_TYPE	111
4.8.6 ADC_KERNEL_USED_COUNT	111
4.8.7 ADC_QM_KERNEL_USED_COUNT	112
4.8.8 ADC_ASIL0_KERNEL_USED_COUNT	112
4.8.9 ADC_ASIL1_KERNEL_USED_COUNT	112
4.8.10 ADC_USED_KERNELS.....	113
4.8.11 ADCx_KERNEL_USED	113
4.8.12 ADC_KERNEL_REQSRC0_USED.....	113
4.8.13 ADC_KERNEL_REQSRC1_USED.....	114
4.8.14 ADC_KERNEL_REQSRC2_USED.....	114
4.8.15 ADCx_REQSRCy.....	114
4.8.16 ADC_REQSRCy	115
4.8.17 ADC_WAIT_ON_STOP	115
4.8.18 ADC_GROUP_EMUX_SCAN	116
4.8.19 ADC_LIMIT_CHECK_USED.....	116
4.8.20 ADC_DMA_MAX_TRANSFER	116

Table of Contents	Page
4.8.21 ADC_MAX_KERNELS	117
4.8.22 ADC_MAX_SHS_CLUSTERS	117
4.8.23 ADC_KERNELS_PER_CLUSTER	117
4.8.24 ADC_ALTERNATE_REF_CH_USED	118
4.8.25 ADC_RESULT_HANDLING_MODE	118
4.8.26 ADC_RESULT_ACCUMULATION	118
4.8.27 ADC_SWOSM_INTERRUPT_HANDLER	119
4.8.28 ADC_TIMER_TRIGG	119
4.8.29 ADC_ERU_TRIGG	120
4.8.30 ADC_ERU_GATING	120
5 Published Parameters	120
5.1 VendorID	120
5.2 ModuleID	121
5.3 InstanceID	121
5.4 ArReleaseMajorVersion	121
5.5 ArReleaseMinorVersion	121
5.6 ArReleaseRevisionVersion	121
6 API Documentation	121
6.1 API Type Definition	122
6.1.1 Adc_ConfigType	122
6.1.2 Adc_KernelConfigType	122
6.1.3 Adc_HwUnitCfgType	123
6.1.4 Adc_GroupCfgType	124
6.1.5 Adc_ChannelCfgType	126
6.1.6 Adc_GlobalCfgType	127
6.1.7 Adc_ChannelType	128
6.1.8 Adc_GroupType	129
6.1.9 Adc_ValueGroupType	129
6.1.10 Adc_ClockSourceType	129
6.1.11 Adc_PrescaleType	130
6.1.12 Adc_ConversionTimeType	130
6.1.13 Adc_SamplingTimeType	130
6.1.14 Adc_VoltageSourceType	130
6.1.15 Adc_ResolutionType	131
6.1.16 Adc_StatusType	131
6.1.17 Adc_TriggerSourceType	131
6.1.18 Adc_GroupConvModeType	131
6.1.19 Adc_GroupPriorityType	132
6.1.20 Adc_GroupDefType	132
6.1.21 Adc_StreamNumSampleType	132
6.1.22 Adc_HwUnitType	132
6.1.23 Adc_StreamBufferModeType	133
6.1.24 Adc_GroupAccessModeType	133
6.1.25 Adc_HwTriggerSignalType	134
6.1.26 Adc_HwTriggerTimerType	134
6.1.27 Adc_TriggSrcArbLevelType	134
6.1.28 Adc_PriorityImplementationType	134
6.1.29 Adc_GroupReplacementType	135
6.1.30 Adc_ChannelRangeSelectType	135
6.1.31 Adc_ResultAlignmentType	135
6.2 API Functions	136
6.2.1 Adc_Init	136
6.2.2 Adc_SetupResultBuffer	138
6.2.3 Adc_DeInit	139
6.2.4 Adc_InitCheck	140
6.2.5 Adc_StartGroupConversion	141
6.2.6 Adc_StopGroupConversion	143

Table of Contents	Page
6.2.7 Adc_ReadGroup	145
6.2.8 Adc_EnableHardwareTrigger	147
6.2.9 Adc_DisableHardwareTrigger	149
6.2.10 Adc_EnableGroupNotification	150
6.2.11 Adc_DisableGroupNotification	151
6.2.12 Adc_GetGroupStatus	152
6.2.13 Adc_GetStreamLastPointer	154
6.2.14 Adc_GetVersionInfo	156
6.2.15 Adc_17_StartGtmTimer	157
6.2.16 Adc_17_StopGtmTimer	158
6.2.17 Adc_17_SetEruTrigger	159
6.2.18 Adc_17_ResetEruTrigger	159
6.2.19 Adc_17_GetStartupCalStatus	160
6.2.20 Adc_17_SoftCalWrLongInit	161
6.2.21 Adc_17_SoftCalWrVagnd	162
6.2.22 Adc_17_GetGroupResult	163
6.2.23 Adc_17_GetChannelStatus	164
6.3 Interrupt Handling	165
6.4 Exclusive Area (EA) Information	167
6.4.1 StartGroup	167
6.4.2 StopGroup	167
6.4.3 ReadGroup	168
6.4.4 EnableHwTrig	168
6.4.5 DisableHwTrig	168
6.4.6 GetGrpStatus	168
6.4.7 GetStreamLastPtr	169
6.4.8 ScheduleStart	169
6.4.9 ScheduleStop	169
6.4.10 ScheduleNext	169
6.4.11 PushQueue	170
6.4.12 PopQueue	170
7 Error Classification	170
8 Example Usage	171
8.1 Configuration of ADC Driver	171
8.1.1 Configuration of VADC Clock	172
8.1.2 Configuration of AdcMaxChConvTimeCount	174
8.1.3 Configuration of ADC Channel Parameters	175
8.1.4 Configuration of SW Trigger Group Parameters	176
8.1.5 Configuration of Timer Trigger Group	179
8.1.6 Sample Configuration of the Driver	181
8.1.7 Sample Configuration to trigger HW group using PWM signal	186
8.1.8 Sample Configuration of limit checking	189
8.1.8.1 ADC Driver Configuration	189
8.1.8.2 Working of limit checking	193
8.1.9 Sample Configuration of the Driver in Non AUTOSAR Result handling mode	193
8.1.9.1 ADC Driver Configuration	194
8.1.9.2 Interrupt priority Configuration	196
8.1.9.3 DMA Library function	197
8.2 Initialization of ADC Driver and initialization Group result buffer pointers	198
8.3 Configuration and Usage of ADC Driver for ADC_PRIORITY_NONE and No Queue	199
8.4 Configuration and Usage of ADC Driver for ADC_PRIORITY_NONE and with Queue	200
8.5 Configuration and Usage of ADC Driver in ADC_PRIORITY_HW mechanism	203
8.6 Configuration and Usage of ADC Driver in ADC_PRIORITY_HW_SW mechanism	203
8.6.1 Effective Group Priority in ADC_PRIORITY_HW_SW mechanism	205
8.7 Adc Group Replacement mechanism in ADC_PRIORITY_HW_SW	205
8.7.1 Abort/Restart mechanism	205
8.8 Group Result Access using Adc_GetStreamLastPointer API	207

Table of Contents	Page
8.9 Configuration and Usage of ADC Driver in EMUX mode.....	208
8.9.1 ADC Driver Configuration.....	208
8.9.2 Interrupt priority Configuration.....	211
8.10 Configuration and Usage of ADC Driver in EMUX SCAN mode.....	212
8.10.1 ADC Driver Configuration.....	212
8.10.2 Interrupt priority Configuration.....	215
8.11 Configuration and Usage of ADC Driver in Same Channel Queuing.....	216
8.11.1 ADC Driver Configuration.....	216
8.11.2 Interrupt priority Configuration.....	219
8.12 Configuration and Usage of ADC Driver in Synchronization of conversion	219
8.12.1 ADC Driver Configuration.....	220
8.12.2 Interrupt priority Configuration.....	223
8.13 Usage of ADC Driver in HW groups with Background Request Source	223
8.14 Usage of ADC Driver in Non AUTOSAR Result handling mode	225
8.15 Configuration and Usage of ADC Driver with Safety	227
8.15.1 ADC Driver Configuration.....	227
8.15.2 Interrupt priority Configuration.....	230
8.15.3 Usage of Safety Group.....	230
8.16 Configuration and Usage of ADC Driver in Result Polling Mode	232
8.16.1 ADC Driver Configuration.....	232
8.16.2 Usage of Result Polling Mode	235
9 Assumptions and Limitations	236
9.1 Assumptions and Deviations from the Software Specification.....	236
9.2 Limitations	237
9.2.1 Software Limitation in ADC_PRIORITY_HW mechanism	238
9.2.2 Software Limitation in ADC_PRIORITY_HW_SW mechanism.....	238
9.2.3 Software Configuration Limitation	239
9.2.4 Software Limitation in ADC_NON_AUTOSAR DMA Mode	239
9.2.5 Software Limitation in Safety feature	239
9.2.6 Software Limitation in ADC_NON_AUTOSAR Result Polling Mode	239
9.3 Hardware Features Not Supported	239

List of Figures

Page

Figure 1	Software Hardware Mapping of ADC Driver	23
Figure 2	Interfaces of SAR ADC driver	28
Figure 3	ADC Group result buffer type (AdcGroupAccessMode - Refer 4.5.15.1)	29
Figure 4	ADC Group with HW trigger type	29
Figure 5	ADC Group with SW trigger type	30
Figure 6	Possible combinations of Group trigger type, conversion mode, Group access mode, and buffer mode	31
Figure 7	Example Group in EMUX SCAN enabled	32
Figure 8	File Structure of ADC Driver.....	33
Figure 9	Result Alignment for Standard Conversions	41
Figure 10	Result Alignment for Accumulated Conversions.....	42
Figure 11	Result Alignment for NonAS DMA Result Handling with No Accumulation	43
Figure 12	AURIX VADC Clock Control Structure	172
Figure 13	Result Monitoring through Standard compare mode limit checking	193
Figure 14	ADC_PRIORITY_NONE and Queuing Enabled mechanism.....	202
Figure 15	Interruption of Groups in HW_SW Priority	204
Figure 16	Abort/Restart mechanism.....	206
Figure 17	ADC Group Result Buffer Access	208
Figure 18	ADC Emux Mode	212
Figure 19	ADC Emux Scan Mode	216
Figure 20	Non AUTOSAR Result Handling.....	227

List of Tables
Page

Table 1	Abbreviations used in this document	20
Table 2	Types of ADC & Not Applicable Parameters in Aurix Devices	25
Table 3	Possible number of channels for Group request sources	31
Table 4	File Contents	33
Table 5	Configuration Class Example 1	35
Table 6	Configuration Class Example 2	35
Table 7	AdcDevErrorDetect	35
Table 8	AdcDeinitApi	36
Table 9	AdcEnableLimitCheck	36
Table 10	AdcEnableQueuing	37
Table 11	AdcEnableStartStopGroupApi	37
Table 12	AdcGrpNotifCapability	38
Table 13	AdcHwTriggerApi	38
Table 14	AdcPriorityImplementation	39
Table 15	AdcReadGroupApi	40
Table 16	AdcResultAlignment	40
Table 17	AdcVersionInfoApi	43
Table 18	AdcUseEmux	44
Table 19	AdcMaxChConvTimeCount	44
Table 20	AdcNonAutosarDmaResultHandling	45
Table 21	AdcResultAccumulation	46
Table 22	AdcPBFixedAddress	47
Table 23	AdcDebugSupport	47
Table 24	AdcSameChannelQueuing	48
Table 25	Specification of AdcMasterSlaveSync	48
Table 26	Specification of AdcAlternateRefMode	49
Table 27	Specification of AdcSampleAndHoldUnitAvailable	50
Table 28	Specification of AdcCcu6Synchronization	50
Table 29	Specification of AdcNonAutosarResultPolling	51
Table 30	Specification of AdcResetSfrAtInit	52
Table 31	Specification of AdcRunningInUser0Mode	52
Table 32	Specification of AdcUserModeInitApiEnable	53
Table 33	Specification of AdcUserModeDeInitApiEnable	53
Table 34	Specification of AdcUserModeRuntimeApiEnable	54
Table 35	Specification of AdcSafetyEnable	54
Table 36	AdcInitCheckApi	55
Table 37	AdcSleepMode	55
Table 38	AdcSystemClock	56
Table 39	AdcEmuxSelectGrp0	56
Table 40	AdcEmuxSelectGrp1	57
Table 41	AdcAnalogClockDivider	57
Table 42	AdcDigitalArbiterClockDivider	58
Table 43	Specification of AdcSHSClockDivider	58
Table 44	Specification of AdcOperationMode	59
Table 45	Specification of AdcCalibrationOrder	60
Table 46	Specification of AdcCalMaxTime	60
Table 47	Specification of AdcCluster0AltRefDef	60
Table 48	Specification of AdcCluster1AltRefDef	61
Table 49	Specification of AdcCluster2AltRefDef	62
Table 50	Specification of AdcRefPrechargeControl	62
Table 51	Specification of AdcLowPowerSupplyVoltage	63
Table 52	Specification of AdcCcu6ModuleRef	63
Table 53	AdcGlobChSampleTime	64
Table 54	AdcGlobChResolution	64
Table 55	AdcGlobEmuxChSampleTime	65
Table 56	AdcClockSource	65
Table 57	AdcHwUnitId	66
Table 58	AdcPrescale	66

List of Tables
Page

Table 59	AdcArbitrationRoundLength	66
Table 60	AdcRequestSource0Prio	67
Table 61	AdcRequestSource1Prio	68
Table 62	AdcRequestSource3Prio	68
Table 63	AdcPostCal	69
Table 64	AdcDmaChannelRef	70
Table 65	Specification of AdcSyncConvMode	70
Table 66	Specification of AdcAcceleratedTiming	70
Table 67	Specification of AdcShortSampleTime	71
Table 68	Specification of AdcSafetySignal	72
Table 69	AdcChannelConvTime	72
Table 70	AdcChannelHighLimit	73
Table 71	AdcChannelId	73
Table 72	AdcChannelLimitCheck	74
Table 73	AdcChannelLowLimit	74
Table 74	AdcChannelRangeSelect	75
Table 75	AdcChannelRefVoltsrcHigh	76
Table 76	AdcChannelRefVoltsrcLow	76
Table 77	AdcAnChannelNum	77
Table 78	Analog Channel Mapping for Aurix HE	78
Table 79	Analog Channel Mapping for Aurix EP	80
Table 80	AdcInputClassSelection	81
Table 81	AdcChannelResolution	82
Table 82	AdcChannelSampTime	82
Table 83	Specification of AdcSyncChannel	82
Table 84	AdcGroupAccessMode	83
Table 85	AdcGroupConversionMode	84
Table 86	AdcGroupId	85
Table 87	AdcGroupPriority	85
Table 88	AdcGroupReplacement	86
Table 89	AdcGroupTriggSrc	87
Table 90	AdcHwTrigSignal	87
Table 91	AdcHwTrigTimer	88
Table 92	AdcNotification	88
Table 93	AdcStreamingBufferMode	89
Table 94	AdcStreamingNumSamples	89
Table 95	AdcGroupDefinition	90
Table 96	AdcGroupRequestSource	91
Table 97	AdcHwTrigType	91
Table 98	AdcHwExtTrigSelect	92
Table 99	AdcEruTriggRef	93
Table 100	AdcEruGateRef	94
Table 101	AdcHwGateSignal	94
Table 102	AdcHwGatePin	95
Table 103	Digital Connections of Aurix	96
Table 104	AdcGtmTimerPeriod	98
Table 105	AdcGtmTriggerRef	99
Table 106	AdcEmuxScanEnable	100
Table 107	AdcChannelEmuxSelect	101
Table 108	Specification of AdcGroupBufferMarker	102
Table 109	Specification of AdcRS3InternalTrigger	102
Table 110	Specification of AdcRS3TriggerSeqCount	103
Table 111	AdcKernelChSampleTime	103
Table 112	AdcKernelChResolution	104
Table 113	AdcKernelEmuxSampleTime	104
Table 114	AdcChannelValueSigned	105
Table 115	AdcGroupFirstChannelFixed	105
Table 116	AdcMaxChannelResolution	106
Table 117	ArMajorVersion	106

List of Tables
Page

Table 118	ArMinorVersion	106
Table 119	ArPatchVersion	107
Table 120	SwMajorVersion	107
Table 121	SwMinorVersion	107
Table 122	SwPatchVersion	108
Table 123	ModuleId	108
Table 124	VendorId	108
Table 125	VendorApilInfix	108
Table 126	Release	109
Table 127	ADC_MAX_GROUPS	109
Table 128	ADCx_QM_KERNEL_INDEX	110
Table 129	ADCx_ASIL0_KERNEL_INDEX	110
Table 130	ADCx_ASIL1_KERNEL_INDEX	111
Table 131	ADC_VALUE_GROUP_TYPE	111
Table 132	ADC_KERNEL_USED_COUNT	111
Table 133	ADC_QM_KERNEL_USED_COUNT	112
Table 134	ADC_ASIL0_KERNEL_USED_COUNT	112
Table 135	ADC_ASIL1_KERNEL_USED_COUNT	112
Table 136	ADC_USED_KERNELS	113
Table 137	ADCx_KERNEL_USED	113
Table 138	ADC_KERNEL_REQSRC0_USED	113
Table 139	ADC_KERNEL_REQSRC1_USED	114
Table 140	ADC_KERNEL_REQSRC2_USED	114
Table 141	ADCx_REQSRCy	114
Table 142	ADC_REQSRCy	115
Table 143	ADC_WAIT_ON_STOP	115
Table 144	ADC_GROUP_MUX_SCAN	116
Table 145	ADC_LIMIT_CHECK_USED	116
Table 146	ADC_DMA_MAX_TRANSFER	116
Table 147	ADC_MAX_KERNELS	117
Table 148	ADC_MAX_SHS_CLUSTERS	117
Table 149	ADC_KERNELS_PER_CLUSTER	117
Table 150	ADC_ALTERNATE_REF_CH_USED	118
Table 151	ADC_RESULT_HANDLING_MODE	118
Table 152	ADC_RESULT_ACCUMULATION	118
Table 153	ADC_SWOSM_INTERRUPT_HANDLER	119
Table 154	ADC_TIMER_TRIGG	119
Table 155	ADC_ERU_TRIGG	120
Table 156	ADC_ERU_GATING	120
Table 157	VendorID	120
Table 158	ModuleID	121
Table 159	InstanceID	121
Table 160	ArReleaseMajorVersion	121
Table 161	ArReleaseMinorVersion	121
Table 162	ArReleaseRevisionVersion	121
Table 163	Adc_ConfigType	122
Table 164	Adc_KernelConfigType	122
Table 165	Adc_HwUnitCfgType	123
Table 166	Adc_GroupCfgType	124
Table 167	Adc_ChannelCfgType	126
Table 168	Adc_GlobalCfgType	127
Table 169	Adc_ChannelType	128
Table 170	Adc_GroupType	129
Table 171	Adc_ValueGroupType	129
Table 172	Adc_ClockSourceType	129
Table 173	Adc_PrescaleType	130
Table 174	Adc_ConversionTimeType	130
Table 175	Adc_SamplingTimeType	130
Table 176	Adc_VoltageSourceType	130

List of Tables
Page

Table 177	Adc_ResolutionType	131
Table 178	Adc_StatusType	131
Table 179	Adc_TriggerSourceType	131
Table 180	Adc_GroupConvModeType	131
Table 181	Adc_GroupPriorityType	132
Table 182	Adc_GroupDefType	132
Table 183	Adc_StreamNumSampleType	132
Table 184	Adc_HwUnitType	132
Table 185	Adc_StreamBufferModeType	133
Table 186	Adc_GroupAccessModeType	133
Table 187	Adc_HwTriggerSignalType	134
Table 188	Adc_HwTriggerTimerType	134
Table 189	Adc_TriggerSrcArbLevelType	134
Table 190	Adc_PriorityImplementationType	134
Table 191	Adc_GroupReplacementType	135
Table 192	Adc_ChannelRangeSelectType	135
Table 193	Adc_ResultAlignmentType	135
Table 194	Specification of Adc_Init API	136
Table 195	Specification of Adc_SetupResultBuffer API	138
Table 196	Specification of Adc_DeInit	139
Table 197	Specification of API Adc_InitCheck	140
Table 198	Specification of Adc_StartGroupConversion	141
Table 199	Specification of Adc_StopGroupConversion	143
Table 200	Specification of Adc_ReadGroup API	145
Table 201	Specification of Adc_EnableHardwareTrigger	147
Table 202	Specification of Adc_DisableHardwareTrigger	149
Table 203	Specification of Adc_EnableGroupNotification	150
Table 204	Specification of Adc_DisableGroupNotification	151
Table 205	Specification of Adc_GetGroupStatus	152
Table 206	Specification of Adc_GetStreamLastPointer	154
Table 207	Specification of Adc_GetVersionInfo	156
Table 208	Specification of Adc_17_StartGtmTimer	157
Table 209	Specification of Adc_17_StopGtmTimer	158
Table 210	Specification of Adc_17_SetEruTrigger	159
Table 211	Specification of Adc_17_ResetEruTrigger	159
Table 212	Specification of Adc_17_GetStartupCalStatus	160
Table 213	Specification of Adc_17_SoftCalWrLongInit	161
Table 214	Specification of Adc_17_SoftCalWrVagnd	162
Table 215	Specification of Adc_17_GetGroupResult	163
Table 216	Specification of Adc_17_GetChannelStatus	164
Table 217	SRN and Interrupt Source Mapping	165
Table 218	Specification of StartGroup	167
Table 219	Specification of StopGroup	167
Table 220	Specification of ReadGroup	168
Table 221	Specification of EnableHwTrig	168
Table 222	Specification of DisableHwTrig	168
Table 223	Specification of GetGrpStatus	168
Table 224	Specification of GetStreamLastPtr	169
Table 225	Specification of ScheduleStart	169
Table 226	Specification of ScheduleStop	169
Table 227	Specification of ScheduleNext	169
Table 228	Specification of PushQueue	170
Table 229	Specification of PopQueue	170
Table 230	Error Classification	170
Table 231	Example - ADC Channel Configuration	175
Table 232	Example – Software, continuous, streaming result access group configuration	177
Table 233	Example – Software, oneshot, single result access group configuration	178
Table 234	Group configuration for timer trigger group	179
Table 235	GtmTriggerForAdc_x configuration	181

List of Tables
Page

Table 236	ATOM0_CH5 configuration	181
Table 237	Container: AdcGeneral.....	181
Table 238	Container: AdcConfigSet.....	182
Table 239	Container: AdcHWUnit.....	182
Table 240	Container: AdcGroup	182
Table 241	Container: AdcChannel	183
Table 242	Container: AdcKernelInputClass.....	184
Table 243	Container: AdcGeneral.....	186
Table 244	Container: AdcConfigSet.....	186
Table 245	Container: AdcHWUnit.....	187
Table 246	Container: AdcGroup	187
Table 247	Container: AdcChannel	188
Table 248	Container: AdcKernelInputClass.....	188
Table 249	PwmGeneral configuration.....	188
Table 250	PwmChannel configuration	189
Table 251	GtmTriggerForAdc_x configuration	189
Table 252	TOM0_CH6 configuration	189
Table 253	Container: AdcGeneral.....	190
Table 254	Container: AdcConfigSet.....	190
Table 255	Container: AdcHWUnit.....	190
Table 256	Container: AdcGroup	190
Table 257	Container: AdcChannel	191
Table 258	Container: AdcGlobInputClass.....	192
Table 259	Container: AdcGeneral.....	194
Table 260	Container: AdcConfigSet.....	194
Table 261	Container: AdcHWUnit.....	194
Table 262	Container: AdcGroup	195
Table 263	Container: AdcChannel	196
Table 264	Container: AdcGlobInputClass.....	196
Table 265	Container: IrqAdc0PrioConfig	196
Table 266	Container: IrqAdc0TosConfig.....	196
Table 267	Container: IrqDmaPrioConfig	196
Table 268	Example to show the effective Group priority	205
Table 269	Example for ADC Group configuration in ADC_PRIORITY_HW_SW	205
Table 270	Container: AdcGeneral.....	208
Table 271	Container: AdcConfigSet.....	209
Table 272	Container: AdcHWUnit.....	209
Table 273	Container: AdcGroup	209
Table 274	Container: AdcChannel	210
Table 275	Container: AdcGlobInputClass.....	211
Table 276	Container: IrqAdc0PrioConfig	211
Table 277	Container: IrqAdc0TosConfig.....	211
Table 278	Container: AdcGeneral.....	212
Table 279	Container: AdcConfigSet.....	213
Table 280	Container: AdcHWUnit.....	213
Table 281	Container: AdcGroup	213
Table 282	Container: AdcChannel	214
Table 283	Container: AdcGlobInputClass.....	215
Table 284	Container: IrqAdc0PrioConfig	215
Table 285	Container: IrqAdc0TosConfig.....	215
Table 286	Container: IrqDmaPrioConfig	215
Table 287	Container: AdcGeneral.....	216
Table 288	Container: AdcConfigSet.....	217
Table 289	Container: AdcHWUnit.....	217
Table 290	Container: AdcGroup	217
Table 291	Container: AdcChannel	218
Table 292	Container: AdcGlobInputClass.....	219
Table 293	Container: IrqAdc0PrioConfig	219
Table 294	Container: IrqAdc0TosConfig.....	219

List of Tables

Page

Table 295	Container: AdcGeneral.....	220
Table 296	Container: AdcConfigSet.....	220
Table 297	Container: AdcHWUnit.....	221
Table 298	Container: AdcGroup	221
Table 299	Container: AdcChannel	222
Table 300	Container: AdcGlobInputClass.....	223
Table 301	Container: IrqAdc0PrioConfig	223
Table 302	Container: IrqAdc0TosConfig.....	223
Table 303	Container: AdcGeneral.....	227
Table 304	Container: AdcSafety	228
Table 305	Container: AdcConfigSet.....	228
Table 306	Container: AdcHWUnit.....	228
Table 307	Container: AdcGroup	228
Table 308	Container: AdcChannel	230
Table 309	Container: AdcGlobInputClass.....	230
Table 310	Container: IrqAdc0PrioConfig	230
Table 311	Container: IrqAdc0TosConfig.....	230
Table 312	Container: AdcGeneral.....	232
Table 313	Container: AdcSafety	233
Table 314	Container: AdcConfigSet.....	233
Table 315	Container: AdcHWUnit.....	233
Table 316	Container: AdcGroup	233
Table 317	Container: AdcChannel	234
Table 318	Container: AdcGlobInputClass.....	235

List of Code Listings

Page

Code Listing 1	Example - ADC HW Unit Configuration Structure in Adc_PBCfg.c.....	173
Code Listing 2	Example - ADC Group Symbolic Name Configuration in Adc_Cfg.h	176
Code Listing 3	Example – ADC Group Definition	176
Code Listing 4	Sample ADC Configuration having one set.....	184
Code Listing 5	Sample Standard compare mode ADC Channel Configuration	192
Code Listing 6	Example – Dma library function call in Non Autosar result handling	197
Code Listing 7	Example – Driver Initialization and Result pointer initialization	198
Code Listing 8	Switching Priority mechanism to No Priority and Queue disabled	200
Code Listing 9	Switching Priority mechanism to No Priority and Queue enabled.....	201
Code Listing 10	Switching Priority mechanism to Hardware Priority.....	203
Code Listing 11	Switching Priority Mechanism to HW_SW Priority.....	205
Code Listing 12	Example – Usage of HW trigger groups with Background Request Source	224
Code Listing 13	Example code of Safety group.....	231
Code Listing 14	Example code of Safety group.....	235

1 Introduction

This User Manual provides information regarding the functionality, configuration parameters and API implementation for the ADC Driver for AUTOSAR Release 4.0.3 [1], Variant post-build. The ADC Driver initializes and provides access to Versatile Analog to Digital converter unit of the microcontroller.

This User Manual is intended to help the user to get acquainted with the ADC Driver implementation for the AURIX VADC Hardware Platform. It is the documentation that describes how to use the ADC Driver.

1.1 Scope

This document addresses the following features of the ADC Driver implementation,

- AURIX microcontroller HW peripherals used to realize the Driver
- File structure of the Driver
- Configuration parameters of the Driver
- Parameters published by the Driver
- API Specification
- Interrupt Handling
- Common use cases with code snippets.
- Limitations and Assumptions

This documentation applies to the ADC Driver Variant: Variant PB

This document addresses the driver implementation for the following variants of Aurix Family

Some of the peripheral units/feature might not be available in all the variants. In such cases, the relevant information should be discarded for the variant where the unit/feature is not applicable.

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 1](#).

Table 1 Abbreviations used in this document

Abbreviation	Explanation
μC	Microcontroller
Acquisition Time	Sample Time + Conversion Time.
ADC	Analog to Digital Conversion
VADC	Versatile Analog to Digital Conversion
ADC Channel	Represents a logical ADC entity bound to one port pin. Multiple ADC entities can be mapped to the same port pin.
ADC Channel Group / ADC Group / AdcGroup	A group of ADC channels linked to the same ADC hardware unit. The conversion of the whole group is triggered by one trigger source.
ADC HW Unit	Represents a microcontroller input electronic device that includes all parts necessary to perform an “analogue to digital conversion”.
ADC Kernel	ADC HW Unit
API	Application Programming Interface
AUTOSAR	AUTomotive Open System Architecture
Conversion Time	Time during which the sampled analogue value is converted into digital representation.
CPU	Central Processing Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer

DMA	Direct Memory Access
ECU	Electronic Control Unit
ERU	External Request Unit
GPIO	General Purpose Input Output
GTM	Generic Timer Module
HW	Hardware
IFX	Infineon Technologies
MCAL	MicroController Abstraction Layer
MCU	MicroController Unit
RAM	Random Access Memory
ROM	Read Only Memory
RS	ADC HW Unit request source. There are 3 RS: RS0 – RS2
Sampling Time, Sample Time	Time during which the analogue value is sampled (e.g. loading the capacitor, ...)
SCU	System Control Unit
SFR	Special Function Register
SHS	Sample and Hold Sequencer
SW	Software
HIS	Hersteller Initiative Software
MO	Marking Options

1.3 References

This section lists down all relevant documents for this User Manual.

Refer to the latest version for the documents without version information.

- [1] Specification of ADC Driver: AUTOSAR_SWS_ADC_Driver.pdf , V4.2.0(Rel 4.0.3)
- [2] Target Data Sheet: tc27x_um_V1.3.1.pdf
- [3] Target Data Sheet: tc26xB_um_v1.2.pdf
- [4] Target Data Sheet: tc29xB_um_v1.2.pdf
- [5] Target Data Sheet: tc23x_tc22x_um_v1.1.pdf
- [6] Target Data Sheet: tc27xC_um_v1.5.pdf
- [7] Target Data Sheet: tc27xD_um_v2.2.pdf
- [8] User's Manual MCU Driver: Aurix_MC-ISAR_UM_MCUDriver.pdf
- [9] User's Manual GTM Driver: Aurix_MC-ISAR_UM_GTMDriver.doc
- [10] User's Manual PORT Driver: Aurix_MC-ISAR_UM_PORTDriver.pdf
- [11] User's Manual IRQ Driver: Aurix_MC-ISAR_UM_IRQDriver.pdf
- [12] User's Manual PWM Driver: Aurix_MC-ISAR_UM_PWMDriver.pdf
- [13] Hardware Errata Analysis: TC27x_MC-ISAR_HWEerrataAnalysis.xls
- [14] Hardware Errata Analysis: TC26x_MC-ISAR_HWEerrataAnalysis.xls
- [15] Hardware Errata Analysis: TC29x_MC-ISAR_HWEerrataAnalysis.xls
- [16] Hardware Errata Analysis: TC23x_MC-ISAR_HWEerrataAnalysis.xls

[17] Hardware Errata Analysis: TC22x_TC21x_MC-ISAR_HWErErrataAnalysis.xls

2 ADC Driver Overview

The AURIX micro controller provides multiple VADC HW Units using the Successive Approximation Register (SAR) principle to convert analog input values (voltages) to discrete digital values. The ADC Driver initializes and controls the internal Versatile Analog to Digital Converter units of the microcontroller. The Driver also provides the facility for the user to initialize the result buffer location to update the converted value. The ADC Driver will allow the configuration of a priority level for each channel group (optional). This implies a prioritization mechanism, implemented in SW, and as supported by the HW.

Aurix has two ADC peripherals, VADC and DSADC. For Autosar ADC driver, only the VADC peripheral is supported. Hence the references to 'ADC' in this document refer to VADC.

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the hardware peripherals and their interaction with the Drivers

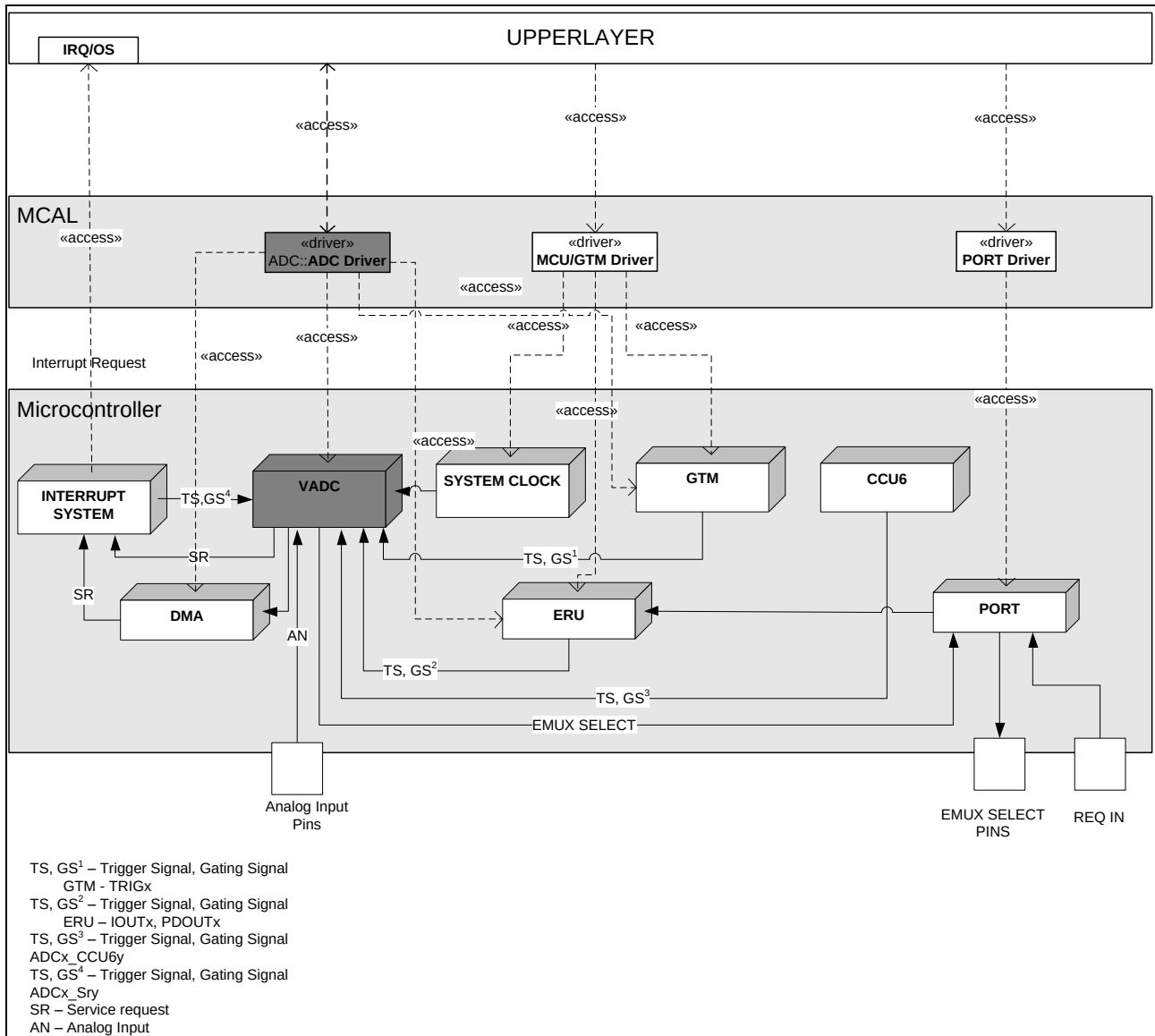


Figure 1 Software Hardware Mapping of ADC Driver

2.1.1 SYSTEM CLOCK

The System Clock is set up via the MCU Driver. This is mandatory for the user to set up an appropriate system clock. Note that the VADC peripheral clock is basically derived from the System Clock configured.

Refer [8]: MCU User Manual for details about MCU Configuration

2.1.2 PORT

The Port Pins which act as input channels to the VADC need to be configured as analog input by PORT driver.

The used request lines for external hardware trigger and Gate signal should be configured for input direction by Port Driver. If EMUX is used, then EMUX select pins needs to be configured for output direction, alternate function by Port Driver.

Refer [10]: PORT User Manual for details about PORT Configuration

2.1.3 ERU

ERU peripheral is required if ERU output signal is configured as the trigger for ADC conversion. Note that this peripheral is also used in the ICU Driver. User must ensure that the same resources are not used at the same time in both the Drivers (ADC and ICU).

2.1.4 CCU6

CCU6 peripheral is required if CCU6 output signal is configured as the trigger for ADC conversion. User should configure/write the CCU6 registers to generate output signal. ADC driver does not configure/write any CCU6 registers.

2.1.5 GTM

GTM peripheral is required if timer triggered periodic conversion is configured as the trigger for VADC conversion. Note that this peripheral is also used in the ICU, GPT and PWM Driver. User must ensure that the same resources are not used at the same time in other modules.

2.1.6 DMA

DMA is used in the ADC Driver when AdcGroup EMUX Scan or Non Autosar Mode is enabled.

2.1.6.1 EMUX SCAN mode DMA usage

On AdcGroup channel conversion complete, VADC triggers DMA to transfer the converted result value to the user buffer. On result transfer completion of all the Group channels for configured Group number of samples, the DMA interrupt is raised. The Group status is then changed to `ADC_STREAM_COMPLETED`. The priority of the service request node i.e. Selected DMA channel's SRN need to be initialized by the user.

2.1.6.2 NON AUTOSAR Result Handling mode DMA usage

The user has to configure the DMA library to use selected DMA channel. So in this case the ADC driver does not handle the results since the user will be directly configuring the DMA driver for the source and destination buffers. On completion of AdcGroup channel conversion, VADC triggers the DMA to transfer the converted result value to the user buffer as configured by the user.

ADC driver enables the AdcGroup last channel interrupt so as to update group and request source data variables, and to call AdcGroup notification if enabled.

2.1.7 VADC

The following list summarizes common features of each VADC kernel:

- Nominal analog supply voltage 5.0 V, operation at 3.3 V with reduced performance
- Input voltage range from 0 V to analog supply voltage
- Standard (VAREF) and alternate (CH0) reference voltage source selectable for each channel to support ratiometric measurements and different signal scales
- 8 independent converters with up to 8 analog input channels arranged in 2 clusters
- External analog multiplexer control, including adjusted sample time and scan support
- Conversion speed and sample time adjustable to adapt to sensors and reference
- Conversion time below 1 μ s (depending on the resolution and sample time)
- Flexible source selection and arbitration
 - Programmable arbitrary conversion sequence (single or repeated)
 - Configurable auto scan conversion (single or repeated) on each converter
 - Configurable auto scan conversion (single or repeated) in the background (all converters)

- Conversions triggered by software, timer events, or external events
- Cancel-inject-restart mode for reduced conversion delay on priority channels
- Powerful result handling
 - Selectable result width of 8/10/12 bits
 - Fast Compare Mode
 - Independent result registers
 - Configurable limit checking against programmable border values
 - Data rate reduction through adding a selectable number of conversion results
 - FIR/IIR filter with selectable coefficients
- Flexible service request generation based on selectable events
- Built-in safety features
 - Configurable register access protection supports safety applications
 - Broken wire detection with programmable default levels
 - Multiplexer test mode to verify signal path integrity
- Support of suspend and power saving modes

2.2 Software Driver Description

The **0** and **Figure 2** show the Application Programming Interface (API) of the Sample&Hold ADC and SAR ADC Driver respectively and its interaction with other software Drivers.

- Development Error Tracer (DET) is required if DET is enabled for the ADC Driver.
- Safe Mcal Report handler is required if Safety is enabled for the ADC Driver.

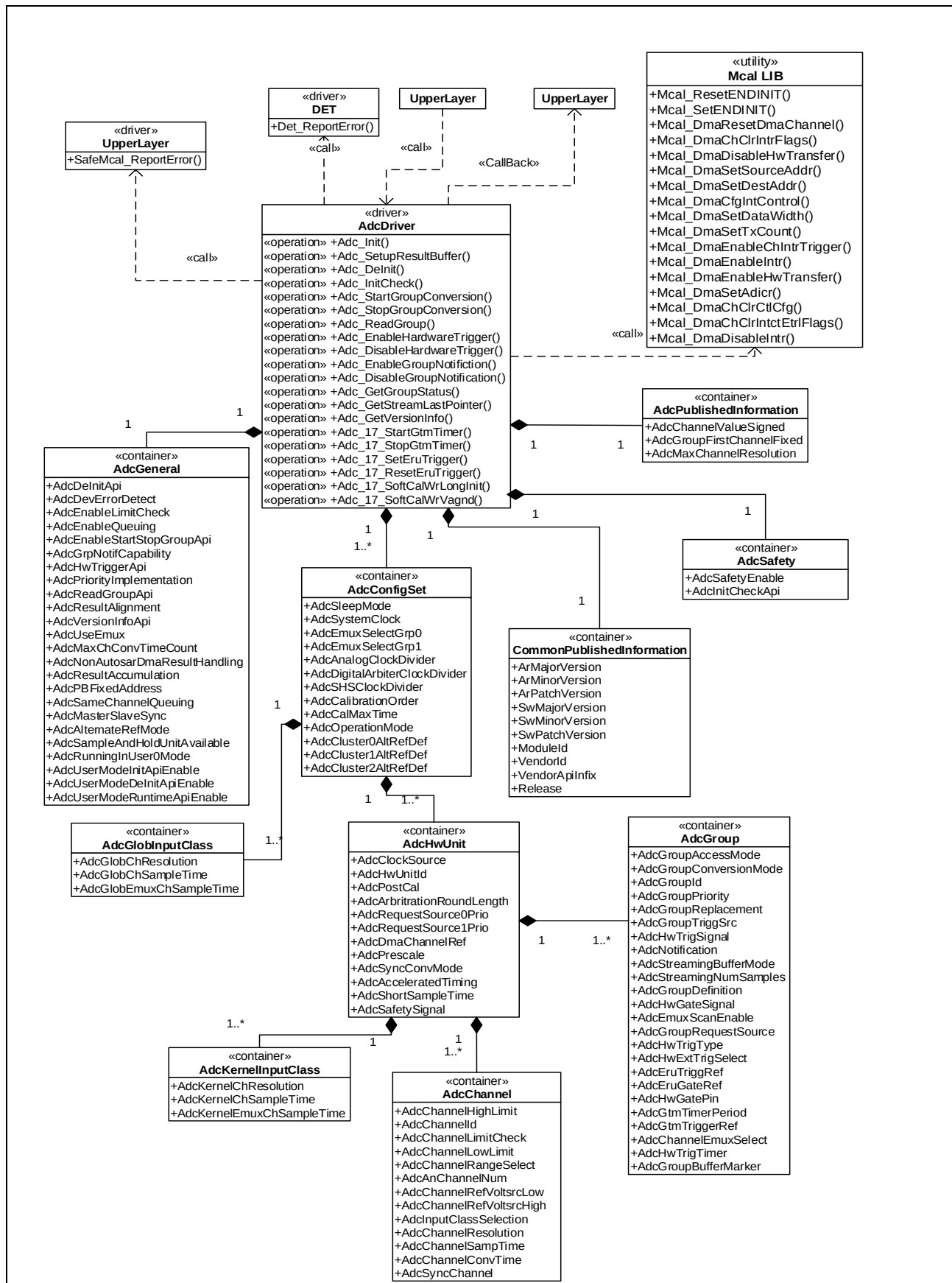
The APIs of the Driver are explained in detail in chapter **5.1**.

The Configuration Data of the Driver is also depicted in the figure. The individual containers and their configuration parameters are explained in detail in chapter **4**.

Table 2 Types of ADC & Not Applicable Parameters in Aurix Devices

Aurix Device	Analog to Digital Converter	Not applicable Parameters
TC27xBC	Sample&Hold ADC	AdcRefPrechargeControl, AdcLowPowerSupplyVoltage, AdcRequestSource3Prio, AdcRS3TriggerSeqCount, AdcRS3InternalTrigger, AdcCcu6Synchronization, AdcCcu6ModuleRef AdcNonAutosarResultPolling AdcResetSfrAtInit
TC26xAB		
TC29xAA		
TC27xCA	SAR ADC	AdcRequestSource3Prio, AdcRS3TriggerSeqCount, AdcRS3InternalTrigger AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable, AdcSHSClockDivider, AdcOperationMode, AdcCalibrationOrder, AdcCalMaxTime, AdcCluster0AltRefDef, AdcCluster1AltRefDef, AdcCluster2AltRefDef, AdcAcceleratedTiming, AdcShortSampleTime, AdcCcu6Synchronization, AdcCcu6ModuleRef
TC26xBA		
TC29xBA		

Aurix Device	Analog to Digital Converter	Not applicable Parameters
TC23xAA TC22xAA		AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable, AdcSHSClockDivider, AdcOperationMode, AdcCalibrationOrder, AdcCalMaxTime, AdcCluster0AltRefDef, AdcCluster1AltRefDef, AdcCluster2AltRefDef, AdcAcceleratedTiming, AdcShortSampleTime, AdcCcu6Synchronization, AdcCcu6ModuleRef
TC27xDB TC26xBB TC29xBB	SAR ADC with metal fix for cross coupling effect	AdcRequestSource3Prio, AdcRS3TriggerSeqCount, AdcRS3InternalTrigger AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable, AdcSHSClockDivider, AdcOperationMode, AdcCalibrationOrder, AdcCalMaxTime, AdcCluster0AltRefDef, AdcCluster1AltRefDef, AdcCluster2AltRefDef, AdcAcceleratedTiming, AdcShortSampleTime
TC23xAB TC22xAB		AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable, AdcSHSClockDivider, AdcOperationMode, AdcCalibrationOrder, AdcCalMaxTime, AdcCluster0AltRefDef, AdcCluster1AltRefDef, AdcCluster2AltRefDef, AdcAcceleratedTiming, AdcShortSampleTime



Interfaces of Sample&Hold ADC driver

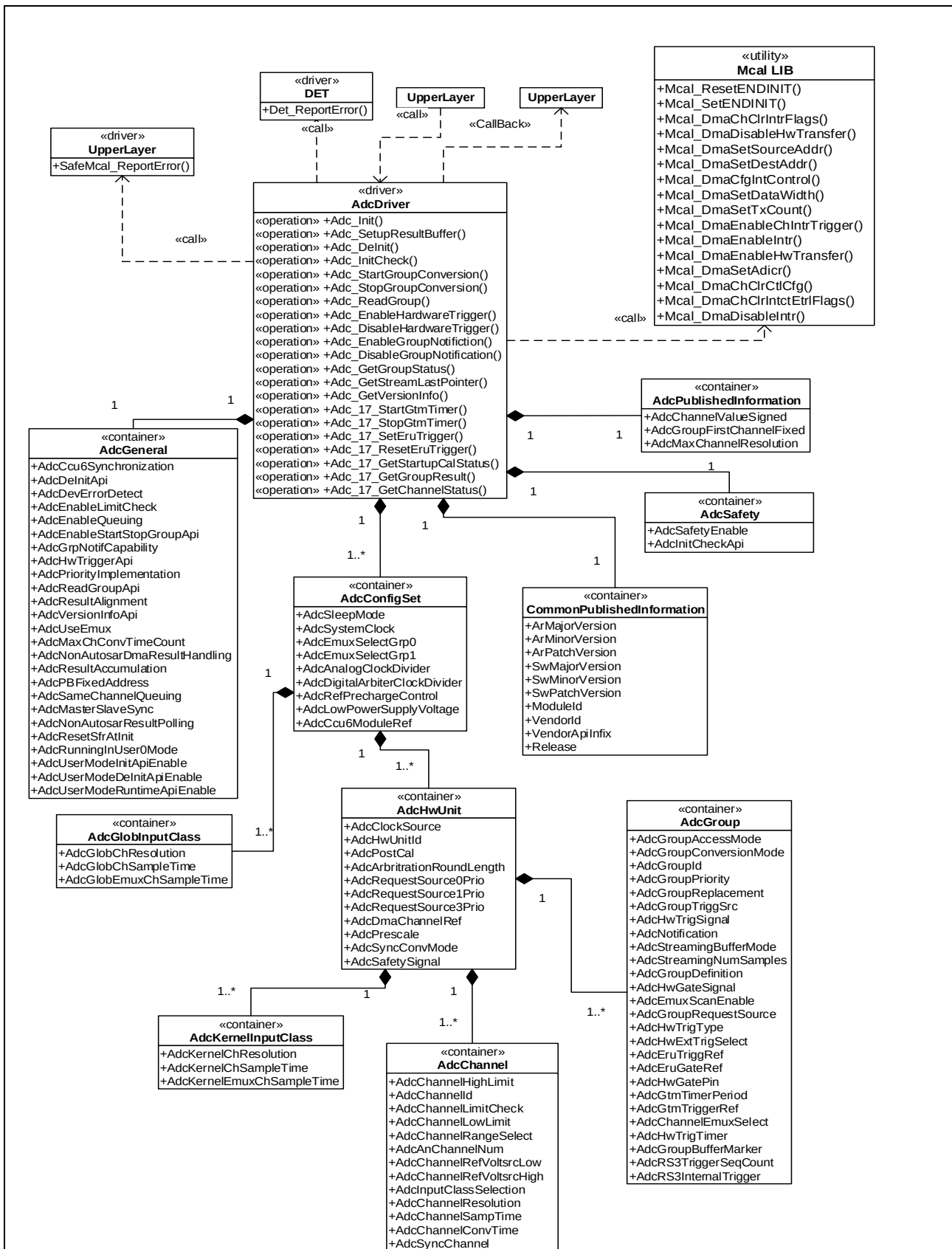


Figure 2 Interfaces of SAR ADC driver

2.2.1 ADC Driver Legend

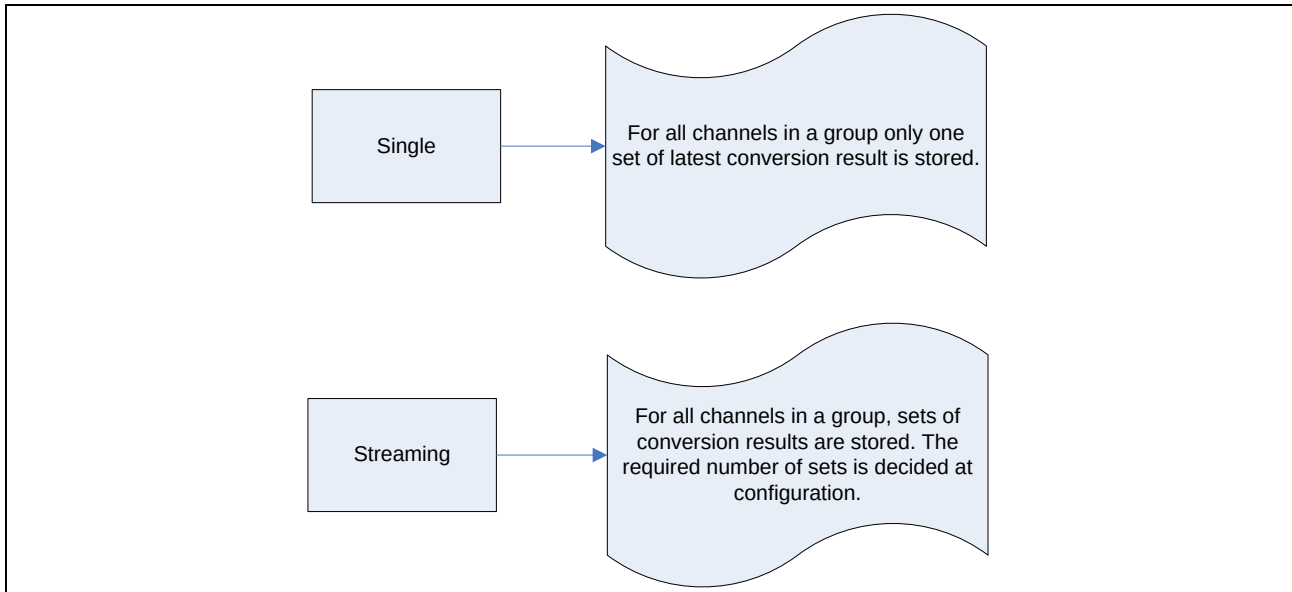


Figure 3 ADC Group result buffer type (AdcGroupAccessMode - Refer 4.5.15.1)

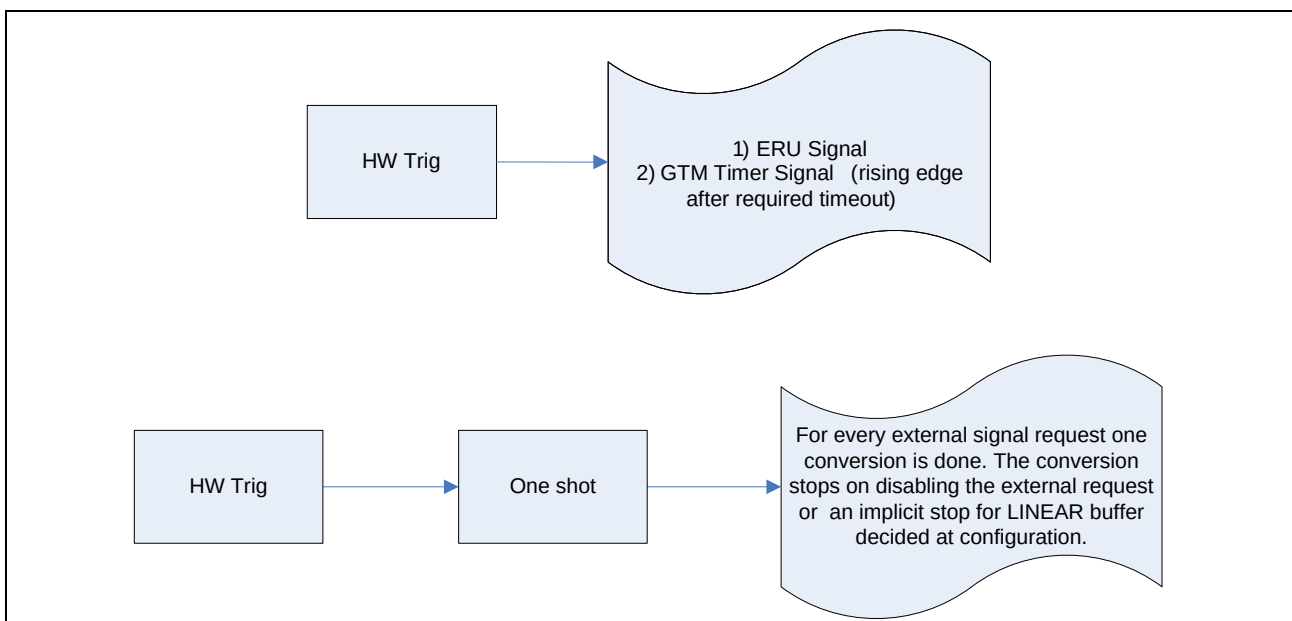


Figure 4 ADC Group with HW trigger type

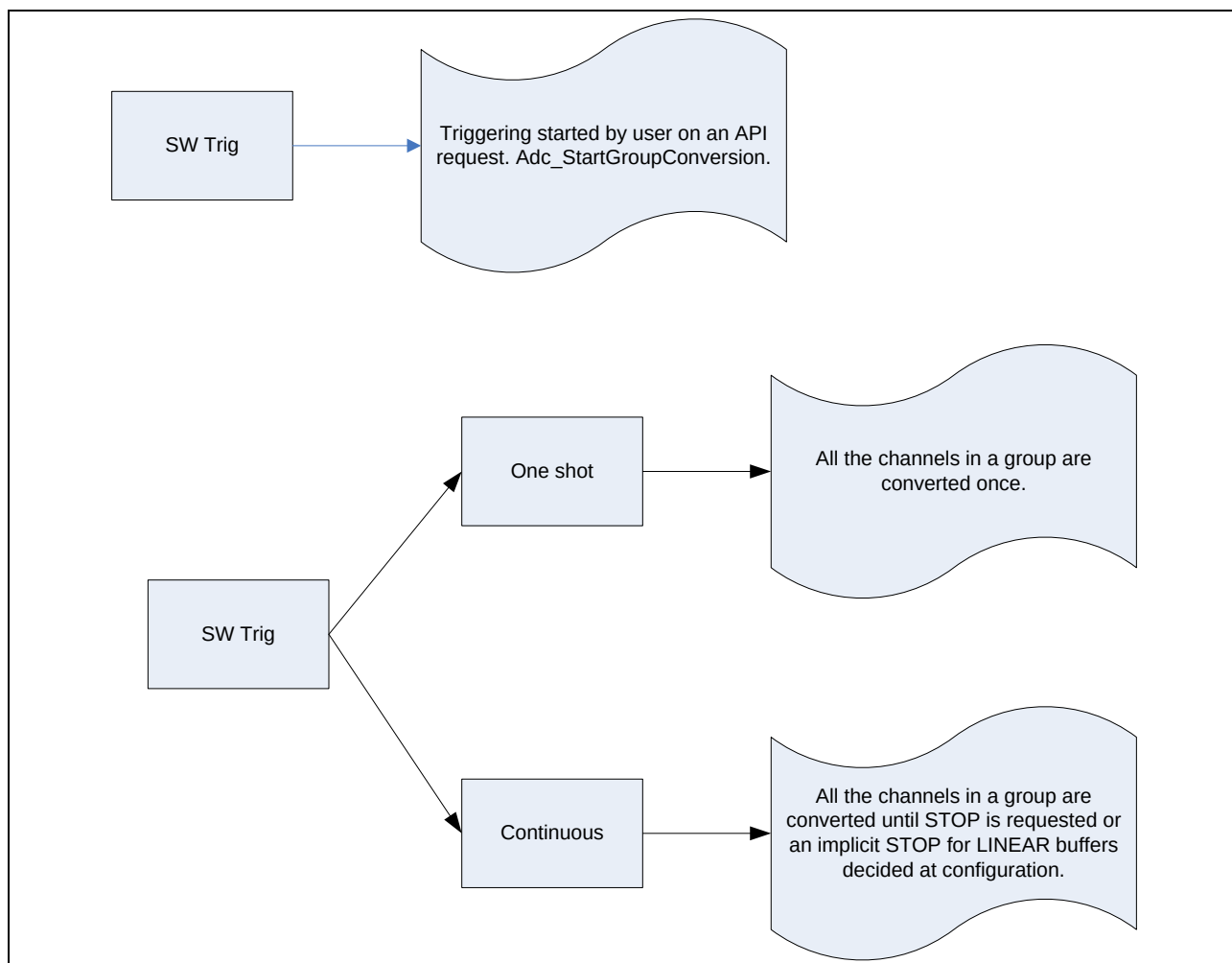


Figure 5 ADC Group with SW trigger type

2.2.2 Possible combinations of Group properties

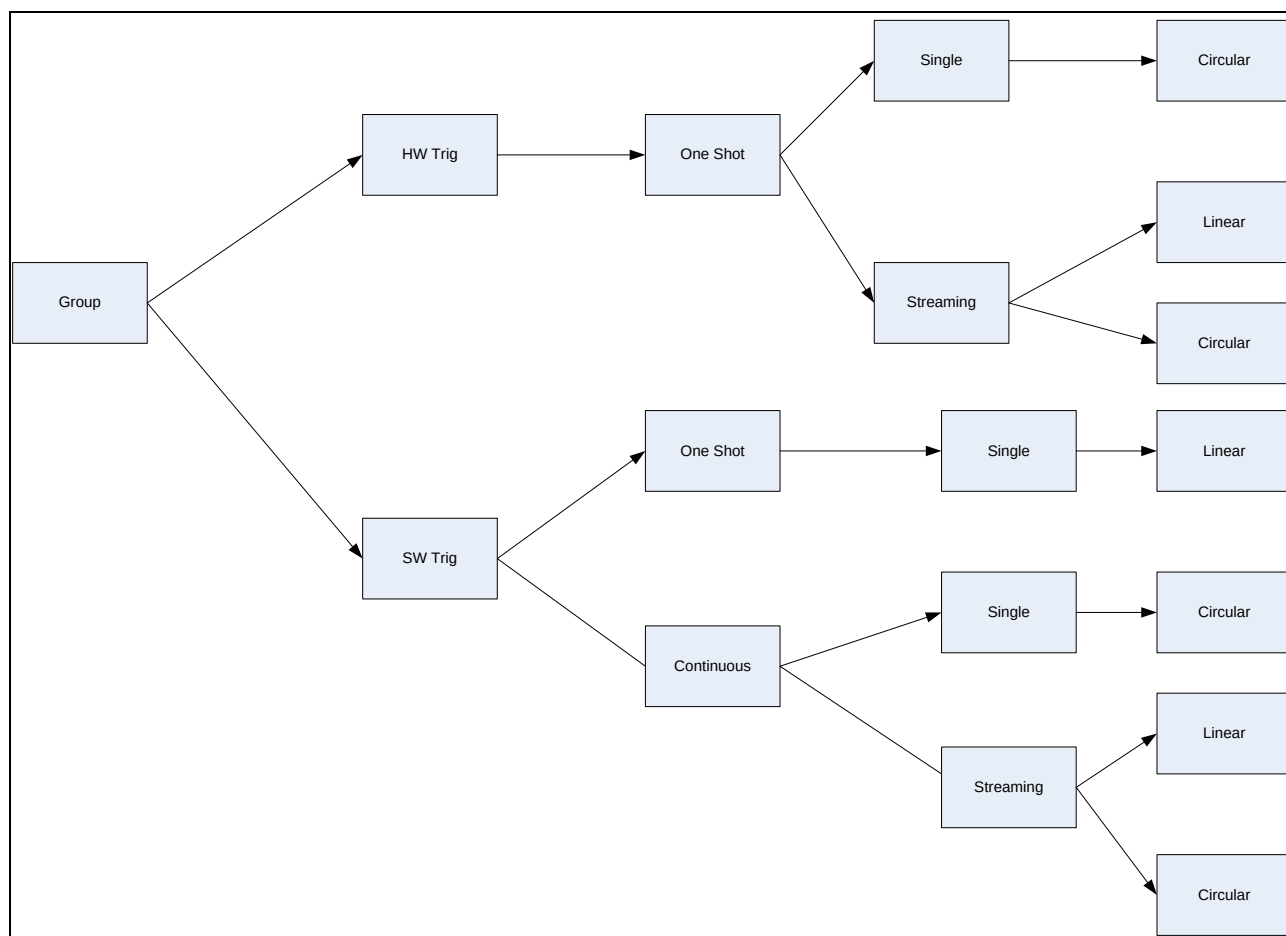


Figure 6 Possible combinations of Group trigger type, conversion mode, Group access mode, and buffer mode

2.2.3 Possible number of channels for Group request sources

Table 3 Possible number of channels for Group request sources

Aurix Device	RS0	RS1	RS2	RS3
TC26x	8	16	16	NA
TC27x/TC29x	8	8	8	NA
TC23x/TC22x	8	16	16	8

2.2.4 EMUX SCAN

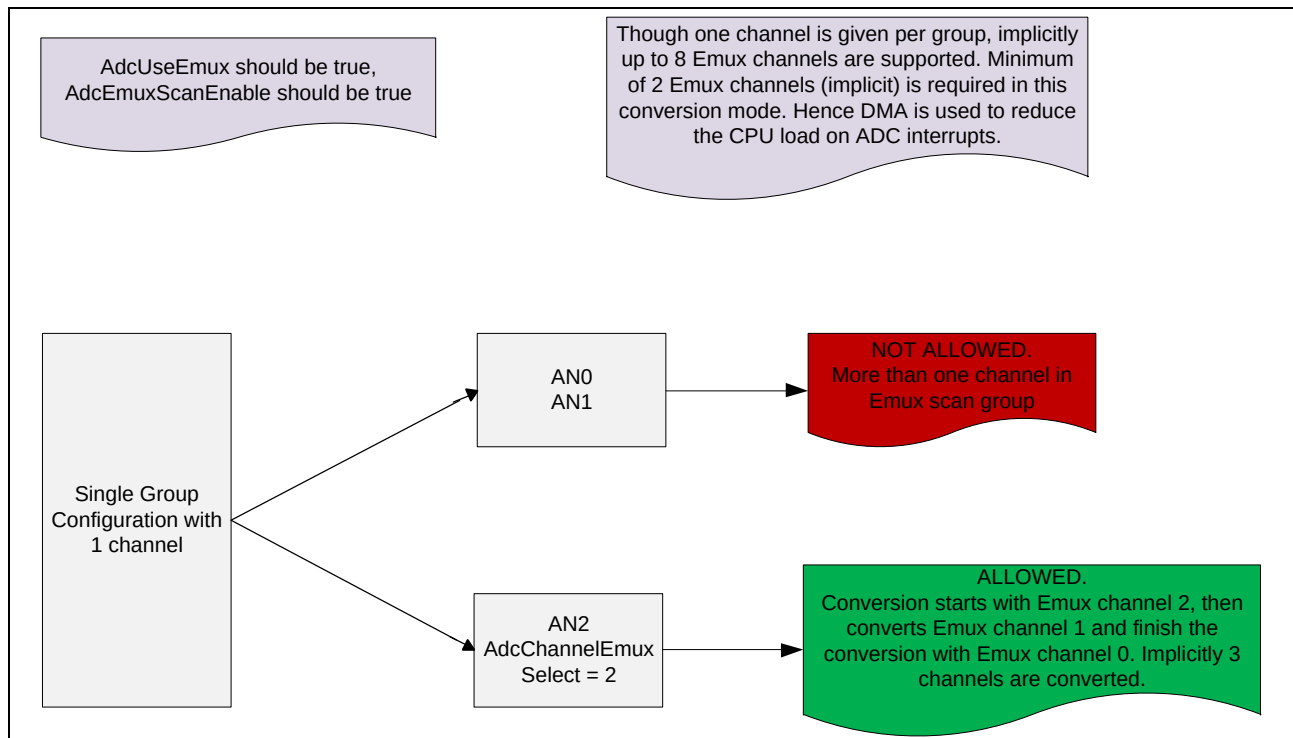


Figure 7 Example Group in EMUX SCAN enabled

3 File Structure

This section provides details about the ADC Driver files and other related files.

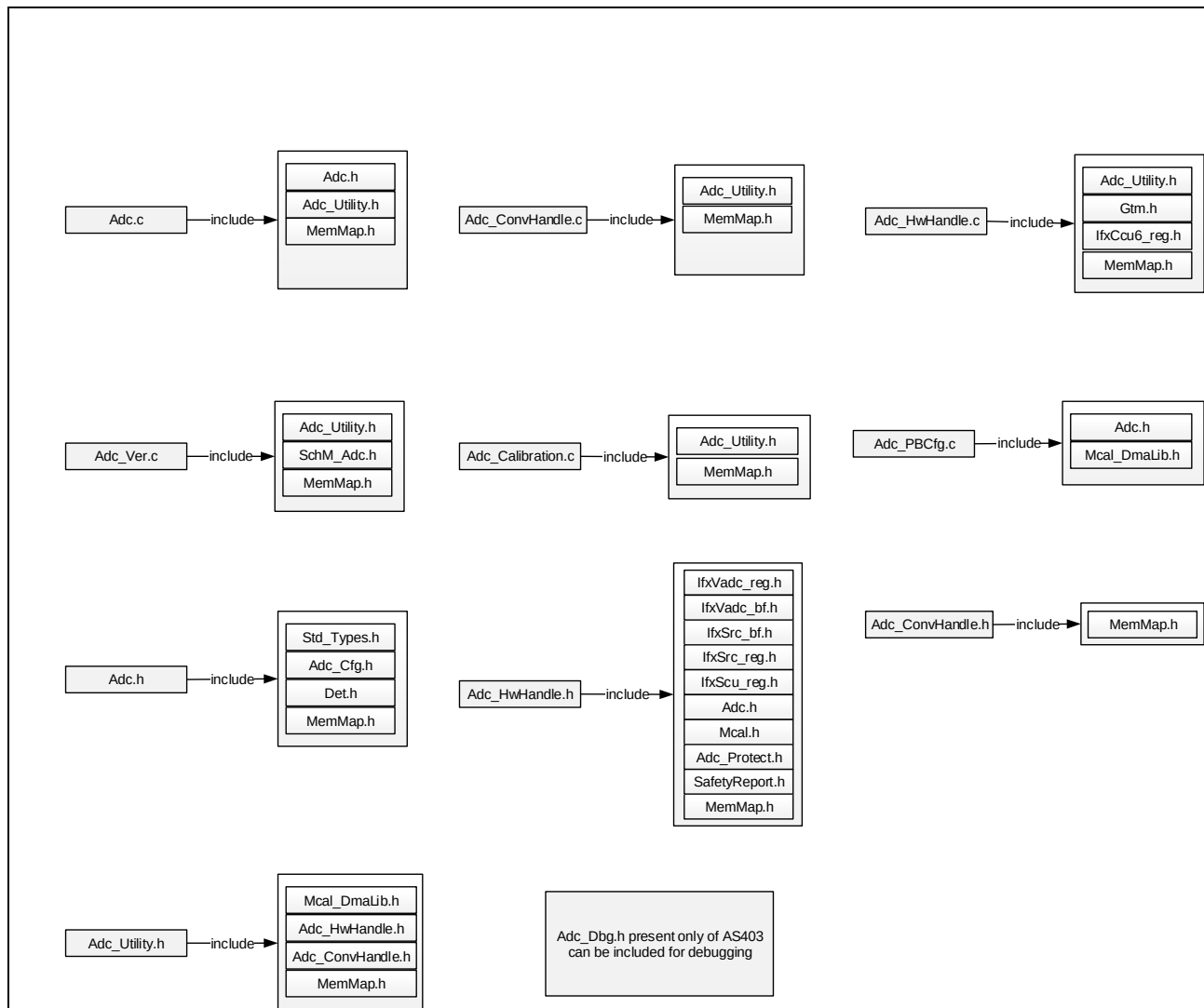


Figure 8 File Structure of ADC Driver

Table 4 File Contents

Platform_Types.h	Platform dependencies are declared here.
Std_Types.h	Standard data types to be used are declared here.
Compiler.h	Compiler dependencies are abstracted here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code within declaration and definition.
Mcal_Compiler.h	Mcal compiler specific functions are abstracted here. This file is included through Mcal.h.
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections
IfxVadc_reg.h	Includes the VADC register definition file for microcontroller.
IfxScu_reg.h	Includes the SCU register definition file for microcontroller.
IfxSrc_reg.h	Includes the SRC register definition file for microcontroller.
Mcal.h	Provides library functions for MCAL layer.
Mcal_DmaLib.h	Provide Dma library functions for MCAL layer

SchM_Adc.h	Functions to enable/disable interrupts are declared here.
Det.h	Provides the functionality of Development Error Tracer
Adc_Dbg.h	Provides the debug variables.
Adc.h	ADC Driver data types and API Functions are exported for the user. ADC Driver Interrupt handlers are exported.
Adc_Utility.h	ADC utility functions and Macros used in Adc.c and Adc_ConvHandle.c are exported.
Adc_ConvHandle.h	Adc conversion handle sub functions are exported.
Adc.c	ADC Driver Function implementation
Adc_ConvHandle.c	ADC conversion handle sub functions of ADC driver.
Adc_Ver.c	Autosar version specific implementation of ADC driver.
Adc_HwHandle.c	ADC hardware access sub functions of ADC driver.
Adc_HwHandle.h	ADC hardware access sub functions of ADC driver are exported.
Adc_Calibration.c	ADC software calibration function implementation. Code available in the file is productive, if Hardware Startup calibration is executed (e.g: TC23xAA, TC27xCA). Code is not productive, if Software calibration is executed (e.g: TC27xBC, TC26xAB, TC29xAA).
Adc_Cfg.h	Contains Driver precompile configuration parameters.
Adc_PBCfg.c	Contains Driver postbuild configuration parameters.
Gtm.h	Import GTM functionality to configure Timer trigger.
SafetyReport.h	Provides the functionality of Safe Mcal Report Error

4 Configuration Documentation

The following chapters summarizes

- All configuration parameters and their description
- Configuration parameter variance w.r.t supported HW
- The type definition used to represent them in the code.

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be frozen before compilation and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

The ADC Driver is delivered as a Variant PB.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Note: Topics [4.1.1](#), [4.1.2](#) should be read for understanding the configuration class field

Table 5 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, Variant PC.

Table 6 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

4.2 Container: AdcGeneral

This container contains the general configuration (parameters) of the ADC Driver software module. Different object files are provided for different settings of these static parameters.

4.2.1 AdcDevErrorDetect

Table 7 AdcDevErrorDetect

Name:	AdcDevErrorDetect
--------------	-------------------

Name:	AdcDevErrorDetect		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True – Development Error Detection is enabled False – Development Error Detection disabled		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Switches the Development Error Detection and Notification ON or OFF. This configuration parameter is named as ADC_DEV_ERROR_DETECT in the configuration file. The value of the parameter ADC_DEV_ERROR_DETECT is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.2 AdcDeinitApi

Table 8 AdcDeinitApi

Name:	AdcDeinitApi		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True: Adc_DeInit() API is enabled. False: Adc_DeInit() API is disabled.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable / disable the API Adc_DeInit() from the driver. This configuration parameter is named as ADC_DEINIT_API in the configuration file. The value of the parameter ADC_DEINIT_API is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.3 AdcEnableLimitCheck

Table 9 AdcEnableLimitCheck

Name:	AdcEnableLimitCheck		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		

Range:	True: Limit checking feature is enabled. False: Limit checking feature is disabled.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable / disable the Limit checking feature from the driver. This configuration parameter is named as ADC_ENABLE_LIMIT_CHECK in the configuration file. The value of the parameter ADC_ENABLE_LIMIT_CHECK is STD_ON / STD_OFF. This parameter is not editable if AdcNonAutosarDmaResultHandling set to True. When a limitcheck enabled group is requested (start/enable) for a conversion, another limit check enabled group of the same ADC kernel should not be requested for the conversion.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcNonAutosarDmaResultHandling		

4.2.4 AdcEnableQueuing

Table 10 AdcEnableQueuing

Name:	AdcEnableQueuing		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True: Queuing mechanism in no priority is Enabled False: Queuing mechanism in no priority is Disabled		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Determines, the queuing mechanism is enabled or disabled. This parameter is editable only if AdcNonAutosarDmaResultHandling is set to false [STD_OFF] and AdcPriorityImplementation is set to ADC_PRIORITY_NONE. This configuration parameter is named as ADC_ENABLE_QUEUEING in the configuration file. The value of the parameter ADC_ENABLE_QUEUEING is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcPriorityImplementation AdcNonAutosarDmaResultHandling		

4.2.5 AdcEnableStartStopGroupApi

Table 11 AdcEnableStartStopGroupApi

Name:	AdcEnableStartStopGroupApi		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True: Adc_StartGroupConversion() and Adc_StopGroupConversion() APIs are enabled. False: Adc_StartGroupConversion() and Adc_StopGroupConversion() APIs are disabled.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Adds / removes the service Adc_StartGroupConversion() and Adc_StopGroupConversion() from the code. This configuration parameter is named as ADC_ENABLE_START_STOP_GROUP_API in the configuration file. The value of the parameter ADC_ENABLE_START_STOP_GROUP_API is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGroupTriggSrc		

4.2.6 AdcGrpNotifCapability

Table 12 AdcGrpNotifCapability

Name:	AdcGrpNotifCapability		
Source:	This parameter is defined by the AUTOSAR		
File:	Adc_Cfg.h		
Range:	True – Group notification mechanism is enabled False – Group notification mechanism is disabled		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Determines, if the group notification mechanism (the functions to enable and disable the notifications) is available at runtime. This configuration parameter is named as ADC_GRP_NOTIF_CAPABILITY in the configuration file. The value of the parameter ADC_GRP_NOTIF_CAPABILITY is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.7 AdcHwTriggerApi

Table 13 AdcHwTriggerApi

Name:	AdcHwTriggerApi
--------------	-----------------

Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True: Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() APIs are enabled. False: Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() APIs are disabled.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Adds / removes the service Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() from the code. This configuration parameter is named as ADC_HW_TRIGGER_API in the configuration file. The value of the parameter ADC_HW_TRIGGER_API is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGroupTriggSrc		

4.2.8 AdcPriorityImplementation

Table 14 AdcPriorityImplementation

Name:	AdcPriorityImplementation		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	ADC_PRIORITY_NONE ADC_PRIORITY_HW ADC_PRIORITY_HW_SW		
Default value:	ADC_PRIORITY_NONE		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Determines the type of prioritization mechanism. The hardware prioritization mechanism (ADC_PRIORITY_HW) uses the ADC hardware features for prioritization of the ADC Groups with Software and Hardware conversion requests. The mixed hardware and software prioritization mechanism (ADC_PRIORITY_HW_SW) uses the ADC hardware features for prioritization of ADC Groups with trigger source hardware and a software implemented prioritization mechanism for ADC Groups with trigger source software. This parameter is significant only if AdcNonAutosarDmaResultHandling is set to False (STD_OFF). When AdcNonAutosarDmaResultHandling = STD_ON, driver will be in ADC_PRIORITY_NONE.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcNonAutosarDmaResultHandling		

4.2.9 AdcReadGroupApi

Table 15 AdcReadGroupApi

Name:	AdcReadGroupApi		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True: Adc_ReadGroup() API is enabled. False: Adc_ReadGroup() API is disabled.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Adds / removes the service Adc_ReadGroup() from the code. This configuration parameter is named as ADC_READ_GROUP_API in the configuration file. The value of the parameter ADC_READ_GROUP_API is STD_ON / STD_OFF. This parameter is not editable if AdcNonAutosarDmaResultHandling is set to True (STD_ON).		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcNonAutosarDmaResultHandling		

4.2.10 AdcResultAlignment

Table 16 AdcResultAlignment

Name:	AdcResultAlignment		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	ADC_ALIGN_LEFT ADC_ALIGN_RIGHT		
Default value:	ADC_ALIGN_RIGHT		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Alignment of ADC raw results in ADC result buffer (left / right alignment). This configuration parameter is named as ADC_RESULT_ALIGNMENT in the configuration file. This parameter should be set to ADC_ALIGN_LEFT, if parameter AdcEnableLimitCheck is set to True. Refer Figure 9 Figure 10 Figure 11 for software result buffer alignment for all the supported features.		
SFRs Modified:	VADC_GxCHCTry Where x is Adc kernel number, y is Analog channel number		
Scope:	Driver		

Dependency: None

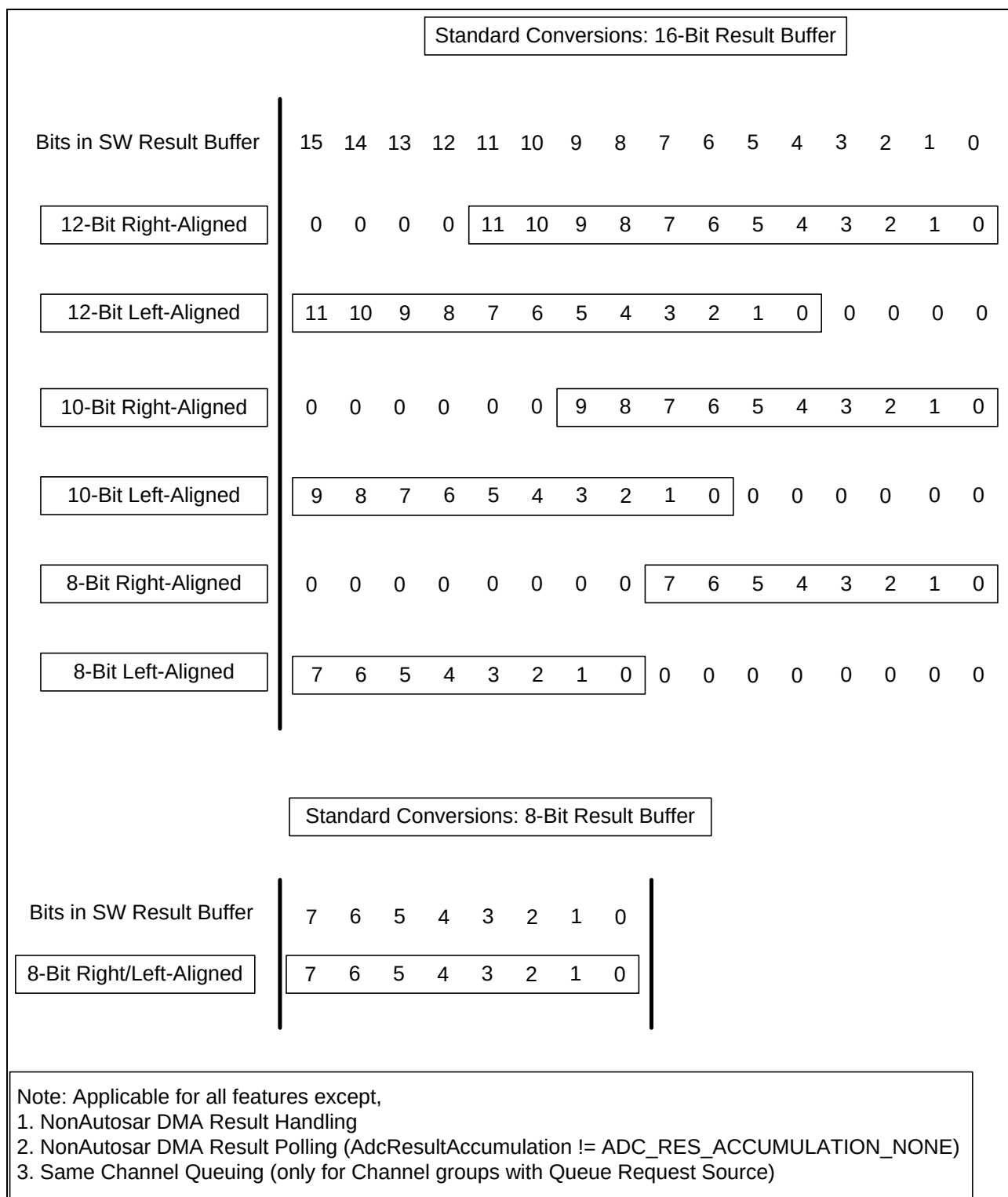


Figure 9 Result Alignment for Standard Conversions

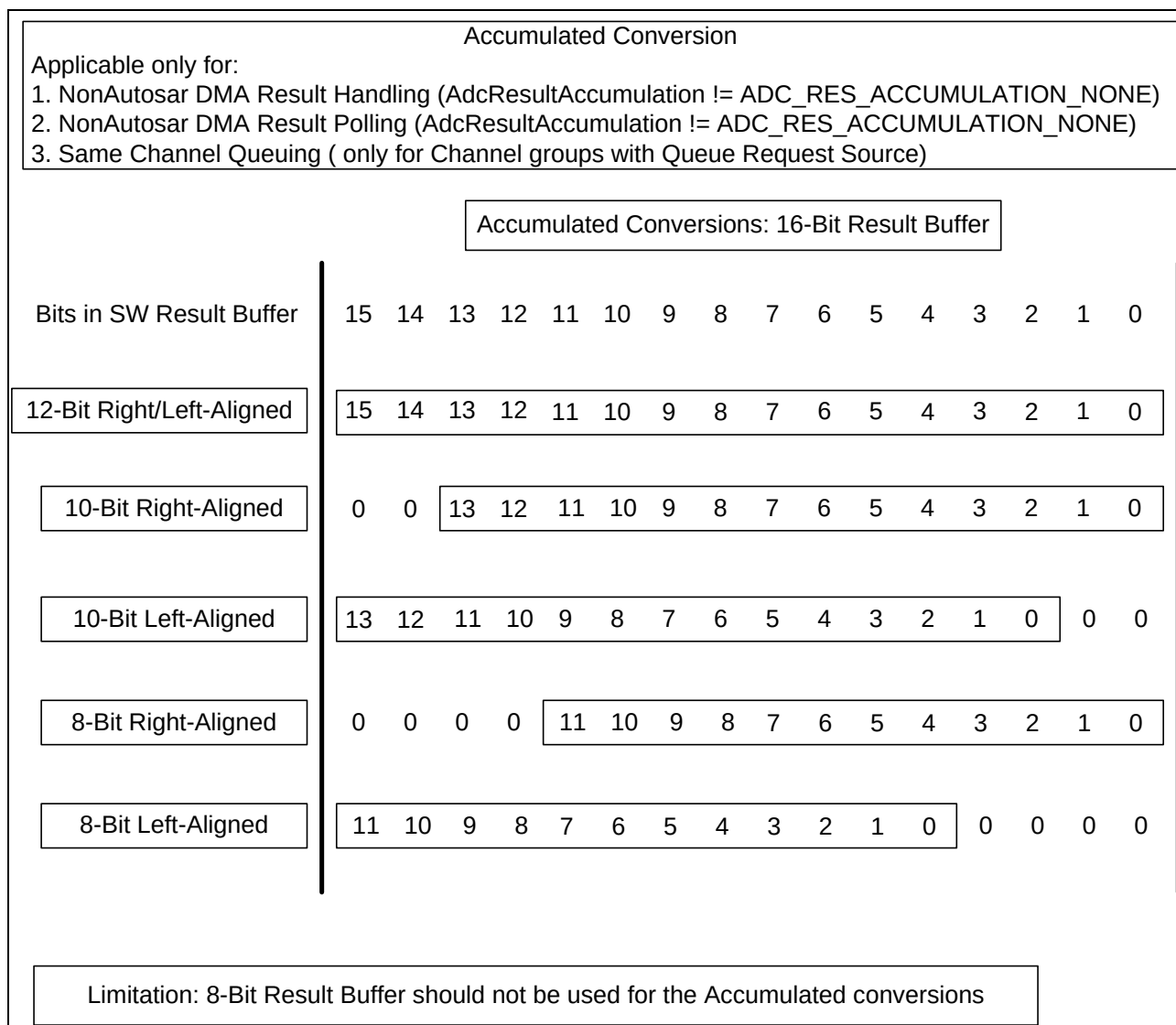


Figure 10 Result Alignment for Accumulated Conversions

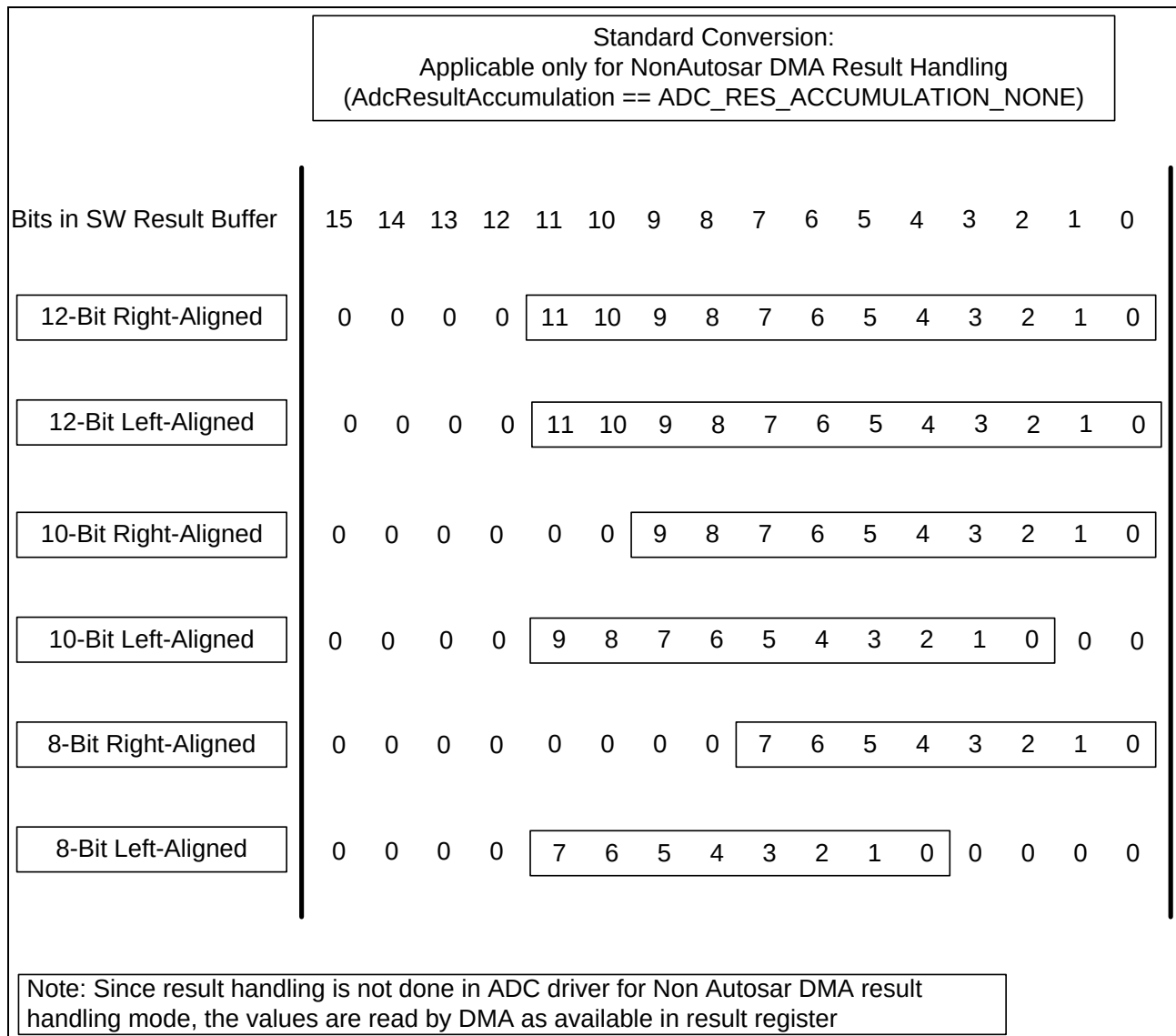


Figure 11 Result Alignment for NonAS DMA Result Handling with No Accumulation

4.2.11 AdcVersionInfoApi

Table 17 AdcVersionInfoApi

Name:	AdcVersionInfoApi		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	True – Version Info API is enabled False – Version Info API disabled		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable / disable the API to read out the modules version information, i.e., Adc_GetVersionInfo()		

	This configuration parameter is named as ADC_VERSION_INFO_API in the configuration file. The value of the parameter ADC_VERSION_INFO_API is STD_ON / STD_OFF.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.2.12 AdcUseEmux

Table 18 AdcUseEmux

Name:	AdcUseEmux		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: ADC External multiplexer is used by the user False: ADC External multiplexer is not used by the user		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Adds / removes the EMUX configuration settings from the code. With the help of External multiplexer, number of analog channels can be increased. Aurix microcontroller provides 3 EMUX select output lines. These pins need to be configured as output EMUX pins using Port Driver to use them as EMUX select pins. With EMUX select line, external 1-out-of-8 multiplexer can be used to extend the number of analog input channels at each analog channel. This configuration parameter is named as ADC_USE_EMUX in the configuration file. The value of the parameter ADC_USE_EMUX is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.13 AdcMaxChConvTimeCount

Table 19 AdcMaxChConvTimeCount

Name:	AdcMaxChConvTimeCount
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h

Range:	16 – 16962, For 1) AdcPriorityImplementation is ADC_PRIORITY_HW_SW 2) AdcPriorityImplementation is ADC_PRIORITY_NONE and AdcEnableQueuing is ON 0 – 16962, For 1) AdcPriorityImplementation is ADC_PRIORITY_HW 2) AdcPriorityImplementation is ADC_PRIORITY_NONE and AdcEnableQueuing is OFF		
Default value:	10000		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Determines, the maximum channel conversion time amongst all the configured channels. This parameter value is required by the ADC Driver during stopping of conversion. Refer 8.1.2 . This will make sure the channel under conversion is completed during stop request and VADC HW Unit goes to Idle state before new start from Driver. If the entered value less than 16, the wait on Stop in the driver is not performed. In such a case user has to ensure that new start of a Group is called only when VADC HW Unit is Idle. Switching off of driver to wait on conversion Stop/Disable is applicable in ADC_PRIORITY_NONE (AdcEnableQueuing is OFF) and ADC_PRIORITY_HW.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcPriorityImplementation, AdcEnableQueuing		

4.2.14 AdcNonAutosarDmaResultHandling

Table 20 AdcNonAutosarDmaResultHandling

Name:	AdcNonAutosarDmaResultHandling		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Non AUTOSAR specific result handling using DMA driver False: AUTOSAR specific result handling.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

Description:	Switches the ADC Group result handling between AUTOSAR specific and Non AUTOSAR specific. To achieve the best interrupt load on CPU, Non AUTOSAR specific result handling mode to be selected. This parameter cannot be configured, if AdcNonAutosarResultPolling is enabled. This configuration parameter is named as ADC_RESULT_HANDLING_MODE in the configuration file. The value of the parameter ADC_RESULT_HANDLING_MODE is ADC_NON_AUTOSAR / ADC_AUTOSAR.
SFRs Modified:	None
Scope:	Driver
Dependency:	AdcNonAutosarResultPolling

4.2.15 AdcResultAccumulation

Table 21 AdcResultAccumulation

Name:	AdcResultAccumulation		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	ADC_RES_ACCUMULATION_NONE: No accumulation of the result. Result register will have only one conversion value. ADC_RES_ACCUMULATION_TWO: Accumulation of the results up to two conversions. ADC_RES_ACCUMULATION_THREE: Accumulation of the results up to three conversions. ADC_RES_ACCUMULATION_FOUR: Accumulation of the results up to four conversions.		
Default value:	ADC_RES_ACCUMULATION_NONE		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	In non AUTOSAR specific result handling mode, the user can select the number of result accumulation over conversions. This value is maintained uniform across the drive for all group channels. For One shot, SW Triggered ADC Group, the result accumulation is always considered to be None irrespective of configured "AdcResultAccumulation". This configuration parameter is named as ADC_RESULT_ACCUMULATION in the configuration file. This parameter is editable only if AdcNonAutosarDmaResultHandling or AdcNonAutosarResultPolling is set to True (STD_ON).		
SFRs Modified:	VADC_GxRCRy Where x is Adc kernel number, y is Analog channel number		
Scope:	Driver		
Dependency:	AdcNonAutosarDmaResultHandling		

4.2.16 AdcPBFixedAddress

Table 22 AdcPBFixedAddress

Name:	AdcPBFixedAddress		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True False		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This configuration parameter allows selecting the fixed address feature. True – AdcPBFixedAddress indicates the configuration parameter set will be in fixed address. The feature supports the optimization of a post build configuration with a fixed address by avoiding unnecessary indirection (HisGen0010). The multiplicity of the AdcConfigSet is 1..1. False – fixed address feature is not used. Hence the optimization based on the requirement HisGen0010 is also not supported. The multiplicity of the AdcConfigSet is 1..*. This configuration parameter is named as ADC_PB_FIXED_ADDRESS in the configuration file. The Value of the parameter ADC_PB_FIXED_ADDRESS is STD_ON / STD_OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.17 AdcDebugSupport

Table 23 AdcDebugSupport

Name:	AdcDebugSupport		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Debug support enabled False: Debug support disabled		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling/disabling the debugging support. This configuration parameter is named as ADC_DEBUG_SUPPORT in the configuration file. The Value of the parameter ADC_DEBUG_SUPPORT is STD_ON / STD_OFF.		
SFRs Modified:	None		

Scope:	Driver
Dependency:	None

4.2.18 AdcSameChannelQueuing

Table 24 AdcSameChannelQueuing

Name:	AdcSameChannelQueuing		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Enables the Queue request source to handle the same channel repeatedly False: Disables the Queue request source to handle the same channel repeatedly		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling/disabling the feature of queuing the same channel repeatedly in Queue request source. This configuration parameter is named as ADC_SAME_CHANNEL_QUEUING in the configuration file. The Value of the parameter ADC_SAME_CHANNEL_QUEUING is STD_ON / STD_OFF. If the parameter is enabled and and AdcGroupRequestSource is REQSRC0_8STG_QUE or REQSRC3_8STG_QUE, the same analog channel can be queued to convert maximum of 4 times in a group, i.e, same analog channel can be added in AdcGroupDefinition for maximum of 4 times in a group. Result of the same channel is accumulated in result register/buffer as many times it is been queued. A channel should be queued equal number of times across the queue groups (REQSRC0_8STG_QUE and REQSRC3_8STG_QUE). Queuing can be in any sequence; allows inserting other channels in between as well. If AdcSameChannelQueuing is set to true (ADC_SAME_CHANNEL_QUEUING = STD_ON), uint16 is selected for for Adc_ValueGroupType.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.19 AdcMasterSlaveSync

Table 25 Specification of AdcMasterSlaveSync

Name:	AdcMasterSlaveSync
Source:	This parameter is defined by IFX.
File:	Adc_Cfg.h

Range:	True: Enables the Synchronization of conversion (Parallel sampling) feature False: Disables the Synchronization of conversion (Parallel sampling) feature		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling/disabling the Synchronization of conversion (Parallel sampling) feature. This configuration parameter is named as ADC_MASTER_SLAVE_SYNC in the configuration file. The Value of the parameter ADC_MASTER_SLAVE_SYNC is STD_ON / STD_OFF. This parameter is editable only if AdcPriorityImplementation is ADC_PRIORITY_NONE and ADC_PRIORITY_HW.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcPriorityImplementation		

4.2.20 AdcAlternateRefMode

Table 26 Specification of AdcAlternateRefMode

Name:	AdcAlternateRefMode		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	ADC_REF_LOCAL_MODE: Channel 0 (CH0) of a Kernel is used as reference ADC_REF_GLOBAL_MODE: Channel 0 (CH0) of a selected kernel Kernel of a cluster is used as reference		
Default value:	ADC_REF_LOCAL_MODE		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling/disabling the alternate reference selection with in the cluster. This configuration parameter is named as ADC_ALTERNATE_REF_MODE in the configuration file. The Value of the parameter ADC_ALTERNATE_REF_MODE is ADC_REF_LOCAL_MODE / ADC_REF_GLOBAL_MODE. This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHSx_CALCTR Where x is SHS cluster number		
Scope:	Driver		
Dependency:	AdcSampleAndHoldUnitAvailable		

4.2.21 AdcSampleAndHoldUnitAvailable

Table 27 Specification of AdcSampleAndHoldUnitAvailable

Name:	AdcSampleAndHoldUnitAvailable		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Enables the configuration parameter and Initialization of the SFRs specific to the Sample & Hold units of ADC False: Disables the configuration parameter and Initialization of the SFRs specific to the Sample & Hold units of ADC		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling/disabling the Sample and Hold units of ADC. This configuration parameter is named as ADC_SAMPLE_HOLD_UNIT in the configuration file. The Value of the parameter ADC_SAMPLE_HOLD_UNIT is STD_ON / STD_OFF. This parameter must be set to true for TC27x BA, BC steps, TC26x AA, AB steps and TC29x AA step. This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.22 AdcCcu6Synchronization

Table 28 Specification of AdcCcu6Synchronization

Name:	AdcCcu6Synchronization		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Enables the CCU6x T13 to generate 20MHz Synchronization signal to all VADCs False: Disables the CCU6x T13 to generate 20MHz Synchronization signal to all VADCs		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

Description:	Pre-processor switch to enable / disable the CCU6x T13 to generate 20MHz Synchronization signal to all VADCs. This configuration parameter is named as ADC_CCU6_SYNCHRONIZATION in the configuration file. The Value of the parameter ADC_CCU6_SYNCHRONIZATION is STD_ON / STD_OFF. This parameter is applicable only for SAR ADC devices with Metal fix for cross coupling effect. (e.g: TC29xBB, TC26xBB and TC27xDB)
SFRs Modified:	VADC_GLOBECFG
Scope:	Driver
Dependency:	None

4.2.23 AdcNonAutosarResultPolling

Table 29 Specification of AdcNonAutosarResultPolling

Name:	AdcNonAutosarResultPolling		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Services Adc_17_GetGroupResult() and Adc_17_GetChannelStatus() can be used. False: Services Adc_17_GetGroupResult() and Adc_17_GetChannelStatus() can not be used.		
Default value:	False		
Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable / disable the services Adc_17_GetGroupResult() and Adc_17_GetChannelStatus() from code. Switches the ADC Group result handling between AUTOSAR specific and Non AUTOSAR polling mode. To achieve the best interrupt load on CPU and to avoid usage of DMA channel to read the results, Non AUTOSAR specific result polling mode to be selected. This parameter cannot be configured, if AdcNonAutosarDmaResultHandling is enabled. This configuration parameter is named as ADC_RESULT_POLLING_MODE in the configuration file. The value of the parameter ADC_RESULT_POLLING_MODE is STD_ON / STD_OFF. This parameter is not applicable for Sample & Hold ADC devices (e.g: TC29xAA, TC26xAB and TC27xBC) This parameter is added based on JIRA 0000051124-40.		
Scope:	Driver		
Dependency:	AdcNonAutosarDmaResultHandling		

4.2.24 AdcResetSfrAtInit

Table 30 Specification of AdcResetSfrAtInit

Syntax:	AdcResetSfrAtInit		
Source:	This parameter is microcontroller specific		
File:	Adc_Cfg.h		
Range:	True: enable the resetting of SFRs used by ADC driver to its reset value in the Adc_Init() API. False: Disable the resetting of SFRs used by ADC driver to its reset value in the Adc_Init() API.		
Default value:	False		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable resetting SFRs used by ADC driver to its reset value in the Adc_Init() API. The Syntax of the generated parameter is, #define ADC_RESET_SFR_INIT (STD_ON) Or #define ADC_RESET_SFR_INIT (STD_OFF) This parameter is added based on JIRA 0000051023-33.		
Scope:	Driver		
Dependency:	None		

4.2.25 AdcRunningInUser0Mode

Table 31 Specification of AdcRunningInUser0Mode

Name:	AdcRunningInUser0Mode		
Source:	This is microcontroller specific parameter.		
File:	Adc_Cfg.h		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--
Description:	If 'AdcRunningInUser0Mode' is enabled then the SFR access mechanism selected by 'AdcUserModeInitApiEnable' or 'AdcUserModeDeInitApiEnable' or 'AdcUserModeRuntimeApiEnable' will run in user mode 0 for the selected API's. If the 'AdcRunningInUser0Mode' switch is not selected and if any of the switches 'AdcUserModeInitApiEnable', 'AdcUserModeDeInitApiEnable', 'AdcUserModeRuntimeApiEnable' are selected then SFR access		

Name:	AdcRunningInUser0Mode
	mechanism will run in user mode 1 for the selected API's If none of the switches are selected then SFR access mechanism will run in Supervisor mode for all API's
Scope:	Driver
Dependency:	If 'AdcRunningInUser0Mode' is enabled then at least any one of the switches 'AdcUserModeInitApiEnable' or 'AdcUserModeDeInitApiEnable' or 'AdcUserModeRuntimeApiEnable' should be enabled.

4.2.26 AdcUserModeInitApiEnable

Table 32 Specification of AdcUserModeInitApiEnable

Name:	AdcUserModeInitApiEnable		
Source:	This is microcontroller specific parameter.		
File:	Adc_Cfg.h		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--
Description:	If AdcUserModeInitApiEnable is enabled then the SFR access mechanism in ADC Init API will run in User mode. Selection of SFR access in User Mode 0 and User mode 1 is based on the switch AdcRunningInUser0Mode. If none of the switches are selected then SFR access mechanism will run in Supervisor mode for Init API		
Scope:	Driver		
Dependency:			

4.2.27 AdcUserModeDeInitApiEnable

Table 33 Specification of AdcUserModeDeInitApiEnable

Name:	AdcUserModeDeInitApiEnable		
Source:	This is microcontroller specific parameter.		
File:	Adc_Cfg.h		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--

Name:	AdcUserModeDeInitApiEnable
Description:	If AdcUserModeDeInitApiEnable is enabled then the SFR access mechanism in ADC De Init API will run in User mode. Selection of SFR access mechanism in User Mode 0 and User mode 1 is based on the switch AdcRunningInUser0Mode. If none of the switches are selected then SFR access mechanism will run in Supervisor mode for DeInit API
Scope:	Driver
Dependency:	

4.2.28 AdcUserModeRuntimeApiEnable

Table 34 Specification of AdcUserModeRuntimeApiEnable

Name:	AdcUserModeRuntimeApiEnable		
Source:	This is microcontroller specific parameter.		
File:	Adc_Cfg.h		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--
Description:	If AdcUserModeRuntimeApiEnable is enabled then the SFR access mechanism in all APIs (other than ADC Init and ADC DeInit APIs) will run in User mode. Selection of SFR Access mechanism in User Mode 0 and User mode 1 is based on the switch AdcRunningInUser0Mode If none of the switches are selected then SFR access mechanism will run in Supervisor mode for all API(other than Init and DeInit)		
Scope:	Driver		
Dependency:			

4.3 Container: AdcSafety

This is the base container that contains the configuration parameters of the safety feature.

4.3.1 AdcSafetyEnable

Table 35 Specification of AdcSafetyEnable

Name:	AdcSafetyEnable
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	True: Safety features enabled for ADC driver False: Safety features disabled for ADC driver
Default value:	False

Configuration Class:	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling the safety features of ADC driver. By enabling this flag, implicitly the following safety features are enabled: - Range check on API parameters are done - ADC configuration structure has a unique marker field which will be verified by the driver - Publish the safety usage of ADC driver for user to set the respective ADC settings This configuration parameter is named as ADC_SAFETY_ENABLE in the configuration file. The Value of the parameter ADC_SAFETY_ENABLE is STD_ON / STD_OFF		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.3.2 AdcInitCheckApi

Table 36 AdcInitCheckApi

Name:	AdcInitCheckApi		
Source:	This parameter is defined by IFX		
File:	Adc_Cfg.h		
Range:	True: Adc_InitCheck() API is enabled False: Adc_InitCheck() API is disabled		
Default value:	False		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety feature Adc_InitCheck() which verifies the initialization done by ADC driver. This configuration parameter is named as ADC_INITCHECK_API in the configuration file. The Value of the parameter ADC_INITCHECK_API is STD_ON / STD_OFF		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcSafetyEnable		

4.4 Container: AdcConfigSet

This is the base container that contains the post-build selectable configuration parameters for ADC Driver.

4.4.1 AdcSleepMode

Table 37 AdcSleepMode

Name:	AdcSleepMode		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	SLEEP_MODE_ACCEPT SLEEP_MODE_REJECT		
Default value:	SLEEP_MODE_ACCEPT		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	ADC Driver accepts/rejects the sleep mode request from MCU Driver depending on configured value.		
SFRs Modified:	VADC_CLC		
Scope:	Driver		
Dependency:	None		

4.4.2 AdcSystemClock

Table 38 AdcSystemClock

Name:	AdcSystemClock		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to parameter of type McuClockSettingConf		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter refers to the system clock configured by MCU driver. With this system clock, the GTM reload value corresponding to the configured timer period for timer triggered Groups is calculated.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	McuClockSettingConf		

4.4.3 AdcEmuxSelectGrp0

Table 39 AdcEmuxSelectGrp0

Name:	AdcEmuxSelectGrp0		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	HWUNIT_ADCx Where x :represents value (Maximum Number of Adc HW Unit -1)		
Default value:	HWUNIT_ADC0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	This parameter is used to route the external multiplexer control signals to the Emux interface of the VADC kernel. Value of the AdcEmuxSelectGrp0 and AdcEmuxSelectGrp1 should not be the same VADC kernel. This parameter is significant only when AdcUseEmux is true.
SFRs Modified:	VADC_EMUXSEL
Scope:	Driver
Dependency:	AdcUseEmux and AdcEmuxSelectGrp1

4.4.4 AdcEmuxSelectGrp1

Table 40 AdcEmuxSelectGrp1

Name:	AdcEmuxSelectGrp1		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	HWUNIT_ADCx Where x :represents value (Maximum Number of Adc HW Unit -1)		
Default value:	HWUNIT_ADC1		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is used to route the external multiplexer control signals to the Emux interface of the VADC kernel. Value of the AdcEmuxSelectGrp0 and AdcEmuxSelectGrp1 should not be the same VADC kernel. This parameter is significant only when AdcUseEmux is true.		
SFRs Modified:	VADC_EMUXSEL		
Scope:	Driver		
Dependency:	AdcUseEmux and AdcEmuxSelectGrp0		

4.4.5 AdcAnalogClockDivider

Table 41 AdcAnalogClockDivider

Name:	AdcAnalogClockDivider		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	00 _H – 1F _H		
Default value:	A _H		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The analog clock f_{ADCI} is used as internal clock for the converter and defines the conversion length and the sample time. 00_H $f_{ADCI} = f_{ADC}$ ($f_{ADC} / 2$ for SAR ADC devices) 01_H $f_{ADCI} = f_{ADC} / 2$		

	$02_H f_{ADCI} = f_{ADC} / 3$... $1F_H f_{ADCI} = f_{ADC} / 32$ This parameter should be configured such that, VADC converter reference clock (f_{ADCI}) value range is 2Mhz to 20Mhz. For the devices with Cross coupling issue the value of the VADC converter reference clock (f_{ADCI}) should be 25MHz.
SFRs Modified:	VADC_GLOBCFG
Scope:	Driver
Dependency:	None

4.4.6 AdcDigitalArbiterClockDivider

Table 42 AdcDigitalArbiterClockDivider

Name:	AdcDigitalArbiterClockDivider		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0 – 3		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The digital clock f_{ADCD} is used for the arbiter and defines the duration of an arbiter round. $00_B f_{ADCD} = f_{ADC}$ $01_B f_{ADCD} = f_{ADC} / 2$ $10_B f_{ADCD} = f_{ADC} / 3$ $11_B f_{ADCD} = f_{ADC} / 4$ This parameter should be configured such that, clock frequency of the arbiter is 25MHz for the devices with Cross coupling issue.		
SFRs Modified:	VADC_GLOBCFG		
Scope:	Driver		
Dependency:	None		

4.4.7 AdcSHSClockDivider

Table 43 Specification of AdcSHSClockDivider

Name:	AdcSHSClockDivider		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0x0 – 0xF		
Default value:	0		
Configuration Class:	Pre-Compile	--	--

	Link Time	--	--
	Post Build	x	Variant PB
Description:	The clock f_{SH} defines the frequency of the clock for the SHS logic. 0000_B $f_{SH} = f_{ADC}$ 0001_B $f_{SH} = f_{ADC} / 2$... 1111_B $f_{SH} = f_{ADC} / 16$ This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHSx_SHSCFG Where x is SHS cluster number		
Scope:	Driver		
Dependency:	None		

4.4.8 AdcOperationMode

Table 44 Specification of AdcOperationMode

Name:	AdcOperationMode		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	NORMAL_MODE FAST_STANDBY_MODE SLOW_STANDBY_MODE		
Default value:	NORMAL_MODE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines the operating mode of the converter in a standalone converter or converter in master mode. User must provide the required wakeup time for the Standby modes. For Fast Standby Mode, wakeup time is approximately 4 us. For Slow Standby Mode, wakeup time is approximately 15 us. This wakeup time can be established by adding it to the intended sample time. For the Standby mode to work, the clock source for f_{ADC} should be $f_{PLL2ERAY}$. i.e, ADC_CLOCK_ERAYPLL2 should be configured in the configuration parameter McuClockSelectADC of module MCU. This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	VADC_GxARBCFG Where x is Adc kernel number		
Scope:	Driver		
Dependency:	None		

4.4.9 AdcCalibrationOrder

Table 45 Specification of AdcCalibrationOrder

Name:	AdcCalibrationOrder		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	ADC_CONVERT_THEN_CAL ADC_CAL_THEN_CONVERT		
Default value:	ADC_CONVERT_THEN_CAL		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines the calibration order of the converter. This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHSx_CALCTR Where x is SHS cluster number		
Scope:	Driver		
Dependency:	AdcSampleAndHoldUnitAvailable		

4.4.10 AdcCalMaxTime

Table 46 Specification of AdcCalMaxTime

Name:	AdcCalMaxTime		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0x0 – 0x3F		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines the maximum time until the next calibration.. $t_{MAX} = 512 / f_{SH} * (CALMAX + 1)$ This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHSx_CALCTR Where x is SHS cluster number		
Scope:	Driver		
Dependency:	AdcSampleAndHoldUnitAvailable		

4.4.11 AdcCluster0AltRefDef

Table 47 Specification of AdcCluster0AltRefDef

Name:	AdcCluster0AltRefDef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	For TC26x: ADC_SAMPLE_HOLD_UNIT0 ADC_SAMPLE_HOLD_UNIT1 For TC27x / TC29x ADC_SAMPLE_HOLD_UNIT0 ADC_SAMPLE_HOLD_UNIT1 ADC_SAMPLE_HOLD_UNIT2 ADC_SAMPLE_HOLD_UNIT3		
Default value:	ADC_SAMPLE_HOLD_UNIT0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines S&H unit whose alternate reference is used for all the channels within the cluster 0. This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHS0_CALCTR		
Scope:	Driver		
Dependency:	AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable		

4.4.12 AdcCluster1AltRefDef

Table 48 Specification of AdcCluster1AltRefDef

Name:	AdcCluster1AltRefDef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	For TC26x ADC_SAMPLE_HOLD_UNIT2 ADC_SAMPLE_HOLD_UNIT3 For TC27x / TC29x ADC_SAMPLE_HOLD_UNIT4 ADC_SAMPLE_HOLD_UNIT5 ADC_SAMPLE_HOLD_UNIT6 ADC_SAMPLE_HOLD_UNIT7		
Default value:	For TC26x: ADC_SAMPLE_HOLD_UNIT2 For TC27x: ADC_SAMPLE_HOLD_UNIT4		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines S&H unit whose alternate reference is used for all the channels within the cluster 1.		

	This parameter is applicable only for Sample&Hold ADC devices.
SFRs Modified:	SHS1_CALCTR
Scope:	Driver
Dependency:	AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable

4.4.13 AdcCluster2AltRefDef

Table 49 Specification of AdcCluster2AltRefDef

Name:	AdcCluster2AltRefDef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	ADC_SAMPLE_HOLD_UNIT8 ADC_SAMPLE_HOLD_UNIT9 ADC_SAMPLE_HOLD_UNIT10		
Default value:	ADC_SAMPLE_HOLD_UNIT8		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines S&H unit whose alternate reference is used for all the channels within the cluster 2. This parameter is applicable only for TC29x and only for Sample&Hold ADC devices. This parameter is editable only if the parameter AdcSampleAndHoldUnitAvailable is set to true.		
SFRs Modified:	SHS2_CALCTR		
Scope:	Driver		
Dependency:	AdcAlternateRefMode, AdcSampleAndHoldUnitAvailable		

4.4.14 AdcRefPrechargeControl

Table 50 Specification of AdcRefPrechargeControl

Name:	AdcRefPrechargeControl		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	USE_VDDM_PRECH_VAREF_ADJ USE_VAREF_CONVERSION		
Default value:	USE_VDDM_PRECH_VAREF_ADJ		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines the Reference Precharge Control of the converter. This parameter is applicable only for the SAR ADC devices.		
SFRs Modified:	GLOBCFG		
Scope:	Driver		

Dependency:	None
-------------	------

4.4.15 AdcLowPowerSupplyVoltage

Table 51 Specification of AdcLowPowerSupplyVoltage

Name:	AdcLowPowerSupplyVoltage		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	POWER_SUPPLY_5V POWER_SUPPLY_3_3V		
Default value:	POWER_SUPPLY_5V		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Adjusts the analog circuitry to the supply voltage used in the application system. This parameter is applicable only for the SAR ADC devices.		
SFRs Modified:	GLOBCFG		
Scope:	Driver		
Dependency:	None		

4.4.16 AdcCcu6ModuleRef

Table 52 Specification of AdcCcu6ModuleRef

Name:	AdcCcu6ModuleRef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to parameter of type Ccu6Configuration in MCU		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter refers to the CCU6x module (T13) used to generate Synchronization signal to all VADCs. This parameter is applicable only if AdcCcu6Synchronization is set to true.		
SFRs Modified:	CCU6x_CLC, CCU6x_T13PR, CCU6x_CC63SR, CCU6x_MODCTR, CCU6x_TCTR0, CCU6x_TCTR4, CCU60_MOSEL		
Scope:	Driver		
Dependency:	AdcCcu6Synchronization		

4.4.17 Container: AdcGlobInputClass

This is a microcontroller specific container. ADC driver have 2 container of this type. Channel resolution and Channel sampling time are the parameters to be configured in this container globally for all the available Adc kernels.

4.4.17.1 AdcGlobChSampleTime

Table 53 AdcGlobChSampleTime

Name:	AdcGlobChSampleTime		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0 – 31		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The time during which the value is sampled, (in clock-cycles). VADC of AURIX provides the duration of the sample phase is programmed individually for each channel via sample time control bit field (STCS). A minimum sample phase of 2 analog clock cycles is extended by the programmed value. sample phase length = $(2 + \text{STC}) / f_{\text{ADCI}}$ f_{ADCI}: Analog clock frequency This parameter is used globally for all the available kernels of VADC.		
SFRs Modified:	VADC_GLOBICLASS0/1		
Scope:	Driver		
Dependency:	AdcAnalogClockDivider		

4.4.17.2 AdcGlobChResolution

Table 54 AdcGlobChResolution

Name:	AdcGlobChResolution		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	CH_RES_8BIT CH_RES_10BIT CH_RES_12BIT		
Default value:	CH_RES_10BIT		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	ADC Channel resolution in bits. This parameter is used globally for all the available kernels of Adc.		
SFRs Modified:	VADC_GLOBICLASS0/1		
Scope:	Driver		
Dependency:	None		

4.4.17.3 AdcGlobEmuxChSampleTime

Table 55 AdcGlobEmuxChSampleTime

Name:	AdcGlobEmuxChSampleTime		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0- 31		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configuration of sampling time (in clock cycles) for Emux channel, i.e. the time during which the value is sampled. VADC of AURIX provides the duration of the sample phase is programmed for Emux channel via sample time control bit field (STCE). A minimum sample phase of 2 analog clock cycles is extended by the programmed value. sample phase length = $(2 + STC) / f_{ADC}$ f_{ADC}: Analog clock frequency This parameter is used globally for all the available kernels of VADC. This parameter is significant only when AdcUseEmux is true.		
SFRs Modified:	VADC_GLOBICLASS0/1		
Scope:	Driver		
Dependency:	Editable only if AdcUseEmux is set to STD_ON		

4.5 Container: AdcHWUnit

This container contains the VADC Hardware unit configuration parameters for each VADC kernel. Data structure Adc_HwUnitCfgType is used to configure all the HW Unit specific parameters.

4.5.1 AdcClockSource

Table 56 AdcClockSource

Name:	AdcClockSource		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	CLK_SRC_SYS_CLK		
Default value:	CLK_SRC_SYS_CLK		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter selects the source for the VADC peripheral clock f_{ADC}. AURIX HW supports only System clock for VADC peripheral.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.2 AdcHwUnitId

Table 57 AdcHwUnitId

Name:	AdcHwUnitId		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	HWUNIT_ADCx Where x :represents value (Maximum Number of Adc HW Unit -1)		
Default value:	HWUNIT_ADC0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is used to arrange the configuration parameters in the configuration structure according to its HW Unit ID. AdcHwUnitId must be unique.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcHwUnitId		

4.5.3 AdcPrescale

Table 58 AdcPrescale

Name:	AdcPrescale		
Source:	This parameter is defined by the AUTOSAR.		
File:	None		
Range:	--		
Default value:	--		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is not supported in Aurix hardware.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.4 AdcArbitrationRoundLength

Table 59 AdcArbitrationRoundLength

Name:	AdcArbitrationRoundLength		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	ARBITRATION_SLOTS_4 ARBITRATION_SLOTS_8 ARBITRATION_SLOTS_16		

	ARBITRATION_SLOTS_20		
Default value:	ARBITRATION_SLOTS_4		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter defines the number of arbitration slots per arbitration round. ARBITRATION_SLOTS_8 must be configured if REQSRC3_8STG_QUE is selected for the parameter AdcGroupRequestSource. ARBITRATION_SLOTS_16 must be configured for the devices with Cross coupling issue (e.g., TC27xCA).		
SFRs Modified:	VADC_GxARBCFG Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcGroupRequestSource		

4.5.5 AdcRequestSource0Prio

Table 60 AdcRequestSource0Prio

Name:	AdcRequestSource0Prio		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	REQSRC_PRIORITY_0_LOWEST REQSRC_PRIORITY_1 REQSRC_PRIORITY_2 REQSRC_PRIORITY_3_HIGHEST		
Default value:	REQSRC_PRIORITY_0_LOWEST		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Priority levels for Request Source. The configuration of priority level for request sources is applicable only for priority mechanism: ADC_PRIORITY_HW and ADC_PRIORITY_HW_SW. In ADC_PRIORITY_NONE, the arbitration level for request sources is fixed to default value. In ADC_PRIORITY_HW implementation, the Groups under the same request source will have same priority levels. Hence the AdcGroupPriority is not significant in ADC_PRIORITY_HW priority mechanism. In ADC_PRIORITY_HW_SW implementation, the Groups under the same request source will be arbitrated by the Driver and between the different trigger sources, the ADC peripheral will arbitrate depending on the request source arbitration level.		
SFRs Modified:	VADC_GxARBPR Where x is Adc kernel number		
Scope:	Driver		

Dependency:	AdcPriorityImplementation
-------------	---------------------------

4.5.6 AdcRequestSource1Prio

Table 61 AdcRequestSource1Prio

Name:	AdcRequestSource1Prio		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	REQSRC_PRIORITY_0_LOWEST REQSRC_PRIORITY_1 REQSRC_PRIORITY_2 REQSRC_PRIORITY_3_HIGHEST		
Default value:	REQSRC_PRIORITY_0_LOWEST		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Priority levels for Request Source. The configuration of priority level for request sources is applicable only for priority mechanism: ADC_PRIORITY_HW and ADC_PRIORITY_HW_SW. In ADC_PRIORITY_NONE, the arbitration level for request sources is fixed to default value. In ADC_PRIORITY_HW implementation, the Groups under the same request source will have same priority levels. Hence the AdcGroupPriority is not significant in ADC_PRIORITY_HW priority mechanism. In ADC_PRIORITY_HW_SW implementation, the Groups under the same request source will be arbitrated by the Driver and between the different trigger sources, the ADC peripheral will arbiter depending on the request source arbitration level.		
SFRs Modified:	VADC_GxARBPR Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcPriorityImplementation		

4.5.7 AdcRequestSource3Prio

Table 62 AdcRequestSource3Prio

Name:	AdcRequestSource3Prio		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	REQSRC_PRIORITY_0_LOWEST REQSRC_PRIORITY_1 REQSRC_PRIORITY_2 REQSRC_PRIORITY_3_HIGHEST		
Default value:	REQSRC_PRIORITY_0_LOWEST		
Configuration Class:	Pre-Compile	--	--

	Link Time	--	--
	Post Build	x	Variant PB
Description:	Priority levels for Request Source. The configuration of priority level for request sources is applicable only for priority mechanism: ADC_PRIORITY_HW and ADC_PRIORITY_HW_SW. In ADC_PRIORITY_NONE, the arbitration level for request sources is fixed to default value. In ADC_PRIORITY_HW implementation, the Groups under the same request source will have same priority levels. Hence the AdcGroupPriority is not significant in ADC_PRIORITY_HW priority mechanism. In ADC_PRIORITY_HW_SW implementation, the Groups under the same request source will be arbitrated by the Driver and between the different trigger sources, the ADC peripheral will arbiter depending on the request source arbitration level. This parameter is available only for TC23x & TC22x		
SFRs Modified:	VADC_GxARBPR Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcPriorityImplementation		

4.5.8 AdcPostCal

Table 63 AdcPostCal

Name:	AdcPostCal		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	POST_CAL_ENABLE POST_CAL_DISABLE		
Default value:	POST_CAL_ENABLE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Enable/Disable Post calibration. Conversion time with post-calibration : $2 \times T_{ADC} + (4 + STC + N) \times T_{ADCI}$ Conversion time without post-calibration : $2 \times T_{ADC} + (2 + STC + N) \times T_{ADCI}$ Where: T_{ADC} – module clock period (same as system clock period) T_{ADCI} – analog clock period (Refer 4.4.5) This parameter is not significant for TC27xBC step device due to the hardware errata ADC_TC.62. Hardware calibration is disabled.		
SFRs Modified:	VADC_GLOBCFG		
Scope:	Driver		
Dependency:	None		

4.5.9 AdcDmaChannelRef

Table 64 AdcDmaChannelRef

Name:	AdcDmaChannelRef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to all available DMA Channels		
Default value:	DMA_CHANNEL0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	In Non AUTOSAR specific result handling mode, the user needs to configure and handle the DMA peripheral using IFX DMA Complex driver. <i>Note: The selected DMA Channel should match with the DMA channel configuration which needs to be used for ADC result handling.</i>		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcNonAutosarDmaResultHandling		

4.5.10 AdcSyncConvMode

Table 65 Specification of AdcSyncConvMode

Name:	AdcSyncConvMode		
Source:	This parameter is defined by IFX.		
File:	Adc_PBCfg.c		
Range:	ADC_MASTER ADC_SLAVE ADC_STANDALONE		
Default value:	ADC_STANDALONE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Conversion mode selection of the Adc kernel for Synchronization of conversion. This parameter is significant only if AdcMasterSlaveSync is set to True (STD_ON).		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcMasterSlaveSync		

4.5.11 AdcAcceleratedTiming

Table 66 Specification of AdcAcceleratedTiming

Name:	AdcAcceleratedTiming
--------------	----------------------

Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	ENABLE DISABLE		
Default value:	DISABLE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Enables/ disables the enhanced conversion timing scheme. This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHSx_TIMCFGy Where x is SHS cluster number & y is SHS unit number within the cluster		
Scope:	Driver		
Dependency:	AdcSampleAndHoldUnitAvailable		

4.5.12 AdcShortSampleTime

Table 67 Specification of AdcShortSampleTime

Name:	AdcShortSampleTime		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0x0 - 0x3F		
Default value:	0x0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines the duration of the sample phase. 00_H Sample time is defined by DIVA and STC 01_H Sample time is $t_{ADC} * 1$... 3F_H Sample time is $t_{ADC} * 63$ This parameter is applicable only for Sample&Hold ADC devices.		
SFRs Modified:	SHSx_TIMCFGy Where x is SHS cluster number & y is SHS unit number within the cluster		
Scope:	Driver		
Dependency:	AdcSampleAndHoldUnitAvailable		

4.5.13 AdcSafetySignal

Table 68 Specification of AdcSafetySignal

Name:	AdcSafetySignal		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	True: Adc Hw Unit is used as Safety signal False: Adc Hw Unit is used as QM signal		
Default value:	False		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The parameter defines the analog channels of the hardware unit is part of ASIL signal or QM signal. This parameter is significant only if AdcSafetyEnable is set to True.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcSafetyEnable		

4.5.14 Container: AdcChannel

This container contains the channel configuration (parameters) depending on the hardware capability. Data type Adc_ChannelCfgType is used to configure all the Channel specific parameters. ADC Driver provides a macro which will accommodate channel configuration parameters in type Adc_ChannelCfgType.

Maximum number of channel configuration under each HW Unit is 255.

Note: Each analog channel can have more than one set of channel configuration.

4.5.14.1 AdcChannelConvTime

Table 69 AdcChannelConvTime

Name:	AdcChannelConvTime		
Source:	This parameter is defined by the AUTOSAR.		
File:	NA		
Range:	--		
Default value:	--		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configuration of conversion time, including sample time (in clock cycles) for each channel, if supported by hardware. This configuration parameter is not supported in Aurix.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.14.2 AdcChannelHighLimit

Table 70 AdcChannelHighLimit

Name:	AdcChannelHighLimit		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	0 – 0xFFFF		
Default value:	0xFF		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	High Limit value used for limit checking. This parameter is significant only if AdcEnableLimitCheck and AdcChannelLimitCheck are set to True (STD_ON). AdcChannelHighLimit value has to be greater than or equal to AdcChannelLowLimit. The parameter is not editable if AdcChannelRangeSelect is ADC_RANGE_UNDER_LOW or ADC_RANGE_NOT_UNDER_LOW.		
SFRs Modified:	VADC_GxBOUND Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcEnableLimitCheck AdcChannelLimitCheck AdcChannelLowLimit AdcChannelRangeSelect		

4.5.14.3 AdcChannelId

Table 71 AdcChannelId

Name:	AdcChannelId		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	0 – x x: is the number of Adc channels configured for a Adc kernel		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Numeric ID of the Channel. This numeric ID represents the array index of its configuration in ADC Channel configuration set.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.14.4 AdcChannelLimitCheck

Table 72 AdcChannelLimitCheck

Name:	AdcChannelLimitCheck		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	True: Limit checking is enabled for a channel False: Limit checking is disabled for a channel.		
Default value:	False		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Pre-processor switch to enable / disable the Limit checking for the Adc Channel. This configuration parameter is named as ADC_CHANNEL_LIMIT_CHECK in the configuration file. The value of the parameter ADC_CHANNEL_LIMIT_CHECK is STD_ON / STD_OFF. This parameter is significant only if AdcEnableLimitCheck is set to True (STD_ON). Adc channels with limit checking feature enabled have to be assigned to ADC group which consists exactly of one limit checking enabled ADC channel. When a limitcheck enabled group is requested (start/enable) for a conversion, another limit check enabled group of the same ADC kernel should not be requested for the conversion.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcEnableLimitCheck AdcGroupDefinition		

4.5.14.5 AdcChannelLowLimit

Table 73 AdcChannelLowLimit

Name:	AdcChannelLowLimit		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	0 – 0xFFFF		
Default value:	0x0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Low Limit value used for limit checking. This parameter is significant only if AdcEnableLimitCheck and AdcChannelLimitCheck are set to True (STD_ON).		

	AdcChannelLowLimit value has to be less than or equal to AdcChannelHighLimit. The parameter is not editable if AdcChannelRangeSelect is ADC_RANGE_OVER_HIGH or ADC_RANGE_NOT_OVER_HIGH
SFRs Modified:	VADC_GxBOUND Where x is Adc kernel number
Scope:	Driver
Dependency:	AdcEnableLimitCheck AdcChannelLimitCheck AdcChannelHighLimit AdcChannelRangeSelect

4.5.14.6 AdcChannelRangeSelect

Table 74 AdcChannelRangeSelect

Name:	AdcChannelRangeSelect		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_RANGE_ALWAYS ADC_RANGE_BETWEEN ADC_RANGE_NOT_BETWEEN ADC_RANGE_OVER_HIGH ADC_RANGE_UNDER_LOW ADC_RANGE_NOT_OVER_HIGH ADC_RANGE_NOT_UNDER_LOW		
Default value:	ADC_RANGE_ALWAYS		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter defines which conversion values are taken into account related to the borders defined with AdcChannelLowLimit and AdcChannelHighLimit. This parameter is significant only if AdcEnableLimitCheck and AdcChannelLimitCheck are set to True (STD_ON). If ADC_RANGE_OVER_HIGH or ADC_RANGE_NOT_OVER_HIGH is selected. User shall configure only the AdcChannelHighLimit. If ADC_RANGE_UNDER_LOW or ADC_RANGE_NOT_UNDER_LOW is selected. User shall configure only the AdcChannelLowLimit.		
SFRs Modified:	VADC_GxCHCTry Where x is Adc kernel number & y is Analog channel number		
Scope:	Driver		
Dependency:	AdcEnableLimitCheck AdcChannelLimitCheck		

4.5.14.7 AdcChannelRefVoltsrcHigh

Table 75 AdcChannelRefVoltsrcHigh

Name:	AdcChannelRefVoltsrcHigh		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	REF_VOLTAGE_VAREF REF_VOLTAGE_CH0		
Default value:	REF_VOLTAGE_VAREF		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Upper reference voltage source for each channel. REF_VOLTAGE_VAREF is used for self calibration of the A/D Converter. Therefore, it must be stable during all conversions, even for those that use another reference voltage. <i>Note: For the channels CH4, CH5, CH6, and CH7 of HWUNIT_ADC0 and the channel CH3 of HWUNIT_ADC4 are always converted referring to the REF_VOLTAGE_VAREF input. For these channels, the alternative reference (REF_VOLTAGE_CH0) should not be used.</i>		
SFRs Modified:	VADC_GxCHCTry Where x is Adc kernel number & y is Analog channel number		
Scope:	Driver		
Dependency:	AdcHwUnitId AdcAnChannelNum		

4.5.14.8 AdcChannelRefVoltsrcLow

Table 76 AdcChannelRefVoltsrcLow

Name:	AdcChannelRefVoltsrcLow		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	REF_VOLTAGE_VAGND		
Default value:	REF_VOLTAGE_VAGND		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Lower reference voltage source for each channel. The value is fixed to VAGND for AURIX VADC peripheral and hence not configurable parameter.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.14.9 AdcAnChannelNum

Table 77 AdcAnChannelNum

Name:	AdcAnChannelNum		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	For HWUNIT_ADC0: 0 – 7 (TC27x/TC29x) & 0 – 15 (TC26x/TC23x/TC22x) For HWUNIT_ADC1: 8 – 15 (TC27x/TC29x) & 16 – 31 (TC26x/TC23x/TC22x) For HWUNIT_ADC2: 16 – 23 (TC27x/TC29x) & 32 – 47 (TC26x) For HWUNIT_ADC3: 24 – 31 (TC27x/TC29x) & 48 – 63 (TC26x) For HWUNIT_ADC4: 32 – 39 For HWUNIT_ADC5: 40 – 47 For HWUNIT_ADC6: 48 – 55 For HWUNIT_ADC7: 56 – 63 For HWUNIT_ADC8: 64 – 71 For HWUNIT_ADC9: 72 – 79 For HWUNIT_ADC10: 80 – 87		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	Analog Input Channel Number. With EMUX OFF, 8 analog inputs for each VADC HW Units are connected. With EMUX ON, external 1-out-of-8 multiplexer can be used to extend the number of analog input channels at each analog channel. Refer the respective Aurix derivative data sheet for the available analog channel. Configuration error will be given if wrong analog channel is configured as "Invalid AdcAnChannelNum for HWUNIT_ADCx. Selected channel number should be from "y" to "z" (Ref Range) only. If the selected channel number is within the range, then the channel is not available for the selected derivative of Aurix". <i>Note: All unused analog input pins should be connected to a fixed potential either VAGND or VAREF to avoid disturbance of active analog inputs.</i> Refer Analog Channel Mapping Table 78 for the available analog channel of the specific devices.
SFRs Modified:	None
Scope:	Driver
Dependency:	AdcHwUnitId

Table 78 Analog Channel Mapping for Aurix HE

HW UM									MCAL ADC UM
Signal		Source							AdcAnChannelNum
TC27x/TC29x	TC26x	TC264	TC265/TC267	TC275	TC277	TC297	TC298	TC299	
G0CH0	G0CH0	AN0	AN0	AN0	AN0	AN0	AN0	AN0	0
G0CH1	G0CH1	AN1	AN1	AN1	AN1	AN1	AN1	AN1	1
G0CH2	G0CH2	AN2	AN2	AN2	AN2	AN2	AN2	AN2	2
G0CH3	G0CH3	AN3	AN3	AN3	AN3	AN3	AN3	AN3	3
G0CH4	G0CH4	AN4	AN4	AN4	AN4	AN4	AN4	AN4	4
G0CH5	G0CH5	AN5	AN5	AN5	AN5	AN5	AN5	AN5	5
G0CH6	G0CH6	AN6	AN6	AN6	AN6	AN6	AN6	AN6	6
G0CH7	G0CH7	AN7	AN7	AN7	AN7	AN7	AN7	AN7	7
G1CH0	G0CH8	AN8	AN8	AN8	AN8	AN8	AN8	AN8	8
G1CH1	G0CH9	-	-	-	AN9	AN9	AN9	AN9	9
G1CH2	G0CH10	AN10	AN10	AN10	AN10	AN10	AN10	AN10	10
G1CH3	G0CH11	AN11	AN11	AN11	AN11	AN11	AN11	AN11	11
G1CH4	G0CH12	AN12	AN12	AN12	AN12	AN12	-	AN12	12
G1CH5	G0CH13	AN13	AN13	AN13	AN13	AN13	-	AN13	13
G1CH6	G0CH14	-	-	-	AN14	AN14	-	AN14	14
G1CH7	G0CH15	-	-	-	AN15	AN15	-	AN15	15
G2CH0	G1CH0	AN16	AN16	AN16	AN16	AN16	AN16	AN16	16
G2CH1	G1CH1	AN17	AN17	AN17	AN17	AN17	AN17	AN17	17
G2CH2	G1CH2	-	AN18	AN18	AN18	AN18	AN18	AN18	18
G2CH3	G1CH3	-	AN19	AN19	AN19	AN19	AN19	AN19	19
G2CH4	G1CH4	AN20	AN20	AN20	AN20	AN20	AN20	AN20	20
G2CH5	G1CH5	AN21	AN21	AN21	AN21	AN21	AN21	AN21	21
G2CH6	G1CH6	-	-	-	AN22	AN22	AN22	AN22	22

G2CH7	G1CH7	-	-	-	AN23	AN23	AN23	AN23	23
G3CH0	G1CH8	AN24, P40.0	AN24, P40.0	AN24, P40.0	AN24, P40.0	AN24, P40.0	AN24, P40.0	AN24, P40.0	24
G3CH1	G1CH9	AN25, P40.1	AN25, P40.1	AN25, P40.1	AN25, P40.1	AN25, P40.1	AN25, P40.1	AN25, P40.1	25
G3CH2	G1CH10	-	AN26, P40.2	AN26, P40.2	AN26, P40.2	AN26, P40.2	AN26, P40.2	AN26, P40.2	26
G3CH3	G1CH11	-	AN27, P40.3	AN27, P40.3	AN27, P40.3	AN27, P40.3	AN27, P40.3	AN27, P40.3	27
G3CH4	G1CH12	-	AN28	AN28	AN28	AN28	AN28	AN28	28
G3CH5	G1CH13	-	AN29	AN29	AN29	AN29	AN29	AN29	29
G3CH6	G1CH14	-	-	-	AN30	AN30	-	AN30	30
G3CH7	G1CH15	-	-	-	AN31	AN31	-	AN31	31
G4CH0	G2CH0	-	AN32, P40.4	AN32, P40.4	AN32, P40.4	AN32, P40.4	AN32, P40.4	AN32, P40.4	32
G4CH1	G2CH1	-	AN33, P40.5	AN33, P40.5	AN33, P40.5	AN33, P40.5	AN33, P40.5	AN33, P40.5	33
G4CH2	G2CH2	-	-	-	AN34	AN34	-	AN34	34
G4CH3	G2CH3	AN35	AN35	AN35	AN35	AN35	-	AN35	35
G4CH4	G2CH4	AN36, P40.6	AN36, P40.6	AN36, P40.6	AN36, P40.6	AN36, P40.6	AN36, P40.6	AN36, P40.6	36
G4CH5	G2CH5	AN37, P40.7	AN37, P40.7	AN37, P40.7	AN37, P40.7	AN37, P40.7	AN37, P40.7	AN37, P40.7	37
G4CH6	G2CH6	AN38, P40.8	AN38, P40.8	AN38, P40.8	AN38, P40.8	AN38, P40.8	-	AN38, P40.8	38
G4CH7	G2CH7	AN39, P40.9	AN39, P40.9	AN39, P40.9	AN39, P40.9	AN39, P40.9	-	AN39, P40.9	39
G5CH0	G2CH8	-	-	-	AN40	AN40	AN40	AN40	40
G5CH1	G2CH9	-	-	-	AN41	AN41	AN41	AN41	41
G5CH2	G2CH10	AN44	AN44	-	AN42	AN42	AN42	AN42	42
G5CH3	G2CH11	AN45	AN45	-	AN43	AN43	AN43	AN43	43
G5CH4	G2CH12	AN46	AN46	AN44	AN44	AN44	-	AN44	44
G5CH5	G2CH13	AN47	AN47	AN45	AN45	AN45	-	AN45	45
G5CH6	G2CH14	AN48	AN48	AN46	AN46	AN46	-	AN46	46
G5CH7	G2CH15	AN49	AN49	AN47	AN47	AN47	-	AN47	47
G6CH0	G3CH0	P00.12	P00.12	P00.12	P00.12	P00.12	P00.12	P00.12	48
G6CH1	G3CH1	-	P00.11	P00.11	P00.11	P00.11	P00.11	P00.11	49
G6CH2	G3CH2	-	P00.10	P00.10	P00.10	P00.10	P00.10	P00.10	50
G6CH3	G3CH3	P00.9	P00.9	P00.9	P00.9	P00.9	P00.9	P00.9	51
G6CH4	G3CH4	P00.8	P00.8	P00.8	P00.8	P00.8	P00.8	P00.8	52
G6CH5	G3CH5	P00.7	P00.7	P00.7	P00.7	P00.7	P00.7	P00.7	53
G6CH6	G3CH6	P00.6	P00.6	-	-	-	-	-	54
G6CH7	G3CH7	P00.5	P00.5	-	-	-	-	-	55
G7CH0	G3CH8	P00.4	P00.4	P00.6	P00.6	P00.6	P00.6	P00.6	56
G7CH1	G3CH9	P00.3	P00.3	P00.5	P00.5	P00.5	P00.5	P00.5	57
G7CH2	G3CH10	P00.2	P00.2	P00.4	P00.4	P00.4	P00.4	P00.4	58
G7CH3	G3CH11	P00.1	P00.1	P00.3	P00.3	P00.3	P00.3	P00.3	59

G7CH4	G3CH12	-	-	P00.2	P00.2	P00.2	P00.2	P00.2	60
G7CH5	G3CH13	-	-	P00.1	P00.1	P00.1	P00.1	P00.1	61
G7CH6	G3CH14	-	-	-	-	-	-	-	62
G7CH7	G3CH15	-	-	-	-	-	-	-	63
G8CH0	-	-	-	-	-	-	AN48	AN48	64
G8CH1	-	-	-	-	-	-	AN49	AN49	65
G8CH2	-	-	-	-	-	-	-	AN50	66
G8CH3	-	-	-	-	-	-	-	AN51	67
G8CH4	-	-	-	-	-	-	AN52	AN52	68
G8CH5	-	-	-	-	-	-	AN53	AN53	69
G8CH6	-	-	-	-	-	-	AN54	AN54	70
G8CH7	-	-	-	-	-	-	AN55	AN55	71
G9CH0	-	-	-	-	-	-	AN56	AN56	72
G9CH1	-	-	-	-	-	-	AN57	AN57	73
G9CH2	-	-	-	-	-	-	-	AN58	74
G9CH3	-	-	-	-	-	-	-	AN59	75
G9CH4	-	-	-	-	-	-	AN60	AN60	76
G9CH5	-	-	-	-	-	-	AN61	AN61	77
G9CH6	-	-	-	-	-	-	-	AN62	78
G9CH7	-	-	-	-	-	-	-	AN63	79
G10CH0	-	-	-	-	-	-	AN64	AN64	80
G10CH1	-	-	-	-	-	-	AN65	AN65	81
G10CH2	-	-	-	-	-	-	-	AN66	82
G10CH3	-	-	-	-	-	-	-	AN67, P40.10	83
G10CH4	-	-	-	-	-	-	AN68, P40.11	AN68, P40.11	84
G10CH5	-	-	-	-	-	-	AN69, P40.12	AN69, P40.12	85
G10CH6	-	-	-	-	-	-	AN70, P40.13	AN70, P40.13	86
G10CH7	-	-	-	-	-	-	AN71, P40.14	AN71, P40.14	87

Table 79 Analog Channel Mapping for Aurix EP

	HW UM		MCAL ADC UM
Signal	Source		AdcAnChannelNum
TC23x/TC22x	TC233 / TC234 / TC237 / TC223 / TC224	TC222	
G0CH0	AN0, P40.0	-	0
G0CH1	AN1, P40.1	-	1
G0CH2	AN2, P40.2	-	2
G0CH3	AN3, P40.3	-	3
G0CH4	AN4, P40.4	-	4
G0CH5	AN5, P40.5	AN5, P40.5	5
G0CH6	AN6, P40.6	AN6, P40.6	6

	HW UM		MCAL ADC UM
G0CH7	AN7, P40.7	AN7, P40.7	7
G0CH8	AN8, P40.8	AN8, P40.8	8
G0CH9	AN9, P40.9	AN9, P40.9	9
G0CH10	AN10, P40.10	AN10, P40.10	10
G0CH11	AN11, P40.11	AN11, P40.11	11
G0CH12	-	-	12
G0CH13	-	-	13
G0CH14	-	-	14
G0CH15	-	-	15
G1CH0	AN12, P41.0	AN12, P41.0	16
G1CH1	AN13, P41.1	AN13, P41.1	17
G1CH2	AN14, P41.2	AN14, P41.2	18
G1CH3	AN15, P41.3	AN15, P41.3	19
G1CH4	AN16, P41.4	AN16, P41.4	20
G1CH5	AN17, P41.5	AN17, P41.5	21
G1CH6	AN18, P41.6	AN18, P41.6	22
G1CH7	AN19, P41.7	-	23
G1CH8	AN20, P41.8	-	24
G1CH9	AN21, P41.9	-	25
G1CH10	AN22, P41.10	-	26
G1CH11	AN23, P41.11	-	27
G1CH12	-	-	28
G1CH13	-	-	29
G1CH14	-	-	30
G1CH15	-	-	31

4.5.14.10 AdcInputClassSelection

Table 80 AdcInputClassSelection

Name:	AdcInputClassSelection		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to parameter of type AdcGlobInputClass and AdcKernellInputClass		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Each VADC HW Unit has 4 input classes (2 Global input classes and 2 Kernel input classes) for configuring channel resolution and sampling time. The selected input class parameter value is taken for the channel resolution and channel sampling time.		
SFRs Modified:	None		
Scope:	Driver		

Name:	AdcInputClassSelection
Dependency:	None

4.5.14.11 AdcChannelResolution

Table 81 AdcChannelResolution

Name:	AdcChannelResolution		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	In AURIX, Channel resolution configuration is moved to Input class container and a reference parameter is provided to select one of 4 input classes.		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Channel resolution in bits.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.14.12 AdcChannelSampTime

Table 82 AdcChannelSampTime

Name:	AdcChannelSampTime		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	In AURIX, Channel sampling time configuration is moved to Input class container and a reference parameter is provided to select one of 4 input classes.		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configuration of sampling time (in clock cycles) for each channel, i.e. the time during which the value is sampled if supported by hardware.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.14.13 AdcSyncChannel

Table 83 Specification of AdcSyncChannel

Name:	AdcSyncChannel
Source:	This parameter is defined by IFX.

File:	Adc_PBCfg.c		
Range:	True: Adc channel is enabled for a synchronized conversion. False: Adc channel is not enabled for a synchronized conversion.		
Default value:	False		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Switch to enable / disable the channel for the Synchronized conversion. This parameter is significant only if AdcMasterSlaveSync is set to True (STD_ON).		
SFRs Modified:	VADC_GxCHCTry Where x is Adc kernel number & y is Analog channel number		
Scope:	Driver		
Dependency:	AdcMasterSlaveSync		

4.5.15 Container: AdcGroup

This container contains the ADC Channel Group configuration parameters. Data structure Adc_GroupCfgType is used to configure all the Group specific parameters.

ADC Channel Groups are sorted in ADC Group configuration structure in the order of their GroupTrigger Source and request source.

Primary sort (Trigger Source): SW groups then HW groups

Secondary sort (Request Source): RS0 to RS3

Note: Maximum number of Groups limited to 32 under each hardware unit.

4.5.15.1 AdcGroupAccessMode

Table 84 AdcGroupAccessMode

Name:	AdcGroupAccessMode		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_ACCESS_MODE_SINGLE ADC_ACCESS_MODE_STREAMING		
Default value:	ADC_ACCESS_MODE_SINGLE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	Type of access mode to group conversion results. In Single access mode, user can have only one set of latest conversion result for the Group channels. In Streaming access mode, user can have Configured AdcStreamingNumSamples sets of conversion results for the Group channels. This parameter is significant if AdcNonAutosarDmaResultHandling is set to False (STD_OFF). ADC_ACCESS_MODE_STREAMING is not significant if AdcGroupTriggSrc is ADC_TRIGG_SRC_SW and AdcGroupConvMode is ADC_CONV_MODE_ONESHOT
SFRs Modified:	None
Scope:	Driver
Dependency:	AdcGroupTriggSrc AdcGroupConvMode AdcNonAutosarDmaResultHandling

4.5.15.2 AdcGroupConversionMode

Table 85 AdcGroupConversionMode

Name:	AdcGroupConversionMode		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_CONV_MODE_ONESHOT ADC_CONV_MODE_CONTINUOUS		
Default value:	ADC_CONV_MODE_ONESHOT		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Type of conversion mode supported by the Driver. ADC_CONV_MODE_CONTINUOUS is significant if AdcGroupTriggSrc is ADC_TRIGG_SRC_SW. ADC_CONV_MODE_CONTINUOUS is not supported, If parameter NonAutosarDmaResultHandling or NonAutosarResultPolling is set to true. Hence ADC_CONV_MODE_CONTINUOUS should not be configured for Non Autosar DMA result handling and Result polling modes. ADC_CONV_MODE_ONESHOT is not available for Group with EMUX SCAN Enabled.		
SFRs Modified:	VADC_GxQINR0 – Queue request source VADC_GxASMR – Auto Scan request source VADC_BRSMR – Background Scan request source Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcGroupTriggSrc AdcEmuxScanEnable		

4.5.15.3 AdcGroupId

Table 86 AdcGroupId

Name:	AdcGroupId		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_Cfg.h		
Range:	0 – 31 for HWUNIT_ADC0 32 – 63 for HWUNIT_ADC1 64 – 95 for HWUNIT_ADC2 96 – 127 for HWUNIT_ADC3 128 – 159 for HWUNIT_ADC4 160 – 191 for HWUNIT_ADC5 192 – 223 for HWUNIT_ADC6 224 – 255 for HWUNIT_ADC7 256 – 287 for HWUNIT_ADC8 288 – 319 for HWUNIT_ADC9 320 – 351 for HWUNIT_ADC10		
Default value:	0		
Configuration Class:	Pre-Compile	x	All Variant
	Link Time	--	--
	Post Build	--	--
Description:	Numeric ID of the ADC Channel Group. The symbolic name of this parameter is to be used in the API. The symbolic name of each ADC Group is a macro defined with the numeric ID. This numeric ID represents the array index of its configuration in ADC Group configuration set. This symbolic name allows accessing Group data. Driver implementation has limited the maximum number of groups to 32 in each VADC HW Unit. Numeric ID of the Group is generated by the configuration tool. Note: Since the Group ID is generated by the configuration tool, the groups in different configurations with same name should have the same group properties but different channels can be included in the groups in this case. AdcGroupId must be in sequence across the groups starting from zero		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcHwUnitId		

4.5.15.4 AdcGroupPriority

Table 87 AdcGroupPriority

Name:	AdcGroupPriority
Source:	This parameter is defined by the AUTOSAR.
File:	Adc_PBCfg.c
Range:	0 – 254
Default value:	0

Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Priority level of the group in its request source. This configuration parameter is valid in case of priority mechanism: ADC_PRIORITY_HW_SW. Since the priority level 255 is assigned to Adc channel group with HW trigger source, there can be maximum of 255 priority levels ranging from 0 to 254. This parameter is significant if AdcPriorityImplementation is ADC_PRIORITY_HW_SW and AdcGroupTriggSrc is ADC_TRIGG_SRC_SW In the ADC Driver priority mechanism the lowest priority is 0. <i>Note: Groups under the same request source will be arbitrated by the Driver and between the different request sources, the VADC peripheral will arbiter depending on the request source arbitration level.</i> <i>Note: If more Groups has same priority level and enters the HW_SW implemented Queue, the implementation will schedule the Groups in the order of its Group ID (Higher Group ID first). This will minimize the ISR overload time.</i>		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcPriorityImplementation AdcGroupTriggSrc		

4.5.15.5 AdcGroupReplacement

Table 88 AdcGroupReplacement

Name:	AdcGroupReplacement		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_GROUP_REPL_ABORT_RESTART		
Default value:	ADC_GROUP_REPL_ABORT_RESTART		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Replacement mechanism, which is used on ADC group level, if a group conversion is interrupted by a group which has a higher priority. In Abort/Restart mechanism, the complete conversion round of the last interrupted group (all group channels) is restarted after the higher priority group conversion is finished. This parameter is significant if AdcPriorityImplementation is ADC_PRIORITY_HW_SW and AdcGroupTriggSrc is ADC_TRIGG_SRC_SW.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcPriorityImplementation AdcHwUnitTriggSrc		

4.5.15.6 AdcGroupTriggSrc

Table 89 AdcGroupTriggSrc

Name:	AdcGroupTriggSrc		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_TRIGG_SRC_SW ADC_TRIGG_SRC_HW		
Default value:	ADC_TRIGG_SRC_SW		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Type of source event that starts a group conversion. Group conversion is started by SW API Call for Groups configured for ADC_TRIGG_SRC_SW Group conversion is started by HW event for Groups configured for ADC_TRIGG_SRC_HW ADC_TRIGG_SRC_HW is not available for Group with AdcGroupConvMode: ADC_CONV_MODE_CONTINUOUS		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGroupConvMode		

4.5.15.7 AdcHwTrigSignal

Table 90 AdcHwTrigSignal

Name:	AdcHwTrigSignal		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_HW_TRIG_RISING_EDGE ADC_HW_TRIG_FALLING_EDGE ADC_HW_TRIG_BOTH_EDGES		
Default value:	ADC_HW_TRIG_RISING_EDGE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configures on which edge of the external request signal the VADC peripheral should react, i.e. start the conversion. If AdcHwTrigType is HW_TRIGG_TIMER, the trigger signal from timer is always selected as raising edge. This parameter is significant if AdcHwTrigType is HW_TRIGG_EXT_REQ.		
SFRs Modified:	VADC_BRSCCTRL, VADC_GxASCTRL, VADC_GxQCTRL0 Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcHwTrigType		

4.5.15.8 AdcHwTrigTimer

Table 91 AdcHwTrigTimer

Name:	AdcHwTrigTimer		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	This parameter is not configurable since there is no embedded timer in VADC HW Unit Timer configuration is through Adc Gtm Timer		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is for Reload value of the ADC module embedded timer (only if supported by hardware),		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.5.15.9 AdcNotification

Table 92 AdcNotification

Name:	AdcNotification		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	Function pointer of type void IoHwAb_Adc_Notification_<GroupID> (void) (Selectable) / Address (Loadable)		
Default value:	NULL_PTR		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Call-back function for each group. The notification functions will have no parameters and no return value. Notification Function name: IoHwAb_Adc_Notification_<GroupID> It has to be implemented by the user. The notification functions are running in interrupt context. It's the responsibility of the user to keep the code of these functions reasonably short. For every group, notification call back has to be configured. This can be function pointer or a NULL Pointer. If for a notification function the NULL pointer is configured, no call-back will be executed. This parameter is significant if AdcGrpNotifCapability is set to true (STD_ON)		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGrpNotifCapability		

4.5.15.10 AdcStreamingBufferMode

Table 93 AdcStreamingBufferMode

Name:	AdcStreamingBufferMode		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	ADC_STREAM_BUFFER_LINEAR ADC_STREAM_BUFFER_CIRCULAR		
Default value:	ADC_STREAM_BUFFER_LINEAR		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configure streaming buffer as "linear buffer" (i.e. the ADC Driver stops the conversion as soon as the stream buffer is full) or as "ring buffer" (wraps around if the end of the stream buffer is reached). This parameter is significant if AdcNonAutosarDmaResultHandling is set to False (STD_OFF). If AdcGroupAccessMode is ADC_ACCESS_MODE_SINGLE, AdcStreamingBufferMode has to be ADC_STREAM_BUFFER_LINEAR for ADC Groups with ADC_TRIGG_SRC_SW, ADC_CONV_MODE_ONESHOT. If AdcGroupAccessMode is ADC_ACCESS_MODE_SINGLE, AdcStreamingBufferMode has to be ADC_STREAM_BUFFER_CIRCULAR for ADC Groups with ADC_TRIGG_SRC_SW, ADC_CONV_MODE_CONTINUOUS / ADC_TRIGG_SRC_HW.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGroupAccessMode AdcNonAutosarDmaResultHandling		

4.5.15.11 AdcStreamingNumSamples

Table 94 AdcStreamingNumSamples

Name:	AdcStreamingNumSamples		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	1-255		
Default value:	1		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	Number of ADC values to be acquired per channel in streaming access mode. In single access mode this parameter assumes value 1, since only one sample per channel is processed. This parameter is significant if <code>AdcNonAutosarDmaResultHandling</code> is set to <code>False (STD_OFF)</code>. If <code>AdcGroupAccessMode</code> is <code>ADC_ACCESS_MODE_SINGLE</code> then <code>AdcStreamingNumSamples</code> is always 1. If <code>AdcGroupAccessMode</code> is <code>ADC_ACCESS_MODE_STREAMING</code> then <code>AdcStreamingNumSamples</code> is greater than 1.
SFRs Modified:	None
Scope:	Driver
Dependency:	<code>AdcGroupAccessMode</code> <code>AdcNonAutosarDmaResultHandling</code>

4.5.15.12 AdcGroupDefinition

Table 95 AdcGroupDefinition

Name:	AdcGroupDefinition		
Source:	This parameter is defined by the AUTOSAR.		
File:	Adc_PBCfg.c		
Range:	Reference to parameter of type <code>AdcChannel</code>		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Assignment of channels to an <code>AdcGroup</code>. Array of channel ID with First element being the number of configured channels in the group. From second element its the channel ID. Channels under the same group can not share the same analog channel number. Group with EMUX SCAN enabled should have only one channel. All channels of a group share the same group configuration (channels can have different channel specific configurations) When <code>AdcNonAutosarDmaResultHandling = STD_ON</code>, the channels in the group have to be in the descending order of their analog channel numbers. Channels under the same group can not share the same analog channel number There can be only one reference to <code>AdcChannel</code> when Group's EMUX SCAN is enabled. If Group configured for <code>AdcGroupRequestSource: REQSRC0_8STG_QUE</code> or <code>REQSRC3_8STG_QUE</code>, then it can have only 8 channels in its <code>AdcGroupDefinition</code>. Maximum number of channels in a group is 8 for TC27x/TC29x and 16 for TC26x/TC23x/TC22x. If <code>AdcSameChannelQueuing</code> is enabled and <code>AdcGroupRequestSource</code> is <code>REQSRC0_8STG_QUE</code> or <code>REQSRC3_8STG_QUE</code>, same analog		

	channel can be queued to convert maximum of 4 times in a group, i.e, same analog channel can be added in AdcGroupDefinition for maximum of 4 times in a group. Result of the same channel is accumulated in result register/buffer as many times it is been queued. A channel should be queued equal number of times across the queue groups (REQSRC0_8STG_QUE and REQSRC3_8STG_QUE). Queing can be in any sequence; allows inserting other channels in between as well.
SFRs Modified:	VADC_GxREFCLR, VADC_GxCEFCLR, VADC_GxVFR, VADC_BRSEL, VADC_GxASSEL, VADC_GxQINR0 Where x is Adc kernel number
Scope:	Driver
Dependency:	AdcAnChannelNum AdcChannelEmuxSelect AdcEmuxScanEnable AdcNonAutosarDmaResultHandling

4.5.15.13 AdcGroupRequestSource

Table 96 AdcGroupRequestSource

Name:	AdcGroupRequestSource		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	REQSRC0_8STG_QUE REQSRC1_NCH_SCAN REQSRC2_BGND_SCAN REQSRC3_8STG_QUE		
Default value:	REQSRC0_8STG_QUE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Conversion request source of Aurix VADC peripheral. If Group configured for AdcGroupRequestSource: REQSRC0_8STG_QUE / REQSRC3_8STG_QUE, then it can have only up to 8 channels in its AdcGroupDefinition. If Group configured for AdcGroupRequestSource: REQSRC1_NCH_SCAN / REQSRC2_BGND_SCAN, then it can have up to maximum group channels i.e. up to 8 channels for TC27x/TC29x and 16 channels for TC26x/TC23x/TC22x in its AdcGroupDefinition. REQSRC3_8STG_QUE is applicable only for TC23x & TC22x.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGroupDefinition		

4.5.15.14 AdcHwTrigType

Table 97 AdcHwTrigType

Name:	AdcHwTrigType		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	HW_TRIGG_EXT_REQ HW_TRIGG_TIMER		
Default value:	HW_TRIGG_EXT_REQ		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configures on which hardware trigger signal is used for triggering the request source. This parameter is significant if AdcGroupTriggSrc is ADC_TRIGG_SRC_HW.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcGroupTriggSrc		

4.5.15.15 AdcHwExtTrigSelect

Table 98 AdcHwExtTrigSelect

Name:	AdcHwExtTrigSelect		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Refer Table 103		
Default value:	ADC0_TRSEL_RS0_RS1_IOUT0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	Configures the request pin for the triggering the request source by External trigger signal. Trigger Types: (a) Trigger using ERU signal In this mode, ADC uses ERU signal lines (IOUTy, y=0 to 7) for triggering the conversions. The ERU signals get initiated from either external port pin lines as per the configuration done in ERU Configuration under MCU. Example: (ADC0_TRSEL_RS0_RS1_IOUT0). Trigger signal of RS2 for all ADC kernel are same, hence it is named as ADCx_TRSEL_RS2_IOUTy (y = supported IOUT signals) (b) Trigger using CCU6 service request In this mode, ADC uses CCU6 service request signal lines for triggering the conversions. ADC driver does not provide any interface to enable CCU6 service request. User has to configure/write CCU6 to enable the service request. (c) Trigger using VADC service request In this mode, ADC uses its own service request signal lines for triggering the conversions. ADC driver uses the service request SR1 of Group Gx (x=0 to 7) to read the result of "N-channel Scan" request source conversion. Hence user has to use the SR1 of Group Gx trigger correctly to trigger other conversion. ADC driver does not provide any interface to enable C0/C1 SR1 service request. User has to configure/write VADC to enable these service requests. (d) Trigger using Gate pin In this mode, ADC uses selected Gate pin for triggering the conversions. (ADC_USE_GATE_PIN_TO_TRIG) This parameter is significant if AdcHwTrigType is HW_TRIGG_EXT_REQ		
SFRs Modified:	VADC_BRSCCTRL, VADC_GxASCTRL, VADC_GxQCTRL0 Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcHwTrigType AdcHwUnitId AdcGroupRequestSource		

4.5.15.16 AdcEruTriggRef

Table 99 AdcEruTriggRef

Name:	AdcEruTriggRef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to parameter of type ERU Unit		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	This is a reference parameter to ERU unit. The configuration of Eru_InputLine at the selected ERU Unit is considered as input to trigger ADC through ERU. It is preferred to select port pin as Eru_InputLine. These port pins needs to be configured for direction as Input. This parameter is significant if AdcHwTrigType is HW_TRIGG_EXT_REQ and AdcHwExtTrigSelect is the signal from ERU
SFRs Modified:	SCU_EICR, SCU_IGCR
Scope:	Driver
Dependency:	AdcHwTrigType AdcHwExtTrigSelect

4.5.15.17 AdcEruGateRef

Table 100 AdcEruGateRef

Name:	AdcEruGateRef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to parameter of type ERU Unit		
Default value:	None		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This is a reference parameter to ERU unit. The configuration of Eru_InputLine at the selected ERU Unit is considered as input to gating ADC through ERU. It is preferred to select port pin as Eru_InputLine. These port pins needs to be configured for direction as Input. For Eru gate, ERU number and OGU (Output Gating Unit) should match with the ADC kernel Number, i.e, if the Adc channel group is in ADC1, then ERU1 should be referenced here. Also OGU1 should be mapped to ERU1 in Eru configuration which is available in MCU. This parameter is significant if AdcHwTrigType is HW_TRIGG_EXT_REQ and AdcHwGatePin is the signal from ERU		
SFRs Modified:	SCU_EICR, SCU_IGCR		
Scope:	Driver		
Dependency:	AdcHwTrigType AdcHwGatePin		

4.5.15.18 AdcHwGateSignal

Table 101 AdcHwGateSignal

Name:	AdcHwGateSignal
Source:	This parameter is defined by IFX
File:	Adc_PBCfg.c

Range:	ADC_GATE_SIGNAL_HIGH ADC_GATE_SIGNAL_LOW ADC_GATE_SIGNAL_NONE		
Default value:	ADC_GATE_SIGNAL_NONE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is gating signal level for request source to be enabled. If ADC_GATE_SIGNAL_HIGH is selected, the corresponding AdcHwGatePin needs to be at high level for the request source to issue conversion. If ADC_GATE_SIGNAL_LOW is selected, the corresponding AdcHwGatePin needs to be at low level for the request source to issue conversion. If ADC_GATE_SIGNAL_NONE is selected, the request source issues conversion if a valid conversion request is pending irrespective of gate pin level. This parameter is significant if AdcGroupTriggSrc is ADC_TRIGG_SRC_HW. The parameter is not editable if AdcHwExtTrigSelect is ADC_USE_GATE_PIN_TO_TRIG. The parameter is not editable if AdcHwTrigType is HW_TRIGG_TIMER.		
SFRs Modified:	VADC_BRSMR, VADC_GxASMR, VADC_GxQMR0 Where x is Adc kernel number		
Scope:	Driver		
Dependency:	AdcGroupTriggSrc		

4.5.15.19 AdcHwGatePin

Table 102 AdcHwGatePin

Name:	AdcHwGatePin		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Refer Table 103		
Default value:	ADC0_GTSEL_RS0_RS1_PDOUT0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	This parameter configures the gating request pin for the triggering the request source. Gate Types: (a) Gate using ERU signal In this mode, ADC uses ERU signal lines (PDOUTy, y=0 to 7) for gating the conversions. The ERU signals get initiated from either external port pin lines as per the configuration done in ERU Configuration under MCU. Example: (ADC0_GTSEL_RS0_RS1_PDOUT0). (b) Gate using GTM signal In this mode, ADC uses GTM signal lines (ADCy_RS0_RS_TIM_TRIG0/1, y=0 to 7 & ADCx_RS2_TIM_TRIG0/1) for gating the conversions. The GTM signals are applicable only if AdcHwTrigType is HW_TRIGG_TIMER and AdcHwExtTrigSelect is ADC_USE_GATE_PIN_TO_TRIG. (c) Gate using CCU6 signal In this mode, ADC uses CCU6 signal lines for gating the conversions. ADC driver does not provide any interface to enable CCU6 gating lines. User has to configure/write CCU6 to enable the gating lines. Gate signal of RS2 for all ADC kernel are same, hence it is named as ADCx. E.g.: ADCx_GTSEL_RS2_TIM_TRIGy (y = 0, 1) This parameter is significant if AdcGroupTriggSrc is ADC_TRIGG_SRC_HW, AdcHwGateSignal is not ADC_GATE_SIGNAL_NONE and AdcHwExtTrigSelect is ADC_USE_GATE_PIN_TO_TRIG.
SFRs Modified:	VADC_BRSCCTRL, VADC_GxASCTRL, VADC_GxQCTRL0 Where x is Adc kernel number
Scope:	Driver
Dependency:	AdcGroupTriggSrc AdcHwGateSignal AdcHwUnitId AdcGroupRequestSource AdcHwExtTrigSelect

Table 103 Digital Connections of Aurix

Derivative	Feature	Availability
TC23x / TC22x	Hardware Unit	2 (0 to 1)
	External Trigger Signals	ADCy_TRSEL_RS0_RS1_RS#_IOUTy (y = 0 - 1)
		ADCx_TRSEL_RS2_IOUT3
		ADCx_TRSEL_RS2_IOUT4
		ADCx_TRSEL_RS2_IOUT6
		ADCx_TRSEL_RS2_IOUT7
		ADCx_TRSEL_RS0_RS1_RS2_RS3_CCU60_SR3
		ADCx_TRSEL_RS0_RS1_RS2_RS3_CCU61_SR3
		ADCy_TRSEL_RS0_RS1_RS3_Gy_SR1 (y = 0 - 1)
		ADCx_TRSEL_RS0_RS1_RS2_RS3_C0_SR1
		ADCx_TRSEL_RS0_RS1_RS2_RS3_C1_SR1

	Gate Signals	ADC_USE_INTERNAL_TRIG_TO_TRIG_RS3
		ADC_USE_GATE_PIN_TO_TRIG
		ADCy_GTSEL_RS0_RS1_RS3_PDOUTy (y = 0 - 1)
		ADCy_GTSEL_RS0_RS1_RS3_TIM_TRIG0 (y = 0 - 1)
		ADCy_GTSEL_RS0_RS1_RS3_TIM_TRIG1 (y = 0 - 1)
		ADCx_GTSEL_RS2_TIM_TRIG0
		ADCx_GTSEL_RS2_TIM_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_RS3_CCU6061_TRIG0
		ADCx_GTSEL_RS0_RS1_RS2_RS3_CCU6061_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_RS3_CCU6061_TRIG2
TC26x	Hardware Unit	4 (0 to 3)
	External Trigger Signals	ADCy_TRSEL_RS0_RS1_IOUTy (y = 0 - 3)
		ADCx_TRSEL_RS2_IOUT3
		ADCx_TRSEL_RS2_IOUT4
		ADCx_TRSEL_RS2_IOUT6
		ADCx_TRSEL_RS2_IOUT7
		ADCx_TRSEL_RS0_RS1_RS2_CCU60_SR3
		ADCx_TRSEL_RS0_RS1_RS2_CCU61_SR3
		ADCy_TRSEL_RS0_RS1_Gy_SR1 (y = 0 - 3)
		ADCx_TRSEL_RS0_RS1_RS2_C0_SR1
		ADCx_TRSEL_RS0_RS1_RS2_C1_SR1
	Gate Signals	ADC_USE_GATE_PIN_TO_TRIG
		ADCy_GTSEL_RS0_RS1_PDOUTy (y = 0 - 3)
		ADCy_GTSEL_RS0_RS1_TIM_TRIG0 (y = 0 - 3)
		ADCy_GTSEL_RS0_RS1_TIM_TRIG1 (y = 0 - 3)
		ADCx_GTSEL_RS2_TIM_TRIG0
		ADCx_GTSEL_RS2_TIM_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG0
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG2
		ADCx_GTSEL_RS2_TIM_TRIG0
		ADCx_GTSEL_RS2_TIM_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG0
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG2
TC27x	Hardware Unit	8 (0 to 7)
	External Trigger Signals	ADCy_TRSEL_RS0_RS1_IOUTy (y = 0 - 7)
		ADCx_TRSEL_RS2_IOUT3
		ADCx_TRSEL_RS2_IOUT4
		ADCx_TRSEL_RS2_IOUT6
		ADCx_TRSEL_RS2_IOUT7
		ADCx_TRSEL_RS0_RS1_RS2_CCU60_SR3
		ADCx_TRSEL_RS0_RS1_RS2_CCU61_SR3
		ADCy_TRSEL_RS0_RS1_Gy_SR1 (y = 0 - 7)

	Gate Signals	ADCx_TRSEL_RS0_RS1_RS2_C0_SR1
		ADCx_TRSEL_RS0_RS1_RS2_C1_SR1
		ADC_USE_GATE_PIN_TO_TRIG
		ADCy_GTSEL_RS0_RS1_PDOUTy (y = 0 - 7)
		ADCy_GTSEL_RS0_RS1_TIM_TRIG0 (y = 0 - 7)
		ADCy_GTSEL_RS0_RS1_TIM_TRIG1 (y = 0 - 7)
		ADCx_GTSEL_RS2_TIM_TRIG0
		ADCx_GTSEL_RS2_TIM_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG0
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG1
TC29x	Hardware Unit	11 (0 to 10)
	External Trigger Signals	ADCy_TRSEL_RS0_RS1_IOUTz (y = 0 - 10, z = 0 - 7)
		ADC8_TRSEL_RS0_RS1_IOUT0
		ADC9_TRSEL_RS0_RS1_IOUT1
		ADC10_TRSEL_RS0_RS1_IOUT2
		ADCx_TRSEL_RS2_IOUT3
		ADCx_TRSEL_RS2_IOUT4
		ADCx_TRSEL_RS2_IOUT6
		ADCx_TRSEL_RS2_IOUT7
		ADCx_TRSEL_RS0_RS1_RS2_CCU60_SR3
		ADCx_TRSEL_RS0_RS1_RS2_CCU61_SR3
		ADCy_TRSEL_RS0_RS1_Gy_SR1 (y = 0 - 10)
		ADCx_TRSEL_RS0_RS1_RS2_C0_SR1
		ADCx_TRSEL_RS0_RS1_RS2_C1_SR1
		ADC_USE_GATE_PIN_TO_TRIG
	Gate Signals	ADCy_GTSEL_RS0_RS1_PDOUTz (y = 0 - 10, z = 0 - 7)
		ADC8_GTSEL_RS0_RS1_PDOUT0
		ADC9_GTSEL_RS0_RS1_PDOUT1
		ADC10_GTSEL_RS0_RS1_PDOUT2
		ADCy_GTSEL_RS0_RS1_TIM_TRIG0 (y = 0 - 10)
		ADCy_GTSEL_RS0_RS1_TIM_TRIG1 (y = 0 - 10)
		ADCx_GTSEL_RS2_TIM_TRIG0
		ADCx_GTSEL_RS2_TIM_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG0
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG1
		ADCx_GTSEL_RS0_RS1_RS2_CCU6061_TRIG2

4.5.15.20 AdcGtmTimerPeriod

Table 104 AdcGtmTimerPeriod

Name:	AdcGtmTimerPeriod
Source:	This parameter is defined by IFX
File:	Adc_PBCfg.c

Range:	Period in millisecond (Float value)		
Default value:	5		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter refers to the time period for the VADC Analog signal to be converted periodically. The value entered is taken in millisecond to configure the microcontroller GTM peripheral. The time value entered is converted in to 16/24-bit reload value. This reload value should be of maximum value 0xFFFF/0xFFFFFFFF based on the selected Timer channel TOM/ATOM. On every rising edge of the timer signal, the conversion is triggered. The timer signal is of period as entered in this field. This parameter is significant if AdcHwTrigType is HW_TRIGG_TIMER. ATOM module is not available for TC23x & TC22x.		
SFRs Modified:	GTM_(A)TOMx_CHy_CN0, GTM_(A)TOMx_CHy_CM0, GTM_(A)TOMx_CHy_CM1, GTM_(A)TOMx_CHy_SR0, GTM_(A)TOMx_CHy_SR1 Where x is (A)TOM module number and y is Channel number		
Scope:	Driver		
Dependency:	AdcHwTrigType		

4.5.15.21 AdcGtmTriggerRef

Table 105 AdcGtmTriggerRef

Name:	AdcGtmTriggerRef		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	Reference to the type of Gtm		
Default value:	--		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	This parameter gives the reference of the TOM/ATOM channel which is configured to trigger the Adc group. This parameter is significant only if AdcHwTrigType is HW_TRIGG_TIMER. Following should be followed for the reference. For Background Source (RS2): GtmTriggerForAdc_2 for TC23x, TC22x and TC21x GtmTriggerForAdc_4 for TC26x GtmTriggerForAdc_8 for TC27x GtmTriggerForAdc_11 for TC29x For all other request sources (RS0, RS1 & RS3): GtmTriggerForAdc_x, where x is ADC Hardware Unit number. Eg. For ADC0, GtmTriggerForAdc_0 should be selected. Name of the referenced parameter "GtmTriggerForAdc_x" should not be modified for the correct generation of ADC configuration.
SFRs Modified:	None
Scope:	Driver
Dependency:	AdcHwTrigType AdcHwUnitId AdcGroupRequestSource

4.5.15.22 AdcEmuxScanEnable

Table 106 AdcEmuxScanEnable

Name:	AdcEmuxScanEnable		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	True: Group EMUX scan mode enabled False: Group EMUX scan mode disabled		
Default value:	False		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is to enable or disable the Group EMUX SCAN mode. This will facilitate the Group with many virtual channels starting from configured AdcChannelEmuxSelect value till 0 connected to the multiplexer. <i>Note: This Group channel's AdcChannelEmuxSelect should be other than 0.</i> For example: Consider AdcChannelEmuxSelect = 3 for analog channel 3 in a group with AdcEmuxScanEnable = TRUE. i.e. this group on conversion will scan 4 channels connected to		

	multiplexer of analog channel 3 starting from select line 3 to 0. This parameter is significant if AdcNonAutosarDmaResultHandling is set to False [STD_OFF]. Also it is significant if AdcUseEmux is ON and AdcGroupConvMode is ADC_CONV_MODE_CONTINUOUS for SW trigger groups. Significant if AdcEmuxSelectGrp0/1 selected for required kernel. Group with EMUX scan mode enabled should have only one channel in a Group. This parameter should be set to false for the group with the limit check enabled channel.
SFRs Modified:	VADC_GxEMUXCTR Where x is Adc kernel number
Scope:	Driver
Dependency:	AdcUseEmux AdcGroupConvMode AdcHwUnitId AdcGroupDefinition AdcNonAutosarDmaResultHandling

4.5.15.23 AdcChannelEmuxSelect

Table 107 AdcChannelEmuxSelect

Name:	AdcChannelEmuxSelect		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0 - 7		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	This parameter is the configuration of channel select line, if external multiplexer (1-out-of-8) is used. This parameter is being used in 2 modes in 2 different ways. Emux Scan Mode : This parameter says the upper count from which it scans down to 0 Eg: 5 => 5 down to 0 Emux Mode: This parameter is used to select the External-channel connected to the inputs of multiplexer. Eg: 5 => select lines are fixed as 101 Futher: if no value is selected: Scan Mode: one chanel is converted/scanned. Emux Mode:ExtCh0 is chosen with select lines being 0,0,0 This parameter is significant only if AdcUseEmux is set to true (STD_ON).		
SFRs Modified:	VADC_GxEMUXCTR Where x is Adc kernel number		

Scope:	Driver
Dependency:	AdcUseEmux

4.5.15.24 AdcGroupBufferMarker

Table 108 Specification of AdcGroupBufferMarker

Name:	AdcGroupBufferMarker		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0 – 255		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The parameter defines the marker value of the Buffer per Adc channel group. The marker is used for validating the correct buffer pointer. This parameter is significant only if AdcSafetyEnable is set to True. Value of this parameter should be uint8 irrespective of the Type, Adc_ValueGroupType. User must write 0x00 for the upper 8bit, if the Adc_ValueGroupType is of Type uint16. E.g., 0x00F0. Buffer Marker needs to be configured even for a QM group, if Safety is enabled.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	AdcSafetyEnable		

4.5.15.25 AdcRS3InternalTrigger

Table 109 Specification of AdcRS3InternalTrigger

Name:	AdcRS3InternalTrigger		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	REQSRC0_8STG_QUE REQSRC1_NCH_SCAN REQSRC2_BGND_SCAN		
Default value:	REQSRC0_8STG_QUE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Define the Internal triggers generated by the respective source events. This parameter is available only for TC23x & TC22x.		
SFRs Modified:	GxTRCTR		
Scope:	Driver		

Dependency:	AdcGroupRequestSource
-------------	-----------------------

4.5.15.26 AdcRS3TriggerSeqCount

Table 110 Specification of AdcRS3TriggerSeqCount

Name:	AdcRS3TriggerSeqCount		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0x0 - 0x3F		
Default value:	0x0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Defines the initial value of the trigger sequence counter TSC and Controls the effect of an incoming internal trigger. TSC is reloaded with the value in TSCSET, when a trigger occurs while TSC = 00H. If the parameter value is 00H, Issue conversion requests immediately. Otherwise, decrement trigger counter. This parameter is available only for TC23x & TC22x.		
SFRs Modified:	GxTRCTR		
Scope:	Driver		
Dependency:	None		

4.5.16 Container: AdcKernelInputClass

This is a microcontroller specific container. Each ADC hardware units have 4 container of this type. Channel resolution and Channel sampling time are the parameters to be configured in this container for the configured ADC hardware unit / kernel.

4.5.16.1 AdcKernelChSampleTime

Table 111 AdcKernelChSampleTime

Name:	AdcKernelChSampleTime		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	0 – 31		
Default value:	0		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB

Description:	The time during which the value is sampled, (in clock-cycles). AURIX provides the duration of the sample phase is programmed individually for each channel via sample time control bit field (STCS). A minimum sample phase of 2 analog clock cycles is extended by the programmed value. sample phase length = $(2 + \text{STC}) / f_{\text{ADCI}}$ f_{ADCI}: Analog clock frequency This is a channel sampling time for the kernel.
SFRs Modified:	VADC_GxICLASS0/1 Where x is Adc kernel number
Scope:	Driver
Dependency:	AdcAnalogClockDivider

4.5.16.2 AdcKernelChResolution

Table 112 AdcKernelChResolution

Name:	AdcKernelChResolution		
Source:	This parameter is defined by IFX		
File:	Adc_PBCfg.c		
Range:	CH_RES_8BIT CH_RES_10BIT CH_RES_12BIT		
Default value:	CH_RES_10BIT		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	ADC Channel resolution in bits. This is a channel resolution for the kernel. CH_RES_12BIT should be selected for the AdcKernelInputClass_0 container, for the errata ADC_TC.62 workaround. This is applicable only for TC27xBC step device.		
SFRs Modified:	VADC_GxICLASS0/1 Where x is Adc kernel number		
Scope:	Driver		
Dependency:	None		

4.5.16.3 AdcKernelEmuxSampleTime

Table 113 AdcKernelEmuxSampleTime

Name:	AdcKernelEmuxSampleTime
Source:	This parameter is defined by IFX
File:	Adc_PBCfg.c
Range:	0- 31
Default value:	0

Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Configuration of sampling time (in clock cycles) for Emux channel, i.e. the time during which the value is sampled. AURIX provides the duration of the sample phase is programmed for Emux channel via sample time control bit field (STCE). A minimum sample phase of 2 analog clock cycles is extended by the programmed value. sample phase length = $(2 + \text{STC}) / f_{\text{ADCI}}$ f_{ADCI}: Analog clock frequency This parameter is a Emux channel sampling time for the kernel.		
SFRs Modified:	VADC_GxICLASS0/1 Where x is Adc kernel number		
Scope:	Driver		
Dependency:	Editable only if AdcUseEmux is set to STD_ON		

4.6 Container: AdcPublishedInformation

This container contains the additional module specific published information. These parameters are non editable.

4.6.1 AdcChannelValueSigned

Table 114 AdcChannelValueSigned

Name:	AdcChannelValueSigned
Source:	This parameter is defined by the AUTOSAR.
File:	Adc_Cfg.h
Range:	True – result is interpreted as signed value False – result is not interpreted as signed value
Default value:	False
Description:	Information whether the result value of the ADC driver has sign information (true) or not (false). If the result is interpreted as signed value it shall apply to C-language rules.
Scope:	Driver
Dependency:	None

4.6.2 AdcGroupFirstChannelFixed

Table 115 AdcGroupFirstChannelFixed

Name:	AdcGroupFirstChannelFixed
Source:	This parameter is defined by the AUTOSAR.
File:	Adc_Cfg.h
Range:	True False
Default value:	False

Name:	AdcGroupFirstChannelFixed
Description:	Information whether the first channel of an ADC Channel group can be configured (false) or is fixed (true) to a value determined by the ADC HW Unit.
Scope:	Driver
Dependency:	None

4.6.3 AdcMaxChannelResolution

Table 116 AdcMaxChannelResolution

Name:	AdcMaxChannelResolution
Source:	This parameter is defined by the AUTOSAR.
File:	Adc_Cfg.h
Range:	12
Default value:	12
Description:	Maximum channel resolution in bits.
Scope:	Driver
Dependency:	None

4.7 Container: CommonPublishedInformation

This container contains the common published information. These parameters are non editable.

4.7.1 ArMajorVersion

Table 117 ArMajorVersion

Name:	ArMajorVersion
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	0 - 255
Default value:	As per the supported Autosar version
Description:	Major version number of AUTOSAR specification on which the appropriate implementation is based on.
Scope:	Driver
Dependency:	None

4.7.2 ArMinorVersion

Table 118 ArMinorVersion

Name:	ArMinorVersion
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	0 - 255
Default value:	As per the supported Autosar version

Name:	ArMinorVersion
Description:	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.
Scope:	Driver
Dependency:	None

4.7.3 ArPatchVersion

Table 119 ArPatchVersion

Name:	ArPatchVersion
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	0 - 255
Default value:	As per the supported Autosar version
Description:	Patch version number of AUTOSAR specification on which the appropriate implementation is based on.
Scope:	Driver
Dependency:	None

4.7.4 SwMajorVersion

Table 120 SwMajorVersion

Name:	SwMajorVersion
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	0 - 255
Default value:	Refer Adc_Cfg.h
Description:	Major version number of the vendor specific implementation of the Driver.
Scope:	Driver
Dependency:	None

4.7.5 SwMinorVersion

Table 121 SwMinorVersion

Name:	SwMinorVersion
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	0 - 255
Default value:	Refer Adc_Cfg.h
Description:	Minor version number of the vendor specific implementation of the Driver.
Scope:	Driver

Name:	SwMinorVersion
Dependency:	None

4.7.6 SwPatchVersion

Table 122 SwPatchVersion

Name:	SwPatchVersion
Source:	This parameter is defined by IFX
File:	Adc_Cfg.h
Range:	0 - 255
Default value:	Refer Adc_Cfg.h
Description:	Patch version number of the vendor specific implementation of the Driver.
Scope:	Driver
Dependency:	None

4.7.7 ModuleId

Table 123 ModuleId

Name:	ModuleId
Source:	This parameter is defined by IFX
File:	Adc.h
Default value:	123
Description:	ADC Driver module ID as described by AUTOSAR.
Scope:	Driver
Dependency:	None

4.7.8 VendorId

Table 124 VendorId

Name:	VendorId
Source:	This parameter is defined by IFX
File:	Adc.h
Default value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list. Vendor is Infineon Technologies.
Scope:	Driver
Dependency:	None

4.7.9 VendorApilInfix

Table 125 VendorApilInfix

Name:	VendorApilInfix
Source:	This parameter is defined by IFX

Name:	VendorApiInfix
File:	Adc.xdm
Default value:	None
Description:	Vendor Specific Name used for extending the API names along with VendorId.
Scope:	Not Applicable for ADC driver
Dependency:	None

4.7.10 Release

Table 126 Release

Name:	Release
Source:	This parameter is defined by IFX
File:	Adc.xdm
Range:	_TRICORE_TC299 _TRICORE_TC298 _TRICORE_TC277 _TRICORE_TC275 _TRICORE_TC267 _TRICORE_TC265 _TRICORE_TC264 _TRICORE_TC233 _TRICORE_TC234 _TRICORE_TC237
Description:	Aurix derivative used for implementation.
Scope:	Driver
Dependency:	None

4.8 Derived Configuration Parameters

These parameters represent the additional information required by the Driver and is automatically calculated by the tool.

*Note: These derived parameters are **precompile** parameters and hence the parameters in the “Dependency” field should not be modified after the build process.*

4.8.1 ADC_MAX_GROUPS

Table 127 ADC_MAX_GROUPS

Name:	ADC_MAX_GROUPS
Type:	#define
File:	Adc_Cfg.h
Description:	It is the maximum among the total number of groups configured in each Adc configuration. For example if there are two post build configurations and 2 HW Units configured among each, with maximum 2, 3, 4 and 5 groups respectively in each HW Units, then ADC_MAX_GROUPS must be

	configured as 5.
Range	0 - 32
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcEnableQueuing

4.8.2 ADCx_QM_KERNEL_INDEX

Table 128 ADCx_QM_KERNEL_INDEX

Name:	ADCx_QM_KERNEL_INDEX
Type:	#define
File:	Adc_Cfg.h
Description:	It is the variable array index of each Adc Hw Unit for the QM Signal. For example if there are two ADC Hw units used, say ADC0 and ADC2, ADC0_QM_KERNEL_INDEX = 0, ADC2_QM_KERNEL_INDEX = 1, ADC1_QM_KERNEL_INDEX = Invalid (0), ADC3_QM_KERNEL_INDEX = Invalid (0)
Range	0 - 10
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwUnitId

4.8.3 ADCx_ASIL0_KERNEL_INDEX

Table 129 ADCx_ASIL0_KERNEL_INDEX

Name:	ADCx_ASIL0_KERNEL_INDEX
Type:	#define
File:	Adc_Cfg.h
Description:	It is the variable array index of each Adc Hw Unit for the ASIL Signal - 0. For example if there are two ADC Hw units used, say ADC0 and ADC2, ADC0_ASIL0_KERNEL_INDEX = 0, ADC2_ASIL0_KERNEL_INDEX = 1, ADC1_ASIL0_KERNEL_INDEX = Invalid (0), ADC3_ASIL0_KERNEL_INDEX = Invalid (0)
Range	0 - 3
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwUnitId

4.8.4 ADCx_ASIL1_KERNEL_INDEX

Table 130 ADCx_ASIL1_KERNEL_INDEX

Name:	ADCx_ASIL1_KERNEL_INDEX
Type:	#define
File:	Adc_Cfg.h
Description:	It is the variable array index of each Adc Hw Unit for the ASIL Signal - 1. For example if there are two ADC Hw units used, say ADC4 and ADC6, ADC4_ASIL1_KERNEL_INDEX = 0, ADC6_ASIL1_KERNEL_INDEX = 1, ADC5_ASIL1_KERNEL_INDEX = Invalid (0), ADC7_ASIL1_KERNEL_INDEX = Invalid (0)
Range	4 - 10
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwUnitId

4.8.5 ADC_VALUE_GROUP_TYPE

Table 131 ADC_VALUE_GROUP_TYPE

Name:	ADC_VALUE_GROUP_TYPE
Type:	#define
File:	Adc_Cfg.h
Description:	Determines the Adc_ValueGroupType to be 8-bit or 16-bit If all the configured ADC channels are of 8 bit resolution, 8-bit is selected for Adc_ValueGroupType. Else, 16-bit is selected. If all the ADC channel's AdcChannelResolution are of 8 bit resolution, ADC_VALUE_GROUP_TYPE is set ADC_VALUE_EIGHT_BIT. Else, ADC_VALUE_GROUP_TYPE is set ADC_VALUE_SIXTEEN_BIT.
Range	ADC_VALUE_EIGHT_BIT ADC_VALUE_SIXTEEN_BIT
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcChannelResolution

4.8.6 ADC_KERNEL_USED_COUNT

Table 132 ADC_KERNEL_USED_COUNT

Name:	ADC_KERNEL_USED_COUNT
Type:	#define

File:	Adc_Cfg.h
Description:	It is the maximum number of ADC HW Units used count value from all the ADC configuration sets.
Range	1 to 8
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.7 ADC_QM_KERNEL_USED_COUNT

Table 133 ADC_QM_KERNEL_USED_COUNT

Name:	ADC_QM_KERNEL_USED_COUNT
Type:	#define
File:	Adc_Cfg.h
Description:	It is the maximum number of ADC HW Units used count value from all the ADC configuration sets for the QM signals
Range	1 to 11
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.8 ADC_ASIL0_KERNEL_USED_COUNT

Table 134 ADC_ASIL0_KERNEL_USED_COUNT

Name:	ADC_ASIL0_KERNEL_USED_COUNT
Type:	#define
File:	Adc_Cfg.h
Description:	It is the maximum number of ADC HW Units of ADC Cluster-0 used count value from all the ADC configuration sets for the ASIL signals
Range	1 to 4
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.9 ADC_ASIL1_KERNEL_USED_COUNT

Table 135 ADC_ASIL1_KERNEL_USED_COUNT

Name:	ADC_ASIL1_KERNEL_USED_COUNT
Type:	#define
File:	Adc_Cfg.h
Description:	It is the maximum number of ADC HW Units of ADC Cluster-1 and ADC Cluster-2 used count value from all the ADC configuration sets for the ASIL signals

Range	1 to 7
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.10 ADC_USED_KERNELS

Table 136 ADC_USED_KERNELS

Name:	ADC_USED_KERNELS
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcHwUnitId, the corresponding macro gets defined.
Range	Bit value = 0 implies the Adc hardware uint is not configured. Bit value = 1 implies the Adc hardware uint is configured. Bit 0 for Adc Hw unit 0, bit 1 for Adc Hw Unit 1 ... Bit 7 for Adc Hw Unit 7.
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwUnitId

4.8.11 ADCx_KERNEL_USED

Table 137 ADCx_KERNEL_USED

Name:	ADCx_KERNEL_USED
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcHwUnitId, the corresponding macro gets defined.
Range	This parameter is either gets defined or not
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwUnitId

4.8.12 ADC_KERNEL_REQSRC0_USED

Table 138 ADC_KERNEL_REQSRC0_USED

Name:	ADC_KERNEL_REQSRC0_USED
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcGroupRequestSource, the corresponding macro gets defined.
Range	Bit value = 0 implies the RS0 is not configured in the Adc Hw Unit.

	Bit value = 1 implies the RS0 is configured in the Adc Hw Unit. Bit 0 for Adc Hw unit 0, bit 1 for Adc Hw Unit 1 ... Bit 7 for Adc Hw Unit 7.
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcGroupRequestSource

4.8.13 ADC_KERNEL_REQSRC1_USED

Table 139 ADC_KERNEL_REQSRC1_USED

Name:	ADC_KERNEL_REQSRC1_USED
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcGroupRequestSource, the corresponding macro gets defined.
Range	Bit value = 0 implies the RS1 is not configured in the Adc Hw Unit. Bit value = 1 implies the RS1 is configured in the Adc Hw Unit. Bit 0 for Adc Hw unit 0, bit 1 for Adc Hw Unit 1 ... Bit 7 for Adc Hw Unit 7.
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcGroupRequestSource

4.8.14 ADC_KERNEL_REQSRC2_USED

Table 140 ADC_KERNEL_REQSRC2_USED

Name:	ADC_KERNEL_REQSRC2_USED
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcGroupRequestSource, the corresponding macro gets defined.
Range:	Bit value = 0 implies the RS2 is not configured in the Adc Hw Unit. Bit value = 1 implies the RS2 is configured in the Adc Hw Unit. Bit 0 for Adc Hw unit 0, bit 1 for Adc Hw Unit 1 ... Bit 7 for Adc Hw Unit 7.
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcGroupRequestSource

4.8.15 ADCx_REQSRCy

Table 141 ADCx_REQSRCy

Name:	ADCx_REQSRCy
--------------	--------------

	Where x : 0 to 10 y : 0 to 3
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcHwUnitId request source, the corresponding macro gets defined as either used or unused.
Range	ADC_REQSRC_USED ADC_REQSRC_UNUSED
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwUnitId AdcGroupRequestSource

4.8.16 ADC_REQSRCy

Table 142 ADC_REQSRCy

Name:	ADC_REQSRCy Where y : 0 to 3
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of request source of any ADC HW Unit, the corresponding macro gets defined as either used or unused.
Range	ADC_REQSRC_USED ADC_REQSRC_UNUSED
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcGroupRequestSource

4.8.17 ADC_WAIT_ON_STOP

Table 143 ADC_WAIT_ON_STOP

Name:	ADC_WAIT_ON_STOP
Type:	#define
File:	Adc_Cfg.h
Description:	Macro to switch STD_ON/STD_OFF the wait for HW unit Idle state in the driver on Stop / Disable request If the configured ADC_MAX_CH_CON_TIME_COUNT is less than the minimum value, ADC_WAIT_ON_STOP is set STD_OFF. In this case, user has to ensure that new start of a Group is called only when ADC HW Unit is Idle. Switching off of driver to wait on stop is applicable in ADC_PRIORITY_NONE (Queue is OFF) and ADC_PRIORITY_HW.
Range	STD_ON STD_OFF

Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	ADC_MAX_CH_CON_TIME_COUNT AdcPriorityImplementation

4.8.18 ADC_GROUP_EMUX_SCAN

Table 144 ADC_GROUP_EMUX_SCAN

Name:	ADC_GROUP_EMUX_SCAN
Type:	#define
File:	Adc_Cfg.h
Description:	Macro which tells if any of the Group uses EMUX scan feature or not.
Range	STD_ON STD_OFF
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcEmuxScanEnable

4.8.19 ADC_LIMIT_CHECK_USED

Table 145 ADC_LIMIT_CHECK_USED

Name:	ADC_LIMIT_CHECK_USED
Type:	#define
File:	Adc_Cfg.h
Description:	Depending on the configuration of AdcChannelLimitCheck, the corresponding macro gets defined.
Range	Bit value = 0 implies the Limit checking is not configured in the Adc Hw Unit. Bit value = 1 implies the Limit checking is configured in the Adc Hw Unit. Bit 0 for Adc Hw unit 0, bit 1 for Adc Hw Unit 1 ... Bit 7 for Adc Hw Unit 7.
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcChannelLimitCheck

4.8.20 ADC_DMA_MAX_TRANSFER

Table 146 ADC_DMA_MAX_TRANSFER

Name:	ADC_DMA_MAX_TRANSFER
Type:	#define
File:	Adc_Cfg.h

Description:	Macro which tells the maximum DMA transfer size from EMUX Scan Groups. value = MAX((Group channel Emux select value + 1) * Group streaming number of samples) The value is generated only when ADC_GROUP_EMUX_SCAN = STD_ON
Range	2 - 2048
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	ADC_GROUP_EMUX_SCAN

4.8.21 ADC_MAX_KERNELS

Table 147 ADC_MAX_KERNELS

Name:	ADC_MAX_KERNELS
Type:	#define
File:	Adc_Cfg.h
Description:	Macro which gives the maximum number of ADC kernels in a microcontroller
Range	11
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.22 ADC_MAX_SHS_CLUSTERS

Table 148 ADC_MAX_SHS_CLUSTERS

Name:	ADC_MAX_SHS_CLUSTERS
Type:	#define
File:	Adc_Cfg.h
Description:	Macro which gives the maximum number of SHS Clusters in a microcontroller
Range	3
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.23 ADC_KERNELS_PER_CLUSTER

Table 149 ADC_KERNELS_PER_CLUSTER

Name:	ADC_KERNELS_PER_CLUSTER
Type:	#define
File:	Adc_Cfg.h

Description:	Macro which gives the number of SHS Units per SHS Clusters in a microcontroller
Range	2 for TC26x 4 for TC27x / TC29x
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.24 ADC_ALTERNATE_REF_CH_USED

Table 150 ADC_ALTERNATE_REF_CH_USED

Name:	ADC_ALTERNATE_REF_CH_USED
Type:	#define
File:	Adc_Cfg.h
Description:	Macro which gives that alternate reference used for any of the channel or not in the configuration
Range	STD_ON STD_OFF
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

4.8.25 ADC_RESULT_HANDLING_MODE

Table 151 ADC_RESULT_HANDLING_MODE

Name:	ADC_RESULT_HANDLING_MODE
Type:	#define
File:	Adc_Cfg.h
Description:	Macro to decide the Adc group result handling to be Autosar mode or Non Autosar mode (DMA is configured by user)
Range	ADC_AUTOSAR ADC_NON_AUTOSAR
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcNonAutosarDmaResultHandling

4.8.26 ADC_RESULT_ACCUMULATION

Table 152 ADC_RESULT_ACCUMULATION

Name:	ADC_RESULT_ACCUMULATION
Type:	#define
File:	Adc_Cfg.h

Description:	Macro which gives the configured value for the result accumulation in the result register
Range	ADC_RES_ACCUMULATION_NONE ADC_RES_ACCUMULATION_TWO ADC_RES_ACCUMULATION_THREE ADC_RES_ACCUMULATION_FOUR
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcNonAutosarDmaResultHandling AdcResultAccumulation

4.8.27 ADC_SWOSM_INTERRUPT_HANDLER

Table 153 ADC_SWOSM_INTERRUPT_HANDLER

Name:	ADC_SWOSM_INTERRUPT_HANDLER
Type:	#define
File:	Adc_Cfg.h
Description:	Macro which switches OSM group ISR handling at compile time. If there are no OSM-SW triggered groups, then the value of this macro is [STD_OFF]. If there are one or more OSM-SW triggered groups, then the values of this macro is [STD_ON]
Range	STD_ON STD_OFF
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcGroupTriggSrc AdcGroupConvMode AdcNonAutosarDmaResultHandling

4.8.28 ADC_TIMER_TRIGG

Table 154 ADC_TIMER_TRIGG

Name:	ADC_TIMER_TRIGG
Type:	#define
File:	Adc_Cfg.h
Description:	Compile time switch for code having timer trigger functionality
Range	STD_ON STD_OFF
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwTriggerApi AdcGroupTriggSrc AdcHwTrigType

4.8.29 ADC_ERU_TRIGG

Table 155 ADC_ERU_TRIGG

Name:	ADC_ERU_TRIGG
Type:	#define
File:	Adc_Cfg.h
Description:	Compile time switch for code having ERU trigger functionality
Range	STD_ON STD_OFF
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwTriggerApi AdcGroupTriggSrc AdcHwTrigType AdcHwExtTrigSelect

4.8.30 ADC_ERU_GATING

Table 156 ADC_ERU_GATING

Name:	ADC_ERU_GATING
Type:	#define
File:	Adc_Cfg.h
Description:	Compile time switch for code having ERU gate functionality
Range	STD_ON STD_OFF
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	AdcHwTriggerApi AdcGroupTriggSrc AdcHwTrigType AdcHwGatePin

5 Published Parameters

This section describes the common published parameters which are published in Adc.h.

5.1 VendorID

Table 157 VendorID

Syntax:	ADC_VENDOR_ID
Value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list. Vendor is Infineon Technologies

5.2 ModuleID

Table 158 ModuleID

Syntax:	ADC_MODULE_ID
Value:	123
Description:	This macro gives the ADC Driver module ID as described by AUTOSAR

5.3 InstanceID

Table 159 InstanceID

Syntax:	ADC_INSTANCE_ID
Value:	0
Description:	This macro gives the ADC Driver instance ID.

5.4 ArReleaseMajorVersion

Table 160 ArReleaseMajorVersion

Syntax:	ADC_AR_RELEASE_MAJOR_VERSION
Value:	Refer Adc_Cfg.h
Description:	Major version number of the AUTOSAR specification.

5.5 ArReleaseMinorVersion

Table 161 ArReleaseMinorVersion

Syntax:	ADC_AR_RELEASE_MINOR_VERSION
Value:	Refer Adc_Cfg.h
Description:	Minor version number of the AUTOSAR specification.

5.6 ArReleaseRevisionVersion

Table 162 ArReleaseRevisionVersion

Syntax:	ADC_AR_RELEASE_REVISION_VERSION
Value:	Refer Adc_Cfg.h
Description:	Revision version number of the AUTOSAR specification.

6 API Documentation

The API documentation describes the syntax and semantics of the API functions as well as it gives some additional implementation specific behavior description not defined in the AUTOSAR ADC Driver specification.

Implementation specific type definitions, which are required for configuration, are also documented.

6.1 API Type Definition

This section describes all the type definitions that are exported by ADC Driver. These type definitions are either used for configuration and/or determine type of API parameters.

6.1.1 Adc_ConfigType

Table 163 Adc_ConfigType

Syntax	Adc_ConfigType	
Type	Structure	
File	Adc.h	
Range	uint32 ConfigMarker	Configuration marker value
	const Adc_KernelConfigType *AdcCfgPtr[ADC_MAX_KERNELS]	Array of pointers to ADC hardware unit configuration structures. ADC_MAX_KERNELS is the derived parameter as per microcontroller derivative.
	const Adc_GlobalCfgType *GlobCfgPtr	Pointers to the Adc global configuration values.
	uint8 SleepMode	ADC Module Sleep mode control. The Microcontroller specific configuration parameter is AdcSleepMode
Description	Data structure containing the pointers to set of each ADC Kernel configuration parameters required for initializing. Pointer to object of this type is passed to API Adc_Init to initialize the ADC Driver.	

6.1.2 Adc_KernelConfigType

Table 164 Adc_KernelConfigType

Syntax	Adc_KernelConfigType	
Type	Structure	
File	Adc.h	
Range	const Adc_HwUnitCfgType *HwUnitCfgPtr	Pointer to the hardware unit configuration structure
	const Adc_ChannelCfgType *ChCfgPtr	Pointer to the ADC Channels configuration set
	const Adc_GroupCfgType *GrpCfgPtr	Pointer to the ADC Groups configuration set
	const uint32 * TriggSrcGroupMask	Pointers to the array of Group mask for SW trigger request sources. Valid only in ADC_PRIORITY_HW_SW mechanism.

	uint8 TotCfgGrps	Total number of ADC Groups configured. The configuration tool derived parameters for this variable are: ADC0_MAX_GROUP ADC1_MAX_GROUP ADC2_MAX_GROUP ADC3_MAX_GROUP ADC4_MAX_GROUP ADC5_MAX_GROUP ADC6_MAX_GROUP ADC7_MAX_GROUP ADC8_MAX_GROUP ADC9_MAX_GROUP ADC10_MAX_GROUP
Description	Data structure containing the set of configuration parameters required for initializing an ADC hardware unit.	

6.1.3 Adc_HwUnitCfgType

Table 165 Adc_HwUnitCfgType

Syntax	Adc_HwUnitCfgType	
Type	Structure	
File	Adc.h	
Range	Uint32 ArbitrationLength	Arbitration Round Length, ANONC : Analog Part Switched On ARBM: The arbiter only runs if at least one conversion request of an enabled request source is pending. The microcontroller specific user configuration parameter is: AdcArbitrationRoundLength
	Adc_TriggerSrcArbLevelType TriggerSrcArbLevel	Priority levels for each request source and the arbitration round length. The microcontroller specific user configuration parameters are: AdcRequestSource0Prio AdcRequestSource1Prio AdcRequestSource2Prio AdcArbitrationRoundLength
	uint32 KernelInputClass[2]	Array of Input class configuration. Each Input class is configured for channel resolution and sampling time. The microcontroller specific user configuration parameter are: Container: AdcInputClass

	uint8 SyncConvMode	Control of Master/Slave conversion mode of the kernels
	uint8 SlaveReady	Control of the Slave Ready signal of the slave kernels.
	uint8 DmaChannel	This parameter is valid only in Non AUTOSAR result handling mode and EMUX SCAN mode. This parameter is implementation specific from user configuration parameter: AdcDmaChannelRef. The selected DMA channel is related to ADC SR line. The corresponding ADC result event is mapped to ADC_SR line.
	uint8 AcceleratedTiming	Decides whether enhanced conversion timing scheme is enabled or disabled
	uint8 ShortSampleTime	Defines the duration of the sample phase
	uint8 SignalType	Defines whether Adc kernel is used for ASIL signal or QM signal
Description	Data structure containing the set of configuration parameters required for initializing the ADC HW Unit (ADC Kernel).	

6.1.4 Adc_GroupCfgType

Table 166 Adc_GroupCfgType

Syntax	Adc_GroupCfgType	
Type	Structure	
File	Adc.h	
Range	Adc_NotifFctPtrType NotifFctPtr	Group notification function pointer
	const Adc_GroupDefType* GroupDefinition	Pointer to assignment of channels to a channel group
	uint32 HwTrigConfig	Contains timers enable bit state. Selection of external request pin and trigger edge, Gate pin, Gate signal, if group is configured for external request trigger type. The microcontroller specific user configuration parameter are: AdcHwTrigType AdcHwExtTrigSelect AdcEruTriggRef AdcHwGateSignal AdcHwGatePin If AdcHwTrigType is HW_TRIGG_EXT_REQ, AdcHwGatePin value is not considered.

uint32 HwTimerValue	If timer trigger is used, this parameter will provide configured time period equivalent reload value and selection of GTM TOM (16-bit) / ATOM (24-bit) channel. The macro will concatenate the reload value and GTM TOM/ATOM channel number to form a 32 bit HwTimerValue The microcontroller specific user configuration parameter are: AdcGtmTimerPeriod AdcGtmTimerSignal SystemClock
uint32 TrigSeqCount	Defines the initial value of the trigger sequence counter TSC and Controls the effect of an incoming internal trigger. – Applicable only for TC23x and TC22x.
uint32 Rs3IntTrigger	Define the Internal triggers generated by the respective source events. – Applicable only for TC23x and TC22x.
uint16 IntChMask	Internal channel mask from group definition - derived from the tool
Adc_ValueGroupType BufferMarker	Contains the Group buffer marker value.
Adc_TriggerSourceType TriggSrc	Group trigger type : SW / HW
uint8 GrpRequestSrc	Selection of ADC HW Unit request source (RS0 – RS2) The microcontroller specific user configuration parameter are: AdcGroupRequestSource
Adc_GroupConvModeType ConvMode	Selected Group conversion mode ADC_CONV_MODE_ONESHOT or ADC_CONV_MODE_CONTINUOUS
Adc_StreamNumSampleType NumSamples	Number of A/D conversion values to be acquired in streaming access mode
Adc_StreamBufferModeType StreamBufferMode	Buffer mode type - configure streaming buffer as "linear buffer" or "circular buffer"
Adc_GroupPriorityType PriorityLevel	Priority level of the Group
Adc_GroupReplacementType GroupReplacement	Group level Replacement mechanism - Selection of Abort/Restart mechanism. Valid only when Group is interruptible.

	uint8 GroupEmuxData	Group channels EMUX select line value Group with channels configured for EMUX – Enable / Disable Group with a Channel Emux scan mode – Enable / Disable. If Emux scan mode enabled, Group channel number and EMUX sampling time are stored Valid only when ADC_USE_EMUX is switched ON. The microcontroller specific user configuration parameters are: AdcUseEmux AdcEmuxScanEnable AdcChannelEmuxSelect
Description	Data structure containing the set of configuration parameters required for initializing the ADC Groups.	

6.1.5 Adc_ChannelCfgType

Table 167 Adc_ChannelCfgType

Syntax	Adc_ChannelCfgType	
Type	Structure	
File	Adc.h	
Range	UInt32 AdcChConfigValue	Adc_VoltageSourceType AdcChannelRefVoltsrcHigh: Upper reference voltage source for each channel. AdcInputClassSelection: Input class selection The microcontroller specific user configuration parameter is: AdcInputClassSelection Adc_channelType AdcAnChannelNum: Analog Input Channel Number The microcontroller specific user configuration parameter is: AdcAnChannelNum The above said parameter values are added to derive the value of AdcChConfigValue (AdcAnChannelNum AdcChannelRefVoltsrcHigh AdcInputClassSelection)
	UInt32 AdcIsChLimitChkEnabled	Decides whether channel limit check is enabled or not.
	UInt32 AdcLimitChkRange	Decides at what scenario the

		event should be raised when limit check is enabled.
	Unit32 AdcLimitChkBnd	Boundary values for limit checking. (Low limit value High limit value)
	Uint32 AdcChBndSelxValue	Decide whether boundary register are selected for comparison or result register in standard comparison mode of limit check. For TC27x & TC2Dx, boundary register is selected always.
	uint8 AdcSyncChannel	Decides whether channel is enabled for synchronization of conversion or not
Description	Structure which handles the Adc channel information.	

6.1.6 Adc_GlobalCfgType

Table 168 Adc_GlobalCfgType

Syntax	Adc_GlobalCfgType	
Type	Structure	
File	Adc.h	
Range	Adc_PrescaleType ClkPrescale	Module Clock control settings: Divider Factor for Analog Internal Clock (f_{ADCI}), Divider Factor for Digital Arbiter Clock (f_{ADCD}).
	Uint32 GlobInputClass[2]	Global Input class selection The microcontroller specific user configuration parameter is: AdcGlobInputClass
	Uint32 PostCalEnable	Control of post calibration The microcontroller specific user configuration parameter is: AdcPostCal
	Uint32 EmuxSelectGrp	Adc Emux Select Group The microcontroller specific user configuration parameter is: AdcEmuxSelectGrp
	Uint32 LowPowerSupply	Defines the Low Power Supply voltage of the system to Adjust the analog circuitry to the supply voltage.
	Uint32 RefPrechargeCtrl	Defines the reference pre charge control of the converter.
	Uint32 T13Clock	Defines the divider value for the Timer T13 input clock –

		Applicable only for the devices with metal fix for cross coupling effect
	Uint32 TrigSel	Defines the selection of trigger output (CCU6061 TRIG1) based on the CCU6 module used – Applicable only for the devices with metal fix for cross coupling effect
	Uint16 PeriodValue	Defines the period value to generate 20MHz signal in CCU6x T13 – Applicable only for the devices with metal fix for cross coupling effect
	Uint16 DutyValue	Defines the duty value of the generated 20MHz signal in CCU6x T13 – Applicable only for the devices with metal fix for cross coupling effect
	Uint8 Ccu6Kernel	Defines the selected CCU6 module – Applicable only for the devices with metal fix for cross coupling effect
	Uint8 OperationMode	Defines the operating mode of the converter in a standalone converter or converter in master mode
	Uint8 SampleHoldClk	Defines the frequency of the clock for the SHS logic
	Uint8 CalibrationOrder	Defines the calibration order of the converter
	Uint8 CalMaxTime	Defines the maximum time until the next calibration
	Uint8 AltRefDef[3]	Defines S&H unit whose alternate reference is be used within the cluster
Description	Configuration structure for all common parameters globally available for all the Adc Kernels.	

6.1.7 Adc_ChannelType

Table 169 Adc_ChannelType

Syntax	Adc_ChannelType
Type	uint8
File	Adc.h
Range	0-255
Description	Numeric ID of ADC Channel. With EMUX OFF, maximum number of analog input channels is 8 for each HW Unit.

	With EMUX ON, the Analog input channels can be extended to 64 channels for each HW Unit. The symbolic name of each ADC Channel is macro defined with the numeric ID. This numeric ID represents the array index of its configuration in ADC channel configuration set.
--	--

6.1.8 Adc_GroupType

Table 170 Adc_GroupType

Syntax	Adc_GroupType
Type	Uint16
File	Adc.h
Range	0-65535
Description	Numeric ID of the ADC Channel Group. The symbolic name of each ADC Group is macro defined with the numeric ID. This numeric ID represents the array index of its configuration in ADC Group configuration set. Driver implementation has limited the maximum number of Groups to 32 under each kernel. To support TC26x, TC27x and TC29x, type uint16 is preferred.

6.1.9 Adc_ValueGroupType

Table 171 Adc_ValueGroupType

Syntax	Adc_ValueGroupType
Type	uint8 / uint16
File	Adc.h
Range	0 - 0xFF / 0 - 0xFFF
Description	AURIX ADC peripheral will support 8-bit, 10-bit and 12-bit conversion resolution. If all the configured ADC channels are of 8 bit resolution, uint8 is selected for Adc_ValueGroupType. Else, uint16 is selected for Adc_ValueGroupType. This is determined by pre compile parameter ADC_VALUE_GROUP_TYPE. If the configuration parameter AdcSameChannelQueuing is set to true (ADC_SAME_CHANNEL_QUEUEING = STD_ON), uint16 is selected for Adc_ValueGroupType. Configurable result buffer (Streaming access or Single access) should be of this type. Thus the dimension (in number of bits) of each buffer element (of type Integer) are uniform, tailored on the largest (in number of bits) channel belonging to the group. The buffer is made of m*n elements, where n is the number of channels belonging to the group, m is the number of samples acquired per channel (i.e. AdcStreamingNumSamples). In single value access mode, m = 1.

6.1.10 Adc_ClockSourceType

Table 172 Adc_ClockSourceType

Syntax	Adc_ClockSourceType
Type	uint8
File	Adc.h
Range	ADC_CLK_SRC_SYS_CLK
Description	Type of clock input for the conversion unit. ADC peripheral of AURIX supports only system clock.

6.1.11 Adc_PrescaleType

Table 173 Adc_PrescaleType

Syntax	Adc_PrescaleType
Type	Uint32
File	Adc.h
Description	Type of clock prescalar factor. Analog and Digital clock divider value type.

6.1.12 Adc_ConversionTimeType

Table 174 Adc_ConversionTimeType

Syntax	Adc_ConversionTimeType
Type	uint8
File	Adc.h
Description	Type of conversion time, i.e, the time during which the sampled analogue value is converted in to a digital representation. This type is not used in Aurix.

6.1.13 Adc_SamplingTimeType

Table 175 Adc_SamplingTimeType

Syntax	Adc_SamplingTimeType
Type	uint8
File	Adc.h
Range	0-0xFF
Description	The time during which the value is sampled, (in clock-cycles)

6.1.14 Adc_VoltageSourceType

Table 176 Adc_VoltageSourceType

Syntax	Adc_VoltageSourceType
Type	uint8
File	Adc.h
Range	ADC_REF_VOLTAGE_VAREF ADC_REF_VOLTAGE_CH0 ADC_REF_VOLTAGE_VAGND
Description	Type of reference voltage source for Upper and Lower reference voltage.

6.1.15 Adc_ResolutionType

Table 177 Adc_ResolutionType

Syntax	Adc_ResolutionType
Type	uint8
File	Adc.h
Range	ADC_CH_RES_8BIT ADC_CH_RES_10BIT ADC_CH_RES_12BIT
Description	Type of channel resolution in number of bits

6.1.16 Adc_StatusType

Table 178 Adc_StatusType

Syntax	Adc_StatusType
Type	uint8
File	Adc.h
Range	ADC_IDLE ADC_BUSY ADC_COMPLETED ADC_STREAM_COMPLETED
Description	Current status of the conversion of the requested ADC Channel group

6.1.17 Adc_TriggerSourceType

Table 179 Adc_TriggerSourceType

Syntax	Adc_TriggerSourceType
Type	uint8
File	Adc.h
Range	ADC_TRIGG_SRC_SW ADC_TRIGG_SRC_HW
Description	Type for configuring the trigger source for an ADC Channel group

6.1.18 Adc_GroupConvModeType

Table 180 Adc_GroupConvModeType

Syntax	Adc_GroupConvModeType
Type	uint8
File	Adc.h
Range	ADC_CONV_MODE_ONESHOT ADC_CONV_MODE_CONTINUOUS
Description	Type for configuring the conversion mode of an ADC Channel group

6.1.19 Adc_GroupPriorityType

Table 181 Adc_GroupPriorityType

Syntax	Adc_GroupPriorityType
Type	uint8
File	Adc.h
Range	0-254
Description	Priority level of the Group. Lowest priority is 0. ADC Driver on AURIX supports 254 priority levels. Priority 255 is assigned to Hw trigger groups.

6.1.20 Adc_GroupDefType

Table 182 Adc_GroupDefType

Syntax	Adc_GroupDefType
Type	Adc_ChannelType
File	Adc.h
Range	Array of ADC Channel IDs.
Description	Type for assignment of channels to a channel Group. Array of channel ID with First element being the number of configured channels in the group. From second element it's the channel ID with second element being the lowest internal channel. Maximum number of channels in a group allowed is 8 for TC27x / TC29x and 16 for TC26x. Channels under the same group can not share the same control register and hence configuration of analog Channel(x) and analog Channel(x+n8) under the same group is not allowed when EMUX is used. The first element of the Group definition will be the total number of channels in a Group.

6.1.21 Adc_StreamNumSampleType

Table 183 Adc_StreamNumSampleType

Syntax	Adc_StreamNumSampleType
Type	uint8
File	Adc.h
Range	1 – 0xFF
Description	Type for the number of samples of a streaming conversion buffer. In Single value access mode, number of samples will be 1. In Streaming access mode, number of samples will be more than 1.

6.1.22 Adc_HwUnitType

Table 184 Adc_HwUnitType

Syntax	Adc_HwUnitType
---------------	----------------

Syntax	Adc_HwUnitType
Type	uint8
File	Adc.h
Range	ADC_HW_UNIT_ADC0 ADC_HW_UNIT_ADC1 ADC_HW_UNIT_ADC2 ADC_HW_UNIT_ADC3 ADC_HW_UNIT_ADC4 ADC_HW_UNIT_ADC5 ADC_HW_UNIT_ADC6 ADC_HW_UNIT_ADC7 ADC_HW_UNIT_ADC8 ADC_HW_UNIT_ADC9 ADC_HW_UNIT_ADC10
Description	Numeric ID of an ADC HW Unit. TC2Dx/TC27x has 8 ADC HW units i.e. ADC0, ADC1, ADC2, ADC3, ADC4, ADC5, ADC6, ADC7 TC26x has 4 ADC HW units i.e. ADC0, ADC1, ADC2, ADC3 TC29x has 11 ADC HW units i.e. ADC0, ADC1, ADC2, ADC3, ADC4, ADC5, ADC6, ADC7, ADC8, ADC9, ADC10 TC23x/TC22x has 2 ADC HW units i.e. ADC0, ADC1

6.1.23 Adc_StreamBufferModeType

Table 185 Adc_StreamBufferModeType

Syntax	Adc_StreamBufferModeType
Type	uint8
File	Adc.h
Range	ADC_STREAM_BUFFER_LINEAR ADC_STREAM_BUFFER_CIRCULAR
Description	Type for configuring the streaming access mode buffer type. ADC_STREAM_BUFFER_LINEAR: The ADC Driver stops the conversion as soon as the stream buffer is full (number of samples reached). ADC_STREAM_BUFFER_CIRCULAR: The ADC Driver continues the conversion even if the stream buffer is full (number of samples reached) by wrapping around the stream buffer itself.

6.1.24 Adc_GroupAccessModeType

Table 186 Adc_GroupAccessModeType

Syntax	Adc_GroupAccessModeType
Type	uint8
File	Adc.h
Range	ADC_ACCESS_MODE_SINGLE ADC_ACCESS_MODE_STREAMING

Syntax	Adc_GroupAccessModeType
Description	Type for configuring the access mode to group conversion results

6.1.25 Adc_HwTriggerSignalType

Table 187 Adc_HwTriggerSignalType

Syntax	Adc_HwTriggerSignalType
Type	uint32
File	Adc.h
Range	Trigger Edges: ADC_HW_TRIG_RISING_EDGE ADC_HW_TRIG_FALLING_EDGE ADC_HW_TRIG_BOTH_EDGE
Description	Type for configuring Trigger pin and on which edge of the hardware trigger signal the Driver should reach, i.e. start the conversion.

6.1.26 Adc_HwTriggerTimerType

Table 188 Adc_HwTriggerTimerType

Syntax	Adc_HwTriggerTimerType
Type	uint32
File	Adc.h
Range	AURIX provides GTM interface for Timer trigger.
Description	Type for configuring reload value and GTM TOM/ATOM channel number of the GTM peripheral timer.

6.1.27 Adc_TriggerSrcArbLevelType

Table 189 Adc_TriggerSrcArbLevelType

Syntax	Adc_TriggerSrcArbLevelType
Type	uint32
File	Adc.h
Range	ADC_REQSRC_PRIORITY_0_LOWEST ADC_REQSRC_PRIORITY_1 ADC_REQSRC_PRIORITY_2 ADC_REQSRC_PRIORITY_3_HIGHEST
Description	Priority levels for HW Trigger source.

6.1.28 Adc_PriorityImplementationType

Table 190 Adc_PriorityImplementationType

Syntax	Adc_PriorityImplementationType
Type	uint8
File	Adc.h

Range	ADC_PRIORITY_NONE ADC_PRIORITY_HW ADC_PRIORITY_HW_SW
Description	Type for configuring the prioritization mechanism. Adc driver on AURIX supports 3 priority levels ADC_PRIORITY_NONE: Priority mechanism is not available ADC_PRIORITY_HW: AURIX ADC HW Unit request source arbitration mechanism is available. ADC_PRIORITY_HW_SW: AURIX ADC HW Unit request source arbitration mechanism along with software implemented priority mechanism is available

6.1.29 Adc_GroupReplacementType

Table 191 Adc_GroupReplacementType

Syntax	Adc_GroupReplacementType
Type	uint8
File	Adc.h
Range	ADC_GROUP_REPL_ABORT_RESTART
Description	Type for Group level replacement mechanism. When Group is interrupted by higher priority, Group channel conversion can be handled by Abort/Restart mechanism. Abort/Restart: The complete conversion round of the interrupted group (all group channels) is restarted after the higher priority group conversion is finished.

6.1.30 Adc_ChannelRangeSelectType

Table 192 Adc_ChannelRangeSelectType

Syntax	Adc_ChannelRangeSelectType
Type	Enum
File	Adc.h
Range	ADC_RANGE_ALWAYS ADC_RANGE_BETWEEN ADC_RANGE_NOT_BETWEEN ADC_RANGE_OVER_HIGH ADC_RANGE_NOT_OVER_HIGH ADC_RANGE_UNDER_LOW ADC_RANGE_NOT_UNDER_LOW
Description	In case of active limit checking: defines which conversion values are taken into account related to the borders defined with AdcChannelLowLimit and AdcChannelHighLimit.

6.1.31 Adc_ResultAlignmentType

Table 193 Adc_ResultAlignmentType

Syntax	Adc_ResultAlignmentType
---------------	-------------------------

Type	Uint8
File	Adc.h
Range	ADC_ALIGN_LEFT ADC_ALIGN_RIGHT
Description	Type for alignment of ADC raw results in ADC result buffer (left/right alignment).

6.2 API Functions

6.2.1 Adc_Init

Table 194 Specification of Adc_Init API

Service name:	Adc_Init	
Syntax:	<pre>void Adc_Init (const Adc_ConfigType *ConfigPtr)</pre>	
Service ID:	0x00	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Non-reentrant	
Parameters (in):	ConfigPtr	Pointer to the Configuration set
Parameters (out):	None	
Return value:	None	
Description:	This service will initialize the ADC peripheral unit and Driver according to the configuration set referenced by ConfigPtr. This function will only initialize the configured resources. Resources that are not contained in the configuration file will not be touched. The following rules regarding initialization of controller registers will apply to this Driver implementations: One-time writable registers that require initialization directly after reset will be initialized by the startup code. By default, notifications and hardware trigger capability will be disabled after Driver initialization (if statically configured as active). All the Group status are set to ADC_IDLE state If development error detection is enabled, the routine will check if the ADC Driver and hardware are already initialized and report error ADC_E_ALREADY_INITIALIZED and return without any action. If called with NULL_PTR as configuration parameter, or if called with other than Adc_ConfigRoot[0] as configuration parameter when ADC_PB_FIXED_ADDRESS is STD_ON, report error ADC_E_PARAM_CONFIG and return without any action. If safety is enabled, plausibility check of ConfigPtr is done to verify if the pointer is valid. If Clock enable failed during initialization, report a safety error ADC_E_CLC_ERROR and return without any action. After calling Adc_Init(), Api Adc_17_SoftCalWrLongInit() or	

	Adc_17_SoftCalWrVagnd() must be called depending on the workaround used in the application.	
Caveats:	None	
Configuration:	None	
DET:	ADC_E_PARAM_CONFIG	Configuration Pointer is a NULL. If ADC_PB_FIXED_ADDRESS is STD_ON and configuration pointer is not equal to Adc_ConfigRoot[0].
	ADC_E_ALREADY_INITIALIZED	ADC Driver already initialized
SAFE MCAL ERROR:	ADC_E_PARAM_CONFIG	Configuration Pointer is a NULL. If ADC_PB_FIXED_ADDRESS is STD_ON and configuration pointer is not equal to Adc_ConfigRoot[0]. If ConfigMarker doesn't match
	ADC_E_CLC_ERROR	Clock enable/disable failed during initialization.
DEM:	None	
Implementation Comments:	- If Queued Request source (RS0) used for conversion, ADC kernel specific service request node SR0 is used. - If Autoscan Request source (RS1) used for conversion, ADC kernel specific service request node SR1 is used. - If Background scan Request source (RS2) used for conversion, common shared service request 0 node SR0 is used for SAR ADC (e.g., TC23x, TC27xCA) and node SR2 is used for SHS ADC (e.g., TC26xAB, TC27xBC, TC29xAA). - If Queued Request source (RS3) used for conversion in ADC kernel 0, common shared service request 0 node SR1 is used. - If Queued Request source (RS3) used for conversion in ADC kernel 1, common shared service request 0 node SR2 is used. - Limit checking group uses ADC kernel specific service request node SR2. - ADC kernel specific service request node SR3 is used for Emux scan enabled for channel groups and Non Autosar Result handling mode. - In case of fixed address configuration usage, i.e. ADC_PB_FIXED_ADDRESS is STD_ON, then Adc_ConfigRoot[0] configuration is used. - CCU60_CLC is initialized even if CCU61 module is only used for ADC to avoid getting trap while accessing CCU60_MOSEL register, since the CCU60_MOSEL register is common for CCU60 and CCU61. This is applicable only for devices with metal fix for cross coupling effect (e.g., TC27xDB, TC26xBB, TC29xBB and TC23xAB). - If Clock enable/disable failed during initialization, report a safety error ADC_E_CLC_ERROR and return without any action. - ADC module (kernel) reset is performed, if the parameter AdcResetSfrAtInit is enabled. SRC (interrupt) register reset is not done in ADC driver. ADC modules environment shall reset the SRC register by calling a function Irq_ClearAdcIntFlags() provided in IRQ driver or with the similar functionality of environment.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided	

configuration parameter `AdcRunningInUser0Mode` is 'False' and `AdcUserModelInitApiEnable` is 'False'.

6.2.2 Adc_SetupResultBuffer

Table 195 Specification of Adc_SetupResultBuffer API

Service name:	Adc_SetupResultBuffer	
Syntax:	Std_ReturnType Adc_SetupResultBuffer (Adc_GroupType Group, Adc_ValueGroupType* DataBufferPtr)	
Service ID:	0x0c	
Sync/ Async:	Synchronous	
Origin:	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC channel group.
	DataBufferPtr	ADC result buffer pointer is initialized with the value of the data buffer pointer
Parameters (out):	None	
Return value:	Std_ReturnType	E_OK: result buffer pointer initialized correctly
		E_NOT_OK: operation failed or development error occurred
Description:	This function is compile time switched off in Non Autosar DMA mode (<code>AdcNonAutosarDmaResultHandling = STD_ON</code>) and Result Polling mode (<code>AdcNonAutosarDmaResultPolling = STD_ON</code>). The function <code>Adc_SetupResultBuffer</code> initialize the result buffer pointer of the selected group with the address value passed as parameter. The ADC module's environment must ensure that the application buffer, which address is passed as parameter in <code>Adc_SetupResultBuffer</code>, has the according size to hold all group channel conversion results and if streaming access is selected, hold these results multiple times as specified with streaming number of sample (<code>AdcStreamingNumSamples</code>). If the configuration parameter <code>AdcSameChannelQueuing</code> is set to true (<code>ADC_SAME_CHANNEL_QUEUING = STD_ON</code>), the repeated same channel entries in configuration parameter <code>AdcGroupDefinition</code> should be consider as individual channels and buffer should be provided for all the entries by the ADC module's environment. If development error detection is enabled, the routine will check for the valid channel group ID. On error, the API will report DET: <code>ADC_E_PARAM_GROUP</code> and return <code>E_NOT_OK</code> without any action. If development error detection is enabled, and the API is called prior to initializing the Driver, the DET: <code>ADC_E_UNINIT</code> will be reported and returns <code>E_NOT_OK</code> without any action. If development error detection is enabled and the API is called for the Group whose conversion is ongoing or Group is in Queue, the DET: <code>ADC_E_BUSY</code> will be reported and returns <code>E_NOT_OK</code> without any	

	action.	
Caveats:	None	
Configuration:	None	
DET:	ADC_E_UNINIT	ADC Driver not initialized
	ADC_E_PARAM_GROUP	Function called with non existing group
	ADC_E_BUSY	Function called while a Group conversion is already running.
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
	ADC_E_PARAM_POINTER	BufferMarker value doesn't match with the configured buffer marker value of the group.
DEM:	None	
Implementation Comments:	- The Groups which are explicitly/implicitly stopped can be started again and the result buffer pointer can be set again. - When new result buffer is initialized for the Group, the data available from previous conversion is reset to 0. - The ADC module's environment shall ensure that no group conversions are started without prior initialization of the according result buffer pointer to point to a valid result buffer. - The ADC module's environment shall ensure that the application buffer, which address is passed as parameter in <code>Adc_SetupResultBuffer</code>, has the according size to hold all group channel conversion results and if streaming access is selected, hold these results multiple times as specified with streaming sample parameter.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'True' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'False'.	

6.2.3 Adc_DelInit

Table 196 Specification of Adc_DelInit

Service name:	Adc_DelInit
Syntax:	<pre>void Adc_DelInit (void)</pre>
Service ID:	0x01
Sync/ Async:	Synchronous
Origin:	Autosar
Re-entrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None

Return value:	None	
Description:	This service will return all ADC peripheral units to a state comparable to their power on reset state. Values of registers which are not writeable are excluded. The service will disable all used interrupts and notifications. If development error detection is enabled, and the service is called while not all groups are either in state ADC_IDLE or state ADC_STREAM_COMPLETED, the function will raise DET: ADC_E_BUSY and return without any action. If development error detection is enabled and calling this function before module has been initialized, will cause a DET: ADC_E_UNINIT. The called service will be left without any action. If Clock disable failed during de-initialization, report a safety error ADC_E_CLC_ERROR and return without any action. The function Adc_DeInit can be pre compile time configurable On/Off by the configuration parameter: ADC_DEINIT_API.	
Caveats:	The ADC module's environment shall not call the function Adc_DeInit while any group is not in state ADC_IDLE.	
Configuration:	This function is available if configuration parameter AdcDeinitApi is ON	
DET:	ADC_E_BUSY	Function called while conversion is running
	ADC_E_UNINIT	Function called prior to initialization
SAFE MCAL ERROR:	ADC_E_CLC_ERROR	Clock enable/disable failed during de-initialization.
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeDeInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeDeInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeDeInitApiEnable is 'False'.	

6.2.4 Adc_InitCheck

Table 197 Specification of API Adc_InitCheck

Syntax:	Std_ReturnType Adc_InitCheck (const Adc_ConfigType *ConfigPtr)		
Service ID:	None		
Sync/Async:	Sync		
Origin	Vendor Specific		
Reentrancy:	Non Reentrant		
Parameters (in):	ConfigPtr	-	Pointer to ADC driver configuration set.
Parameters (out):	None		
Return value:	E_OK – Initialization comparison successful E_NOT_OK – Initialization not properly done by ADC driver		
Description:	This API verifies the initialization done by ADC driver against the configuration pointer only. This API is available only if ADC_SAFETY_ENABLE and ADC_INITCHECK_API are STD_ON.		

Preconditions:	Adc_Init() should be called in a different task than the caller calling this API.
Timing Constraints:	None
Caveats:	None
Configuration:	This function is available if configuration parameter AdcSafetyEnable is ON
DET:	None
SAFE MCAL ERROR:	None
DEM:	None
Implementation Comments:	It will check against all SFRs or variables initialized by Adc_Init(). It does not modify any SFR/variable, only a read operation is done. If any failure in comparison, it reports an error. To avoid multiple if-else conditions and increase code complexity, each comparison will be done by a XOR of value against the inverted value of the intended value or vice versa. The comparison result should always be 0xFFFFFFFF (all 1s). This will also improve runtime since multiple checks are avoided if no failures exist. Failure happening has less probability and can be reported well within the diagnostic test interval.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModelInitApiEnable is 'False'.

6.2.5 Adc_StartGroupConversion

Table 198 Specification of Adc_StartGroupConversion

Service name:	Adc_StartGroupConversion	
Syntax:	<pre>void Adc_StartGroupConversion (Adc_GroupType Group)</pre>	
Service ID:	0x02	
Sync/ Async:	Asynchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Group.
Parameters (out):	None	
Return value:	None	
Description:	This service will start the conversion of all channels of the requested ADC Channel group. Depending on the group configuration single-shot or continuous conversion is started. The service will reset the internal result buffer pointer to the result buffer base address which was configured with Adc_SetupResultBuffer API. Starting a group conversion will NOT automatically enable the	

notification mechanism for that group (this has to be done by a `Adc_EnableGroupNotification` API call).

The service can be pre-compile time configurable On/Off by the configuration parameter `ADC_ENABLE_START_STOP_GROUP_API`.

If development error detection is enabled for the ADC Driver, calling this function with a non-existing channel group ID will cause a development error `ADC_E_PARAM_GROUP`. The service will then be left without any action.

If development error detection is enabled for the ADC Driver, calling this function on a group with trigger source configured as hardware will cause a development error `ADC_E_WRONG_TRIGG_SRC`. The service will then be left without any action.

If development error detection is enabled and the priority mechanism is disabled and the queuing is disabled, when called while any of the groups, which can not be implicitly stopped, is not in state `ADC_IDLE`, the service will raise DET: `ADC_E_BUSY` and return without any action.

If development error detection is enabled and the priority mechanism is disabled and the queuing is disabled, when called while any of the groups, which can be implicitly stopped, is not in state `ADC_IDLE` and not in state `ADC_STREAM_COMPLETED`, the service will raise DET: `ADC_E_BUSY` and return without any action.

Note: Groups which can be implicitly stopped are:

- *Software triggered groups configured in one-shot, single-access mode*
- *Software triggered groups configured in continuous, linear streaming access mode*
- *Hardware triggered groups configured in one-shot, linear streaming access mode*

If development error detection is enabled and the priority mechanism is enabled, when called while a group, which can not be implicitly stopped, is not in state `ADC_IDLE`, the service will raise DET: `ADC_E_BUSY` and return without any action.

If development error detection is enabled and the priority mechanism is enabled, when called while a group, which can be implicitly stopped, is not in state `ADC_IDLE` and not in state `ADC_STREAM_COMPLETED`, the service will raise DET: `ADC_E_BUSY` and return without any action.

If development error detection is enabled and the priority mechanism is disabled and the queuing is enabled, when called while a group, which can not be implicitly stopped, is not in state `ADC_IDLE`, the service raises DET: `ADC_E_BUSY` and return without any action.

If development error detection is enabled and the priority mechanism is disabled and the queuing is enabled, when called while a group, which can be implicitly stopped, is not in state `ADC_IDLE` and not in state `ADC_STREAM_COMPLETED`, the service will raise DET: `ADC_E_BUSY` and return without any action.

If development error detection is enabled, when called prior to initializing the result buffer pointer with function `Adc_SetupResultBuffer`, the service will raise DET: `ADC_E_BUFFER_UNINIT` and return without any action.

If development error detection is enabled and service is called prior to

	initializing the Driver, the DET: ADC_E_UNINIT will be reported and return without any action. If NonAutosarDmaReultPolling is enabled, Autosar specific statuses ADC_IDLE, ADC_BUSY, ADC_COMPLETED, and ADC_STREAM_COMPLETED are not supported.	
Caveats:	The ADC module's environment shall only call Adc_StartGroupConversion for groups configured with software trigger source.	
Configuration:	This function is available if configuration parameter AdcEnableStartStopGroupApi is ON	
DET:	ADC_E_UNINIT	ADC Driver not initialized
	ADC_E_PARAM_GROUP	Function called with non existing group
	ADC_E_WRONG_TRIGG_SRC	Function called for a group configured for hardware trigger source
	ADC_E_CAL_ONGOING	Power up calibration is on going
	ADC_E_BUSY	Function called while a conversion is already running
	ADC_E_BUFFER_UNINIT	Function called while result buffer pointer is not initialized
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
DEM:	None	
Implementation Comments:	- Group must be in ADC_IDLE state before requesting for Start conversion again. Exceptions to this rule are groups which are implicitly stopped because of the selected conversion mode (linear buffer with streaming access mode or one-shot conversion mode with single access). These groups can also be restarted while the group is in state ADC_STREAM_COMPLETED. - If Software trigger source is free, other Group under same trigger source can be started for conversion in ADC_PRIORITY_HW and ADC_PRIORITY_HW_SW. - The ADC module's environment shall check the integrity, if several calls for the same ADC group are used during runtime in different tasks or ISR's.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.	

6.2.6 Adc_StopGroupConversion

Table 199 Specification of Adc_StopGroupConversion

Service name:	Adc_StopGroupConversion
----------------------	-------------------------

Syntax:	<pre>void Adc_StopGroupConversion (Adc_GroupType Group)</pre>	
Service ID:	0x03	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	None	
Return value:	None	
Description:	When the ADC Channel Group is in one-shot and software-trigger mode, the service will stop an ongoing conversion of the group and remove any start/restart requests of the group from the queue, if queuing is enabled. When the ADC Channel Group is in continuous-conversion and software-trigger mode, the service will stop an ongoing conversion of the group and remove any start/restart requests of the group from the queue, if queuing is enabled. Stopping a group conversion will automatically disable group notification for the requested group. <i>Note: Groups which are implicitly stopped will not disable the group notification until Adc_StopGroupConversion is called.</i> The function Adc_StopGroupConversion will set the group status to state ADC_IDLE. The function Adc_StopGroupConversion can be pre compile time configurable On/Off by the configuration parameter ADC_ENABLE_START_STOP_GROUP_API. If development error detection is enabled and the service is called for non-existing Group ID, the service will raise DET: ADC_E_PARAM_GROUP and return without any action. If development error detection is enabled and the service is called for the group with trigger source configured as hardware, the service will raise DET: ADC_E_WRONG_TRIGG_SRC and return without any action. If development error detection is enabled, when called while the group is in state ADC_IDLE, the service will raise DET: ADC_E_IDLE and return without any action. <i>Note: For groups which are implicitly stopped (groups with conversion mode one-shot or groups with linear streaming buffer mode), state is ADC_STREAM_COMPLETED until results are accessed with Adc_ReadGroup or Adc_GetStreamLastPointer API functions or until group is explicitly stopped by Adc_StopGroupConversion API.</i> If development error detection is enabled and when service is called prior to initializing the module, the service will raise DET: ADC_E_UNINIT and return without any action.	
Caveats:	The ADC module's environment shall only call the function Adc_StopGroupConversion for groups configured with trigger source software.	
Configuration:	This function is available if configuration parameter AdcEnableStartStopGroupApi is ON	

DET:	ADC_E_UNINIT	ADC Driver not initialized
	ADC_E_PARAM_GROUP	Function called with non existing group
	ADC_E_WRONG_TRIGG_SRC	Function called for a group configured for hardware trigger source
	ADC_E_IDLE	Function called while Group conversion is not started.
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
DEM:	None	
Implementation Comments:	Group will go to ADC_IDLE state by explicit stop API call or by reading the Group result data. These Groups when again requested for stop will raise DET: ADC_E_IDLE. <i>Note: All groups which are started with Adc_StartGroupConversion should also be stopped with Adc_StopGroupConversion, before they are started again to reset the group status to ADC_IDLE. Exceptions to this rule are groups which are implicitly stopped because of the selected conversion mode (linear buffer with streaming access mode or one-shot conversion mode with single access). These groups can also be restarted while the group is in state ADC_STREAM_COMPLETED.</i>	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.	

6.2.7 Adc_ReadGroup

Table 200 Specification of Adc_ReadGroup API

Service name:	Adc_ReadGroup	
Syntax:	Std_ReturnType Adc_ReadGroup (Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr) 	
Service ID:	0x04	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC channel group.
	DataBufferPtr	ADC results of all channels of the selected group are stored in the data buffer addressed with the pointer.
Parameters (out):	None	

Return value:	Std_ReturnType	E_OK: result are available and written to the data buffer E_NOT_OK: no results are available or DET occurred.
Description:	This function is compile time switched off in Non Autosar mode (AdcNonAutosarDmaResultHandling = ON). The service will read the latest available conversion results of the requested group. The service will read the raw converted values without further scaling. The read values are aligned as per the configuration (Left/Right). The service will mask out all information or diagnostic bits provided by the conversion but not belonging to the conversion results themselves. Calling function Adc_ReadGroup while group status is ADC_STREAM_COMPLETED will trigger a state transition to ADC_BUSY for continuous conversion modes (single access mode or circular streaming buffer mode) and hardware triggered groups in single access mode or circular streaming access mode. Calling function Adc_ReadGroup while group status is ADC_STREAM_COMPLETED will trigger a state transition to ADC_IDLE for software triggered conversion modes which automatically stop the conversion (streaming buffer with linear access mode or one-shot conversion mode with single access) and for the hardware triggered conversion mode in combination with linear streaming access mode. Calling function Adc_ReadGroup while Group status is ADC_COMPLETED will trigger a state transition to ADC_BUSY. If development error detection is enabled, when called while the group status is ADC_IDLE and no result data available from previous conversion after implicit/explicit Stop, the function Adc_ReadGroup will raise DET: ADC_E_IDLE, return E_NOT_OK without any action. If development error detection is enabled, and service is called with the non-existing Group ID will raise DET: ADC_E_PARAM_GROUP and return E_NOT_OK without any action. If development error detection is enabled, when called prior to initializing the driver, the service will raise DET: ADC_E_UNINIT and return E_NOT_OK without any action.	
Caveats:	None	
Configuration:	The service can be pre-compile configurable On/Off by the configuration parameter AdcReadGroupApi.	
DET:	ADC_E_UNINIT	ADC Driver not initialized
	ADC_E_PARAM_GROUP	Function called with non existing group
	ADC_E_IDLE	Function called while group status is ADC_IDLE
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
	ADC_E_PARAM_POINTER	BufferMarker value doesn't match with the configured buffer marker value of the group.
DEM:	None	
Implementation Comments:	When Group status is ADC_BUSY state and with no matter of result availability (i.e. Group status ADC_BUSY after fresh start / after result read), the Adc_ReadGroup API call will not report any	

	DET instead will return E_NOT_OK. If Group has stopped conversion by explicit/Implicit stop. The Group result buffer can be initialized with new location using <code>Adc_SetupResultBuffer</code>. When new result buffer is initialized for the Group, no data will be read and the read service will raise a DET: <code>ADC_E_IDLE</code>
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'True' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'False'.

6.2.8 Adc_EnableHardwareTrigger

Table 201 Specification of Adc_EnableHardwareTrigger

Service name:	Adc_EnableHardwareTrigger	
Syntax:	<pre>void Adc_EnableHardwareTrigger (Adc_GroupType Group)</pre>	
Service ID:	0x05	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	None	
Return value:	None	
Description:	The function <code>Adc_EnableHardwareTrigger</code> will enable the hardware trigger for the requested ADC Channel group. A group with trigger source hardware, whose trigger was enabled with <code>Adc_EnableHardwareTrigger</code>, will execute the group channel conversions, whenever a trigger event occurs. The service will reset the internal result buffer pointer to the result buffer base address which was configured with <code>Adc_SetupResultBuffer</code> API. If development error detection is enabled and if the priority mechanism is disabled and queuing disabled, when called while any group with trigger source SW is not in state <code>ADC_IDLE</code>, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_BUSY</code> and return without any action. If development error detection is enabled and if the HW trigger for the group is already enabled, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_BUSY</code> and return without any action. If development error detection is enabled and if the channel group ID is invalid, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_PARAM_GROUP</code> and return without any action.	

	If development error detection is enabled and if the group is configured for software API trigger mode, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_WRONG_TRIGG_SRC</code> and return without any action. If development error detection is enabled and if a HW group is erroneously configured for continuous conversion mode, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_WRONG_CONV_MODE</code> and return without any action. <i>Note: SW groups configured in continuous conversion mode will raise DET: <code>ADC_E_WRONG_TRIGG_SRC</code> instead.</i> If development error detection is enabled and if called prior to initializing the driver, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_UNINIT</code> and return without any action. If development error detection for the ADC module is enabled and when called prior to initializing the result buffer pointer with function <code>Adc_SetupResultBuffer</code>, the function <code>Adc_EnableHardwareTrigger</code> will raise DET: <code>ADC_E_BUFFER_UNINIT</code>. If <code>NonAutosarDmaReultPolling</code> is enabled, Autosar specific statuses <code>ADC_IDLE</code>, <code>ADC_BUSY</code>, <code>ADC_COMPLETED</code>, and <code>ADC_STREAM_COMPLETED</code> are not supported.	
Caveats:	The ADC module's environment shall only call the function <code>Adc_EnableHardwareTrigger</code> for groups configured in hardware trigger mode. The ADC module's environment shall guarantee that no concurrent conversions take place on the same HW Unit (happening of different hardware triggers at the same time).	
Configuration:	The function <code>Adc_EnableHardwareTrigger</code> can be pre-compile time configurable On/Off by the configuration parameter <code>ADC_HW_TRIGGER_API</code>.	
DET:	<code>ADC_E_UNINIT</code>	ADC Driver not initialized
	<code>ADC_E_PARAM_GROUP</code>	Function called with non existing group
	<code>ADC_E_WRONG_TRIGG_SRC</code>	Function called for a group configured for Software trigger source
	<code>ADC_E_WRONG_CONV_MODE</code>	Function is called for a group configured in continuous conversion mode.
	<code>ADC_E_CAL_ONGOING</code>	Power up calibration is on going
	<code>ADC_E_BUSY</code>	Function called while a conversion is already running
	<code>ADC_E_BUFFER_UNINIT</code>	Function called while result buffer pointer is not initialized
SAFE MCAL ERROR:	<code>ADC_E_PARAM_GROUP</code>	Function called with wrong group
DEM:	None	
Implementation Comments:	- If the priority mechanism is disabled and queuing enabled, the driver will not allow Group to be enabled until the ADC HW Unit is free and Queue is empty. - Group must be in <code>ADC_IDLE</code> state before requesting for Enable conversion again. Exceptions to this rule are groups which are implicitly stopped because of the selected conversion mode (linear buffer with streaming access mode). These groups can also be restarted while the group is in state <code>ADC_STREAM_COMPLETED</code>.	

	• If Hardware trigger source is free, other Group under same trigger source can be started for conversion in ADC_PRIORITY_HW and ADC_PRIORITY_HW_SW. • The ADC module's environment shall check the integrity, if several calls for the same ADC group are used during runtime in different tasks or ISR's.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.

6.2.9 Adc_DisableHardwareTrigger

Table 202 Specification of Adc_DisableHardwareTrigger

Service name:	Adc_DisableHardwareTrigger	
Syntax:	<pre>void Adc_DisableHardwareTrigger (Adc_GroupType Group)</pre>	
Service ID:	0x06	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	None	
Return value:	None	
Description:	The function Adc_DisableHardwareTrigger will disable the hardware trigger for the requested ADC Channel group. The function Adc_DisableHardwareTrigger will remove any queued start/restart request for the requested ADC Channel group if queuing is enabled. The function Adc_DisableHardwareTrigger will abort an ongoing conversion. If enabled, the function Adc_DisableHardwareTrigger will disable the notification mechanism for the requested group. The function Adc_DisableHardwareTrigger will set the group status to state ADC_IDLE. If development error detection is enabled and if the channel group ID is non-existing, the service will raise DET: ADC_E_PARAM_GROUP and return without any action If development error detection is enabled and if the group is configured for software API trigger mode, the service will raise DET: ADC_E_WRONG_TRIGG_SRC and return without any action. If development error detection is enabled and if a HW group is erroneously configured for continuous conversion mode, the service will raise DET:	

	ADC_E_WRONG_CONV_MODE and return without any action. <i>Note: SW groups configured in continuous conversion mode will raise DET: ADC_E_WRONG_TRIGG_SRC instead.</i> If development error detection is enabled and if the group is not enabled (with a previous call of Adc_EnableHardwareTrigger), the service will raise DET: ADC_E_IDLE and return without any action. If development error detection is enabled and if called prior to initializing the ADC module, Adc_DisableHardwareTrigger will raise DET: ADC_E_UNINIT and return without any action.	
Caveats:	The ADC module's environment shall only call the function Adc_DisableHardwareTrigger for groups configured in hardware trigger mode.	
Configuration:	The function Adc_DisableHardwareTrigger will be pre-compile time configurable On/Off by the configuration parameter AdcHwTriggerApi.	
DET:	ADC_E_UNINIT	ADC Driver not initialized
	ADC_E_PARAM_GROUP	Function called with non existing group
	ADC_E_WRONG_TRIGG_SRC	Function called for a group configured for Software trigger source
	ADC_E_WRONG_CONV_MODE	Function is called for a group configured in continuous conversion mode.
	ADC_E_IDLE	Function called while Group conversion is not Enabled.
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
DEM:	None	
Implementation Comments:	Group will go to ADC_IDLE state by explicit Disable API call or by reading the Group result data. These Groups when again requested for Disable will raise DET: ADC_E_IDLE.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.	

6.2.10 Adc_EnableGroupNotification

Table 203 Specification of Adc_EnableGroupNotification

Service name:	Adc_EnableGroupNotification
Syntax:	<pre>void Adc_EnableGroupNotification (Adc_GroupType Group)</pre>
Service ID:	0x07
Sync/ Async:	Synchronous

Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	None	
Return value:	None	
Description:	The function <code>Adc_EnableGroupNotification</code> will enable the notification mechanism for the requested ADC Channel group. If development error detection is enabled and if the channel group ID is non-existing, the service will raise DET: <code>ADC_E_PARAM_GROUP</code> and return without any action If development error detection is enabled and if the group notification function pointer is NULL, the service will raise DET: <code>ADC_E_NOTIF_CAPABILITY</code> and return without any action. If development error detection is enabled and if called prior to initializing the ADC module, the service will raise DET: <code>ADC_E_UNINIT</code> and return without any action.	
Caveats:	None	
Configuration:	The function <code>Adc_EnableGroupNotification</code> will be pre-compile time configurable On/Off by the configuration parameter <code>AdcGrpNotifCapability</code> .	
DET:	<code>ADC_E_UNINIT</code>	ADC Driver not initialized
	<code>ADC_E_PARAM_GROUP</code>	Function called with non existing group
	<code>ADC_E_NOTIF_CAPABILITY</code>	Notification function pointer of requested group is NULL
SAFE MCAL ERROR:	<code>ADC_E_PARAM_GROUP</code>	Function called with wrong group
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'True' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'False'.	

6.2.11 Adc_DisableGroupNotification

Table 204 Specification of `Adc_DisableGroupNotification`

Service name:	<code>Adc_DisableGroupNotification</code>
Syntax:	<pre>void Adc_DisableGroupNotification (Adc_GroupType Group)</pre>
Service ID:	0x08
Sync/ Async:	Synchronous

Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	None	
Return value:	None	
Description:	The function <code>Adc_DisableGroupNotification</code> will disable the notification mechanism for the requested ADC Channel group. If development error detection is enabled and if the channel group ID is non-existing, the function <code>Adc_DisableGroupNotification</code> will raise DET: <code>ADC_E_PARAM_GROUP</code> and return without any action. If development error detection is enabled and if the group notification function pointer is NULL, the service will raise DET: <code>ADC_E_NOTIF_CAPABILITY</code> and return without any action. If development error detection is enabled: and if called prior to initializing the ADC module, the service will raise DET: <code>ADC_E_UNINIT</code> and return without any action.	
Caveats:	None	
Configuration:	ADC101: The function <code>Adc_DisableGroupNotification</code> will be pre-compile time configurable On/Off by the configuration parameter <code>AdcGrpNotifCapability</code> .	
DET:	<code>ADC_E_UNINIT</code>	ADC Driver not initialized
	<code>ADC_E_PARAM_GROUP</code>	Function called with non existing group
	<code>ADC_E_NOTIF_CAPABILITY</code>	Notification function pointer of requested group is NULL
SAFE MCAL ERROR:	<code>ADC_E_PARAM_GROUP</code>	Function called with wrong group
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'True' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'False'.	

6.2.12 Adc_GetGroupStatus

Table 205 Specification of `Adc_GetGroupStatus`

Service name:	<code>Adc_GetGroupStatus</code>
Syntax:	<pre>Adc_StatusType Adc_GetGroupStatus (Adc_GroupType Group)</pre>
Service ID:	0x09
Sync/ Async:	Synchronous

Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	None	
Return value:	Adc_StatusType	Conversion status for the requested group.
Description:	This service returns the conversion status of the requested ADC Channel group. This service returns ADC_IDLE if it is called before the conversion of the requested group has been started or due to the state transition after reading the result data. This service returns ADC_BUSY: • If it is called while the first conversion of the requested group is still ongoing (continuous conversion mode). • If trigger is enabled for group configured for HW trigger source. • Once Adc_StartGroupConversion is called for group configured for SW trigger source • Due to the state transition after reading the result data The service returns ADC_COMPLETED: • If it is called after a conversion round (not the final one) of the requested group has been finished. The service returns ADC_STREAM_COMPLETED: • If it is called in single access mode after one conversion round is completed. • If it is called in streaming access mode after the number of conversion rounds of the requested group have been finished, to fill the streaming buffer completely. The service provides atomic access to the status data by the use of atomic instructions. To guarantee consistent returned values, it is assumed that ADC group conversion is always started (or enabled in case of HW group) successfully by SW before status polling begins. If development error detection for the ADC module is enabled and if the channel group ID is non-existing, the service will raise DET: ADC_E_PARAM_GROUP and return ADC_IDLE without any action. If development error detection is enabled and if called prior to initializing the ADC module, the service will raise DET: ADC_E_UNINIT and return ADC_IDLE without any action. The ADC module's environment shall guarantee the consistency of the data that has been read by checking the return value of Adc_GetGroupStatus. The Api is not available if parameter AdcNonAutosarResultPolling is enabled.	
Caveats:	None	
Configuration:	None	
DET:	ADC_E_UNINIT	ADC Driver not initialized
	ADC_E_PARAM_GROUP	Function called with non existing group
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
DEM:	None	
Implementation Comments:	None	

I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'True' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'False'.
------------------	--

6.2.13 Adc_GetStreamLastPointer

Table 206 Specification of `Adc_GetStreamLastPointer`

Service name:	<code>Adc_GetStreamLastPointer</code>	
Syntax:	<pre>Adc_StreamNumSampleType Adc_GetStreamLastPointer (Adc_GroupType Group, Adc_ValueGroupType** PtrToSamplePtr)</pre>	
Service ID:	0x0B	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Reentrant	
Parameters (in):	Group	Numeric ID of requested ADC Channel group.
Parameters (out):	PtrToSamplePtr	Pointer to result buffer pointer.
Return value:	<code>Adc_StreamNumSampleType</code>	Number of valid samples per channel.

Description:	This function is compile time switched off in Non Autosar DMA mode (AdcNonAutosarDmaResultHandling = ON) and Result Polling mode (AdcNonAutosarDmaResultPolling = ON). The function Adc_GetStreamLastPointer will set the pointer, passed as parameter (PtrToSamplePtr) to point in the ADC result buffer to the latest result of the first group channel of the last completed conversion round. All values which the ADC driver stores in the ADC result buffer, are left without further scaling and will be aligned as per the configuration. The function Adc_GetStreamLastPointer will return the number of valid samples per channel, stored in the ADC result buffer. <i>Note: Valid samples are in the ADC result buffer when the group is in state ADC_COMPLETED or ADC_STREAM_COMPLETED. In state ADC_BUSY or ADC_IDLE the value 0 is returned.</i> <i>Note: The return value is 1 for groups with single access mode configuration, if valid samples are stored in the ADC result buffer.</i> When called while the group status is ADC_BUSY (a conversion of the group is in progress), the function Adc_GetStreamLastPointer will set the pointer, passed as parameter (PtrToSamplePtr), to NULL and return 0. The ADC module's environment shall guarantee the consistency of the data that has been read by checking the return value of Adc_GetGroupStatus. Calling function Adc_GetStreamLastPointer while group status is ADC_STREAM_COMPLETED will trigger a state transition to ADC_BUSY for continuous conversion modes (single access mode or circular streaming buffer mode) and hardware triggered groups in single access mode or circular streaming access mode. Calling function Adc_GetStreamLastPointer while group status is ADC_STREAM_COMPLETED will trigger a state transition to ADC_IDLE for software conversion modes which automatically stop the conversion (streaming buffer with linear access mode or one-shot conversion mode with single access) and for the hardware triggered conversion mode in combination with linear streaming access mode. Calling function Adc_GetStreamLastPointer while group status is ADC_COMPLETED will trigger a state transition to ADC_BUSY. If development error detection is enabled and when called while the group status is ADC_IDLE and the group conversion was not started (no results are available from previous conversions), the function Adc_GetStreamLastPointer will raise DET: ADC_E_IDLE, set the pointer, passed as parameter (PtrToSamplePtr), to NULL and return 0. If development error detection for the ADC module is enabled and if the group ID is non-existent, the service will raise DET: ADC_E_PARAM_GROUP, set the pointer, passed as parameter (PtrToSamplePtr), to NULL and return 0 without any further action. If development error detection for the ADC module is enabled and if called prior to initializing the driver, the service will raise DET: ADC_E_UNINIT, set the pointer, passed as parameter (PtrToSamplePtr), to NULL and return 0 without any further action.
Caveats:	None
Configuration:	None
DET:	ADC_E_UNINIT ADC Driver not initialized

	ADC_E_PARAM_GROUP	Function called with non existing group
	ADC_E_IDLE	Function called while group status is ADC_IDLE
SAFE MCAL ERROR:	ADC_E_PARAM_GROUP	Function called with wrong group
DEM:	None	
Implementation Comments:	- When Group status is ADC_BUSY state and with no matter of result availability (i.e. Group status ADC_BUSY after fresh start / after result read), the Adc_GetStreamLastPointer API call will not report any DET instead will return 0 and out parameter is set to NULL. - If Group has stopped conversion by explicit/Implicit stop. The Group result buffer can be initialized with new location using Adc_SetupResultBuffer. When new result buffer is initialized for the Group, no data will be read and the read service will raise a DET: ADC_E_IDLE - Range check is not done for the output parameter PtrToSamplePtr, since it has second level of dereferencing	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.	

6.2.14 Adc_GetVersionInfo

Table 207 Specification of Adc_GetVersionInfo

Service name:	Adc_GetVersionInfo	
Syntax:	<pre>void Adc_GetVersionInfo (Std_VersionInfoType *versioninfo)</pre>	
Service ID:	0x0A	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	VersionInfoPtr	Pointer to where to store the version information of this Driver.
Return value:	None	
Description:	This service returns the version information of this Driver. The version information includes: - Module Id. - Vendor Id. - Vendor specific version numbers (BSW00407). If development error detection is enabled: The function	

Service name:	Adc_GetVersionInfo	
	Adc_GetVersionInfo raises the DET: ADC_E_PARAM_POINTER if the parameter versioninfo passed as NULL.	
Caveats:	None	
Configuration:	This function will be pre-compile time configurable On/Off by the configuration parameter: AdcVersionInfoApi	
DET:	ADC_E_PARAM_POINTER	parameter versioninfo passed as NULL
SAFE MCAL ERROR:	None	
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.	

6.2.15 Adc_17_StartGtmTimer

Table 208 Specification of Adc_17_StartGtmTimer

Syntax:	<pre>void Adc_17_StartGtmTimer (Adc_GroupType Group)</pre>		
Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Reentrant		
Parameters (in):	Group	Adc_GroupType	Adc channel group number
Parameters (out):	None	-	-
Return value:	void	-	-
Description:	Configures the GTM timer when Group is configured for Timer trigger type and AdcGroupRequestSource is REQSRC2_BGND_SCAN		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	None		
Implementation Comments:	Selected GTM TOM / ATOM Channel are the resources used as timer This API is pre-compile time switched off if there are no HW trigger groups with Timer trigger. Available if (ADC_HW_TRIGGER_API == STD_ON && ADC_TIMER_TRIGG ==		

	STD_ON) This API should be called to start the timer trigger from Gtm if Adc channel groups, AdcGroupRequestSource is REQSRC2_BGND_SCAN
Design Decisions:	GTM modules functions are called to set the timer values.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.

6.2.16 Adc_17_StopGtmTimer

Table 209 Specification of Adc_17_StopGtmTimer

Syntax:	void Adc_17_StopGtmTimer (Adc_GroupType Group)		
Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Reentrant		
Parameters (in):	Group	Adc_GroupType	Adc channel group number
Parameters (out):	None	-	-
Return value:	void	-	-
Description:	Service stops the running GTM timer channel if AdcGroupRequestSource is REQSRC2_BGND_SCAN		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	None		
Implementation Comments:	This API is pre-compile time switched off if there are no HW trigger groups withTimer trigger types. i.e., the function is available only if (ADC_HW_TRIGGER_API == STD_ON && ADC_TIMER_TRIGG == STD_ON) This API should be called to stop the timer trigger from Gtm if Adc channel groups, AdcGroupRequestSource is REQSRC2_BGND_SCAN		
Design Decisions:	None		
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and		

AdcUserModeRuntimeApiEnable is 'False'.

6.2.17 Adc_17_SetEruTrigger

Table 210 Specification of Adc_17_SetEruTrigger

Syntax:	voidAdc_17_SetEruTrigger (Adc_GroupType Group)		
Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Reentrant		
Parameters (in):	Group	Adc_GroupType	Adc channel group number
Parameters (out):	None	-	-
Return value:	void	-	-
Description:	Configures the ERU module when Group is configured for Eru trigger/gate type and AdcGroupRequestSource is REQSRC2_BGND_SCAN		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	None		
Implementation Comments:	This API is pre-compile time switched off if there are no HW trigger groups with External trigger. Available if (ADC_HW_TRIGGER_API == STD_ON && ADC_ERU_TRIGG == STD_ON) This API should be called to start the external trigger/gate from Eru if Adc channel group, AdcGroupRequestSource is REQSRC2_BGND_SCAN.		
Design Decisions:	None		
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.		

6.2.18 Adc_17_ResetEruTrigger

Table 211 Specification of Adc_17_ResetEruTrigger

Syntax:	voidAdc_17_ResetEruTrigger (Adc_GroupType Group)
----------------	---

Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Reentrant		
Parameters (in):	Group	Adc_GroupType	Adc channel group number
Parameters (out):	None	-	-
Return value:	void		-
Description:	Resets the ERU register configured for triggering/gating Adc channel groups if AdcGroupRequestSource is REQSRC2_BGND_SCAN		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	None		
Implementation Comments:	This API is pre-compile time switched off if there are no HW trigger groups with External trigger. i.e., the function is available only if (ADC_HW_TRIGGER_API == STD_ON && ADC_ERU_TRIGG == STD_ON). This API should be called to stop the external trigger/gate from Eru if Adc channel groups, AdcGroupRequestSource is REQSRC2_BGND_SCAN		
Design Decisions:	None		
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.		

6.2.19 Adc_17_GetStartupCalStatus

Table 212 Specification of Adc_17_GetStartupCalStatus

Syntax:	Std_ReturnType Adc_17_GetStartupCalStatus(void)		
Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Reentrant		
Parameters (in):	Void	-	-
Parameters (out):	None	-	-
Return value:	Std_ReturnType		E_OK: Starup calibration is completed. E_NOT_OK: Starup calibration is ongoing
Description:	Function checks the end of startup calibration.		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		

Error Handling:	None
Implementation Comments:	API checks the end of the startup calibration and returns E_OK if the startup calibration is completed for all the enabled Adc kernels. It returns E_NOT_OK if startup calibration is not completed for any one the Adc kernels. API Adc_17_GetStartupCalStatus() returns E_NOT_OK, if the configured AdcConfigSets does not have identical Adc kernels. Eg: if AdcConfigSet_0 has ADC_HW_UNIT0 & ADC_HW_UNIT1, AdcConfigSet_1 has ADC_HW_UNIT0 & ADC_HW_UNIT2, then Adc_17_GetStartupCalStatus() always returns E_NOT_OK for both the AdcConfigSets. API Adc_17_GetStartupCalStatus() is not available for TC27xBC, TC26xAA, TC26xAB and TC29xAA devices, since the hardware calibration is disabled due to hardware errata ADC_TC.62 workaround implementation.
Design Decisions:	None
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.

6.2.20 Adc_17_SoftCalWrLongInit

Table 213 Specification of Adc_17_SoftCalWrLongInit

Syntax:	void Adc_17_SoftCalWrLongInit(uint8 AdcKernel)		
Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Non-Reentrant		
Parameters (in):	AdcKernel	uint8	Adc hardware unit number
Parameters (out):	None	-	-
Return value:	None	-	
Description:	Function executes the Start Up and Post calibration of the ADC.		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	None		
Implementation Comments:	Api Adc_17_SoftCalWrLongInit() must be called for each available kernel which needs a software calibration. The calibration function must be called with the Adc kernel number as parameter. Make sure that the Adc kernel is available in the device. Api Adc_17_SoftCalWrLongInit() switches always between standard and alternate reference. Api Adc_17_SoftCalWrLongInit() should be called for each available kernel of the device after Initialization (Adc Init()). After this, it is sufficient to call the function		

	each millisecond or higher time, e.g. before or after a finished auto scan. Api <code>Adc_17_SoftCalWrLongInit()</code> is implemented based on the ADC_TC.62 Errata workaround (Application note AP32246_SW_calibration_v2_0_0.pdf). Api <code>Adc_17_SoftCalWrLongInit()</code> is given as evaluation only. Api <code>Adc_17_SoftCalWrLongInit()</code> uses the Alias feature to redirect conversion requests for CH0 and CH1 to other channel numbers (in this case to the negative and positive reference voltage). Therefore the CH0 and CH1 should not be used in HW triggered conversions to make sure that the CH0 and CH1 are not redirected to the reference voltage. Also make sure that until the function is finished, CH0 and CH1 are not used (e.g. by software via interrupt and so on). Request source Queue must be free while using API <code>Adc_17_SoftCalWrLongInit()</code>, since it also uses Queue request source for the conversion. After calling <code>Adc_Init()</code>, Api <code>Adc_17_SoftCalWrLongInit()</code> or <code>Adc_17_SoftCalWrVagnd()</code> must be called depending on the workaround used in the application. This Api is applicable only for Sample&Hold ADC devices.
Design Decisions:	None
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'True' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter <code>AdcRunningInUser0Mode</code> is 'False' and <code>AdcUserModeRuntimeApiEnable</code> is 'False'.

6.2.21 Adc_17_SoftCalWrVagnd

Table 214 Specification of Adc_17_SoftCalWrVagnd

Syntax:	<code>void Adc_17_SoftCalWrVagnd(uint8 AdcKernel)</code>		
Service ID:	None		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Non-Reentrant		
Parameters (in):	AdcKernel	uint8	Adc hardware unit number
Parameters (out):	None	-	-
Return value:	None		-
Description:	Function executes the Start Up and Post calibration of the ADC.		
Preconditions:	None		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	None		
Implementation	Api <code>Adc_17_SoftCalWrVagnd()</code> must be called for each available kernel which		

Comments:	needs a software calibration. The calibration function must be called with the Adc kernel number as parameter. Make sure that the Adc kernel is available in the device. Api Adc_17_SoftCalWrVagnd() performs always a conversion to VAGND before a usable conversion. Api Adc_17_SoftCalWrVagnd() should be called for each available kernel of the device after Initialization (Adc_Init()). After this, it is sufficient to call the function each millisecond or higher time, e.g. before or after a finished auto scan. Api Adc_17_SoftCalWrVagnd() is implemented based on the ADC_TC.62 Errata workaround (Application note AP32246_SW_calibration_v2_0_0.pdf). Api Adc_17_SoftCalWrVagnd() is given as evaluation only. Api Adc_17_SoftCalWrVagnd() uses the Alias feature to redirect conversion requests for CH0 and CH1 to other channel numbers (in this case to the negative and positive reference voltage). Therefore the CH0 and CH1 should not be used in HW triggered conversions to make sure that the CH0 and CH1 are not redirected to the reference voltage. Also make sure that until the function is finished, CH0 and CH1 are not used (e.g. by software via interrupt and so on). Request source Queue must be free while using API Adc_17_SoftCalWrVagnd(), since it also uses Queue request source for the conversion. After calling Adc_Init(), Api Adc_17_SoftCalWrLongInit() or Adc_17_SoftCalWrVagnd() must be called depending on the workaround used in the application. This Api is applicable only for Sample&Hold ADC devices.		
Design Decisions:	None		
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.		

6.2.22 Adc_17_GetGroupResult

Table 215 Specification of Adc_17_GetGroupResult

Syntax:	Std_ReturnType Adc_17_GetGroupResult (Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr)		
Service ID:	0x2E		
Sync/Async:	Synchronous		
Origin	Vendor Specific		
Reentrancy:	Reentrant		
Parameters (in):	Group	Adc_GroupType	Numeric ID of requested ADC channel group.
Parameters (out):	DataBufferPtr	Adc_ValueGroupType*	ADC results of all channels of the selected group are stored in the data

		buffer addressed with the pointer.
Return value:	Std_ReturnType	E_OK: No DET or Safety error occurred E_NOT_OK: DET or Safety error occurred
Description:	Function is to read the latest available conversion results of the requested group in a polling mode (JIRA 0000051124-40). This API is not available for Sample&Hold ADC devices.	
Preconditions:	The function is pre compile time configurable On/Off by the configuration parameter ADC_RESULT_POLLING_MODE	
Timing Constraints:	None	
Caveats:	None	
Configuration:	AdcNonAutosarResultPolling	
Error Handling:	DET: ADC_E_PARAM_GROUP is reported if invalid group id is passed SAFETY ERROR: ADC_E_PARAM_GROUP is reported if invalid group id is passed ADC_E_PARAM_POINTER is reported if group buffer mask doesn't match.	
Implementation Comments:	1. The function is pre compile time configurable On/Off by the configuration parameter ADC_RESULT_POLLING_MODE 2. The return status is used only upon the DET and SAFETY switched ON. With DET OFF and SAFETY OFF, it always return E_OK	
AlgorithmDescription:	1. Extract the ADC Kernel ID and Group number from input parameter Group 2. API parameter checking is enabled only if DET is ON and SAFETY is ON. DETs are serviced by Det_ReportError service. SAFETY Errors are serviced by SafeMcal_ReportError service. 3. If error is detected, the DET / SAFETY are reported and return E_NOT_OK without any action. Else, Read is serviced. 4. Read the result register of the corresponding channels of the requested group 5. The Read value is aligned according to the configuration parameter ADC_RESULT_ALIGNMENT. 6. Does not check or clear the Autosar specific ADC Group Result Status on read 7. Return with E_OK	
Design Decisions:	None	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.	

6.2.23 Adc_17_GetChannelStatus

Table 216 Specification of Adc_17_GetChannelStatus

Syntax:	Std_ReturnType Adc_17_GetChannelStatus(uint8 ChannelNumber)
Service ID:	0x2F
Sync/Async:	Synchronous
Origin	Vendor Specific

Reentrancy:	Reentrant		
Parameters (in):	uint8	ChannelNumber	Numeric id of analog channel number
Parameters (out):	None	-	-
Return value:	Std_ReturnType		E_OK: Channel conversion is completed. E_NOT_OK: Channel conversion is ongoing
Description:	Function is to check the end of requested analog channel conversion in a polling mode (JIRA 0000051124-40) This API is not available for Sample&Hold ADC devices.		
Preconditions:	The function is pre compile time configurable On/Off by the configuration parameter ADC_RESULT_POLLING_MODE		
Timing Constraints:	None		
Caveats:	None		
Configuration:	None		
Error Handling:	SAFETY ERROR: ADC_E_PARAM_CHANNEL is reported if invalid channel number is passed		
Implementation Comments:	None		
AlgorithmDescription:	1. Read the register GxRESy.VF 2. Return E_OK if the register bitfield value is 1. 3. Return E_NOT_OK if the register bitfield value is 0.		
Design Decisions:	None		
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter AdcRunningInUser0Mode is 'True' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter AdcRunningInUser0Mode is 'False' and AdcUserModeRuntimeApiEnable is 'False'.		

6.3 Interrupt Handling

After every group conversion round, the corresponding interrupt service routine is serviced and the conversion round results are updated in the result buffer initialized by the user using Adc_SetupResultBuffer API. The Group status transition and decision on implicit stop happens in ISR.

In HW_SW priority or Queue mode, the scheduler functions are called from ISR.

For the Group with EMUX SCAN feature, DMA is used for result transfer and hence DMA interrupt is used when Group state reaches ADC_STREAM_COMPLETED.

Interrupts are not used if parameter AdcNonAutosarResultPolling is set to true (enabled).

Table 217 SRN and Interrupt Source Mapping

Service Request node	Events that calls the ISR	ISR function that needs to be called (Exported in Adc.h file)
ADCx_SR0, where x = 0 – 10	Handles the interrupts from Group that uses Request Source 0 of ADCx (applicabler for all devices) and the Request Source 3 of ADC1 (applicable only for Aurix EP devices).	Adc_IsrSrn0AdcRS0

Service Request node	Events that calls the ISR	ISR function that needs to be called (Exported in Adc.h file)
	This ISR is required if request source 0 is Configured by any of the Groups or request source 3 is configured in ADC1. Report safety error ADC_E_INTR_SRC_ERROR in case of interrupt source is wrong	
ADCx_SR1, where x = 0 – 10	Handles the interrupts from Group that uses Request Source 1 of ADCx. This ISR is required if request source 1 is Configured by any of the Groups. Report safety error ADC_E_INTR_SRC_ERROR in case of interrupt source is wrong	Adc_IsrSrn1AdcRS1
ADCCG0_SR2	Handles the interrupts from Group that uses Request Source 2 of all ADC kernels. This ISR is required if request source 2 is Configured by any of the Groups. This is applicable only for Sample&Hold ADC devices.	Adc_IsrSrn2CG0AdcRS2
ADCCG0_SR0	Handles the interrupts from Group that uses Request Source 2 of all ADC kernels. This ISR is required if request source 2 is Configured by any of the Groups. This is applicable only for SAR ADC devices.	Adc_IsrSrn0CG0AdcRS2
ADCCG0_SR1	Handles the interrupts from Group that uses Request Source 3 of ADC0. This ISR is required if request source 3 is Configured by any of the Groups. Report safety error ADC_E_INTR_SRC_ERROR in case of interrupt source is wrong This is applicable only for TC23x & TC22x.	Adc_IsrSrn1CG0AdcRS3
ADCx_SR2, where x = 0 – 10	Handles the interrupts from Limit check enabled Groups of ADCx.	Adc_IsrSrn2AdcChEvtnt

Service Request node	Events that calls the ISR	ISR function that needs to be called (Exported in Adc.h file)
	This ISR is required if limit checking is enabled for any of the Groups. Report safety error ADC_E_INTR_SRC_ERROR in case of interrupt source is wrong	
DMAMEx_CHy_SR	Handles the interrupts from Group with EMUX Scan feature This ISR is required if any of the ADC Group is configured for EMUX SCAN	Adc_IsrDmaSr
ADCx_SR3 (x = 0 to 10)	Event triggers the selected DMA channel to start the data transfer in EMUX SCAN and Non Autosar result handling mode.	-

6.4 Exclusive Area (EA) Information

The exclusive areas (critical sections) are provided for any of the following reason:

- 1) To protect shared resources from the module interrupts. The resources are used in ISR's.
- 2) To protect shared resources from another API / same API.

Note: All the following Exclusive Areas belong to the point 1.

6.4.1 StartGroup

Table 218 Specification of StartGroup

API Names:	Adc_StartGroupConversion
Resource Name:	Adc_BgndScanData Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables have to be checked and updated in a critical section only in the case of DET == STD_ON. This is because the status check is done only in the Development code

6.4.2 StopGroup

Table 219 Specification of StopGroup

API Names:	Adc_StopGroupConversion
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal

ISR Name:	NA
Comment	The Status variables have to be checked in a critical section.

6.4.3 ReadGroup

Table 220 Specification of ReadGroup

API Names:	Adc_ReadGroup
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables have to be checked and updated in a critical section only in the case of DET == STD_ON. This is because the status check is done only in the Development code

6.4.4 EnableHwTrig

Table 221 Specification of EnableHwTrig

API Names:	Adc_EnableHardwareTrigger
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables have to be checked and updated in a critical section only in the case of DET == STD_ON. This is because the status check is done only in the Development code

6.4.5 DisableHwTrig

Table 222 Specification of DisableHwTrig

API Names:	Adc_DisableHardwareTrigger
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables have to be checked in a critical section.

6.4.6 GetGrpStatus

Table 223 Specification of GetGrpStatus

API Names:	Adc_GetGroupStatus
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA

Comment	The Status variables read in a critical section.
---------	--

6.4.7 GetStreamLastPtr

Table 224 Specification of GetStreamLastPtr

API Names:	Adc_GetStreamLastPointer
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status and valid conversion result variables are read and updated in a critical section.

6.4.8 ScheduleStart

Table 225 Specification of ScheduleStart

API Names:	Adc_ISchedulerOnStart
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables are read and updated in a critical section.

6.4.9 ScheduleStop

Table 226 Specification of ScheduleStop

API Names:	Adc_ISchedulerOnStop
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables are read and updated in a critical section.

6.4.10 ScheduleNext

Table 227 Specification of ScheduleNext

API Names:	Adc_IScheduleNext
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal Adc_IPrepareGrpForStart Adc_IStartSwConversionRS
ISR Name:	NA
Comment	During a start of the group no other request for start or stop are allowed

CONFIDENTIAL

6.4.11 PushQueue

Table 228 Specification of PushQueue

API Names:	Adc_IPushOnQueue
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal
ISR Name:	NA
Comment	The Status variables are read and updated in a critical section.

6.4.12 PopQueue

Table 229 Specification of PopQueue

API Names:	Adc_IPopFromQueue
Resource Name:	Adc_AsilSignal0 Adc_AsilSignal1 Adc_QmSignal Adc_IStopConversionRequestSource Adc_IResetTrigSrcData Adc_IScheduleFromQueue
ISR Name:	NA
Comment	During the Group status check, interrupt should not be serviced. To avoid the same group or any other Group to be started when its status is set to Idle but not yet stopped in the ADC HW

7 Error Classification

Table 230 Error Classification

Error Code	Error Description	Relevance (Development / Production / Safe Macal Error)	Value (Hex)
ADC_E_UNINIT	Adc_Init has not been called prior to another function call.	Development	0x0A
ADC_E_BUSY	Adc_StartGroupConversion /Adc_EnableHardwareTrigger were called while another conversion is already running. Adc_DeInit was called while a conversion is still ongoing	Development	0x0B
ADC_E_IDLE	Adc_StopGroupConversion / Adc_DisableHardwareTrigger called while no conversion was running. Adc_ReadGroup /	Development	0x0C

	Adc_GetStreamLastPointer called when the group is idle and there is no previous conversion result.		
ADC_E_ALREADY_INITIALIZED	Adc_Init has been called while ADC is already initialized	Development	0x0D
ADC_E_PARAM_CONFIG	Adc_Init has been called with incorrect configuration parameter (configuration pointer is NULL_PTR for post-build configuration)	Development / Safe Mcal Error	0x0E
ADC_E_PARAM_POINTER	Adc_SetupResultBuffer or Adc_GetVersionInfo called with invalid data buffer pointer. NULL_PTR passed.	Development / Safe Mcal Error	0x14
ADC_E_PARAM_GROUP	Invalid group ID requested	Development / Safe Mcal Error	0x15
ADC_E_WRONG_CONV_MODE	Adc_EnableHardwareTrigger or Adc_DisableHardwareTrigger called on a group with conversion mode configured as continuous	Development	0x16
ADC_E_WRONG_TRIGG_SRC	Adc_StartGroupConversion or Adc_StopGroupConversion called on a group configured for hardware trigger source. Adc_EnableHardwareTrigger or Adc_DisableHardwareTrigger called on a group configured for software trigger source	Development	0x17
ADC_E_NOTIF_CAPABILITY	Enable/disable notification function for a group whose configuration set has no notification available	Development	0x18
ADC_E_BUFFER_UNINIT	Significant if AdcNonAutosarDmaResultHandling is [OFF]. Conversion started and result buffer pointer is not initialized	Development	0x19
ADC_E_PARAM_CHANNEL	Invalid analog channel number passed to Adc_17_GetChannelStatus()	Safe Mcal Error	0x29

8 Example Usage

This chapter describes how the ADC Driver can be configured and how to use different API provided by it.

8.1 Configuration of ADC Driver

The detailed description of the Configuration concept and configuration parameters are provided in chapter 4. The usage of the Driver involves

Configuration of other Drivers:

1. Configuration of System Clock. This Configuration is done using the MCU Driver. [\[8\]](#)
2. Port Pin configuration for external trigger pins and EMUX Select pins. [\[10\]](#)

3. ERU Configuration if External trigger is used, using the MCU Driver. [8]
4. GTM Configuration if Timer trigger is used. [9]

ADC Configuration:

1. AdcGeneralConfiguration 4.2
2. AdcHwUnitConfiguration 4.5
3. AdcGroupConfiguration 4.5.14.13
4. AdcChannelConfiguration 4.5.10

8.1.1 Configuration of VADC Clock

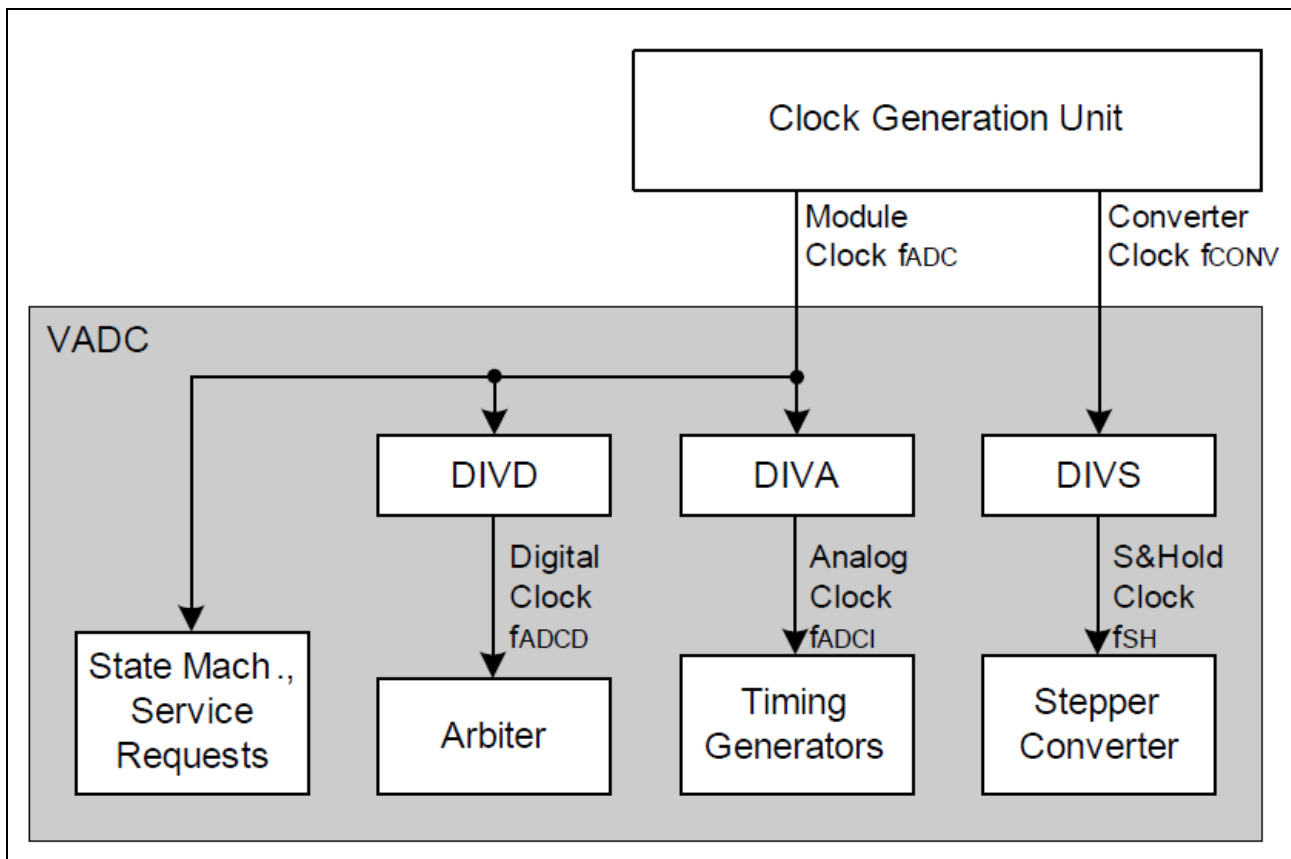


Figure 12 AURIX VADC Clock Control Structure

The configuration of AdcAnalogClockDivider [4.4.5] and AdcDigitalArbiterClockDivider [4.4.6] will give the required Analog clock and Digital clock for VADC HW Unit. AdcSHSClockDivider [4.4.7] defines the clock for SHS logic.

Example:

- $f_{\text{SYS}} = f_{\text{CLC}} = 80 \text{ MHz}$
- $f_{\text{ADC}} = f_{\text{SYS}}$
- $f_{\text{ADCI}} = f_{\text{ADC}} / 4 = 20 \text{ MHz}$ (GLOBCFG.DIVA = 3)

The analog clock f_{ADCI} is used as basic converter clock.

The digital clock f_{ADCD} is used for the arbiter and defines the duration of an arbiter round.

The Sample and Hold clock f_{SH} is used as clock for SHS logic and converter.

Code Listing 1 Example - ADC HW Unit Configuration Structure in Adc_PBCfg.c

```

001:  /*
002:                                     Container: AdcHWUnitConfiguration
003:  */
004:  /*
005:  Configuration: ADC HW unit configuration structure
006:  */
007:  /*Configuration set 1 */
008:  /* ADC 0 hardware unit configuration */
009:  static const Adc_HwUnitCfgType Adc_kKernel0HwCfg0 =
010:  {
011:      /*
012:          Arbitration Round Length: 00b - 11b
013:          ANONC : 11b
014:          ARBM  : 1b
015:      */
016:      Adc_lConfigArbitration((uint32)ADC_ARBITRATION_SLOTS_4),
017:
018:      /* Trigger source arbitration level control */
019:      /* HW trigger sources participate in arbitration only when
020:         priority mechanism PRIORITY_HW or PRIORITY_HW_SW considered
021:         Default value for each trigger source is : REQSRC_PRIORITY_0_LOWEST
022:         In No priority, the priority levels of trigger source data shall be
         set to default value.
023:      */
024:      Adc_lArbLvlCtrl(
025:          ADC_REQSRC_PRIORITY_3_HIGHEST,
026:          ADC_REQSRC_PRIORITY_1,
027:          ADC_REQSRC_PRIORITY_0_LOWEST
028:      ),
029:
030:      /*Adc Kernel Input class configuration */
031:      {
032:          Adc_lInpClass
033:          ( (uint32)0,
034:            (uint32)ADC_CH_RES_10BIT,
035:            (uint32)0
036:          ),
037:          Adc_lInpClass
038:          ( (uint32)10,
039:            (uint32)ADC_CH_RES_12BIT,
040:            (uint32)0
041:          )
042:      },
043:
044:
045:      /* Control of post calibration */
046:      (uint8)ADC_POST_CAL_ENABLE,
047:      (uint8)(ADC_DMA_CH_INVALID)
048:  };/* Adc HwCfg */

```

8.1.2 Configuration of AdcMaxChConvTimeCount

This parameter value is required by the ADC Driver during stopping of conversion.

This will make sure the channel under conversion is completed during stop request and ADC HW Unit goes to idle state before new conversion starts by Driver.

Note: If the entered value of AdcMaxChConvTimeCount is less than 16, the wait on Stop/Disable in the driver is not performed. In such a case user has to ensure that new start of a Group is called only when ADC HW Unit is Idle.

Switching off of driver to wait on stop is applicable in ADC_PRIORITY_NONE (AdcEnableQueuing is OFF) and ADC_PRIORITY_HW only.

Calculating the AdcMaxChConvTimeCount:

$$\text{Conversion time} = 2 * t_{\text{ADC}} + (4 + \text{STC} + n) * t_{\text{ADCI}} + \text{SL}$$

Where,

$n = 8, 10, 12$ for n bit resolution

t_{ADC} = ADC module clock = system clock

t_{ADCI} = Analog clock

STC = Sample time control (Value - 0 to 256)

SL = Sampling phase length

$$\text{Sample phase length, SL} = (2 + \text{STC}) / f_{\text{ADCI}}$$

where STC: Sample time control (Value - 0 to 256)

f_{ADCI} : Analog clock frequency

$$1/f_{\text{ADCI}} = t_{\text{ADCI}}$$

Maximum SL value:

$$\text{STC} = 256$$

$$t_{\text{ADCI}} = 32 * t_{\text{ADC}}$$

$$t_{\text{ADC}} = t_{\text{SYS}} \text{ (System clock time)}$$

$$\text{No. of sampling time count} = 32 * t_{\text{SYS}} (2+256) / t_{\text{SYS}} = 8256$$

Minimum SL value:

$$\text{STC} = 0$$

$$t_{\text{ADCI}} = 1 * t_{\text{ADC}}$$

$$t_{\text{ADC}} = t_{\text{SYS}} \text{ (System clock time)}$$

$$\text{No. of sampling time count} = 1 * t_{\text{SYS}} (2+0) / t_{\text{SYS}} = 2$$

Maximum conversion time value:

$$t_{\text{ADC}} = t_{\text{SYS}} \text{ (System clock time)}$$

$$t_{\text{ADCI}} = 32 * t_{\text{ADC}}$$

$$\text{STC} = 256$$

$n = 12$ bit resolution

$$\text{Maximum SL value} = 8256$$

No. of conversion time count = $t_{SYS} (2 + (4 + 256 + 12)32) / t_{SYS} + SL = 8706 + 8256$
 $= 16962$

Minimum conversion time value:

$t_{ADC} = t_{SYS}$ (System clock time)

$t_{ADCI} = 1 * t_{ADC}$

STC = 0

n = 8 bit resolution

Minimum SL value = 2

No. of conversion time count = $t_{SYS} (2 + (4 + 0 + 8)1) / t_{SYS} + SL = 14 + 2 = 16$

The conversion time is composed of the sample time count configured as `AdcKernelChSampleTime` [4.5.16.1] or `AdcGlobChSampleTime` [4.4.17.1], the ADC clock period which is same as system clock period, and Analog clock period.

Conversion time = $2 \times T_{ADC} + (4 + STC + n) \times T_{ADCI}$. Where n = 8, 10 or 12 for n – bit resolution.

The conversion time t_c is related with the system clock to get the `AdcMaxChConvTimeCount`.

Note: In above calculations, the final value is divided by t_{SYS} , this indicates the arrived value should be read in terms of t_{SYS} . Ex: if conversion time count is 16 then the actual result is 16 times t_{SYS} .

For Example:

If System clock frequency, $f_{SYS} = 60\text{MHz}$

Conversion time, $t_c = 10\mu\text{Sec}$

`AdcMaxChConvTimeCount` = $t_c / (1 / f_{SYS}) = 600$

8.1.3 Configuration of ADC Channel Parameters

Table 231 Example - ADC Channel Configuration

AdcChannelId/ Channel Symbolic name	AdcChannel0	AdcChannel1	AdcChannel2
AdcChannelId	0	1	2
AdcAnChannelNum	0	5	7
AdcChannelSampTime	10	20	30
AdcChannelResolution	ADC_CH_RES_8BIT	ADC_CH_RES_12BIT	ADC_CH_RES_10BIT
AdcChannelRefVoltsrsrcHigh	ADC_REF_VOLTAGE_VAREF	ADC_REF_VOLTAGE_VAREF	ADC_REF_VOLTAGE_CH0

`AdcChannelSampTime` and `AdcChannelResolutions` are taken from Referred `AdcInputClassSelection`

The symbolic name of each ADC Channel is a macro defined with the numeric Channel ID (`AdcChannelId`). This numeric ID represents the array index of its configuration in ADC Channel configuration set. This symbolic name allows accessing Channel data.

The symbolic name and `AdcChannelId` of the ADC Channel is configurable by the user.

8.1.4 Configuration of SW Trigger Group Parameters

The symbolic name of each ADC Group is a macro defined with the numeric ID. This numeric ID represents the array index of its configuration in ADC Group configuration set. This symbolic name allows accessing Group data.

The symbolic name of the ADC Group is configurable by the user. The AdcGroupId will be automatically generated by the configuration tool.

ADC Groups are sorted in ADC Group configuration structure in the order of their trigger type and request source starting from Software trigger groups, RS0 to RS2 and then followed by Hardware trigger groups, RS0 to RS2. The configuration tool will automatically arrange the Groups in the above mentioned order.

Code Listing 2 Example - ADC Group Symbolic Name Configuration in Adc_Cfg.h

```

049:  /*****
050:  **                               Global Symbols                               **
051:  *****/
052:
053:  /*GROUP SYMBOLIC NAME */
054:
055:  /* Configuration set 1 */
056:
057:  /* ADC0 HW Unit */
058:
059:  #ifndef AdcGroup_0 /* to prevent double declaration */
060:  #define AdcGroup_0  ((Adc_GroupType)0)
061:  #endif /* #ifdef AdcGroup_0 */
062:
063:  #ifndef AdcGroup_1 /* to prevent double declaration */
064:  #define AdcGroup_1  ((Adc_GroupType)1)
065:  #endif /* #ifdef AdcGroup_1 */
066:
067:  #ifndef AdcGroup_2 /* to prevent double declaration */
068:  #define AdcGroup_2  ((Adc_GroupType)2)
069:  #endif /* #ifdef AdcGroup_2 */

```

Code Listing 3 Example – ADC Group Definition

```

070:  /*
071:  Container: AdcGroupConfiguration
072:  */
073:  /*
074:  Configuration: ADC Group Definitions
075:  Array of channel Id (i.e. Channel Index of channel configuration
    structure) ending with Delimiter for each Adc Channel Group.
076:  Maximum number of channels in a group shall be 16.
077:  Channels under the same group can not share the same CHCTR register and
    hence configuration of Channel(x) and Channel(x+n16) under the same
    group is not allowed.
078:  */
079:  /*Configset 1 */
080:  /* ADC 0 unit configuration */

```

Code Listing 3 Example – ADC Group Definition

```

081:
082:   static const Adc_GroupDefType Adc_kCfg0Kernel0AdcGroup_0[] =
083:   {
084:       2U,
085:
086:       2U ,
087:       1U
088:   };
089:   static const Adc_GroupDefType Adc_kCfg0Kernel0AdcGroup_1[] =
090:   {
091:       1U,
092:
093:       0U
094:   };
095:   static const Adc_GroupDefType Adc_kCfg0Kernel0AdcGroup_2[] =
096:   {
097:       1U,
098:
099:       1U
0100:  };

```

Table 232 Example – Software, continuous, streaming result access group configuration

Configuration Parameter	Value	Description
Container Name	AdcGroup_0	The container name will give the Group symbolic name.
AdcGroupId	Not configurable	Numeric ID is generated by the config tool. The container name is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data.
AdcGroupPriority	5	Applicable only when ADC_PRIORITY_HW_SW mechanism is considered
AdcGroupDefinition	Reference to: AdcChannel0 AdcChannel1	This is a reference parameter to AdcChannel
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is Software
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	NA	Not configurable for SW trigger groups
AdcGroupConvMode	ADC_CONV_MODE_CONTINUOUS	Continuous conversion
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	10	10 samples of each channel is storable in streaming buffer

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After 10 sample conversion, conversion for the Group will stop.
AdcHwExtTrigSelect	NA	Not configurable for SW trigger groups
AdcHwGateSignal	NA	
AdcHwGatePin	NA	
AdcGtmTimerPeriod	NA	
AdcGtmTimerRef	NA	
AdcNotification	IoHwAb_Adc_Notification_AdcGroup0	Group notification function name
AdcGroupReplacement	ADC_GROUP_REPL_ABORT_RESTART	Applicable only when ADC_PRIORITY_HW_SW mechanism is considered and for ADC_TRIGG_SRC_SW
AdcEruTriggRef	NA	Not applicable for SW trigger groups
AdcEruGateRef	NA	Not applicable for SW trigger groups
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	OFF	Emux Scan mode disabled

Table 233 Example – Software, oneshot, single result access group configuration

Configuration Parameter	Value	Description
Container Name	AdcGroup_1	The container name will give the Group symbolic name.
AdcGroupId	Not configurable	Numeric ID is generated by the config tool. The container name is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data.
AdcGroupPriority	0	lowest priority when ADC_PRIORITY_HW_SW priority mechanism is considered
AdcGroupDefinition	Reference to: AdcChannel0	This is a reference parameter to AdcChannel
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is Software
AdcGroupRequestSource	REQSRC0_8STG_QUE	ADC HW Unit Request source 0 is selected
AdcHwTrigType	NA	Not configurable for SW trigger groups
AdcGroupConvMode	ADC_CONV_MODE_ONESHOT	Oneshot conversion. After one conversion round, the Group request source will be free for other requests.

Configuration Parameter	Value	Description
AdcGroupAccessMode	ADC_ACCESS_MODE_SINGLE	Result buffer will hold only one set of result
AdcStreamingNumSamples	1	1 samples of each channel is storable in result buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	Default for oneshot, SW trigger groups
AdcHwExtTrigSelect	NA	Not configurable for SW trigger groups
AdcHwGateSignal	NA	
AdcHwGatePin	NA	
AdcGtmTimerPeriod	NA	
AdcGtmTimerRef	NA	
AdcNotification	IoHwAb_Adc_Notification_AdcGroup1	Group notification function name
AdcGroupReplacement	ADC_GROUP_REPL_ABORT_RESTART	Applicable only when ADC_PRIORITY_HW_SW mechanism is considered and for ADC_TRIGG_SRC_SW
AdcEruTriggRef	NA	Not applicable for SW trigger groups
AdcEruGateRef	NA	Not applicable for SW trigger groups
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	OFF	Emux Scan mode disabled

8.1.5 Configuration of Timer Trigger Group

Consider a timer trigger Group: AdcTimerTriggGrp which uses GTM timer ATOM/TOM channels to trigger the conversion on periodic basis.

Table 234 shows the example for timer trigger Group.

Table 234 Group configuration for timer trigger group

Configuration Parameter	Value	Description
Container Name	AdcTimerTriggGrp	The container name will give the Group symbolic name.
AdcGroupId	Not configurable	Numeric ID is generated by the config tool. The container name is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data.
AdcGroupPriority	NA	Not configurable parameter groups with trigger type: ADC_TRIGG_SRC_HW
AdcGroupDefinition	Reference to: AdcChannel2	This is a reference parameter to AdcChannel
AdcGroupTriggSrc	ADC_TRIGG_SRC_HW	Trigger source type is hardware
AdcGroupRequestSource	REQSRC2_BGND_SCAN	ADC HW Unit Request source 2 is selected

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
AdcHwTrigType	HW_TRIGG_TIMER	Timer signal for triggering the request source is selected.
AdcGroupConvMode	ADC_CONV_MODE_ONESHOT	One conversion round for each external event
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	2	2 samples of each channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_CIRCULAR	The new converted values are stored in result buffer by overwriting the old sets.
AdcHwTrigSignal	Not configurable	Only the raising edge of the timer signal will trigger the ADC HW Unit for conversion.
AdcHwExtTrigSelect	NA	Not required for timer trigger groups
AdcHwGateSignal	NA	Not required for timer trigger groups
AdcHwGatePin	ADCx_GTSEL_RS2_TIM_TRIG0	Selected GTM timers ATOM/TOM channels output used as gate. This pin needs to be configured for Output. ADC's RS2 TRIG0 gate signal from GTM. Selected gate signal is used as trigger source.
AdcGtmTimerPeriod	2.5	The Timer period for Group periodic conversion. Unit in milliseconds.
AdcGtmTimerRef	Reference to GtmTriggerForAdc_8	Selection of GTM ATOM/TOM channel. GtmTriggerForAdc_8 is common for all the available ADC kernels (ADC0 to ADC7) RS2 trigger.
AdcNotification	IoHwAb_Adc_Notification_AdcTimerTriggGrp	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter for groups with trigger type: ADC_TRIGG_SRC_HW
AdcEruTriggRef	NA	Not applicable for timer trigger groups
AdcEruGateRef	NA	Not applicable for timer trigger groups
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	OFF	Emux Scan mode disabled

As per the Group configuration, the GTM ATOM/TOM channel used for periodic triggering of ADC0 RS2. The ATOM/TOM channels are configured under GTM complex driver for trigger output from timer on GTM TRIG0. Refer GTM User manual [\[9\]](#)

GtmTriggerForAdc x Configuration:

Table 235 GtmTriggerForAdc_x configuration

Configuration Parameter	Value	Description
GtmTrigger0Select	TRIG_6_ATOM0_CH5	ATOM0 Channel5 timer channel used as Adc trigger.

ATOM0_CH5 Configuration:

Table 236 ATOM0_CH5 configuration

Configuration Parameter	Value
AtomChannelUsage	USED_BY_GTM_DRIVER
AtomChannelEnable	AT_START
AtomChannelOutputControl	ENABLE_OUTPUT_AT_START
AtomChannelOutputSignalLevel	HIGH
AtomChannelPortPinSelect	TOUT15_SEL_C_PORT0_PIN6
AtomChannelModeSelect	ATOM_SIG_OUTPUT_MODE_PWM
AtomChUpdateEnableOnCn0Reset	TRUE
AtomChannelClockSelect	GTM_CONFIGURABLE_CLOCK_0
AtomChannelCounterCn0Reset	ON_COMPARE_MATCH_ON_CHANNEL

Note: Other configuration parameters should be left as default values.

To get the timer output on TRIG0, the selected Port pin should be configured as direction out and pin mode as GTM_TOUT (corresponding ALT output) in Port configuration. Refer Port driver [10].

Divider values of the selected clock should be set in Gtm configuration. Refer Gtm driver [9].

8.1.6 Sample Configuration of the Driver

Requirement:

User requires one ADC Group having two ADC channels and to be triggered by external event. The result of 10 triggers needs to be converted and stored in the buffer and hence streaming result buffer – linear mode is used. The result buffer size must be $m \times n = 10 \times 2$

For optimization purpose, user chooses No priority mechanism. Since there is only one Group with hardware trigger source, it is optimal to switch OFF AdcEnableStartStopGroupApi.

The configuration of each of the container is as shown below:

Table 237 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueueing	OFF
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	OFF
AdcEnableStartStopGroupApi	OFF

Configuration Parameter	Value
AdcHwTriggerApi	ON
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcResultAccumulation	NA
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcPBFixedAddress	OFF

Table 238 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	NA
AdcEmuxSelectGrp1	NA

Table 239 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA

Table 240 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcExtEventGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel1 AdcChannel2	This is a reference parameter. The first channel in the Group definition must be the least internal channel amongst the group channels
AdcGroupTriggSrc	ADC_TRIGG_SRC_HW	Trigger source type is hardware
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	HW_TRIGG_EXT_REQ	External request for triggering the request source

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
		is selected.
AdcGroupConvMode	ADC_CONV_MODE_ONESHOT	One conversion round for each external event
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	10	10 samples of each channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	ADC_HW_TRIG_RISING_EDGE	Raising edge of the signal on trigger pin will trigger the ADC HW Unit for conversion.
AdcHwExtTrigSelect	ADC0_TRSEL_RS0_RS1_IOUT0	Selections of ERU signal for triggering the request source.
AdcHwGateSignal	ADC_GATE_SIGNAL_NONE	The gate for request source is set to be open on pending request.
AdcHwGatePin	NA	Since gate signal is none.
AdcGtmTimerPeriod	NA	Not configurable parameter for External event trigger group
AdcGtmTimerRef	NA	Not configurable parameter for External event trigger group
AdcEruTriggRef	Reference to: ERU0	This is a reference parameter. The selected ERU configurations are used for triggering the Adc group conversion.
AdcEruGateRef	NA	Not applicable if AdcHwGateSignal is ADC_GATE_SIGNAL_NONE
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	NA	Not applicable for Emux disabled groups
AdcNotification	IoHwAb_Adc_Notification_AdcExtEventGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE

Table 241 Container: AdcChannel

Configuration Parameter	AdcChannel1 value	AdcChannel2 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel1	AdcChannel2
AdcChannelId	0	1
AdcAnChannelNum	1	2

Configuration Parameter	AdcChannel1 value	AdcChannel2 value
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcChannelRefVoltsrcLow	REF_VOLTAGE_VAGND	REF_VOLTAGE_VAGND
AdcInputClassSelection	Refer to AdcKernelInputClass_0	Refer to AdcKernelInputClass_1
AdcChannelLimitCheck	NA	NA
AdcChannelHighLimit	NA	NA
AdcChannelLowLimit	NA	NA
AdcChannelRangeSelect	NA	NA

Table 242 Container: AdcKernelInputClass

Configuration Parameter	AdcKernelInputClass_0	AdcKernelInputClass_1
AdcKernelChSampleTime	10	20
AdcKernelChResolution	8-bit	10-bit
AdcKernelEmuxChSampTime	NA	NA

In Post build, there can be more than one AdcConfiguration set. User has to send the address of any one configuration set referenced by ConfigPtr as input pointer to Adc_Init API.

The API Adc_Init will initialize the ADC hardware units and driver according to the configuration set referenced by ConfigPtr.

Code Listing 4 will give the sample ADC Configuration set

Code Listing 4 Sample ADC Configuration having one set

```

0101:  /*
0102:             Container: Adc Kernel Configuration
0103:  */
0104:  /*
0105:  Configuration : Adc_KernelConfigType
0106:  Configuration of ADC:
0107:  Pointer to Adc Hw Unit configuration
0108:  Pointer to the array of channel configuration
0109:  Pointer to the array of group configuration
0110:  Pointer to the array of Group mask in each trigger source
0111:  Total number of configured groups
0112:  */
0113:  /*Configset 1 */
0114:  /* ADC 0  configuration */
0115:  static const Adc_KernelConfigType  Adc_kKernel0Config0 =
0116:  {
0117:      /* Pointer to Adc Hw Unit configuration */
0118:      &Adc_kKernel0HwCfg0,
0119:
0120:      /* Pointer to the array of channel configuration */
0121:      &Adc_kKernel0ChCfg0[0U],
0122:
0123:      /* Pointer to the array of group configuration */
0124:      &Adc_kKernel0GrpCfg0[0U],
0125:
0126:

```

Code Listing 4 Sample ADC Configuration having one set

```

0127:      /* Total number of configured groups */
0128:      1U
0129:  };/* Adcx Configuration */
0130:
0131:
0132:
0133:  /*
0134:              Container: AdcConfiguration
0135:  */
0136:  /*
0137:  Configuration : Adc_ConfigType
0138:  Configuration of ADC:
0139:  Pointer to Adc Hw Unit configuration
0140:  Pointer to the array of channel configuration
0141:  Pointer to the array of group configuration
0142:  Pointer to the array of Group mask in each trigger source
0143:  Total number of configured groups
0144:  */
0145:  const Adc_ConfigType  Adc_ConfigRoot[1] =
0146:  {
0147:      /*Configset 1 */
0148:      {
0149:          {
0150:              /* Pointer to Adc0 configuration */
0151:              &Adc_kKernel0Config0,
0152:
0153:              /* Pointer to Adc1 configuration */
0154:              NULL_PTR,
0155:
0156:              /* Pointer to Adc2 configuration */
0157:              NULL_PTR,
0158:
0159:              /* Pointer to Adc3 configuration */
0160:              NULL_PTR,
0161:
0162:              /* Pointer to Adc4 configuration */
0163:              NULL_PTR,
0164:
0165:              /* Pointer to Adc5 configuration */
0166:              NULL_PTR,
0167:
0168:              /* Pointer to Adc6 configuration */
0169:              NULL_PTR,
0170:
0171:              /* Pointer to Adc7 configuration */
0172:              NULL_PTR
0173:          },
0174:
0175:      /* Pointer to Adc Global configuration value */

```

Code Listing 4 Sample ADC Configuration having one set

```
0176:    &Adc_kGlobCfg0,
0177:    /* Module sleep mode control */
0178:    ADC_SLEEP_MODE_ACCEPT
0179:    }
0180: }; /* AdcConfiguration */
```

8.1.7 Sample Configuration to trigger HW group using PWM signal

Requirement:

User requires one ADC Group having two ADC channels and to be triggered by external event (Pwm signal). The result of 10 triggers needs to be converted and stored in the buffer and hence streaming result buffer – linear mode is used. The result buffer size must be $m \times n = 10 \times 2$

For optimization purpose, user chooses No priority mechanism. Since there is only one Group with hardware trigger source, it is optimal to switch OFF AdcEnableStartStopGroupApi.

The configuration of each of the container is as shown below:

Table 243 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	OFF
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	OFF
AdcEnableStartStopGroupApi	OFF
AdcHwTriggerApi	ON
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcResultAccumulation	NA
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcPBFixedAddress	OFF

Table 244 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	NA

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcEmuxSelectGrp1	NA

Table 245 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA

Table 246 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcExtPwmGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel1 AdcChannel2	This is a reference parameter. The first channel in the Group definition must be the least internal channel amongst the group channels
AdcGroupTriggSrc	ADC_TRIGG_SRC_HW	Trigger source type is hardware
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	HW_TRIGG_EXT_REQ	External request for triggering the request source is selected.
AdcGroupConvMode	ADC_CONV_MODE_ONESHOT	One conversion round for each external event
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	10	10 samples of each channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	ADC_HW_TRIG_RISING_EDGE	Raising edge of the signal on trigger pin will trigger the ADC HW Unit for conversion.
AdcHwExtTrigSelect	ADC_USE_GATE_PIN_TO_TRIG	Selected gate signal is used as trigger source.
AdcHwGateSignal	NA	The gate for request source is set to be open on pending request.
AdcHwGatePin	ADC0_GTSEL_RS0_RS1_TIM_TRIG0	Selected GTM timers ATOM/TOM channels output used as gate. This pin needs to be configured for Output for the PWM signal to trigger the Adc group.

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
AdcGtmTimerPeriod	NA	Not configurable parameter for External event trigger group
AdcGtmTimerRef	NA	Not configurable parameter for External event trigger group
AdcEruTriggRef	NA	Not configurable parameter if AdcHwExtTrigSelect is ADC_USE_GATE_PIN_TO_TRIG
AdcEruGateRef	NA	Not applicable if AdcHwGateSignal is ADC_GATE_SIGNAL_NONE
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	NA	Not applicable for Emux disabled groups
AdcNotification	IoHwAb_Adc_Notification_AdcExtEventGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE

Table 247 Container: AdcChannel

Configuration Parameter	AdcChannel1 value	AdcChannel2 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel1	AdcChannel2
AdcChannelId	0	1
AdcAnChannelNum	1	2
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcChannelRefVoltsrcLow	REF_VOLTAGE_VAGND	REF_VOLTAGE_VAGND
AdcInputClassSelection	Refer to AdcKernelInputClass_0	Refer to AdcKernelInputClass_1
AdcChannelLimitCheck	NA	NA
AdcChannelHighLimit	NA	NA
AdcChannelLowLimit	NA	NA
AdcChannelRangeSelect	NA	NA

Table 248 Container: AdcKernelInputClass

Configuration Parameter	AdcKernelInputClass_0	AdcKernelInputClass_1
AdcKernelChSampleTime	10	20
AdcKernelChResolution	8-bit	10-bit
AdcKernelEmuxChSampTime	NA	NA

As per the Group configuration, the GTM ATOM/TOM channel used for periodic triggering of ADC0 RS1. Here the periodic time is defined by the period of the PWM signal. The ATOM/TOM channels are configured in GTM to trigger output from PWM signal on GTM TRIG0. Refer GTM User manual [9] and PWM user manual [12].

Pwm Configuration:

Table 249 PwmGeneral configuration

Configuration Parameter	Value
-------------------------	-------

Configuration Parameter	Value
PwmSetDutyCycle	ON
PwmDeInitApi	ON

Table 250 PwmChannel configuration

Configuration Parameter	Value
PwmChannelClass	PWM_FIXED_PERIOD
PwmChannelId	0
PwmDutycycleDefault	16384 (50%)
PwmIdleState	PWM_HIGH
PwmPeriodDefault	10000 (Actual time period required to trigger ADC group)
PwmPolarity	PWM_HIGH
PwmAssignedHwUnit	Reference to TOM0_CH6

Note: Other configuration parameters can be left as default values. User should initialize the Pwm driver to start the Pwm signal on TOM0_CH6 output pin.

GtmTriggerForAdc_x Configuration:

Table 251 GtmTriggerForAdc_x configuration

Configuration Parameter	Value	Description
GtmTrigger0Select	TRIG_1_TOM0_CH6	TOM0 Channel6 timer channel used as Adc trigger.

TOM0_CH6 Configuration:

Table 252 TOM0_CH6 configuration

Configuration Parameter	Value
TomChannelUsage	USED_BY_OTHER_DRIVERS_OR_UNUSED
TomChannelPortPinSelect	TOUT106_SELA_PORT10_PIN4
TomChannelClockSelect	GTM_FIXED_CLOCK_0

Note: Other configuration parameters should be left as default values.

To get the timer output on TRIG0, the selected Port pin should be configured as direction out and pin mode as GTM_TOUT (corresponding ALT output) in Port configuration. Refer Port driver [10].

Divider values of the selected clock should be set in Gtm configuration. Refer Gtm driver [9].

8.1.8 Sample Configuration of limit checking

Requirement:

User requires one ADC Group having one limit checking enabled ADC channel to be triggered by software and in continuous conversion mode.

8.1.8.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 253 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcEnableLimitCheck	ON
AdcResultAlignment	ADC_ALIGN_LEFT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcResultAccumulation	None
AdcPBFixedAddress	OFF

Table 254 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	NA
AdcEmuxSelectGrp1	NA

Table 255 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA

Table 256 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcStdLimitChkGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
		in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel5	This is a reference parameter.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConvMode	ADC_CONV_MODE_CONTINUOUS	Continuous conversion for the SW trigger
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	2	2 samples of a channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	NA	Not applicable for Emux disabled groups
AdcNotification	IoHwAb_Adc_Notification_AdcLimitChkGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE

Table 257 Container: AdcChannel

Configuration Parameter	AdcChannel5 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel5

Configuration Parameter	AdcChannel5 value
AdcChannelId	0
AdcAnChannelNum	5
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF
AdcInputClassSelection	Refer to AdcGlobInputClass_0
AdcChannelLimitCheck	ON
AdcChannelHighLimit	1023
AdcChannelLowLimit	512
AdcChannelRangeSelect	ADC_RANGE_BETWEEN

Table 258 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	2
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	NA

Code Listing 5 Sample Standard compare mode ADC Channel Configuration

```

0181:  /*
0182:                                     Container: AdcChannelConfiguration
0183:  */
0184:  /*
0185:  Configuration: ADC Channel configuration structure
0186:  */
0187:  /*Configuration set 1 */
0188:  /* ADC 0 Channel configuration */
0189:  static const Adc_ChannelCfgType Adc_kKernel0ChCfg0[]=
0190:  {
0191:  {
0192:      /* AdcChannel5 configuration */
0193:      Adc_lConfigChannel(
0194:          ADC_REF_VOLTAGE_VAREF,
0195:          (2U),
0196:          (0x5U)
0197:      ),
0198:      ADC_RES_ACCUMULATION_NONE,
0199:      ADC_CHANNEL_LIMIT_CHECK,
0200:      (uint32)ADC_RANGE_BETWEEN,
0201:      Adc_lCfgChBoundry(
0202:          ((uint32)1023 << (uint32)4),
0203:          ((uint32)512 << (uint32)4)
0204:      ),
0205:      BNDSELU_BNDSELL /* value of BNDSELX bitfields */
0206:  }
0207: }; /* Adc0_ChCfg0 */

```

8.1.8.2 Working of limit checking

Standard compare mode limit checking compares the converted result value against the user configured boundary values and updates the result buffer with the converted value if the converted value is within the configured range.

The **Figure 13** explains the result monitoring of standard compare limit checking.

As per configuration **Table 257**,

- Upper Boundary is 1023.
- Lower Boundary is 512.
- Range is between the boundary values.
- If the conversion is within the boundary values, result buffer is updated with the converted result, group status is updated and notification function is been called if enabled.
- As per **Figure 13** conversions Conv 2 & Conv 4 results are within the configured range. Group status and the result buffer are updated in these conversions.
- Conv 1 & Conv 3 results are out of the configured range and the conversion is neglected.

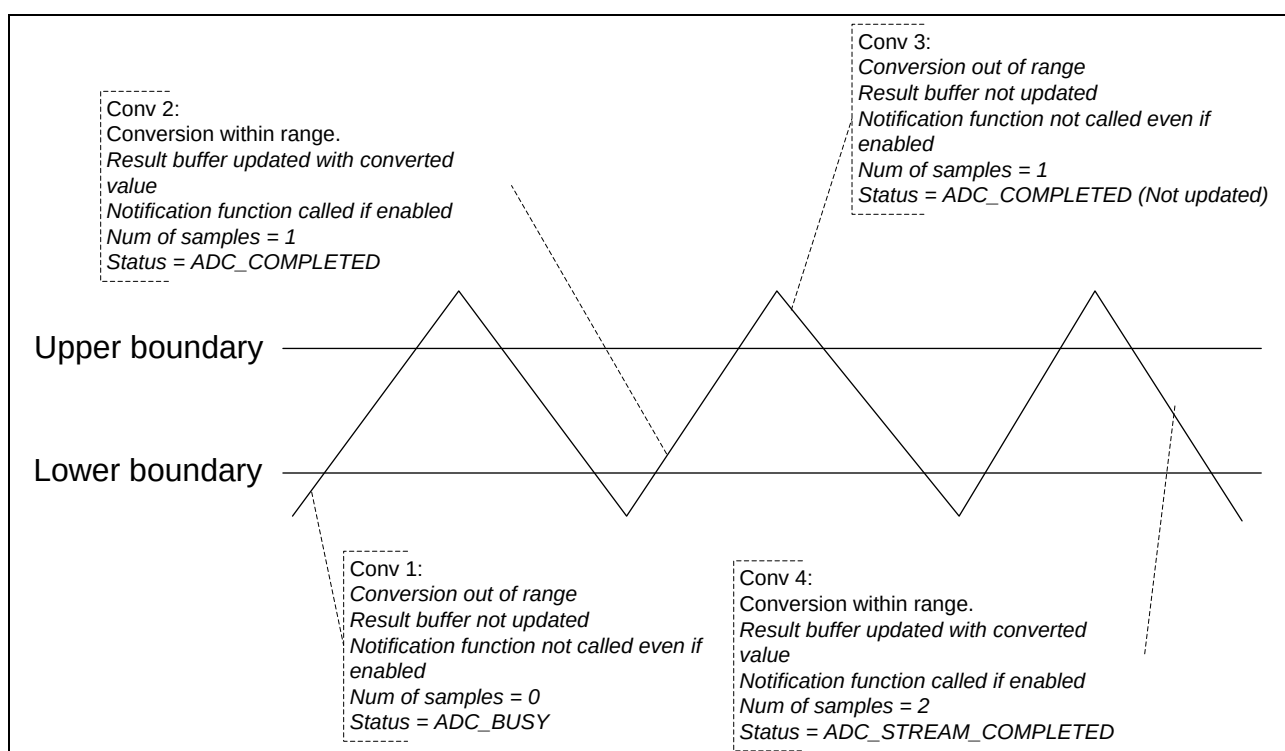


Figure 13 Result Monitoring through Standard compare mode limit checking

8.1.9 Sample Configuration of the Driver in Non AUTOSAR Result handling mode

Requirement:

User requires one ADC Group having two ADC channels to be triggered by software and in continuous conversion mode.

DMA Move Engine 0 Channel 2 is required for ADC result transfer.

ADC Group result to be at variable "AdcResult []"

For Non Autosar result handling mode the priority is ADC_PRIORITY_NONE.

The configuration of each of the container is as shown below:

8.1.9.1 ADC Driver Configuration

Table 259 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	NA
AdcNonAutosarDmaResultHandling	ON
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcResultAccumulation	None
AdcPBFixedAddress	OFF

Table 260 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	NA
AdcEmuxSelectGrp1	NA

Table 261 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	Reference to DMA_CHANNEL4

CONFIDENTIAL
Example Usage
Table 262 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcSwTriggGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel5 AdcChannel0	This is a reference parameter. The channels are arranged in Descending order.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConversionMode	ADC_CONV_MODE_ONESHOT	One Shot conversion for the SW trigger. Continuous conversion for the SW trigger is not supported in Non Autosar DMA result handling mode
AdcGroupAccessMode	NA	Not configurable parameter in Non AUTOSAR Result handling mode
AdcStreamingNumSamples	NA	Not configurable parameter in Non AUTOSAR Result handling mode
AdcStreamingBufferMode	NA	Not configurable parameter in Non AUTOSAR Result handling mode
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	NA	Not applicable for Emux disabled groups
AdcNotification		No notification function for this Group

Configuration Parameter	Value	Description
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE

Table 263 Container: AdcChannel

Configuration Parameter	AdcChannel5 value	AdcChannel0 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel5	AdcChannel0
AdcChannelId	1	0
AdcAnChannelNum	5	0
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcInputClassSelection	Refer to AdcGlobInputClass_0	Refer to AdcGlobInputClass_0

Table 264 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	10
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	NA

8.1.9.2 Interrupt priority Configuration

Interrupt priorities are configured in Irq module. Refer [\[11\]](#).

Table 265 Container: IrqAdc0PrioConfig

Configuration Parameter	Value
IrqAdc0SR0Prio	NA
IrqAdc0SR1Prio	NA
IrqAdc0SR2Prio	NA
IrqAdc0SR3Prio	4 (Configured Dma channel number, Eg: DMA_CHANNEL0 = 0 DMA_CHANNEL1 = 1 DMA_CHANNEL2 = 2 DMA_CHANNEL3 = 3 DMA_CHANNEL4 = 4)

Table 266 Container: IrqAdc0TosConfig

Configuration Parameter	Value
IrqAdc0SR0Tos	NA
IrqAdc0SR1Tos	NA
IrqAdc0SR2Tos	NA
IrqAdc0SR3Tos	DMA

Table 267 Container: IrqDmaPrioConfig

Configuration Parameter	Value
-------------------------	-------

Configuration Parameter	Value
IrqDmaCh4SRPrio	50 (Non zero value)

8.1.9.3 DMA Library function

The DMA Library has to be used by the user in Non AutoSAR Result handling mode. For this example, DMA Channel 4 has to be passed as parameter to DMA library functions.

Code Listing 6 Example – Dma library function call in Non Autosar result handling

```

0208: /* Non Autosar result handling */
0209: #define ADC_DMA_ADR_CTR      ((uint32)0x0000FF89)
0210:
0211: uint32 Address;
0212: uint32 SrcAddress;
0213: Dma_ChannelType DmaChannel;
0214:
0215: /* Set Source Address */
0216: SrcAddress = (uint32)&(VADC_G0RES0.Reg);
0217:
0218: DmaChannel = DMA_CHANNEL4;
0219:
0220: Address = (uint32)&Adc_NonASDmaBuffer[0U];
0221:
0222: /* Set Source Address */
0223: Mcal_DmaSetSourceAddr(DmaChannel, SrcAddress);
0224:
0225: /* Set Destination Address */
0226: Mcal_DmaSetDestAddr(DmaChannel, Address);
0227:
0228: /* Clear CHCR register */
0229: Mcal_DmaChClrCtlCfg(DmaChannel);
0230:
0231: /* Clear interrupt on DMA Channel */
0232: Mcal_DmaChClrIntrFlags(DmaChannel);
0233:
0234: /* Set Interrupt Control and Interrupt Pointer */
0235: Mcal_DmaCfgIntControl(DmaChannel, 2U);
0236:
0237: /* Set DMA Channel control register */
0238: /* After the configured number of transfer, DMA interrupt is generated
    */
0239: Mcal_DmaSetDataWidth(DmaChannel, 1U);
0240: Mcal_DmaSetTxCount(DmaChannel, 1U);
0241:
0242: Mcal_DmaCfgNoOfMovesPerTransfer(DmaChannel, 2U);
0243: /* Enable DMA Interrupt */
0244: Mcal_DmaEnableChIntrTrigger(DmaChannel);
0245: Mcal_DmaEnableIntr(DmaChannel);
0246:
0247: /* DMA Address control */
0248: Mcal_DmaSetAdicr(DmaChannel, ADC_DMA_ADR_CTR);

```

Code Listing 6 Example – Dma library function call in Non Autosar result handling

```

0249:
0250:  /* Enable ADC HW trigg on selected DMA Channel */
0251:  Mcal_DmaEnableHwTransfer(DmaChannel);
0252:
0253:  /* Start the AdcSWGroup group */
0254:  Adc_StartGroupConversion(AdcSWGroup);
0255:
0256:  Adc_EnableGroupNotification(AdcSWGroup);
0257:
0258:  while(Mcal_DmaGetTransferIntStatus(DmaChannel));
0259:
0260:  while( (Adc_GetGroupStatus(AdcSWGroup)) != ADC_STREAM_COMPLETED);
0261:
0262:  Adc_DisableGroupNotification(AdcSWGroup);
0263:  /* Print the latest converted result of the channels as pointed by
0264:     Demo_ResultPtr
0265:  */
0266:  printf("\nADC Conversion Results (Raw Value): \n");
0267:  printf("AN0 : %4x\n",Adc_NonASDmaBuffer[0]);
0268:  printf("AN1 : %4x\n",Adc_NonASDmaBuffer[1]);
0269:
0270:  /* Check if the Group has implicitly stopped and status is IDLE
0271:     before Stop request
0272:  */
0273:  if( (Adc_GetGroupStatus(AdcSWGroup)) != ADC_IDLE)
0274:  {
0275:      /* Stop the conversion which will reset the status and result buffer
0276:         and notification is disabled
0277:      */
0278:      Adc_StopGroupConversion(AdcSWGroup);
0279:  }
0280:
0281:  /* Disable ADC HW trigg on DMA Channel */
0282:  Mcal_DmaDisableHwTransfer(DmaChannel);

```

8.2 Initialization of ADC Driver and initialization Group result buffer pointers

ADC Hardware and Driver are initialized by Adc_Init API call. In Post build, the input parameter i.e. ConfigPtr will point to the configuration set for which the Initialization is required.

After Initializing the Driver, the user needs to configure the Group result buffer pointer via Adc_SetupResultBuffer which is used by ADC Driver. The ADC Driver stores all group conversion results to result buffer addressed with the result buffer pointer. One buffer is required for each group. The buffer size depends on the number of group channels, the group access mode and from the number of streaming samples, if streaming access mode is selected.

User should ensure that no group conversions are started without prior initialization of the result buffer.

Code Listing 7 gives the example Driver initialization and Result pointer initialization

Code Listing 7 Example – Driver Initialization and Result pointer initialization

```

0283:  /* Result buffer definitions for each ADC Group */

```

Code Listing 7 Example – Driver Initialization and Result pointer initialization

```

0284:
0285:  /* Result buffer for AdcGroup0 */
0286:  /* Streaming access result group: 4*2 */
0287:  Adc_ValueGroupType ADC_SW_GRP_RES[8];
0288:
0289:  /* Result buffer for AdcGroup1 */
0290:  /* Single access result group: 1*1 */
0291:  Adc_ValueGroupType ADC_CHIN_GRP_RES;
0292:
0293:  /* Result buffer for AdcGroup2 */
0294:  /* Streaming access result group: 10*1 */
0295:  Adc_ValueGroupType ADC_TIM_GRP_RES[10];
0296:
0297:  /* Initialize the ADC Hw unit and Driver */
0298:  Adc_Init(&Adc_ConfigRoot[0]);
0299:
0300:  /* Initialize the Group result buffers */
0301:  Adc_SetupResultBuffer(AdcGroup0,ADC_SW_GRP_RES);
0302:  Adc_SetupResultBuffer(AdcGroup1,ADC_CHIN_GRP_RES);
0303:  Adc_SetupResultBuffer(AdcGroup2,ADC_TIM_GRP_RES);

```

8.3 Configuration and Usage of ADC Driver for ADC_PRIORITY_NONE and No Queue

1. The user can disable the ADC Driver priority mechanism as a whole and can set AdcEnableQueuing OFF. In this scenario, all ADC Channel Groups get the same priority. Also, there is no suspending/aborting of ongoing conversions. Queuing mechanism is disabled and hence only one conversion is handled at a time.
2. AURIX ADC conversion request source arbitration levels are fixed to same priority level REQSRC_PRIORITY_0_LOWEST so that they don't participate in the ADC HW unit source arbitration. ADC request source priority levels are not a configurable parameter for user.
3. All the Groups will have the same priority level and hence AdcGroupPriority [\[4.5.15.4\]](#) and AdcGroupReplacement [\[4.5.15.5\]](#) are not a configurable parameter by the user.
4. In this mechanism, conversion of ADC Group should be started one at a time. When once a group is started and if second request for conversion of any group occurs, the request will not be handled, instead a DET: ADC_E_BUSY will be reported.
5. A software trigger group configured for Oneshot, and having a group status of ADC_STREAM_COMPLETED is allowed to start the conversion without stopping the group (using Stop API). After start, the status of the Group is set to ADC_BUSY and the conversion result for previous conversion is not available for the user.
6. A conversion request for a group configured in ADC_TRIGG_SRC_SW, ADC_CONV_MODE_CONTINUOUS, ADC_ACCESS_MODE_SINGLE / ADC_ACCESS_MODE_STREAMING - ADC_STREAM_BUFFER_CIRCULAR needs to be stopped before requesting conversion for another group, even when the Group status is ADC_STREAM_COMPLETED.
7. A conversion request for a group configured in ADC_TRIGG_SRC_HW, ADC_ACCESS_MODE_SINGLE / ADC_ACCESS_MODE_STREAMING - ADC_STREAM_BUFFER_CIRCULAR needs to be disabled before requesting conversion for another group, even when the Group status is ADC_STREAM_COMPLETED.
8. A conversion request for a group configured in ADC_TRIGG_SRC_HW, ADC_ACCESS_MODE_STREAMING - ADC_STREAM_BUFFER_LINEAR and having Group status ADC_STREAM_COMPLETED is allowed to Enable the conversion without Disabling the group (using

Disable API). After Enable, the status of the Group is set to ADC_BUSY and the conversion result for previous conversion is not available for the user.

Using Static configuration parameter, priority mechanism is switched to ADC_PRIORITY_NONE and Queue disabled

Code Listing 8 Switching Priority mechanism to No Priority and Queue disabled

```
0304:  /*
0305:                Container : AdcDriverGeneralConfiguration
0306:  */
0307:
0308:  /* Configuration: ADC_PRIORITY_IMPLEMENTATION
0309:  Determines the type of prioritization mechanism
0310:  */
0311:  #define ADC_PRIORITY_IMPLEMENTATION  (ADC_PRIORITY_NONE)
0312:
0313:  /* Configuration: ADC_ENABLE_QUEUEING
0314:  Determines, if the queuing mechanism is active in case of
0315:  priority mechanism disabled.
0316:  Note: If priority mechanism is enabled,
0317:  queuing mechanism is always active and the parameter
0318:  ADC_ENABLE_QUEUEING is not evaluated.
0319:  - if ON, Queuing mechanism in no priority is Enabled
0320:  - if OFF, Queuing mechanism in no priority is Disabled
0321:  */
0322:  #define ADC_ENABLE_QUEUEING  (STD_OFF)
```

8.4 Configuration and Usage of ADC Driver for ADC_PRIORITY_NONE and with Queue

1. The user can disable the ADC Driver priority mechanism as a whole and can set AdcEnableQueuing ON. In this scenario, all ADC Channel Groups get the same priority. Also, there is no suspending/aborting of ongoing conversions. Queuing mechanism is enabled and hence queuing of Groups with ADC_TRIGG_SRC_SW conversion requests is possible and will service the groups in a 'first come first served' order.
2. Groups with ADC_TRIGG_SRC_HW can not enter the Queue. The Adc_EnableHardwareTrigger request for these Groups is only serviced when ADC Hardware unit is free and Software Queue is empty.
3. AURIX ADC conversion request source arbitration levels are fixed to same priority level REQSRC_PRIORITY_0_LOWEST so that they don't participate in the ADC HW unit source arbitration. ADC request source priority levels are not a configurable parameter for user.
4. All the Groups will have the same priority level and hence AdcGroupPriority [4.5.15.4] and AdcGroupReplacement [4.5.15.5] are not a configurable parameter for the user.
5. In this mechanism, all the ADC Groups with TRIGG_SRC_SW can be requested for conversion. They are serviced in 'first come first served' order.
6. A software trigger group configured for Oneshot, and having a group status of ADC_STREAM_COMPLETED is allowed to start the conversion without stopping the group (using Stop API). After start, the status of the Group is set to ADC_BUSY and the conversion result for previous conversion is not available for the user.
7. A conversion request for a group configured in ADC_TRIGG_SRC_SW, ADC_CONV_MODE_CONTINUOUS, ADC_ACCESS_MODE_SINGLE / ADC_ACCESS_MODE_STREAMING - ADC_STREAM_BUFFER_CIRCULAR needs to be stopped for the next group in Queue to start A/D conversion, even when the Group status is ADC_STREAM_COMPLETED.
8. A conversion request for a group configured in ADC_TRIGG_SRC_HW, ADC_ACCESS_MODE_SINGLE / ADC_ACCESS_MODE_STREAMING - ADC_STREAM_BUFFER_CIRCULAR needs to be disabled for

CONFIDENTIAL
Example Usage

the next group in Queue to start A/D conversion, even when the Group status is ADC_STREAM_COMPLETED.

9. A conversion request for a group configured in ADC_TRIGG_SRC_HW, ADC_ACCESS_MODE_STREAMING - ADC_STREAM_BUFFER_LINEAR and having Group status ADC_STREAM_COMPLETED and when the Queue is empty, it is allowed to enable the conversion without disabling the group (using Disable API). After Enable, the status of the Group is set to ADC_BUSY and the conversion result for previous conversion is not available for the user.

Using Static configuration parameter, priority mechanism is switched to ADC_PRIORITY_NONE and Queue enabled

Code Listing 9 Switching Priority mechanism to No Priority and Queue enabled

```

001:  /*
002:                               Container : AdcDriverGeneralConfiguration
003:  */
004:
005:  /* Configuration: ADC_PRIORITY_IMPLEMENTATION
006:  Determines the type of prioritization mechanism
007:  */
008:  #define ADC_PRIORITY_IMPLEMENTATION  (ADC_PRIORITY_NONE)
009:  /* Configuration: ADC_ENABLE_QUEUEING
010:  Determines, if the queuing mechanism is active in case of
011:  priority mechanism disabled.
012:  Note: If priority mechanism is enabled,
013:  queuing mechanism is always active and the parameter
014:  ADC_ENABLE_QUEUEING is not evaluated.
015:  - if STD_ON, Queuing mechanism in no priority is Enabled
016:  - if STD_OFF, Queuing mechanism in no priority is Disabled
017:  */
018:  #define ADC_ENABLE_QUEUEING  (STD_ON)

```

Figure 14 shows the order in which ADC Driver schedules the software Groups in no priority and Queue enabled mechanism

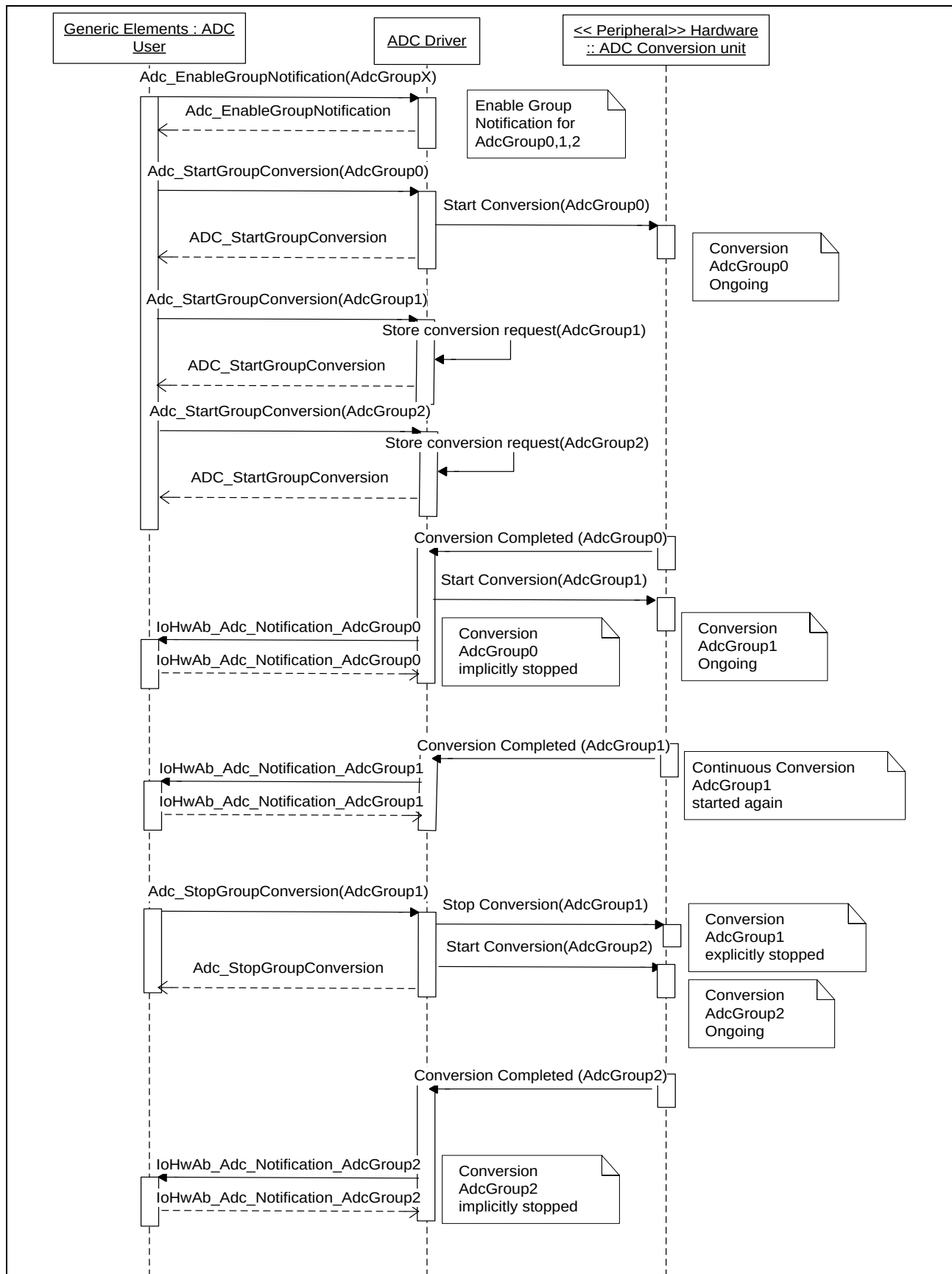


Figure 14 ADC_PRIORITY_NONE and Queuing Enabled mechanism

8.5 Configuration and Usage of ADC Driver in ADC_PRIORITY_HW mechanism

1. As an optimization, the user of the ADC Driver can restrict the range of available priorities to those that are already provided by the underlying hardware.
2. ADC Driver for AURIX ADC HW unit uses 3 conversion request sources and arbitration of the conversion is handled by the hardware. However, the priority of each conversion request source can be programmed individually. The priority of a request determines the order in which pending conversion requests from different sources are performed.
3. ADC Groups under the same request source will have same priority level and hence `AdcGroupPriority` [4.5.15.4] and `AdcGroupReplacement` [4.5.15.5] is not a configurable parameter for the user.
4. Groups under different conversion request sources can be started simultaneously. Hence, maximum of 3 Adc Groups can be requested for conversion simultaneously and these groups should be configured for different conversion request source.
5. The higher priority group in continuous conversion needs to be stopped using Stop/Disable API, for next priority level group to be converted.
6. ADC Groups configured for different Conversion trigger sources `AdcHwUnitTriggSrc` but sharing some of the ADC internal channels can not be started simultaneously. In such a case DET: ADC_E_BUSY is raised.

Using Static configuration parameter, priority mechanism is switched to ADC_PRIORITY_HW

Code Listing 10 Switching Priority mechanism to Hardware Priority

```
001:  /*
002:                Container : AdcDriverGeneralConfiguration
003:  */
004:  /* Configuration: ADC_PRIORITY_IMPLEMENTATION
005:  Determines the type of prioritization mechanism
006:  */
007:  #define ADC_PRIORITY_IMPLEMENTATION  (ADC_PRIORITY_HW)
```

8.6 Configuration and Usage of ADC Driver in ADC_PRIORITY_HW_SW mechanism

1. The ADC Driver priority mechanism will allow configuration up to 256 priority levels (0..255). AURIX hardware will not fully support the prioritization mechanism and hence this priority mechanism is handled by implemented software along with the available priorities to those that are already provided by the underlying hardware.
2. ADC Driver for AURIX ADC HW unit uses 3 conversion request sources and arbitration of the conversion is handled by the hardware. However, the priority of each conversion request source can be programmed individually. The priority of a request source determines the order in which pending conversion requests from different sources are performed.
3. Additionally, the configuration of Priority levels for each group becomes necessary by the configuration parameter `AdcGroupPriority` [4.5.15.4] and `AdcGroupReplacement` [4.5.15.5]. For `AdcGroupPriority`, the lowest priority is 0.
4. The combination of request source arbitration level and `AdcGroupPriority` forms the effective Group priority [8.6.1]
5. Groups under different conversion request sources can be started simultaneously. Hence, maximum of 3 Adc Groups of different trigger sources can be enabled in hardware for conversion simultaneously.
6. Groups under same trigger source are arbitrated by the implemented software. Higher priority Groups will always be enabled in the Hardware and the rest will be in implemented software queue.
7. Higher priority Group will interrupt the ongoing lower priority group conversion. In such situation, the lower priority group can be aborted.

8. Groups with ADC_TRIGG_SRC_HW can not enter the software implemented Queue. Hence Adc_EnableHardwareTrigger request for these Groups are only serviced when its respective trigger source is free.
9. If many ADC Groups of **same priority** are in software queue, the order in which the groups are serviced for start of conversion is by the AdcGroupID and not by First Come First Service basis. The requirement ADC417 is deviated to minimize the ISR overload time.

For Example,

AdcGroup0 – Group Priority 2, Group ID = 0 (ADC_TRIGG_SRC_SW)

AdcGroup1 – Group Priority 2, Group ID = 1 (ADC_TRIGG_SRC_SW)

AdcGroup2 – Group Priority 4, Group ID = 2 (ADC_TRIGG_SRC_SW)

ADC HW unit is busy with the conversion of AdcGroup2. Request for AdcGroup0 is called first and is put in software queue. Next request for AdcGroup1 is also put into the software queue. On conversion complete/ stopping the group conversion for AdcGroup2, ADC Driver will schedule next group from software queue. Even though AdcGroup0 is requested before AdcGroup1, AdcGroup1 is scheduled next as its Group ID is greater than the group ID of AdcGroup0.

10. The ADC Driver will not store software conversion requests for a group, whose group status is not equal to ADC_IDLE.
11. It is important that the user has to stop the higher priority group in continuous conversion using Stop/Disable API, to give the chance for next priority level group to be converted.
12. ADC Groups sharing some of the ADC internal channels can not serviced by Start/Enable request. In such a case DET: ADC_E_BUSY is raised.

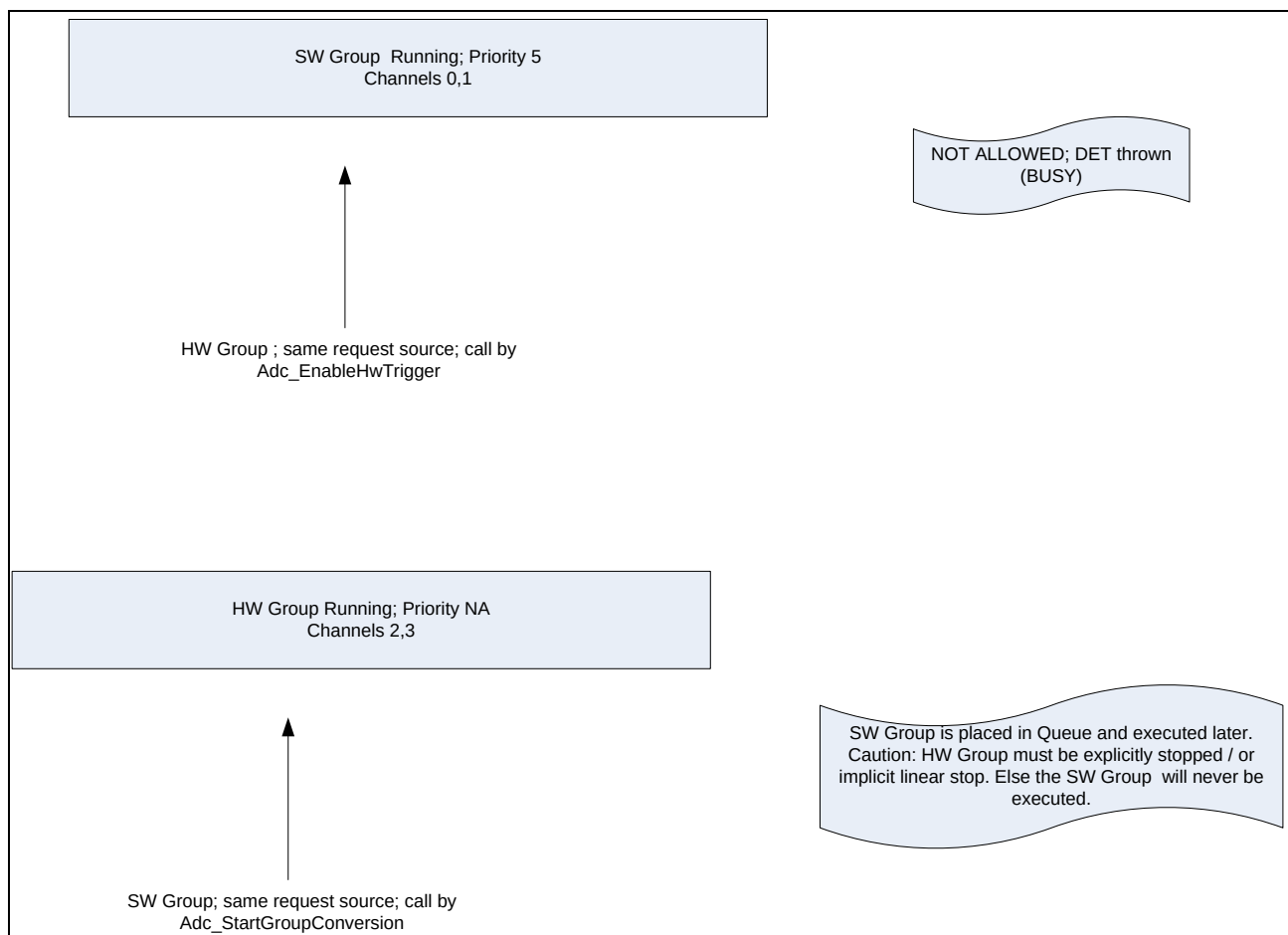


Figure 15 Interruption of Groups in HW_SW Priority

Using static configuration parameter, priority mechanism is switched to ADC_PRIORITY_HW_SW.

Code Listing 11 Switching Priority Mechanism to HW_SW Priority

```
008:  /*
009:           Container : AdcDriverGeneralConfiguration
010:  */
011: 061:  /* Configuration: ADC_PRIORITY_IMPLEMENTATION
012: 062:  Determines the type of prioritization mechanism
013: 063:  */
014: 064:  #define ADC_PRIORITY_IMPLEMENTATION (ADC_PRIORITY_HW_SW)
```

8.6.1 Effective Group Priority in ADC_PRIORITY_HW_SW mechanism

Effective Group priority is the combination of Group ADC HW Unit specific request source arbitration levels as configured [4.5.5], [4.5.6] and AdcGroupPriority [4.5.15.4]

Each Group under different HW Unit specific request sources can be enabled in the ADC Peripheral. The ADC Peripheral will arbitrate as per the ADC HW Unit specific request source arbitration level.

The priority of a request source is named as source arbitration level and it determines the order in which pending conversion requests from different sources are performed.

Each source should have an individual priority level. Nevertheless, if several conversion request sources have been programmed to the same priority level, the first detected source within this group of identical levels is taken into account. The Groups under same ADC HW Unit specific request source are arbitrated by ADC Driver software.

For Example:

Table 268 Example to show the effective Group priority

AdcGroupId / ADC Group Symbolic name	Group request source	Group trigger source arbitration level (x)	AdcGroupPriority (y)	Effective priority E = Concatenate(x,y)
AdcGroup4	RS2	0 (Lowest)	4	04
AdcGroup1	RS1	1	8	18
AdcGroup2	RS0	3 (Highest)	9	39

From Table 268 we can say that AdcGroup2 is of highest priority and AdcGroup4 is of lowest priority.

8.7 Adc Group Replacement mechanism in ADC_PRIORITY_HW_SW

Abort/Restart mechanism is applicable for Groups with in the same HW unit specific trigger source amongst ADC_TRIGG_SRC_SW [4.5.15.6]

8.7.1 Abort/Restart mechanism

In Abort/Restart mechanism, the complete conversion round of the interrupted group (all group channels) is restarted after the higher priority group conversion is finished.

For Example:

Consider two groups in oneshot conversion mode and with ADC_TRIGG_SRC_SW are as configured below for AdcGroupReplacement in ADC_PRIORITY_HW_SW:

Table 269 Example for ADC Group configuration in ADC_PRIORITY_HW_SW

AdcGroupId / Group Symbolic name	Group channels	Effective Group priority	AdcGroupReplacement mechanism
AdcGroup0	C0,C1,C2,C3,C4,C5	Low	ADC_GROUP_REPL_ABORT_RESTART
AdcGroup1	C6,C7,C9	High	ADC_GROUP_REPL_ABORT_RESTART

When AdcGroup0 conversion started, AdcGroup1 is requested by the user. Since AdcGroup1 priority is higher as compared to AdcGroup0, the Conversion round of the interrupted Group i.e. AdcGroup0 is aborted from conversion. Only when once the conversion of AdcGroup1 is completed, the AdcGroup0 is restarted the conversion by the Driver for all Group channels. After the completion of AdcGroup0 conversion, the results of all the channels of conversion round are available to the user.

Figure 16 explains the example scenario

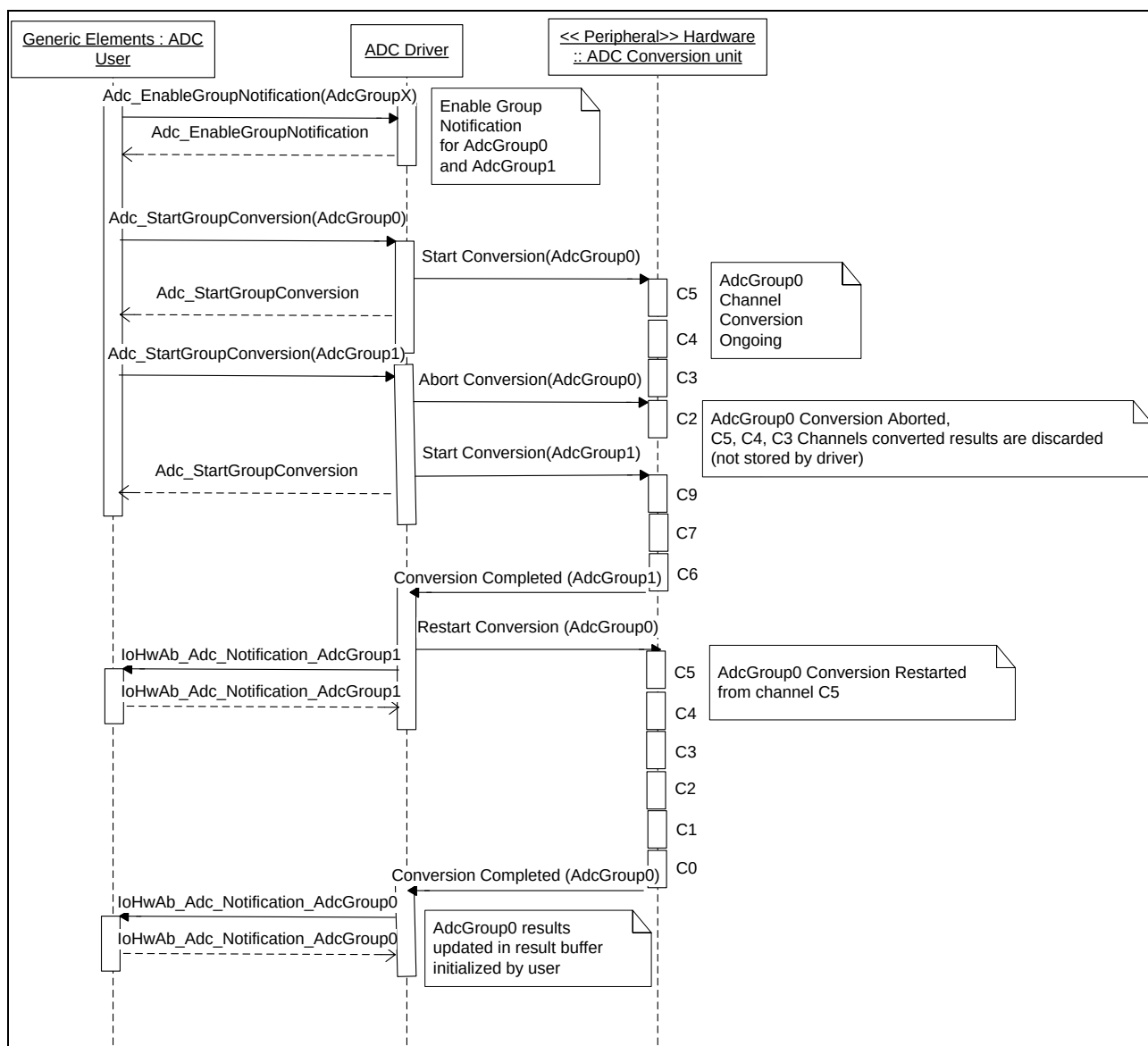


Figure 16 Abort/Restart mechanism

8.8 Group Result Access using `Adc_GetStreamLastPointer` API

ADC result buffer is of type `Adc_ValueGroupType` and will be of Size: $M \times N$.

M: Number of samples acquired per channel

N: Number of channels belonging to the group

This $M \times N$ buffer should be initialized by the user (both in normal and streaming result access mode) using `Adc_SetupResultBuffer` API [\[6.2.2\]](#)

Channel results are arranged in the order of the Group definition `AdcGroupDefinition` [\[4.5.15.5\]](#) and ADC Group result buffer will be pointing to the first channel in the Group definition

For Example:

M = 4 samples per channel, N = 10 channels configured for a Group

Initial buffer pointer (Base address), initialized by the user (`DataBufferPtr`) will point to the first channel of the group

After sampling, the pointer will point to the latest conversion result of the first channel.

When Group in `ADC_IDLE/ADC_BUSY` state, the last conversion result location will be set to `NULL_PTR` and number of conversion result available will be 0.

The user can access the result buffer using `Adc_GetStreamLastPointer` API. The results are given below.

- Before conversion start; `SamplePtr` = `NULL` and return value = 0
- On one round of conversion completion; `SamplePtr` = Base Buffer Pointer and the return value = 1
- On second round of conversion completion; `SamplePtr` = Buffer Pointer 2 and return value = 2
- On fourth round of conversion completion; `SamplePtr` = Buffer Pointer 4 and return value = 4

Using the sample pointer from `Adc_GetStreamLastPointer` API, the user can access the latest conversion results of all the channels of Group and the history of certain number of conversions.

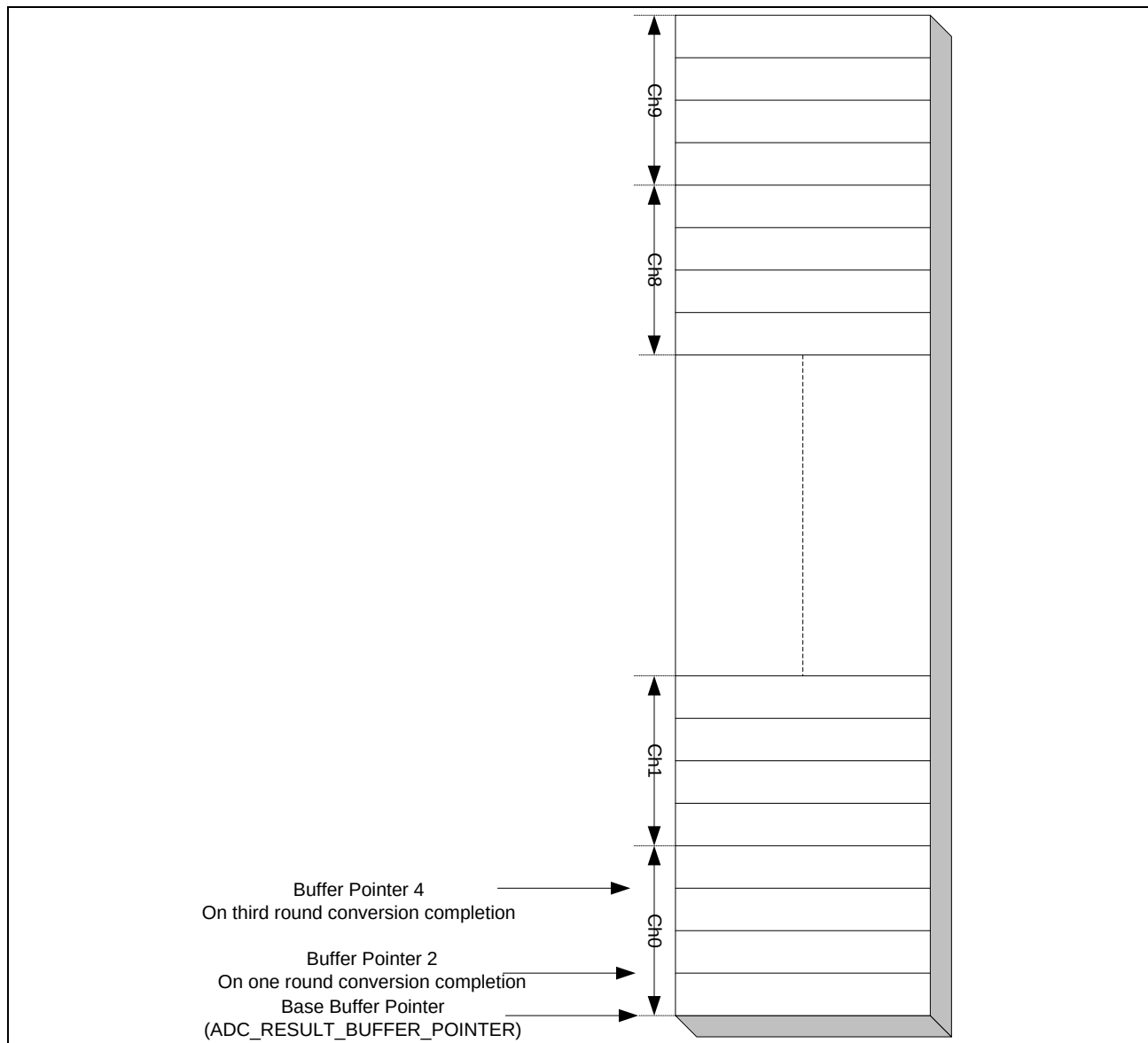


Figure 17 ADC Group Result Buffer Access

8.9 Configuration and Usage of ADC Driver in EMUX mode

1. The EMUX mode group works as a normal Autosar Adc channel group. The only difference is, an external analog channel is connected through EMUX.
2. In this feature, on each conversion round, the EMUX select line is kept constantly in the configured value. This will facilitate to select the same analog channel connected to the multiplexer always.

8.9.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 270 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcUseEmux	ON
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcResultAccumulation	None
AdcPBFixedAddress	OFF

Table 271 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	HWUNIT_ADC0
AdcEmuxSelectGrp1	HWUNIT_ADC1

Table 272 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA

Table 273 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcEmuxGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to:	This is a reference

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
	AdcChannel3 AdcChannel5 AdcChannel0 AdcChannel2	parameter.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC0_8CH_QUE	ADC HW Unit Request source 0 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConvMode	ADC_CONV_MODE_CONTINUOUS	Continuous conversion for the SW trigger
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	2	2 samples of a channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcNotification	IoHwAb_Adc_Notification_AdcEmuxScanGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcEmuxScanEnable	OFF	Emux Scan mode enabled/disabled
AdcChannelEmuxSelect	5	Emux select line value to select the external analog channel.

Table 274 Container: AdcChannel

Configuration Parameter	AdcChannel5 value	AdcChannel3/0/2 value
Container name (The container	AdcChannel5	AdcChannel3/0/2

Configuration Parameter	AdcChannel5 value	AdcChannel3/0/2 value
name will give the Channel symbolic name.)		
AdcChannelId	1	0/2/3
AdcAnChannelNum	5	3/0/2
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcInputClassSelection	Refer to AdcGlobInputClass_0	Refer to AdcGlobInputClass_0

Table 275 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	2
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	2

8.9.2 Interrupt priority Configuration

Interrupt priorities are configured in Irq module. Refer [\[11\]](#).

Table 276 Container: IrqAdc0PrioConfig

Configuration Parameter	Value
IrqAdc0SR0Prio	45 (Non zero value)
IrqAdc0SR1Prio	NA
IrqAdc0SR2Prio	NA
IrqAdc0SR3Prio	NA

Table 277 Container: IrqAdc0TosConfig

Configuration Parameter	Value
IrqAdc0SR0Tos	CPU0
IrqAdc0SR1Tos	NA
IrqAdc0SR2Tos	NA
IrqAdc0SR3Tos	NA

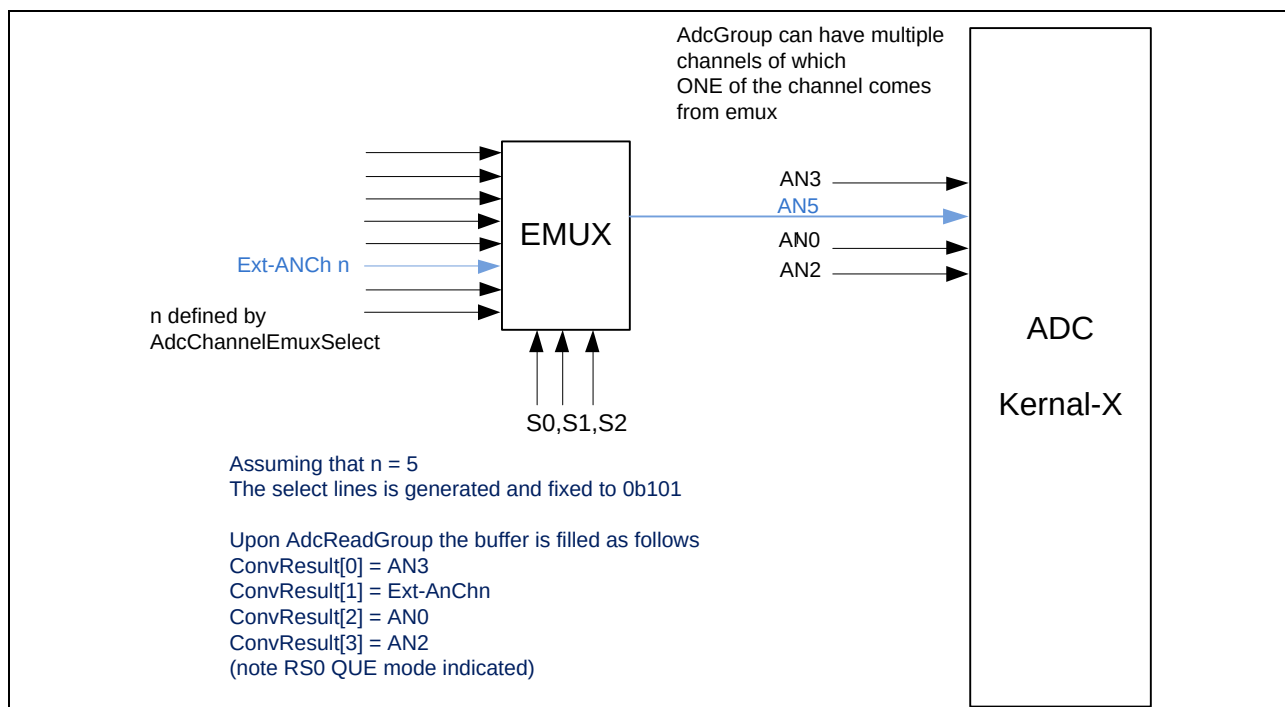


Figure 18 ADC Emux Mode

8.10 Configuration and Usage of ADC Driver in EMUX SCAN mode

1. The EMUX SCAN mode can be enabled for any two ADC Kernels per configuration.
2. In this feature, on each conversion round, the EMUX select line is automatically decremented by 1 from configured value. This will facilitate to scan all the analog channels (up to 8) connected to the multiplexer. The channel control values (i.e. reference voltage, input class selection, result register selection) for all the scanning channels will remain same.
3. Group with EMUX SCAN in SW Trigger type needs to be configured for continuous conversion mode only because of ADC HW Unit behavior.
4. For Group with channel EMUX SCAN mode, the DMA is used to transfer the converted result from result register to Driver Internal buffer (Adc_DmaBuffer). User configured DMA channel is used and is triggered by ADCx_SR3
5. DMA will interrupt the CPU on transfer completion through selected DMA channels service request line.
6. A Group with channel EMUX SCAN mode cannot be suspended in PRIORITY_HW_SW.
7. A Group with EMUX SCAN mode, the status of Group is stalled at ADC_BUSY till conversion for streaming number of samples is completed. The channel results are stored in internal buffer using DMA. On DMA interrupt, the Group status is updated to ADC_STREAM_COMPLETED and Channel Results are available in user buffer.
8. Notification function is called only if the conversion for streaming number of samples is completed. i.e., as per below example, for 25 streaming number of sample conversion, notification function is called only once.

8.10.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 278 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcUseEmux	ON
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcResultAccumulation	None
AdcPBFixedAddress	OFF

Table 279 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	HWUNIT_ADC0
AdcEmuxSelectGrp1	HWUNIT_ADC1

Table 280 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	Reference to DMA_CHANNEL24

Table 281 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcEmuxScanGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to:	This is a reference

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
	AdcChannel5	parameter.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConvMode	ADC_CONV_MODE_CONTINUOUS	Continuous conversion for the SW trigger
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	25	25 samples of a channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcNotification	IoHwAb_Adc_Notification_AdcEmuxScanGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcEmuxScanEnable	ON	Emux Scan mode enabled/disabled
AdcChannelEmuxSelect	7	Emux select line value to select the external analog channels.

Table 282 Container: AdcChannel

Configuration Parameter	AdcChannel5 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel5
AdcChannelId	0
AdcAnChannelNum	5

Configuration Parameter	AdcChannel5 value
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF
AdcChannelRefVoltsrcLow	REF_VOLTAGE_VAGND
AdcInputClassSelection	Refer to AdcGlobInputClass_0
AdcChannelLimitCheck	OFF
AdcChannelHighLimit	NA
AdcChannelLowLimit	NA
AdcChannelRangeSelect	NA

Table 283 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	2
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	2

8.10.2 Interrupt priority Configuration

Interrupt priorities are configured in Irq module. Refer [\[11\]](#).

Table 284 Container: IrqAdc0PrioConfig

Configuration Parameter	Value
IrqAdc0SR0Prio	NA
IrqAdc0SR1Prio	NA
IrqAdc0SR2Prio	NA
IrqAdc0SR3Prio	24 (Configured Dma channel number, Eg: DMA_CHANNEL0 = 0 DMA_CHANNEL1 = 1 DMA_CHANNEL2 = 2 DMA_CHANNEL3 = 3)

Table 285 Container: IrqAdc0TosConfig

Configuration Parameter	Value
IrqAdc0SR0Tos	NA
IrqAdc0SR1Tos	NA
IrqAdc0SR2Tos	NA
IrqAdc0SR3Tos	DMA

Table 286 Container: IrqDmaPrioConfig

Configuration Parameter	Value
IrqDmaCh24SRPrio	55 (Non zero value)

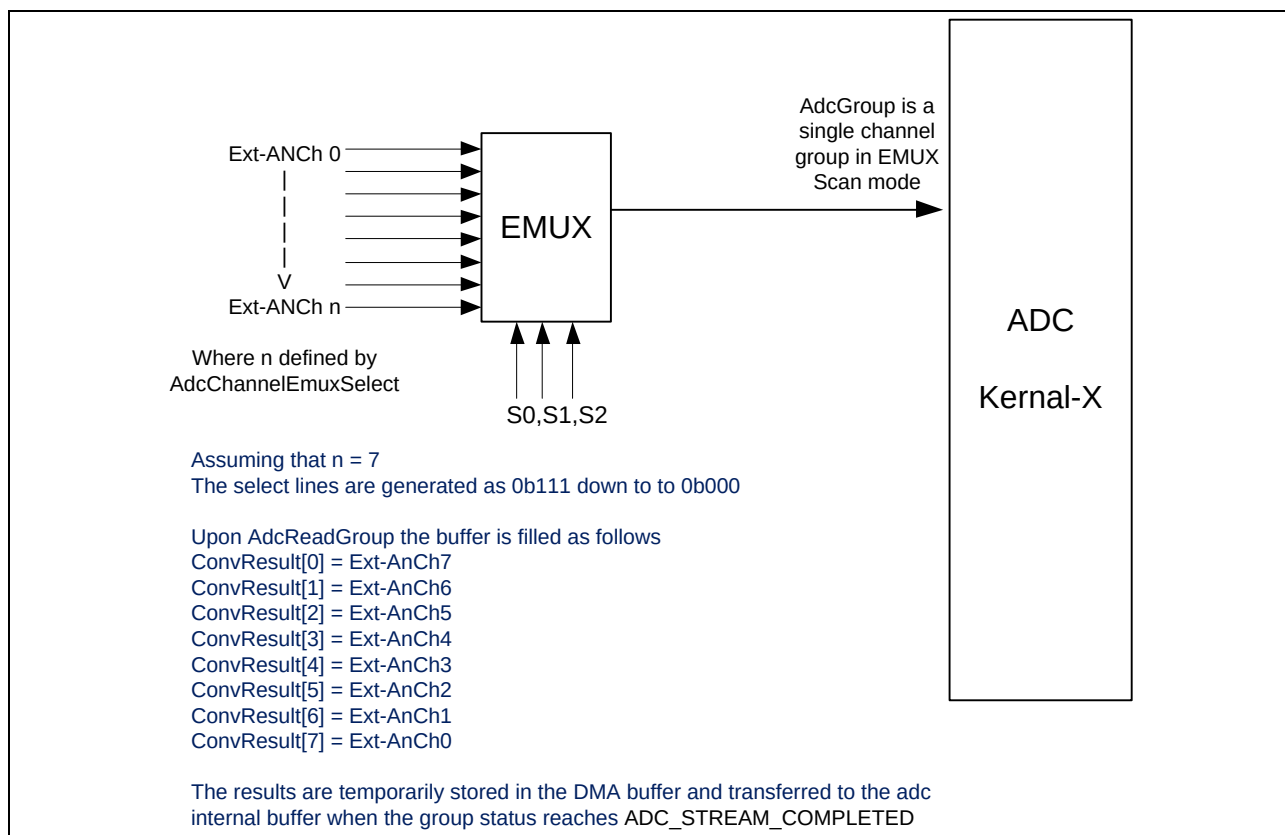


Figure 19 ADC Emux Scan Mode

8.11 Configuration and Usage of ADC Driver in Same Channel Queuing

1. Same channel Queuing is applicable only for the Request source REQSRC0_8CH_QUE.
2. In this feature, the same analog channel can be queued to convert maximum of 4 times in a group, i.e., same analog channel can be added in AdcGroupDefinition for maximum of 4 times in a group.
3. Standard Data Reduction Mode (Data Accumulation) is used for the same channel queuing mode.
4. Result of the same channel is accumulated in result register/buffer as many times it is been queued. i.e., as per below configuration, result of AdcChannel0 is accumulated 3 times and AdcChannel1 is accumulated 2 times in a result buffer.
5. In this feature, the repeated same channel entries in configuration parameter AdcGroupDefnition should be consider as individual channels and buffer should be provided for all the entries by the ADC module's environment. As per below configuration the result buffer size should be 5. The accumulated results are available in all the buffers.

8.11.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 287 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	ON
AdcGrpNotifCapability	ON
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	ON
AdcResultAccumulation	None
AdcPBFixedAddress	OFF

Table 288 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	NA
AdcEmuxSelectGrp1	NA

Table 289 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA

Table 290 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcSameChQueGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel0 AdcChannel1 AdcChannel0	This is a reference parameter.

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
	AdcChannel1 AdcChannel0	
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC0_8CH_QUE	ADC HW Unit Request source 0 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConvMode	ADC_CONV_MODE_ONESHOT	Continuous conversion for the SW trigger
AdcGroupAccessMode	ADC_ACCESS_MODE_SINGLE	Streaming buffer is used
AdcStreamingNumSamples	1	1 samples of a channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcNotification	IoHwAb_Adc_Notification_AdcSameChQueGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcEmuxScanEnable	OFF	Emux Scan mode enabled/disabled
AdcChannelEmuxSelect	NA	Emux select line value to select the external analog channel.

Table 291 Container: AdcChannel

Configuration Parameter	AdcChannel0 value	AdcChannel1 value
Container name (The container name will give the Channel symbolic	AdcChannel0	AdcChannel1

Configuration Parameter	AdcChannel0 value	AdcChannel1 value
name.)		
AdcChannelId	0	1
AdcAnChannelNum	0	1
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcInputClassSelection	Refer to AdcGlobInputClass_0	Refer to AdcGlobInputClass_0

Table 292 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	2
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	2

8.11.2 Interrupt priority Configuration

Interrupt priorities are configured in Irq module. Refer [\[11\]](#).

Table 293 Container: IrqAdc0PrioConfig

Configuration Parameter	Value
IrqAdc0SR0Prio	45 (Non zero value)
IrqAdc0SR1Prio	NA
IrqAdc0SR2Prio	NA
IrqAdc0SR3Prio	NA

Table 294 Container: IrqAdc0TosConfig

Configuration Parameter	Value
IrqAdc0SR0Tos	CPU0
IrqAdc0SR1Tos	NA
IrqAdc0SR2Tos	NA
IrqAdc0SR3Tos	NA

8.12 Configuration and Usage of ADC Driver in Synchronization of conversion

- Two Synchronization groups with four ADC kernels per synchronization groups are available in TC27x.
- Only one ADC kernel can be configured as Master per Synchronization group. Maximum of three kernels can be configured as Slave per Synchronization group.
- The AdcGroup of the Master kernel updates the result buffer such that results of all Master kernel channels are updated first and then the results of Slave kernel channels are updated. Applicable Slave kernels results are kept in the ascending order.

For e.g. ADC2 is configured as a Master kernel and ADC0 and ADC3 are configured as Slave kernels, AdcGroup of ADC2 contains 2 channels CH0 and CH1. Where, CH0 is configured for synchronization. While updating the result buffer, the ADC driver updates as below.

```
ADC2_CH0
ADC2_CH1
ADC0_CH0
ADC3_CH0
```

- In this feature, size of the buffer of ADC group which has synchronization of conversion enabled channel of Master Kernel, should be calculated as follows. Buffer Size = (Num of Channels in ADC group +

(Synchronization channels * Num of Slave kernels)). As per below configuration the result buffer size should be 4.

5. The parameter `AdcStreamingNumSamples` value should be 1, if `AdcSyncChannel` enabled `AdcChannel` is added in an `AdcGroup`.
6. If `AdcKernelInputClass` is selected for Synchronization of conversion enabled channel of Master kernel, the channels of Slave kernels selects the respective kernel's `AdcKernelInputClass` configured values.

8.12.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 295 Container: AdcGeneral

Configuration Parameter	Value
<code>AdcDevErrorDetect</code>	ON
<code>AdcGrpNotifCapability</code>	ON
<code>AdcUseEmux</code>	OFF
<code>AdcPriorityImplementation</code>	ADC_PRIORITY_NONE
<code>AdcEnableQueuing</code>	NA
<code>AdcMaxChConvTimeCount</code>	10000(Default)
<code>AdcVersionInfoApi</code>	ON
<code>AdcDeinitApi</code>	ON
<code>AdcEnableStartStopGroupApi</code>	ON
<code>AdcHwTriggerApi</code>	OFF
<code>AdcReadGroupApi</code>	ON
<code>AdcNonAutosarDmaResultHandling</code>	OFF
<code>AdcEnableLimitCheck</code>	OFF
<code>AdcResultAlignment</code>	ADC_ALIGN_RIGHT
<code>AdcDebugSupport</code>	OFF
<code>AdcSameChannelQueuing</code>	ON
<code>AdcResultAccumulation</code>	None
<code>AdcPBFixedAddress</code>	OFF
<code>AdcMasterSlaveSync</code>	ON
<code>AdcAlternateRefMode</code>	ADC_REF_LOCAL_MODE

Table 296 Container: AdcConfigSet

Configuration Parameter	Value
<code>AdcSleepMode</code>	SLEEP_MODE_ACCEPT
<code>AdcAnalogClockDivider</code>	10
<code>AdcDigitalClockDivider</code>	0
<code>AdcSystemClock</code>	Reference to McuClockSettingConfig
<code>AdcSHSClockDivider</code>	0
<code>AdcOperationMode</code>	NORMAL_MODE
<code>AdcCalibrationOrder</code>	ADC_CONVERT_THEN_CAL
<code>AdcCalMaxTime</code>	0x3F
<code>AdcCluster0AltRefDef</code>	NA
<code>AdcCluster1AltRefDef</code>	NA

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcEmuxSelectGrp0	NA
AdcEmuxSelectGrp1	NA

Table 297 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC2
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA
AdcSyncConvMode	ADC_MASTER
AdcAcceleratedTiming	DISABLE
AdcShortSampleTime	0x0

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	NA
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA
AdcSyncConvMode	ADC_SLAVE
AdcAcceleratedTiming	DISABLE
AdcShortSampleTime	0x0

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC3
AdcArbitrationRoundLength	NA
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA
AdcSyncConvMode	ADC_SLAVE
AdcAcceleratedTiming	DISABLE
AdcShortSampleTime	0x0

Table 298 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcSynchGroup	The container name will give the Group symbolic

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
		name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel0 AdcChannel1	This is a reference parameter.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConvMode	ADC_CONV_MODE_ONESHOT	Continuous conversion for the SW trigger
AdcGroupAccessMode	ADC_ACCESS_MODE_SINGLE	Streaming buffer is used
AdcStreamingNumSamples	1	1 samples of a channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcNotification	IoHwAb_Adc_Notification_AdcSynchGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcEmuxScanEnable	OFF	Emux Scan mode enabled/disabled
AdcChannelEmuxSelect	NA	Emux select line value to select the external analog channel.

Table 299 Container: AdcChannel

Configuration Parameter	AdcChannel0 value	AdcChannel1 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel0	AdcChannel1
AdcChannelId	0	1
AdcAnChannelNum	0	1
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcSyncChannel	ON	OFF
AdcInputClassSelection	Refer to AdcGlobInputClass_0	Refer to AdcGlobInputClass_0

Table 300 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	2
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	2

8.12.2 Interrupt priority Configuration

Interrupt priorities are configured in Irq module. Refer [\[11\]](#).

Table 301 Container: IrqAdc0PrioConfig

Configuration Parameter	Value
IrqAdc0SR0Prio	NA
IrqAdc0SR1Prio	46 (Non zero value)
IrqAdc0SR2Prio	NA
IrqAdc0SR3Prio	NA

Table 302 Container: IrqAdc0TosConfig

Configuration Parameter	Value
IrqAdc0SR0Tos	NA
IrqAdc0SR1Tos	CPU0
IrqAdc0SR2Tos	NA
IrqAdc0SR3Tos	NA

8.13 Usage of ADC Driver in HW groups with Background Request Source

1. HW trigger groups with Background Request Source (REQSRC2_BGND_SCAN) of different kernel (ADC HW Unit) are allowed to enable for conversion at the same time.
2. Only one Hardware triggered group with RS2 (Background Request Source) can be requested for conversion per Adc kernel.
3. It is recommended to use the identical groups (identical group configuration except AdcGroupDefinition) across the kernel, if the user required converting the HW trigger groups with Background Request Source of different kernels.
4. When Hardware triggered groups with RS2 (Background Request Source) from different ADC kernels are requested for conversion, the Request source expect only the trigger line of the first group which is enabled (Adc_EnableHardwareTrigger) for conversion. Other groups trigger lines are not considered. User should start the trigger by calling Adc_17_StartGtmTimer () or Adc_17_SetEruTrigger () APIs.
5. When Hardware triggered groups with RS2 (Background Request Source) from different ADC kernels are requested to stop the conversion, the Request source is stopped only after the last group is disabled

(Adc_DisableHardwareTrigger). User should stop the trigger by calling Adc_17_StopGtmTimer () or Adc_17_ResetEruTrigger () APIs.

Code Listing 12 Example – Usage of HW trigger groups with Background Request Source

```

001:     Std_ReturnType AdcBufferSetupStatus[4];
002:     Adc_ValueGroupType ADC_SW_GRP_RES_K0[8];
003:     Adc_ValueGroupType ADC_SW_GRP_RES_K1[8];
004:     Adc_ValueGroupType ADC_SW_GRP_RES_K2[8];
005:     Adc_ValueGroupType ADC_SW_GRP_RES_K5[8];
006:     static Adc_StreamNumSampleType Demo_AdcNoSamp[4];
007:
008:     /* Identical groups of different kernels are converted using RS2 */
009:
010:     /* Initialise the Result buffer for AdcHWBgndGroupK0 */
011:     AdcBufferSetupStatus[0] =
        Adc_SetupResultBuffer(AdcHWBgndGroupK0,ADC_SW_GRP_RES_K0);
012:     /* Initialise the Result buffer for AdcHWBgndGroupK1 */
013:     AdcBufferSetupStatus[1] =
        Adc_SetupResultBuffer(AdcHWBgndGroupK1,ADC_SW_GRP_RES_K1);
014:     /* Initialise the Result buffer for AdcHWBgndGroupK2 */
015:     AdcBufferSetupStatus[2] =
        Adc_SetupResultBuffer(AdcHWBgndGroupK0,ADC_SW_GRP_RES_K2);
016:     /* Initialise the Result buffer for AdcHWBgndGroupK5 */
017:     AdcBufferSetupStatus[3] =
        Adc_SetupResultBuffer(AdcHWBgndGroupK1,ADC_SW_GRP_RES_K5);
018:
019:     Adc_EnableGroupNotification(AdcHWBgndGroupK0);
020:     Adc_EnableGroupNotification(AdcHWBgndGroupK1);
021:     Adc_EnableGroupNotification(AdcHWBgndGroupK2);
022:     Adc_EnableGroupNotification(AdcHWBgndGroupK5);
023:
024:     /* Start the AdcHWBgndGroupK0 group */
025:     Adc_EnableHardwareTrigger(AdcHWBgndGroupK0);
026:     /* Start the AdcHWBgndGroupK1 group */
027:     Adc_EnableHardwareTrigger(AdcHWBgndGroupK1);
028:     /* Start the AdcHWBgndGroupK2 group */
029:     Adc_EnableHardwareTrigger(AdcHWBgndGroupK2);
030:     /* Start the AdcHWBgndGroupK5 group */
031:     Adc_EnableHardwareTrigger(AdcHWBgndGroupK5);
032:
033:     /* Start the trigger Signal */
034:     Adc_17_StartGtmTimer(AdcHWBgndGroupK0);
035:     /* Adc_17_SetEruTrigger(AdcHWBgndGroupK0); */
036:
037:     while( (Adc_GetGroupStatus(AdcHWBgndGroupK0)) !=
        ADC_STREAM_COMPLETED);
038:     while( (Adc_GetGroupStatus(AdcHWBgndGroupK1)) !=
        ADC_STREAM_COMPLETED);
039:     while( (Adc_GetGroupStatus(AdcHWBgndGroupK2)) !=
        ADC_STREAM_COMPLETED);
040:     while( (Adc_GetGroupStatus(AdcHWBgndGroupK5)) !=
        ADC_STREAM_COMPLETED);

```

Code Listing 12 Example – Usage of HW trigger groups with Background Request Source

```

041:
042:     Demo_AdcNoSamp[0] = (Std_ReturnType)Adc_ReadGroup(AdcHWBgndGroupK0,
    &ADC_SW_GRP_RES_K0[0]);
043:     Demo_AdcNoSamp[1] = (Std_ReturnType)Adc_ReadGroup(AdcHWBgndGroupK1,
    &ADC_SW_GRP_RES_K1[0]);
044:     Demo_AdcNoSamp[2] = (Std_ReturnType)Adc_ReadGroup(AdcHWBgndGroupK2,
    &ADC_SW_GRP_RES_K2[0]);
045:     Demo_AdcNoSamp[3] = (Std_ReturnType)Adc_ReadGroup(AdcHWBgndGroupK5,
    &ADC_SW_GRP_RES_K5[0]);
046:
047:     /* Stop the conversion which will reset the status and result buffer
048:        and notification is disabled */
049:     if( (Adc_GetGroupStatus(AdcHWBgndGroupK0)) != ADC_IDLE)
050:     {
051:         Adc_DisableHardwareTrigger(AdcHWBgndGroupK0);
052:     }
053:     if( (Adc_GetGroupStatus(AdcHWBgndGroupK1)) != ADC_IDLE)
054:     {
055:         Adc_DisableHardwareTrigger(AdcHWBgndGroupK1);
056:     }
057:     if( (Adc_GetGroupStatus(AdcHWBgndGroupK2)) != ADC_IDLE)
058:     {
059:         Adc_DisableHardwareTrigger(AdcHWBgndGroupK2);
060:     }
061:     if( (Adc_GetGroupStatus(AdcHWBgndGroupK2)) != ADC_IDLE)
062:     {
063:         Adc_DisableHardwareTrigger(AdcHWBgndGroupK2);
064:     }
065:     /* Print the latest converted result of the channel */
066:     printf("\nADC Conversion Results (Raw Value): \n");
067:     printf("K0 : %4x\n",ADC_SW_GRP_RES_K0[0]);
068:     printf("K1 : %4x\n",ADC_SW_GRP_RES_K1[0]);
069:     printf("K2 : %4x\n",ADC_SW_GRP_RES_K2[0]);
070:     printf("K5 : %4x\n",ADC_SW_GRP_RES_K5[0]);
071:
072:     /* Stop the trigger Signal */
073:     Adc_17_StopGtmTimer(AdcHWBgndGroupK0);
074:     /* Adc_17_ResetEruTrigger(AdcHWBgndGroupK0); */

```

8.14 Usage of ADC Driver in Non AUTOSAR Result handling mode

1. When configured as Non AutoSAR mode, the Driver priority will be considered as ADC_PRIORITY_NONE and Queuing is disabled.
2. The Group status can range between ADC_IDLE, ADC_BUSY, and ADC_STREAM_COMPLETED. The AUTOSAR specific status "ADC_COMPLETED" is not supported.
3. The APIs Adc_SetupResultBuffer, Adc_ReadGroup, Adc_GetStreamLastPointer are compile time switched off.
4. Parameters AdcReadGroupApi, AdcGroupAccessMode, AdcStreamingBufferMode, AdcStreamingNumSamples, AdcEmuxScanEnable, AdcEnableQueuing and AdcEnableLimitCheck are not configurable.
5. Group EMUX SCAN mode can't be used.

6. The DMA driver needs to be configured by the user for DMA channel selected for ADC Group Result handling (as in ADC Driver config). Selection of available DMA channels is provided for each ADC hardware unit through the configuration parameter "AdcDmaChannelRef".
7. DMA channel is to be configured for HW trigger.
8. The user can select the number of result accumulation over conversions. The selection is provided by pre-compile configuration parameter "AdcResultAccumulation". This value is maintained uniform across the driver for all group channels. The accumulated results are available in the result register. After the configured number of result accumulation, the result event is generated which triggers the DMA to copy the data. "AdcResultAccumulation" is configurable for 2 to 4 conversions.
9. For One shot, SW Triggered ADC Group, the result accumulation is always considered to be None irrespective of configured "AdcResultAccumulation". i.e., Accumulated conversion is not supported for the SW triggered ADC Group in Non Autosar DMA Result Handling mode.
10. Continuous conversion is not supported for the SW triggered ADC groups in Non Autosar DMA Result Handling mode, hence continuous conversion shall not be configured by user for the SW triggered ADC groups in Non Autosar DMA Result Handling mode.

For the example selected in section [8.1.9](#), following points give the use case along with the sequence diagram [Figure 20](#).

1. The ADC Group conversion is initiated by calling ADC Driver API `Adc_StartGroupConversion`.
2. Once the Group conversion is completed, the DMA channel will be triggered to transfer the result from ADC channel result register to the destination buffer.
3. If the group analog channels are not in sequence, the missing channel result needs be ignored in the user buffer. It is up to the user to configure DMA channel for data transfer.
For Example: Consider the Group analog channels from section [8.1.9](#): 5, 0. The user can configure the DMA channel to move the analog channel results from Channel 0 to Channel 5. In the DMA destination buffer, the user needs to ignore the values of Channel 1 to Channel 4.
4. DMA channel is configured for Transaction completion interrupt. In this ISR the user can program for ADC result handling.
5. In case of continuous conversion, after the configured number of conversions API `Adc_StopGroupConversion` is called to stop the conversion. This can be done from the DMA Transaction completion ISR.
6. DMA Reset channel API: 'Dma_ResetChannel' is to be called when DMA transaction is not complete. This can happen when: In ADC continuous conversion the ongoing conversion is stopped and DMA transaction is incomplete.
7. For new Start/Enable of ADC conversion, DMA transfer should be enabled.

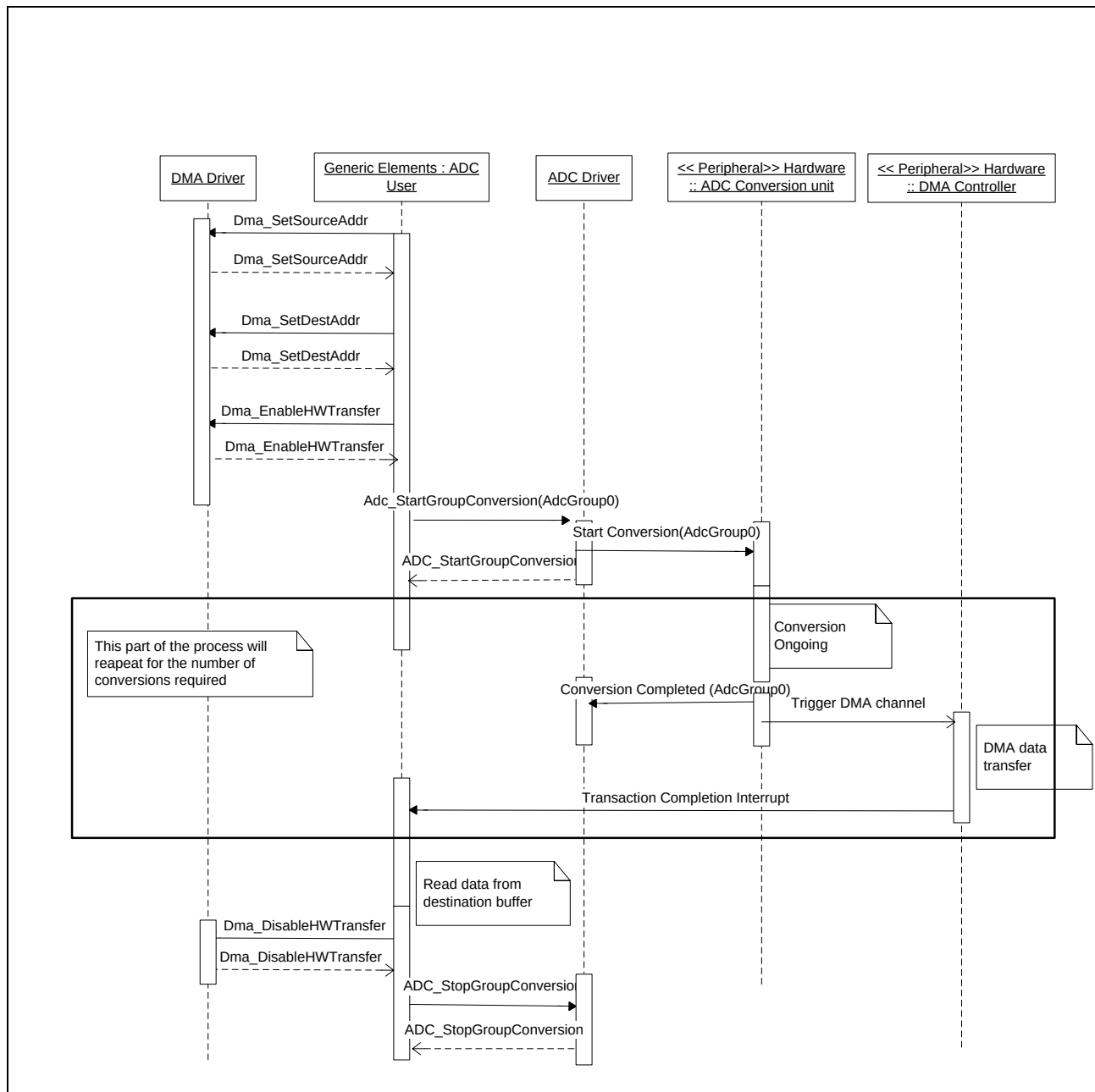


Figure 20 Non AUTOSAR Result Handling

8.15 Configuration and Usage of ADC Driver with Safety

8.15.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 303 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	OFF
AdcGrpNotifCapability	ON

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	ON
AdcNonAutosarDmaResultHandling	OFF
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcResultAccumulation	None
AdcPBFixedAddress	OFF
AdcSampleAndHoldUnitAvailable	OFF

Table 304 Container: AdcSafety

Configuration Parameter	Value
AdcSafetyEnable	True
AdcInitCheckApi	True

Table 305 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	HWUNIT_ADC0
AdcEmuxSelectGrp1	HWUNIT_ADC1

Table 306 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_DISABLE
AdcDmaChannelRef	NA
AdcSafetySignal	True

Table 307 Container: AdcGroup

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
Container Name	AdcSafeGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcGroupDefinition	Reference to: AdcChannel0	This is a reference parameter.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC0_8CH_QUE	ADC HW Unit Request source 0 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConvMode	ADC_CONV_MODE_CONTINUOUS	Continuous conversion for the SW trigger
AdcGroupAccessMode	ADC_ACCESS_MODE_STREAMING	Streaming buffer is used
AdcStreamingNumSamples	2	2 samples of a channel is storable in streaming buffer
AdcStreamingBufferMode	ADC_STREAM_BUFFER_LINEAR	After the conversion completion of configured number of samples, the Group is disabled from conversion.
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcNotification	Adc_Notif_AdcSafeGroup	Group notification function name
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE
AdcEmuxScanEnable	OFF	Emux Scan mode enabled/disabled
AdcChannelEmuxSelect	NA	Emux select line value to select the external analog channel.
AdcGroupBufferMarker	0xc8	

Table 308 Container: AdcChannel

Configuration Parameter	AdcChannel0 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel0
AdcChannelId	1
AdcAnChannelNum	0
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF
AdcInputClassSelection	Refer to AdcGlobInputClass_0

Table 309 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	2
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	2

8.15.2 Interrupt priority Configuration

Interrupt priorities are configured in Irq module. Refer [\[11\]](#).

Table 310 Container: IrqAdc0PrioConfig

Configuration Parameter	Value
IrqAdc0SR0Prio	45 (Non zero value)
IrqAdc0SR1Prio	NA
IrqAdc0SR2Prio	NA
IrqAdc0SR3Prio	NA

Table 311 Container: IrqAdc0TosConfig

Configuration Parameter	Value
IrqAdc0SR0Tos	CPU0
IrqAdc0SR1Tos	NA
IrqAdc0SR2Tos	NA
IrqAdc0SR3Tos	NA

8.15.3 Usage of Safety Group

1. Refer the chapter 9 for CoU of the ADC driver.
2. Parameter range checks are performed for all Api.
3. Variables specific to the ASIL and QM signal paths are separated and the respective variables are placed in different memory sections.
4. Variables of Safety signal enabled ADC HW units of ADC Cluster-0 are placed in one memory section.
5. Variables of Safety signal enabled ADC HW units of ADC Cluster-1 and Cluster-2 are placed in one memory section.
6. Variables of other ADC HW units (QM signal) of ADC Cluster-0, Cluster-1 and Cluster-2 are placed in one memory section
7. If ASIL/QM signals are requested for conversion, variable of the respective ASIL/QM signals are identified and used.
8. Api Adc_SetupResultBuffer and Adc_ReadGroup verify the buffers by comparing the configured Buffer Marker with the value present in the first index of the passed buffer.

9. `Adc_ValueGroupType` is of Type `uint8`, the configured `AdcGroupBufferMarker` value should be written to the buffer passed to the Api `Adc_SetupResultBuffer` and `Adc_ReadGroup`. E.g. if value parameter `AdcGroupBufferMarker` is `0xC8`, then the value `0xC8` should be written to the first index of the passed buffer (`ResultBuffer[0] = 0xC8`).
10. `Adc_ValueGroupType` is of Type `uint16`, the configured `AdcGroupBufferMarker` value should be written to the buffer passed to the Api `Adc_SetupResultBuffer` and `Adc_ReadGroup`. E.g. if value parameter `AdcGroupBufferMarker` is `0xC8`, then the value `0x00C8` should be written to the first index of the passed buffer (`ResultBuffer[0] = 0x00C8`).
11. Buffer Marker needs to be configured even for a QM group, if Safety is enabled.
12. `Adc_GetStreamLastPointer` Api parameter range check is not done for the output parameter `PtrToSamplePtr`, since it has second level of dereferencing.

Code Listing 13 Example code of Safety group

```

001:  Adc_ValueGroupType Adc_GrpResBuff[2];
002:  Adc_ValueGroupType Adc_GrpReadBuff[2];
003:  Std_ReturnType AdcBufferSetupStatus;
004:  Std_ReturnType AdcReadStatus;
005:
006:  /* Initialize the ADC Driver */
007:  Adc_Init(&Adc_ConfigRoot[0]);
008:
009:  while( (Adc_17_GetStartupCalStatus()) != E_OK);
010:
011:  #if (ADC_INITCHECK_API == STD_ON )
012:
013:  if(Adc_InitCheck(&Adc_ConfigRoot[0]) != E_OK)
014:  {
015:      printf("\n Adc_InitCheck FAILED \n");
016:  }
017:  else
018:  {
019:      printf("\n Adc_InitCheck PASSED \n");
020:  }
021:
022:  #endif /* (ADC_INITCHECK_API == STD_ON ) */
023:
024:  Adc_GrpResBuff[0] = 0x00C8;
025:
026:  /* Initialise the Result buffer for AdcSafeGroup */
027:  AdcBufferSetupStatus =
    Adc_SetupResultBuffer(AdcSafeGroup,Adc_GrpResBuff);
028:
029:  if(AdcBufferSetupStatus == E_OK)
030:  {
031:      printf("\n Adc_GrpResBuff buffer is setup for AdcSafeGroup");
032:      /* Initialise the no of converted samples to 0 before starting the
033:      conversion
034:      */
035:      /* Start the AdcSafeGroup group */
036:      Adc_StartGroupConversion(AdcSafeGroup);
037:
038:      Adc_EnableGroupNotification(AdcSafeGroup);

```


Code Listing 13 Example code of Safety group

```

039:
040:     while( (Adc_GetGroupStatus(AdcSafeGroup)) != ADC_STREAM_COMPLETED);
041:
042:     Adc_GrpReadBuff[0] = 0x00C8;
043:
044:     /* Read the result of Adc Group */
045:     AdcReadStatus =
046:     (Std_ReturnType)Adc_ReadGroup(AdcSafeGroup,Adc_GrpReadBuff);
047:
048:     Adc_DisableGroupNotification(AdcSafeGroup);
049:     /* Print the result of the channels */
050:     printf("\n ADC Conversion Results (Raw Value): \n");
051:     printf("AN0 : %4x\n",Adc_GrpReadBuff[0]);
052:     printf("AN0 : %4x\n",Adc_GrpReadBuff[1]);
053:     /* Check if the Group has implicitly stopped and status is IDLE
054:     before Stop request
055:     */
056:     if( (Adc_GetGroupStatus(AdcSafeGroup)) != ADC_IDLE)
057:     {
058:         /* Stop the conversion which will reset the status and result
059:         buffer
060:         and notification is disabled
061:         */
062:         Adc_StopGroupConversion(AdcSafeGroup);
063:     }
064:     else
065:     {
066:         printf("\nError in Adc_GrpResBuff buffer setup for AdcSafeGroup");
067:     }

```

8.16 Configuration and Usage of ADC Driver in Result Polling Mode

8.16.1 ADC Driver Configuration

The configuration of each of the container is as shown below:

Table 312 Container: AdcGeneral

Configuration Parameter	Value
AdcDevErrorDetect	OFF
AdcGrpNotifCapability	OFF
AdcUseEmux	OFF
AdcPriorityImplementation	ADC_PRIORITY_NONE
AdcEnableQueuing	NA
AdcMaxChConvTimeCount	10000(Default)
AdcVersionInfoApi	ON
AdcDeinitApi	ON

CONFIDENTIAL
Example Usage

Configuration Parameter	Value
AdcEnableStartStopGroupApi	ON
AdcHwTriggerApi	OFF
AdcReadGroupApi	OFF
AdcNonAutosarDmaResultHandling	OFF
AdcEnableLimitCheck	OFF
AdcResultAlignment	ADC_ALIGN_RIGHT
AdcDebugSupport	OFF
AdcSameChannelQueuing	OFF
AdcResultAccumulation	None
AdcPBFixedAddress	OFF
AdcSampleAndHoldUnitAvailable	OFF
AdcNonAutosarDmaResultPolling	ON
AdcResetSfrAtInit	OFF

Table 313 Container: AdcSafety

Configuration Parameter	Value
AdcSafetyEnable	True
AdcInitCheckApi	True

Table 314 Container: AdcConfigSet

Configuration Parameter	Value
AdcSleepMode	SLEEP_MODE_ACCEPT
AdcAnalogClockDivider	10
AdcDigitalClockDivider	0
AdcSystemClock	Reference to McuClockSettingConfig
AdcEmuxSelectGrp0	NA
AdcEmuxSelectGrp1	NA

Table 315 Container: AdcHWUnit

Configuration Parameter	Value
AdcHwUnitId	HWUNIT_ADC0
AdcArbitrationRoundLength	ARBITRATION_SLOTS_4
AdcClockSource	CLK_SRC_SYS_CLK
AdcRequestSource0Prio	NA – for ADC_PRIORITY_NONE
AdcRequestSource1Prio	
AdcPostCal	POST_CAL_ENABLE
AdcDmaChannelRef	NA

Table 316 Container: AdcGroup

Configuration Parameter	Value	Description
Container Name	AdcSwTriggGroup	The container name will give the Group symbolic name.
AdcGroupPriority	NA	Not configurable parameter in ADC_PRIORITY_NONE

CONFIDENTIAL
Example Usage

Configuration Parameter	Value	Description
AdcGroupDefinition	Reference to: AdcChannel5 AdcChannel0	This is a reference parameter. The channels are arranged in Descending order.
AdcGroupTriggSrc	ADC_TRIGG_SRC_SW	Trigger source type is software
AdcGroupRequestSource	REQSRC1_NCH_SCAN	ADC HW Unit Request source 1 is selected
AdcHwTrigType	NA	Not configurable parameter in SW trigger.
AdcGroupConversionMode	ADC_CONV_MODE_ONESHOT	One Shot conversion for the SW trigger. Continuous conversion for the SW trigger is not supported in Non Autosar result Polling mode
AdcGroupAccessMode	NA	Not configurable parameter in Non AUTOSAR Result handling mode
AdcStreamingNumSamples	NA	Not configurable parameter in Non AUTOSAR Result handling mode
AdcStreamingBufferMode	NA	Not configurable parameter in Non AUTOSAR Result handling mode
AdcHwTrigSignal	NA	Not configurable parameter in SW Trigger.
AdcHwExtTriggSelect	NA	Not configurable parameter in SW Trigger.
AdcHwGateSignal	NA	Not configurable parameter in SW Trigger.
AdcHwGatePin	NA	Not configurable parameter in SW Trigger.
AdcGtmTimerPeriod	NA	Not configurable parameter in SW Trigger
AdcGtmTimerRef	NA	Not configurable parameter in SW Trigger
AdcEruTriggRef	NA	Not configurable parameter in SW Trigger
AdcEruGateRef	NA	Not configurable parameter in SW Trigger
AdcChannelEmuxSelect	NA	Not applicable for Emux disabled groups
AdcEmuxScanEnable	NA	Not applicable in Result Polling mode
AdcNotification	NA	Not applicable in Result Polling mode
AdcGroupReplacement	NA	Not configurable parameter in ADC_PRIORITY_NONE

Table 317 Container: AdcChannel

Configuration Parameter	AdcChannel5 value	AdcChannel0 value
-------------------------	-------------------	-------------------

Configuration Parameter	AdcChannel5 value	AdcChannel0 value
Container name (The container name will give the Channel symbolic name.)	AdcChannel5	AdcChannel0
AdcChannelId	1	0
AdcAnChannelNum	5	0
AdcChannelRefVoltsrcHigh	REF_VOLTAGE_VAREF	REF_VOLTAGE_VAREF
AdcInputClassSelection	Refer to AdcGlobInputClass_0	Refer to AdcGlobInputClass_0

Table 318 Container: AdcGlobInputClass

Configuration Parameter	AdcGlobInputClass_0
AdcGlobChSampleTime	10
AdcChannelResolution	12-bit
AdcGlobEmuxChSampleTime	NA

8.16.2 Usage of Result Polling Mode

- When configured as Non AutoSAR Polling mode, the Driver priority will be considered as ADC_PRIORITY_NONE and Queuing is disabled.
- The AUTOSAR specific group status ADC_IDLE, ADC_BUSY and ADC_STREAM_COMPLETED and ADC_COMPLETED are not supported.
- The APIs Adc_SetupResultBuffer, Adc_ReadGroup, Adc_GetStreamLastPointer, Adc_EnableGroupNotification, Adc_DisableGroupNotification and Adc_GetGroupStatus are compile time switched off.
- Parameters AdcReadGroupApi, AdcGrpNotifCapability, AdcGroupAccessMode, AdcStreamingBufferMode, AdcStreamingNumSamples, AdcEmuxScanEnable, AdcEnableQueuing, AdcSameChannelQueuing and AdcEnableLimitCheck are not configurable.
- Group EMUX SCAN mode can't be used.
- The user can select the number of result accumulation over conversions. The selection is provided by pre-compile configuration parameter "AdcResultAccumulation". This value is maintained uniform across the driver for all group channels. The accumulated results are available in the result register. "AdcResultAccumulation" is configurable for 2 to 4 conversions.
- For One shot, SW Triggered ADC Group, the result accumulation is always considered to be None irrespective of configured "AdcResultAccumulation". i.e., Accumulated conversion is not supported for the SW triggered ADC Group in Result Polling mode.
- Continuous conversion is not supported for the SW triggered ADC groups in Result Polling mode, Hence continuous conversion shall not be configured by user for the SW triggered ADC groups in Result Polling mode.

Code Listing 14 Example code of Safety group

```

001:  Adc_ValueGroupType Adc_GrpReadBuff[2];
002:  Std_ReturnType AdcBufferSetupStatus;
003:  Std_ReturnType AdcReadStatus;
004:
005:  /* Initialize the ADC Driver */
006:  Adc_Init(&Adc_ConfigRoot[0]);
007:
008:  while( (Adc_17_GetStartupCalStatus()) != E_OK);
009:
010:  #if (ADC_INITCHECK_API == STD_ON )
011:
012:  if(Adc_InitCheck(&Adc_ConfigRoot[0]) != E_OK)

```

Code Listing 14 Example code of Safety group

```

013:  {
014:      printf("\n Adc_InitCheck FAILED \n");
015:  }
016:  else
017:  {
018:      printf("\n Adc_InitCheck PASSED \n");
019:  }
020:
021:  #endif /* (ADC_INITCHECK_API == STD_ON ) */
022:
023:  /* Initialise the no of converted samples to 0 before starting the
024:      conversion
025:  */
026:  /* Start the AdcSafeGroup group */
027:  Adc_StartGroupConversion(AdcSafeGroup);
028:
029:  while( (Adc_17_GetChannelStatus(0U)) != E_OK);
030:
031:  Adc_GrpReadBuff[0] = 0x00C8;
032:
033:  /* Read the result of Adc Group */
034:  AdcReadStatus =
    (Std_ReturnType)Adc_17_GetGroupResult(AdcSafeGroup,Adc_GrpReadBuff);
035:
036:  /* Print the result of the channels */
037:  printf("\n ADC Conversion Results (Raw Value): \n");
038:  printf("AN0 : %4x\n",Adc_GrpReadBuff[0]);
039:  printf("AN5 : %4x\n",Adc_GrpReadBuff[1]);
040:
041:  Adc_StopGroupConversion(AdcSafeGroup);
042:

```

9 Assumptions and Limitations

9.1 Assumptions and Deviations from the Software Specification

Assumption - : To maintain a same xdm across TC27xCA (TC26xBA, TC29xBA & TC23xAA) and TC27xDB (TC26xBB, TC29xBB & TC23xAB) step the parameter, AdcCcu6Synchronization, is added to make a configuration difference. The parameter, AdcCcu6ModuleRef is editable only if AdcCcu6Synchronization is true.

The ADC driver deviations from the specification are listed below:

1. We are deviating from the use of TRUE, FALSE to using it as STD_ON, STD_OFF for the configuration switches.
2. In ADC_PRIORITY_HW_SW mechanism, If more Groups has same priority level and enters the HW SW implemented Queue, the implementation will schedule the Groups in the order of its Group ID rather than 'first come first served' order. The requirement ADC417 is deviated to minimize the ISR overload time.
3. In ADC_PRIORITY_HW_SW mechanism, if a group with the same priority as the currently converting group is started for the conversion, the currently converting group continues to convert and the started conversion is placed in the software queue.

4. Deviation from ADC316: The `AdcStreamingBufferMode` needs to be configured even in Single Access mode. This is done since, in Single Access mode, the the buffer is assumed to be circular in case of continuous conversions, and linear in case of Oneshot conversions. The possible combinations of the Group trigger type, conversion mode, access mode and buffer mode is depicted in [Figure 6](#).
5. Deviation from ADC345, ADC430, and ADC435_Conf: Suspend Resume mechanism not supported.
6. Deviation from ADC019_Conf: Channel resolution configuration moved to `AdcGlobInputClass` and `AdcKernelInputclass` container and a reference parameter `AdcInputClassSelection` provided to select one of 4 input classes.
7. Deviation from ADC088_Conf: Configuration parameter `AdcPrescale` is not supported. The same functionality is achieved using the parameter `AdcAnalogClockDivider`.
8. Deviation from ADC290_Conf: Configuration parameter `AdcChannelSampTime` is not supported. The same functionality is achieved using parameters `AdcGlobChSampleTime` and `AdcKernelSampleTime`.
9. Deviation from ADC289, ADC287_Conf: Range of the configuration parameter `AdcGroupPriority` is restricted to 0 – 254, since only 32 groups per hardware unit are allowed to configure and the Hw trigger groups priorities are set to 255 (High priority) in `ADC_PRIORITY_HW_SW` mechanism.
10. Deviation from ADC453_Conf, ADC454_Conf, ADC455_Conf, and ADC456_Conf: Configuration parameters are implemented as Variant post build parameter, since the VADC hardware supports channel wise limitchecking.

9.2 Limitations

1. Maximum number of groups can be 32 in each ADC HW Unit
2. Maximum number of channel configurations can be 256 in each ADC HW Unit
3. A group can have maximum of 8 channels.
4. Interrupt latency/execution time problems - There may be errors if interrupt is not handled fast enough.
5. ADC HW unit configured for faster conversion time: ADC Group configured for Software trigger, Continuous conversion mode may complete the conversion before completion interrupt service routine. In this scenario, the program execution control will always be in ISR. The next statement of the application will never get the chance for execution.
6. The above scenario can even happen for Groups using Timer trigger source and external request trigger source. In case of Timer trigger source, execution control will never come out of ISR when timer period is small and ADC HW is triggered and the conversion finishes before the completion of ISR. In case of external request trigger source, execution control will never come out of ISR when external requesting is triggering the conversion and A/D conversion is finished before the ISR execution is completed.
7. Any two ADC hardware units can be enabled for using external multiplexer.
8. Group with EMUX SCAN enabled
 - `AdcGroupDefination` should contain one channel with `Emux select line` configured
 - Group should be configured for continuous conversion mode
 - The channel under EMUX SCAN Group should have EMUX select value greater than 0.
9. When a Group with EMUX SCAN enabled is requested for conversion, then the other Group with EMUX SCAN enabled is not accepted for conversion in any priority mechanism except No Priority Queue mechanism (DET: `ADC_E_BUSY` will be raised)
10. Hardware triggered groups are not supported for EMUX SCAN feature.
11. EMUX SCAN enabled group can not have Limit checking enabled channel.
12. In EMUX SCAN mode, Notification function is called only if the conversion for streaming number of samples is completed.
13. When a Software triggered group with RS2 (Backround Request Source) is requested for conversion, then the other Software/Hardware triggered group with RS2 is not accepted for conversion in any priority mechanism, irrespective of the `Adc channel group kernels` (DET: `ADC_E_BUSY` will be raised).
14. When a Hardware triggered group with RS2 (Backround Request Source) is requested for conversion, then the other Software triggered group with RS2 is not accepted for conversion in any priority mechanism, irrespective of the `Adc channel group kernels` (DET: `ADC_E_BUSY` will be raised).
15. When Hardware triggered group with RS2 (Backround Request Source) is requested for conversion, then the other Hardware triggered group with RS2 of the same ADC kernel should not be called for conversion.
16. When Hardware triggered groups with RS2 (Backround Request Source) from different ADC kernels are requested for conversion, the Request source expect only the trigger line of the first group which is

- enabled (Adc_EnableHardwareTrigger) for conversion. User should start the trigger by calling Adc_17_StartGtmTimer () or Adc_17_SetEruTrigger () APIs.
17. When Hardware triggered groups with RS2 (Background Request Source) from different ADC kernels are requested to stop the conversion, the Request source is stopped only after the last group is disabled (Adc_DisableHardwareTrigger). User should stop the trigger by calling Adc_17_StopGtmTimer () or Adc_17_ResetEruTrigger () APIs.
 18. The Group notification depends on RS priority as well as the interrupt node priority (does not always follow the priorities of the groups).
 19. All the API execution should be from lower priority as compared to ADC ISRs.
 20. Hardware trigger Groups of an ADC kernel which shares the same trigger unit (Eg: ERU trigger) should not be enabled for the conversion or wait for the trigger at the same time.
 21. When a limitcheck enabled group is requested (start/enable) for a conversion, another limit check enabled group of the same ADC kernel should not be requested for the conversion.
 22. API Adc_17_GetStartupCalStatus() returns E_NOT_OK, if the configured AdcConfigSets does not have identical Adc kernels. Eg: if AdcConfigSet_0 has ADC_HW_UNIT0 & ADC_HW_UNIT1, AdcConfigSet_1 has ADC_HW_UNIT0 & ADC_HW_UNIT2, then Adc_17_GetStartupCalStatus() always returns E_NOT_OK for both the AdcConfigSets.

9.2.1 Software Limitation in ADC_PRIORITY_HW mechanism

1. Groups with the same request source AdcGroupRequestSource will have the same priority level as per request source priority.
2. There can be only one request by the same request source Group. In case if there is a Second request by the group with same request source, a DET: ADC_E_BUSY will be reported.
3. ADC Groups configured for different conversion request sources but sharing some of the ADC internal channels cannot be started simultaneously. Here we are overruling the priority mechanism. In such scenario, Start/Enable service will report DET: ADC_E_BUSY.

For example,

Consider two ADC Groups configured as:

AdcGroup0-RS2, Group Definition {15, 10}

AdcGroup1-RS1, Group Definition {5, 15}

AdcGroup0 and AdcGroup1 cannot be requested for conversion simultaneously. If AdcGroup0 is requested for conversion and then immediately request for conversion for AdcGroup1 is received, AdcGroup1 is not serviced, instead a DET: ADC_E_BUSY will be reported even if its request source priority level is higher than AdcGroup0 request source priority level. Here we are overruling the priority mechanism because of internal channel sharing by the requested groups.

9.2.2 Software Limitation in ADC_PRIORITY_HW_SW mechanism

1. ADC Groups sharing some of the ADC internal channels cannot be enabled for conversion and will not enter the Software implemented Queue. Here we are overruling the priority mechanism. In such scenario, Start/Enable service will report DET: ADC_E_BUSY.

For example,

Consider two ADC Groups configured as:

AdcGroup0-RS2 – Priority 0, Group Definition {15, 10}

AdcGroup1-RS0 – Priority 2, Group Definition {5, 15}

AdcGroup0 and AdcGroup1 cannot be requested for conversion simultaneously. If AdcGroup0 is requested for conversion and then immediately request for conversion for AdcGroup1 is received, AdcGroup1 is not serviced, instead a DET: ADC_E_BUSY will be reported even if its Group priority level is higher than AdcGroup0 trigger source priority level. Here we are overruling the priority mechanism because of internal channel sharing by the requested groups.

9.2.3 Software Configuration Limitation

1. AdcChannelId for Adc Channels should be in a continuous sequence i.e. (0,1,2,3,...)
2. AdcGroupId for ADC Groups should be in a continuous sequence i.e. (0,1,2,3,...)
3. AdcGroupId for ADC Groups are generated by the configuration tool. User has to refer the AdcGroupId by their symbolic names. The symbolic names are configurable by the user.
4. ADC Groups are sorted in ADC Group configuration structure in the order of their trigger type and request source starting from software trigger groups of RS0 to RS2 and then followed by hardware trigger groups of RS0 to RS2. The configuration tool will automatically arrange the Groups in the order of their AdcGroupRequestSource
5. The first element of the Group definition [6.1.20] should be the number of configured channels in the group. From Second element will give the channel ID.

For example,

A group with channels (say) {4, 6, 7, 3, and 1}; first element of Group definition is the total number of channels which is 5.

9.2.4 Software Limitation in ADC_NON_AUTOSAR DMA Mode

1. Priority mechanism is disabled. The driver is always in ADC_PRIORITY_NONE with Queue disabled.
2. The Group channels have to be arranged in the descending order of their analog channel numbers.
3. EMUX Scan can not be used.
4. Limit checking can not be used.

9.2.5 Software Limitation in Safety feature

1. Background source can't be used for any of the group.
2. Emux Scan can't be used for any of the group.
3. Non Autosar DMA result handling mode can't be used.

9.2.6 Software Limitation in ADC_NON_AUTOSAR Result Polling Mode

1. Priority mechanism is disabled. The driver is always in ADC_PRIORITY_NONE with Queue disabled.
2. EMUX Scan can not be used.
3. Limit checking can not be used.
4. Notification functions are not supported
5. DMA channels are not used to transfer the conversion result values to the user defined location.
6. Interrupts are not used to update the status.
7. Autosar specific ADC_IDLE, ADC_BUSY, ADC_COMPLETED, ADC_STREAM_COMPLETED are not used.
8. The APIs Adc_SetupResultBuffer, Adc_ReadGroup, Adc_GetStreamLastPointer, Adc_EnableGroupNotification, Adc_DisableGroupNotification and Adc_GetGroupStatus are compile time switched off.
9. Parameters AdcReadGroupApi, AdcGrpNotifCapability, AdcGroupAccessMode, AdcStreamingBufferMode, AdcStreamingNumSamples, AdcEmuxScanEnable, AdcEnableQueuing and AdcEnableLimitCheck are not configurable.

9.3 Hardware Features Not Supported

This section lists down all the AURIX VADC hardware features that are not supported by the ADC Driver. These features are out of the scope of ADC Driver's specification.

1. FIFO of result register as not applicable by AUTOSAR requirement
2. Data Modification not supported.
3. Alias feature.
4. Broken wire detectection.

5. Multiplexer Diagnostics.
6. Converter Diagnostics.
7. Fast compare mode.

www.infineon.com